

Associate Architect 양성과정 · Architecture 설계

Day 2 · Architecture 스타일 (2) & Views

Distributed(2) / Deployment Architecture · Architecture Views · SEI View Model 문서화

- 1.7 Distributed Architecture (2) · 1.8 Deployment Architecture · 1.9 Summary
- 2.1 Architecture Views (4+1 / Siemens / SEI)
- 2.2 Documenting Architecture using SEI View Model
- 2.3 Hierarchical Multi-View Model & C4 Model

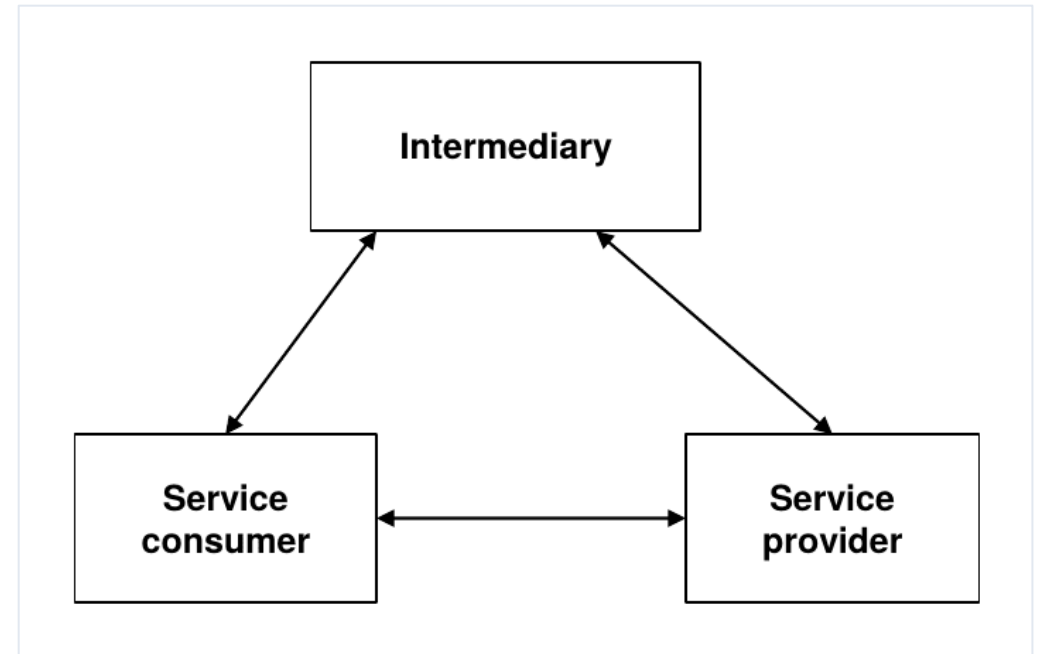
SOA Style — 구성 요소와 제약

■ Elements

- Service provider: 공개된 interface 로 하나 이상의 서비스 제공, 서비스 정보를 intermediary(service registry)에 등록
- Intermediary component: 서비스 정보를 잠재 소비자에게 공개, provider 와 consumer 사이 메시지를 라우팅·변환, 상호작용 조정
- Service consumer: 서비스를 직접 또는 intermediary 를 통해 호출

■ Constraints

- Platform 독립성
- Stateless 서비스
- 표준화된 통신

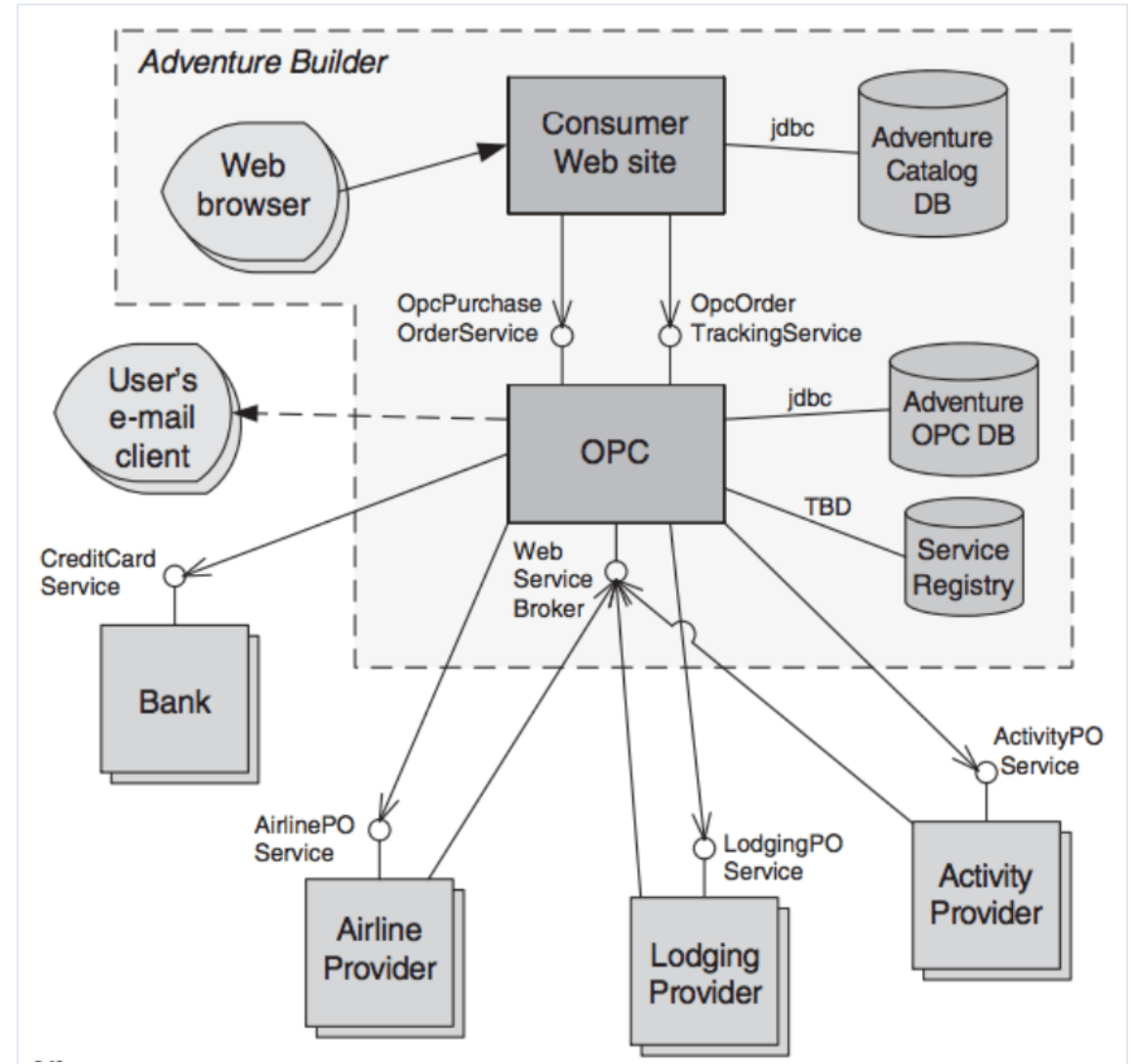


Relations: Intermediary – Service consumer – Service provider

SOA 예제 — Adventure Builder

■ 여행 상품 구성 시스템 (Bass et al., SAiP 3rd ed.)

- 외부 service provider(Airline, Lodging, Activity Provider)와 상호작용해 휴가 상품을 구성하고, Bank 서비스로 결제를 처리한다
- 중앙의 OPC(Order Processing Center) component 가 내·외부 service consumer 와 provider 사이의 상호작용을 조정한다
- Web Service Broker 와 Service Registry 가 intermediary 역할
- 연결 종류: HTTP/HTTPS, SOAP call, Data access(jdbc), SMTP



SOA Consequences: Benefits 상세

■ Modifiability 와 진화(Evolution)

- Stateless 서비스 특성 덕분에 각 서비스 component 는 독립적이다
- 공개된 interface 만 바뀌지 않으면 서비스 구현 변경이 application 에 영향을 주지 않는다 → SOA 소프트웨어는 진화·갱신이 훨씬 쉽다

■ Interoperability (상호운용성)

- 기술적으로 어떤 client 나 서비스든 platform, 기술, vendor, 언어와 무관하게 다른 서비스에 접근할 수 있다

■ Reusability (재사용성)

- 어떤 서비스든 다른 서비스가 재사용할 수 있다
- 서비스의 client 는 공개 interface 만 알면 되므로 서비스 조합 (composition)과 통합이 쉬워진다
- → SOA 기반 business application 개발이 시간·비용 면에서 훨씬 효율적이다

■ Scalability 와 부하 분산

- 각 서비스는 서로 다른 process, 심지어 다른 컴퓨터에서 실행될 수 있다
- Server 사용률이 한계에 가까워지면 컴퓨터를 추가해 일부 서비스를 새 컴퓨터로 옮겨 부하를 나눌 수 있다

SOA Consequences: 장점과 약점

■ Benefits (장점)

- Modifiability 와 Evolution — stateless 서비스, interface 유지 시 구현 변경 자유
- Interoperability — platform·기술·vendor·언어에 무관
- Reusability — 공개 interface 만 알면 서비스 조합·통합 가능
- Scalability 와 Workload Distribution — 서비스를 다른 process/컴퓨터로 분산

■ Weaknesses (약점)

- SOA 기반 시스템은 대체로 구축이 복잡하다
- 독립적인 서비스의 진화(evolution)를 내가 통제할 수 없다
- Middleware 로 인한 성능 overhead 가 있다
 - 서비스가 성능 병목이 될 수 있고, 보통 성능을 보장하지 않는다

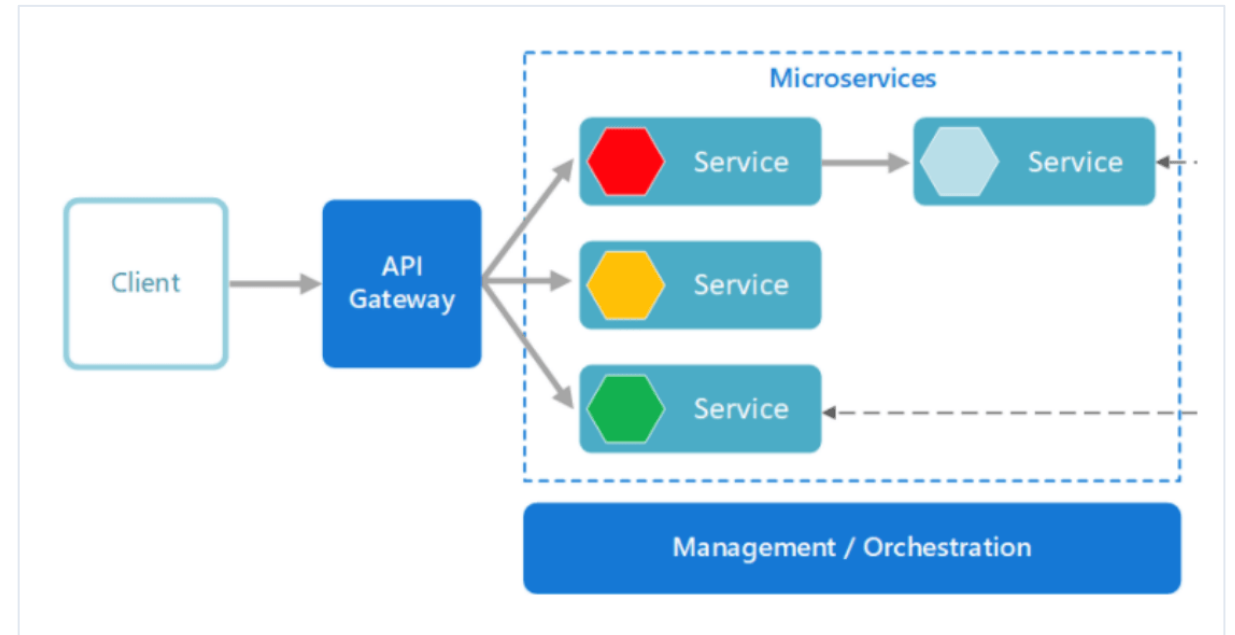
Microservice Architecture 개요

■ 정의

- 작고 자율적인(autonomous) 서비스들의 집합으로 구성되는 Architecture
- 각 서비스는 자기 완결적이며 하나의 business capability 를 구현한다

■ 구성 요소 (그림)

- Client → API Gateway → 여러 Service
- Management / Orchestration 이 서비스 배치와 운영을 담당
- DevOps 팀이 각 서비스를 독립적으로 관리



Microservice Architecture (출처: Microsoft Azure Architecture Guide)

Microservice Style 의 요소 (1) — Microservice

■ 각 Microservice 의 특징

- 별도의 codebase 를 가지며, 소규모 개발팀이 관리할 수 있다
- 독립적으로 배포(deploy)할 수 있다
 - 전체 application 을 다시 빌드·배포하지 않고도 기존 서비스를 갱신할 수 있다
- 자신의 데이터나 외부 상태를 스스로 저장(persist)한다
 - 별도의 data layer 가 데이터 저장을 담당하던 전통적 모델과 다르다
- 잘 정의된 API 로 서로 통신하며, 각 서비스의 내부 구현은 다른 서비스에 숨겨진다
- 같은 기술 stack, library, framework 를 공유할 필요가 없다

핵심 Microservice = 독립 codebase + 독립 배포 + 독립 데이터 + API 통신

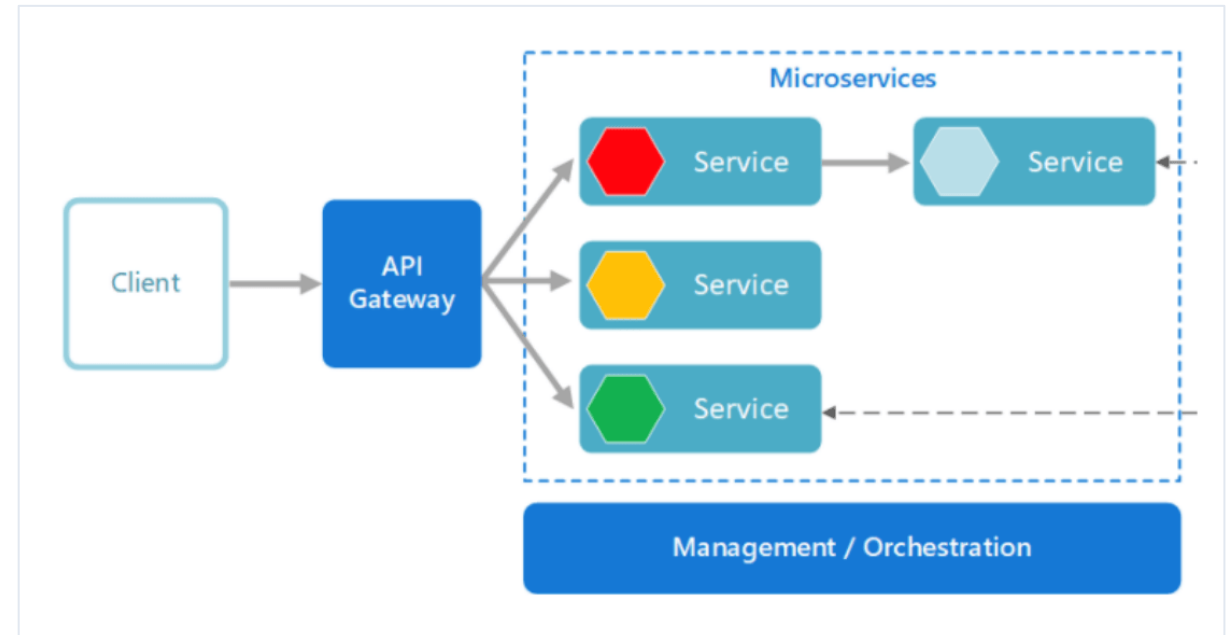
Microservice Style 의 요소 (2) — Gateway 와 Orchestration

■ API Gateway

- Client 의 진입점(entry point)
- Client 는 API gateway 를 호출하고, gateway 가 back end 의 적절한 서비스로 호출을 전달한다

■ Management / Orchestration

- 서비스를 node 에 배치(placing)
- 장애 식별(identifying failures)
- Node 간 서비스 재배포치(rebalancing) 등

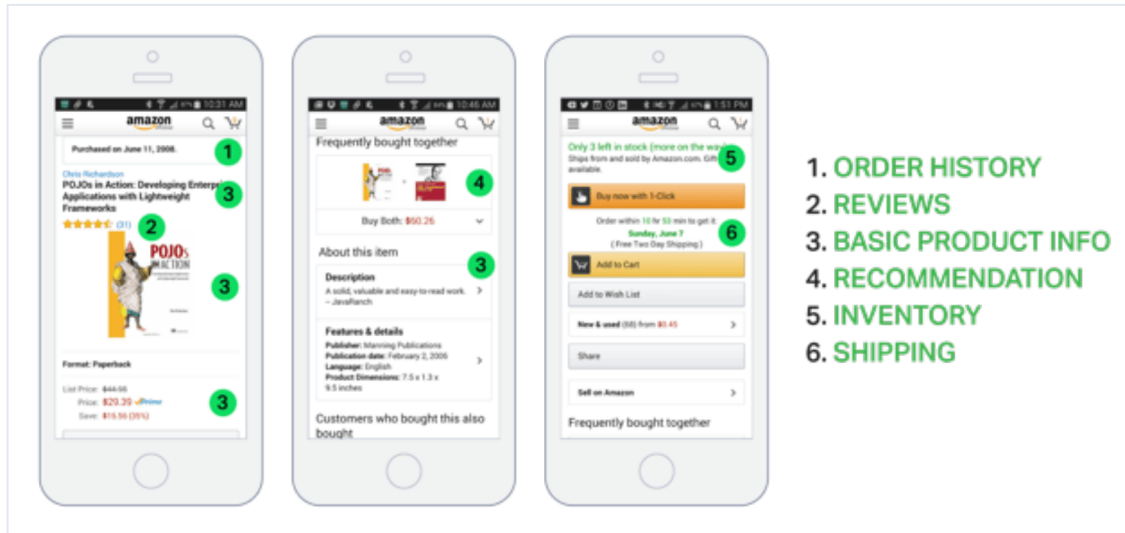


API Gateway 와 Management/Orchestration

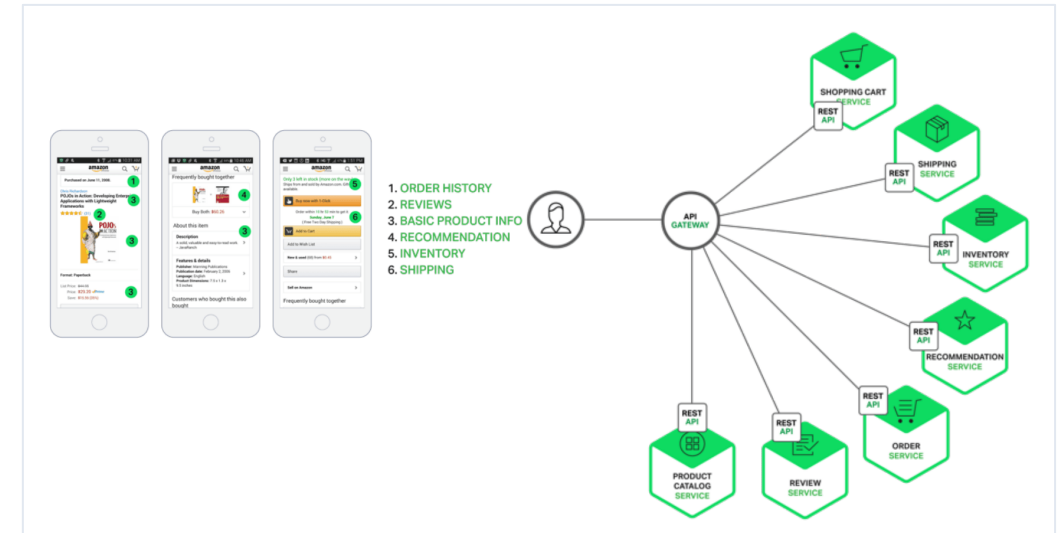
Microservice 예제 — RESTful Web Service

■ 온라인 쇼핑 앱

- 모바일 앱 화면의 각 영역이 서로 다른 서비스에서 온다: 1. Order History, 2. Reviews, 3. Basic Product Info, 4. Recommendation, 5. Inventory, 6. Shipping
- 각 서비스(Product Catalog, Review, Order, Recommendation, Inventory, Shipping, Shopping Cart 등)는 REST API 로 API Gateway 에 연결된다



화면 영역별 서비스 매핑



API Gateway 와 REST 서비스들

Microservice Consequences: Benefits 상세

■ Agility (민첩성)

- 서비스를 독립적으로 배포하므로 bug fix 와 기능 release 관리가 쉽다
- 전체 application 을 재배포하지 않고 서비스 하나만 갱신하고, 문제가 생기면 되돌릴(roll back) 수 있다

■ 새 기능 추가 용이

- 코드와 data store 를 공유하지 않아 의존성이 최소화되고, 새 기능을 추가하기 쉽다

■ Fault isolation (장애 격리)

- 한 microservice 가 죽어도 upstream 서비스가 장애를 올바르게 처리하도록 설계되어 있다면 전체 application 이 중단되지 않는다

■ Scalability · Data isolation

- 서비스를 독립적으로 확장할 수 있어, 자원이 더 필요한 subsystem 만 scale out 할 수 있다
- Schema 변경이 단일 microservice 에만 영향을 주므로 schema update 가 훨씬 쉽다

Microservice Consequences: Weaknesses 상세

■ Complexity (복잡성)

- 개별 서비스는 단순해지지만 시스템 전체는 더 복잡해진다

■ Testing (테스트)

- 서비스가 다른 서비스에 의존하면, 도구 지원 없이 의존성을 고려한 테스트가 어렵다

■ Lack of governance (거버넌스 부족)

- 기술 stack 을 공유하지 않아도 되는 것은 장점이지만, 언어와 framework 가 너무 다양해져 유지보수가 어려워질 수 있다

■ Network congestion and latency

- 작고 세분화된 서비스가 많아지면 서비스 간 통신이 늘어난다
- 서비스 의존 chain 이 너무 길어지면 latency 가 문제가 된다

■ Data integrity (데이터 무결성)

- 각 microservice 가 자기 데이터를 따로 저장하므로 데이터 일관성(consistency) 유지가 어렵다

■ Versioning (버전 관리)

- 서비스 갱신이 이에 의존하는 서비스를 깨뜨리면 안 된다
- 여러 서비스가 동시에 갱신될 수 있어, 신중히 설계하지 않으면 backward/forward 호환성 문제가 생긴다

Microservice Consequences: 장점과 약점

■ Benefits (장점)

- Agility — 독립 배포로 bug fix·release·roll back 이 쉽다
- 새 기능 추가 용이 — 코드·data store 비공유로 의존성 최소화
- Fault isolation — 서비스 하나의 장애가 전체를 멈추지 않는다
- Scalability — 필요한 subsystem 만 독립적으로 scale out
- Data isolation — schema 변경이 한 서비스에만 영향

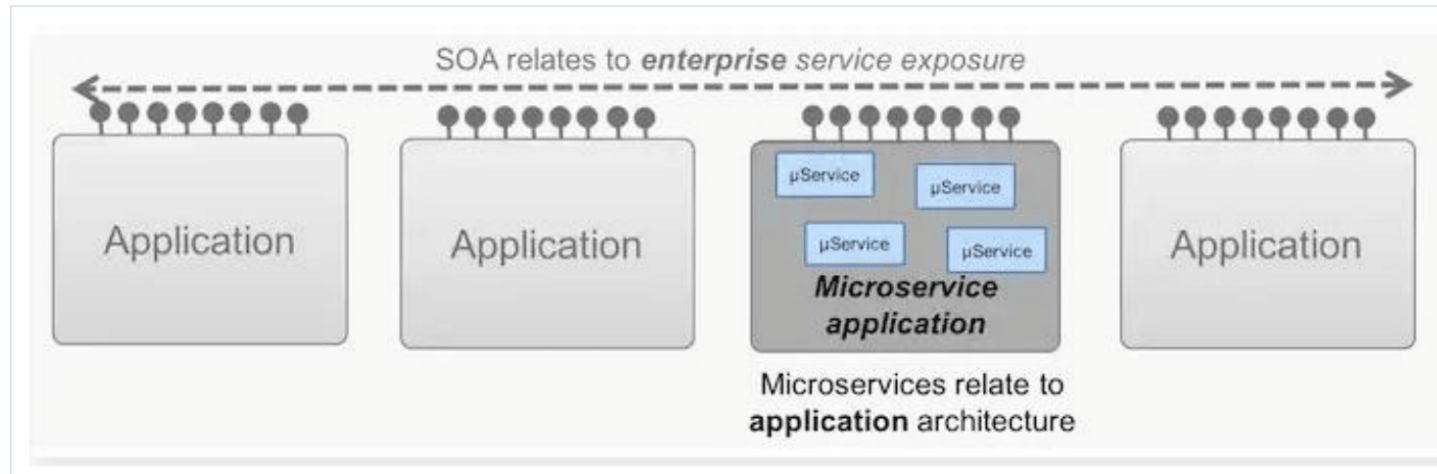
■ Weaknesses (약점)

- Complexity — 서비스는 단순, 전체 시스템은 복잡
- Testing — 서비스 의존성 때문에 테스트가 어렵다
- Lack of governance — 언어·framework 난립으로 유지보수 어려움
- Network congestion and latency — 서비스 간 통신 증가
- Data integrity — 서비스별 데이터 저장으로 일관성 유지 어려움
- Versioning — backward/forward 호환성 문제

SOA vs. Microservice Architecture

■ 핵심 차이는 범위(scope)

- SOA 는 enterprise 범위 — 여러 application 이 공유하는 서비스를 기업 차원에서 공개(enterprise service exposure)
- Microservice 는 application 범위 — 하나의 application 을 작은 서비스들로 나누어 구성
- (출처: IBM Cloud Blog, SOA vs. Microservices)



SOA 는 enterprise 수준, Microservice 는 application 수준의 서비스 분해

핵심 SOA: enterprise scope / Microservice: application scope

1.8

Deployment Architecture

시스템을 어디에, 어떻게 배치하고 실행할 것인가를 정하는 스타일

- Deployment Architecture 의 정의와 요소
- Container-Based Style
- Cloud-Based Style
- Edge-Based Style

Deployment Architecture(배치 Architecture) 개요

■ 정의

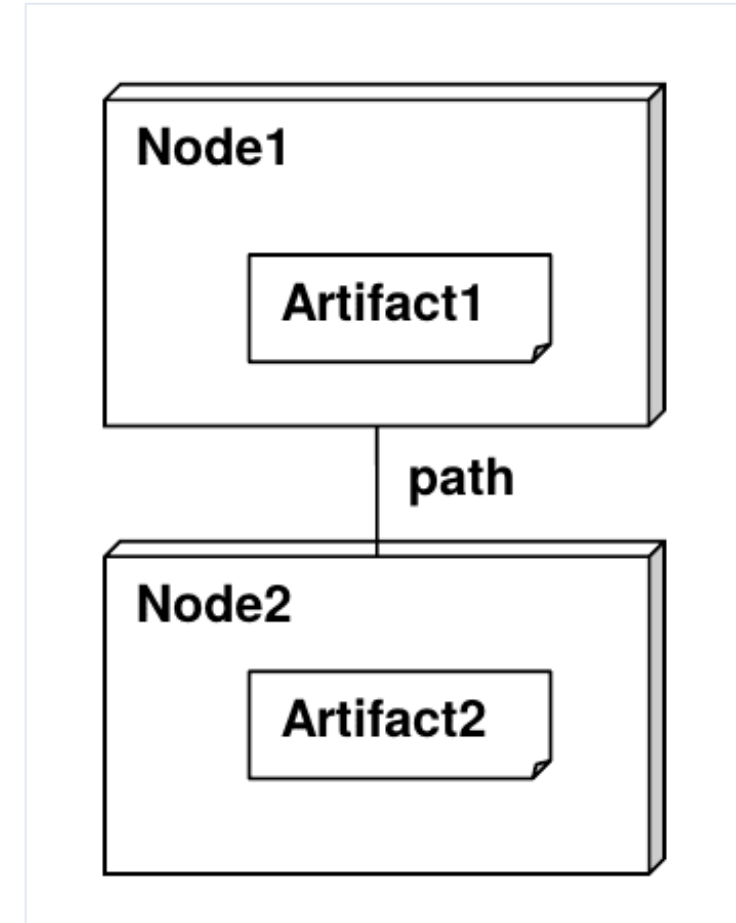
- 시스템이 어떻게, 어디서 실행되는지를 명세한다
 - 물리적·가상적 환경, 소프트웨어 artifact 의 분산 배치, 그리고 그들 사이의 통신을 정의해 원하는 성능·확장성·신뢰성을 달성

■ Elements / Relations

- Deployment Node: artifact 가 배치되어 실행되는 물리적 또는 논리적 환경
- Deployable Artifact: 배치 가능한 소프트웨어 패키지
- Communication Path: deployment node 사이의 네트워크 연결
- Mapping: artifact 를 node 에 할당(assignment)

■ 분류

- Container-Based Style: 가볍고 이식성 높은 runtime 환경
- Cloud-Based Style: 완전 관리형(fully managed) 분산 배치
- Edge-Based Style: 낮은 latency 를 위해 데이터 소스 가까이에서 연산



Node 와 Artifact, 그리고 이를 잇는 path

Container-Based Style 개요

■ Container 란?

- Application 과 그 의존성(dependencies)을 하나로 패키징한다
- 격리된 user space 에서 실행된다
- 여러 장비에서 일관된 runtime 동작을 보장한다
- 가상 머신(VM)보다 빨리 시작되고 자원을 적게 쓴다

■ Container-Based Deployment Architecture Style

- Application 을 container 로 패키징하고, container orchestration platform(Kubernetes, ECS 등)을 통해 배포·확장·운영하는 스타일

핵심 Container 는 '앱 + 의존성' 을 한 덩어리로 묶어 어디서나 같은 방식으로 실행하는 배치 단위이다

Container-Based Style — Elements

■ Deployment Node (Container Host)

- Container 가 실행되는 물리적 또는 가상 머신으로, Docker engine 같은 container runtime 을 갖춘다

■ Deployable Unit (Container Instance)

- Orchestrator 가 deployment node 에 배치해 실행하는 runtime 인스턴스

■ Deployable Artifact (Container Image)

- Container 실행에 필요한 모든 것을 정의한 패키지 명세(이미지)

■ Orchestrator / Container Registry

- Orchestrator: 여러 deployment node 에 걸쳐 runtime unit 을 스케줄링·배포·확장·관리하는 제어 시스템
- Container Registry: container image 를 저장·관리·공유하는 중앙 저장소 (Docker Hub, GCR, ECR 등)

Container-Based Style — Relations 와 Constraints

■ Relations

- Deployment Relation: Container Registry → Container Host/Orchestrator
 - registry 의 image 를 orchestrator 가 요청하거나 host runtime 이 내려받아 container 로 실행
- Execution Relation: Container Instance → Container Host
 - container 는 host 의 OS kernel 을 공유하면서 격리된 환경에서 실행
- Management Relation: Orchestrator → Container Instance
 - 여러 container 의 lifecycle(생성·확장·복구·종료)과 네트워킹, service discovery 를 관리

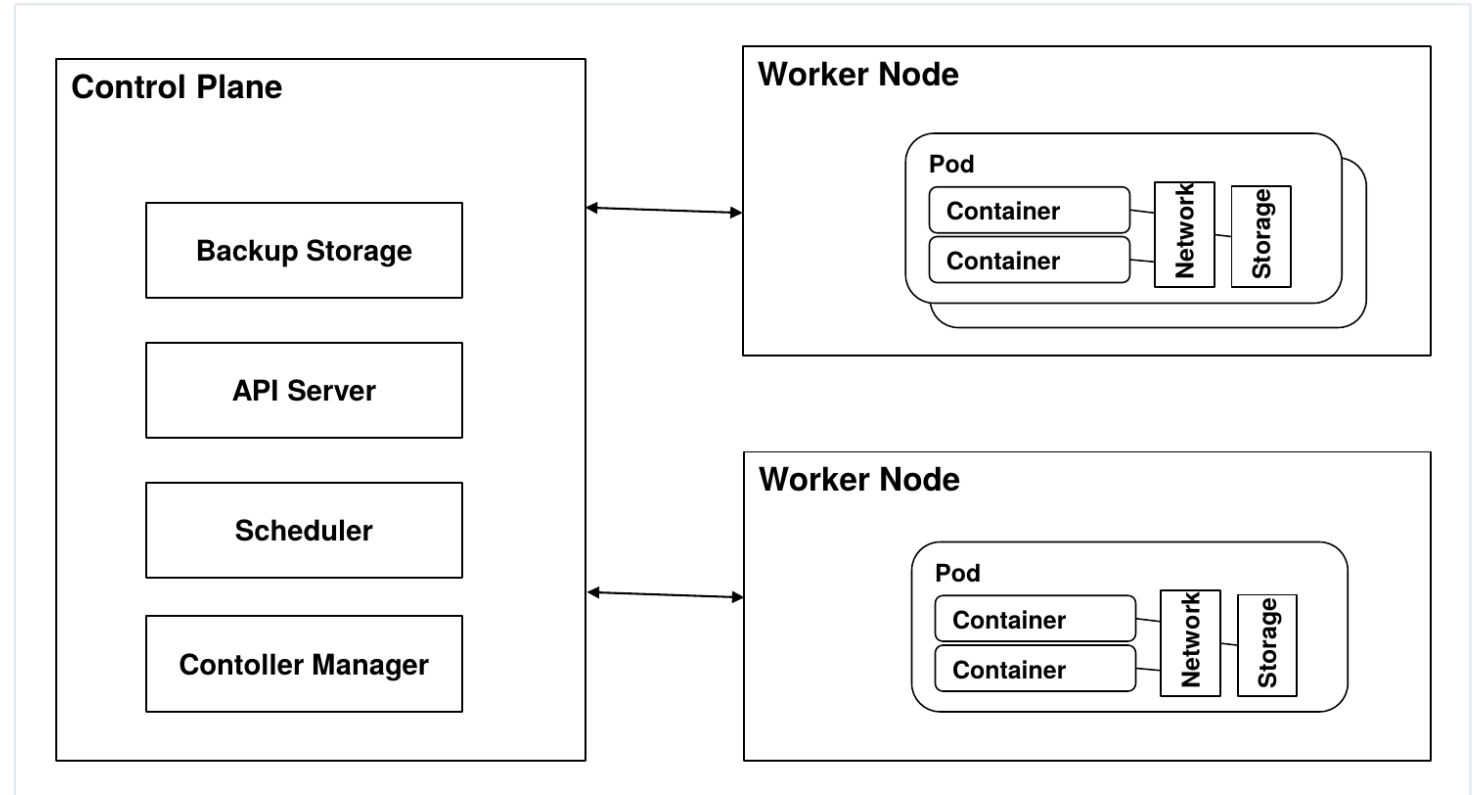
■ Constraints

- Immutability(불변성): 배포된 image 는 수정하지 않는다. 갱신 시 새 image 를 빌드해 기존 container 를 교체(rolling update)
- Statelessness Design(권장): container 는 상태를 저장하지 않게 설계하고, 영속 데이터는 Volume, DB, 외부 storage 에 둔다
 - container 를 자유롭게 버리고 재배포할 수 있어 resilience 가 높아진다

Container 예제 — Kubernetes Architecture

■ Kubernetes 구성

- Control Plane(Orchestrator): API Server, Scheduler, Controller Manager, Backup Storage
- Worker Node(Deployment Node): 하나 이상의 Pod 를 실행
- Pod(Deployable Unit): 하나 이상의 Container 와 Network, Storage 를 묶은 단위



Control Plane 이 여러 Worker Node 의 Pod 를 관리한다

Container 플랫폼 비교: Kubernetes · ECS · Docker

- 같은 요소가 플랫폼마다 다른 이름으로 불린다 — 요소의 역할을 기준으로 대응시켜 보면 이해가 쉽다

요소	Kubernetes	ECS	Docker
Deployment Node	Worker Node	EC2 / Fargate Node	Docker Host
Deployable Unit	Pod	Task	Container
Deployable Artifact	Pod Spec + Image	Task Definition + Image	Image
Orchestrator	Kubernetes Control Plane	ECS Control Plane	없음 (X)
Container Registry	Docker Hub, GCR/ACR/ECR	Amazon ECR (Elastic Container Registry)	Docker Hub

Container Consequences: Benefits 상세

■ 이식성과 환경 일관성

- "내 컴퓨터에서는 되는데..." 문제를 해결한다
- Container runtime 만 있으면 on-premises, AWS, Azure, GCP 어디서나 실행되어 인프라 이전이 쉽다

■ 효율성과 경량성

- 낮은 overhead: VM 과 달리 guest OS 전체를 포함하지 않는다
- 비용 절감: 더 적은 하드웨어로 더 많은 application 을 실행할 수 있다

■ 빠른 배포와 확장성

- CI/CD 가속: container image 가 배포 단위를 표준화한다
- 탄력적 확장: Kubernetes 같은 orchestration 도구로 인스턴스를 빠르고 유연하게 확장

■ 격리와 모듈성

- 의존성 충돌 방지: 한 container 의 library 가 다른 container 에 영향을 주지 않는다
- Microservice 지원: 각 서비스를 캡슐화해 독립 배포하기에 이상적인 단위

Container Consequences: 장점과 약점

■ Benefits (장점)

- Portability & Environmental Consistency — 어디서나 같은 동작
- Efficiency & Lightweight — guest OS 없음, 하드웨어 비용 절감
- Rapid Deployment & Scalability — CI/CD 가속, 탄력적 확장
- Isolation & Modularity — 의존성 충돌 방지, microservice 에 적합

■ Weaknesses (약점)

- OS kernel 의존성과 호환성 한계 — host kernel 을 공유하므로 Windows container 를 Linux host 에서 실행할 수 없다
- 운영 복잡성 증가 — 대규모 배포에는 orchestration, 네트워킹, service discovery, 모니터링, 보안 설정이 필요
- 보안과 격리 우려 — kernel 공유로 VM 보다 격리가 약해 세심한 설정과 추가 hardening 이 필요
- Orchestrator 학습 곡선 — Kubernetes 는 특히 monolithic/VM 환경에서 전환하는 팀에 큰 학습·운영 부담

Cloud-Based Style 개요

■ Cloud-Based Deployment Architecture Style

- Cloud computing 환경의 서비스를 활용해 소프트웨어 시스템을 구성·배치하는 방법을 정의하는 Deployment Architectural Style
- Elasticity(탄력성), Scalability(확장성), High Availability(고가용성) 같은 cloud 의 특성을 최대한 살리도록 소프트웨어 artifact 를 cloud node 에 배치하는 데 초점을 둔다

핵심 Cloud 의 강점(탄력성·확장성·고가용성)이 살아나도록 artifact 를 cloud node 에 배치한다

Cloud-Based Style — Elements

■ Software Artifacts

- 배치할 모든 소프트웨어 구성 요소: application 코드, library, 설정 파일, database schema

■ Deployment Nodes

- Cloud Resources: cloud vendor 가 제공하는 컴퓨팅·스토리지·네트워킹 자원
 - IaaS(VM/Container), PaaS(App Services, Lambda/Functions 같은 managed service), FaaS(serverless function)
- Managed Services: cloud 가 관리해 주는 서비스
 - Database(RDS, DynamoDB, Cosmos DB), Messaging Queue(SQS, Kafka), Load Balancer(ALB, ELB), API Gateway

■ Infrastructure-as-Code (IaC) Artifacts

- 인프라를 정의하고 provisioning 하는 script 나 설정 파일 (Terraform, CloudFormation, ARM Template 등)
- 배치의 자동화와 재현성(reproducibility)을 보장한다

Cloud-Based Style — Relations

■ Deployment Relation

- Software Artifact 는 Deployment Node(Cloud Resource, Managed Service) 위에 배치되거나 이를 소비(consume)한다
- 예: microservice artifact 를 container orchestration platform(EKS/AKS/GKE)의 node 에 배치

■ Provisioning Relation

- IaC Artifact 가 Deployment Node 를 생성(provision)하고 구성한다
- IaC Artifact 로 여러 Cloud Resource node(예: web server 인스턴스)를 수요에 따라 scale in/out 하게 할 수 있다

핵심 Artifact 는 node 에 '배치'되고, IaC 는 node 를 '만든다'

Cloud 예제 (1) —고가용성 3-Tier 웹 애플리케이션

■ IaaS/PaaS Hybrid

- 전통적인 3-tier 웹 application 을 cloud 의 managed service 와 multi-AZ 구성으로고가용성 있게 배치한 예

요소 (Elements)	아티팩트 (Artifacts)	노드 배치 (Node Placement)
Presentation (프론트엔드)	웹 자산(HTML, CSS, JS)	CDN(Content Delivery Network) 및 S3/Cloud Storage 에 배포
Application (백엔드)	웹 서버, API 코드	Auto-Scaling Group 내의 VM(EC2/Compute Engine) 또는 Managed Container(EKS/GKE/ECS)에 배포. 여러 가용 영역(Availability Zones, AZs)에 분산
Data (데이터베이스)	데이터베이스 스키마 및 데이터	Managed DB Service(RDS/Cloud SQL/Cosmos DB)를 Multi-AZ 구성으로 배포
Networking	로드 밸런서, VPC	ELB/ALB 를 통해 트래픽을 AZ 간 분산, WAF(Web Application Firewall) 사용

Cloud 예제 (2) — 마이크로서비스 배포

■ Container/PaaS 중심

- 수십~수백 개의 microservice 를 Kubernetes cluster 와 managed service 위에 배치한 예

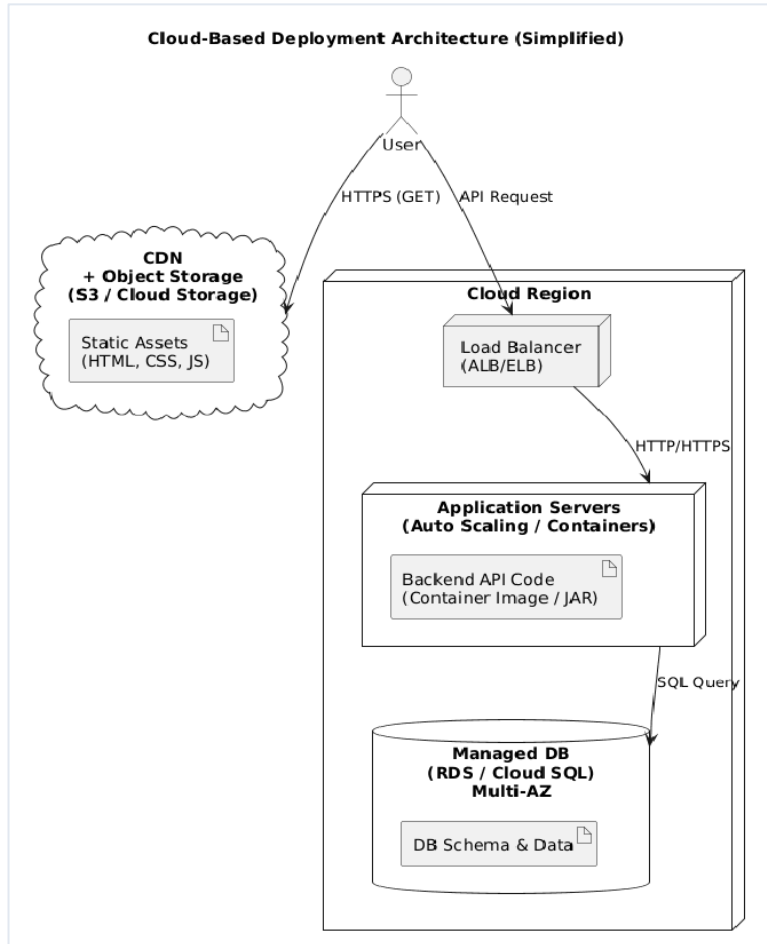
요소 (Elements)	아티팩트 (Artifacts)	노드 배치 (Node Placement)
Services	수십~수백 개의 독립적인 마이크로서비스 (Containerized)	Kubernetes Cluster(EKS/GKE/AKS) 노드에 배포. 각 서비스는 독립적인 Pod 로 관리
API Gateway	서비스 라우팅 정의	API Gateway Service(AWS API Gateway / Azure API Management)를 사용해 모든 트래픽의 단일 진입점 역할
Communication	비동기 메시지	Managed Message Queue/Stream(SQS, Kafka, Pub/Sub) 노드에 배포
Data Stores	서비스별 DB	서비스의 특성에 맞는 다양한 Managed DB(Relational, NoSQL)를 사용

Cloud 예제 (3) — 서버리스 이벤트 중심 Architecture

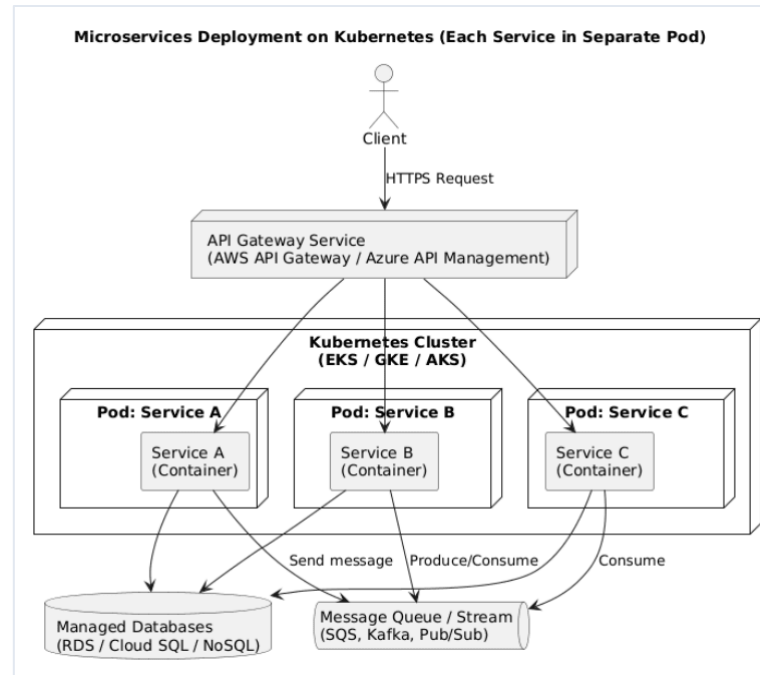
■ FaaS 중심

- 서버 관리를 cloud 에 위임하고, 이벤트가 발생할 때만 함수를 실행하는 serverless 배치 예

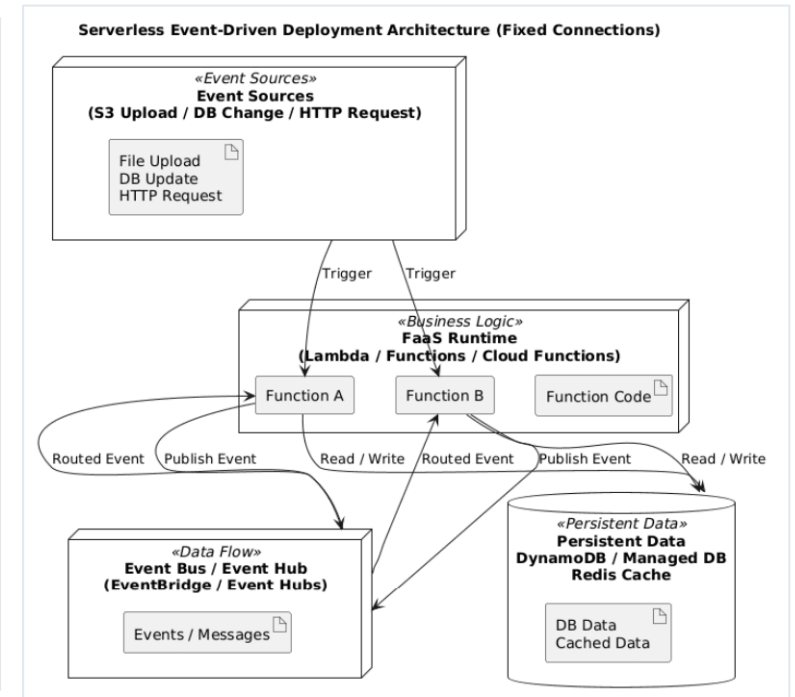
요소 (Elements)	아티팩트 (Artifacts)	노드 배치 (Node Placement)
Business Logic	개별 함수 코드 (Function Code)	FaaS Runtime(AWS Lambda / Azure Functions / Cloud Functions)에 배포. 노드 (서버) 관리는 클라우드에 위임
Event Sources	파일 업로드, DB 변경, HTTP 요청	Storage(S3/Blob), Managed DB, API Gateway 등이 이벤트 트리거 노드 역할
Data Flow	이벤트/메시지	Event Bus/Hub(EventBridge, Event Hubs)를 사용해 함수 간 또는 외부 시스템으로 이벤트 라우팅
Persistent Data	데이터베이스, 캐시	Managed NoSQL DB(DynamoDB) 또는 Caching Service(Redis) 사용



고가용성 3-Tier



Microservice on Kubernetes



Serverless Event-Driven

Cloud-Based Consequences: 장점과 약점

■ Benefits (장점)

- High Availability & Fault Tolerance — multi-AZ/region 배치, 자동 장애 감지와 복구
- Auto Scalability — 트래픽이 많으면 늘리고 적으면 줄여 비용 효율적으로 운영
- Managed Service 활용 — DB, messaging, 보안, logging 을 맡겨 운영 부담 감소
- 보안과 compliance 향상 — cloud 제공자의 탄탄한 보안 인프라, IAM, key 관리

■ Weaknesses (약점)

- Vendor Lock-in — 특정 vendor 서비스에 강하게 의존해 이전이 어렵다
- 복잡성 증가 — VPC, IAM 정책, 네트워크/보안 설정이 Architecture 를 복잡하게 만든다
- 예측하기 어려운 비용 — auto-scaling 과 사용량 기반 과금으로 비용 예측이 어렵다
- Latency 민감성 — hybrid 연결이나 region 간 통신이 latency 를 유발
- 높은 데이터 전송 비용 — region 간·cloud 간 데이터 전송이 비쌀 수 있다

Edge-Based Architecture 개요

■ 배경

- 네트워크 가장자리(edge)의 IoT 장치가 늘면서 data center 에서 처리해야 할 데이터가 폭증하고, 네트워크 대역폭 요구가 한계에 이르고 있다

■ Edge computing 이란

- 연산과 데이터 저장을 데이터가 생기는 곳(source) 가까이로 가져오는 분산 컴퓨팅 패러다임
- 응답 시간을 줄이고 대역폭을 절약할 수 있다
- 대부분의 연산을 cloud 의 서버가 아니라 장치(Edge)에서 수행한다는 뜻

■ Cloud vs. Edge

- Cloud computing 은 big data 를 다루고, edge computing 은 센서나 사용자가 실시간으로 만들어 내는 "instant data" 를 다룬다

핵심 Edge = 데이터가 생기는 곳 바로 옆에서 처리해 latency 와 대역폭을 아낀다

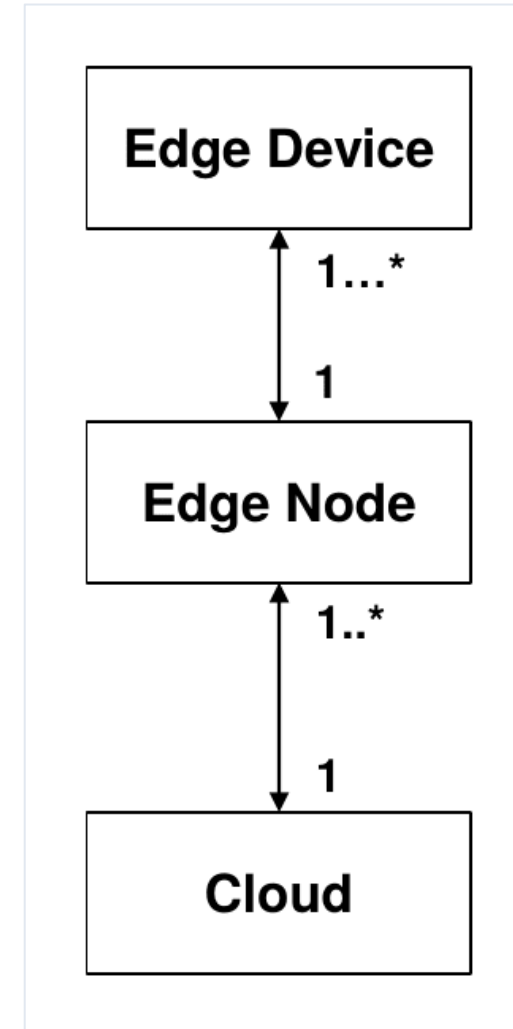
Edge-Based Style — 구성 요소

■ Elements

- Edge Devices: 온도 센서, 카메라, 차량 라이다, 스마트 미터 등
 - 주로 데이터를 '생성' 하는 역할
- Edge Node: 로컬 CPU/GPU/TPU 등 비교적 강한 연산 자원
 - 데이터 전처리, ML 모델 추론, 로컬 의사결정·제어 등을 수행
- Cloud / Remote Data Center: edge 에서 처리하기 어려운, 큰 컴퓨팅 파워가 필요한 서비스를 처리

■ Relations

- Edge Device(1..*) — Edge Node(1) — Cloud(1): 여러 장치가 한 edge node 에, 여러 edge node 가 하나의 cloud 에 연결된다

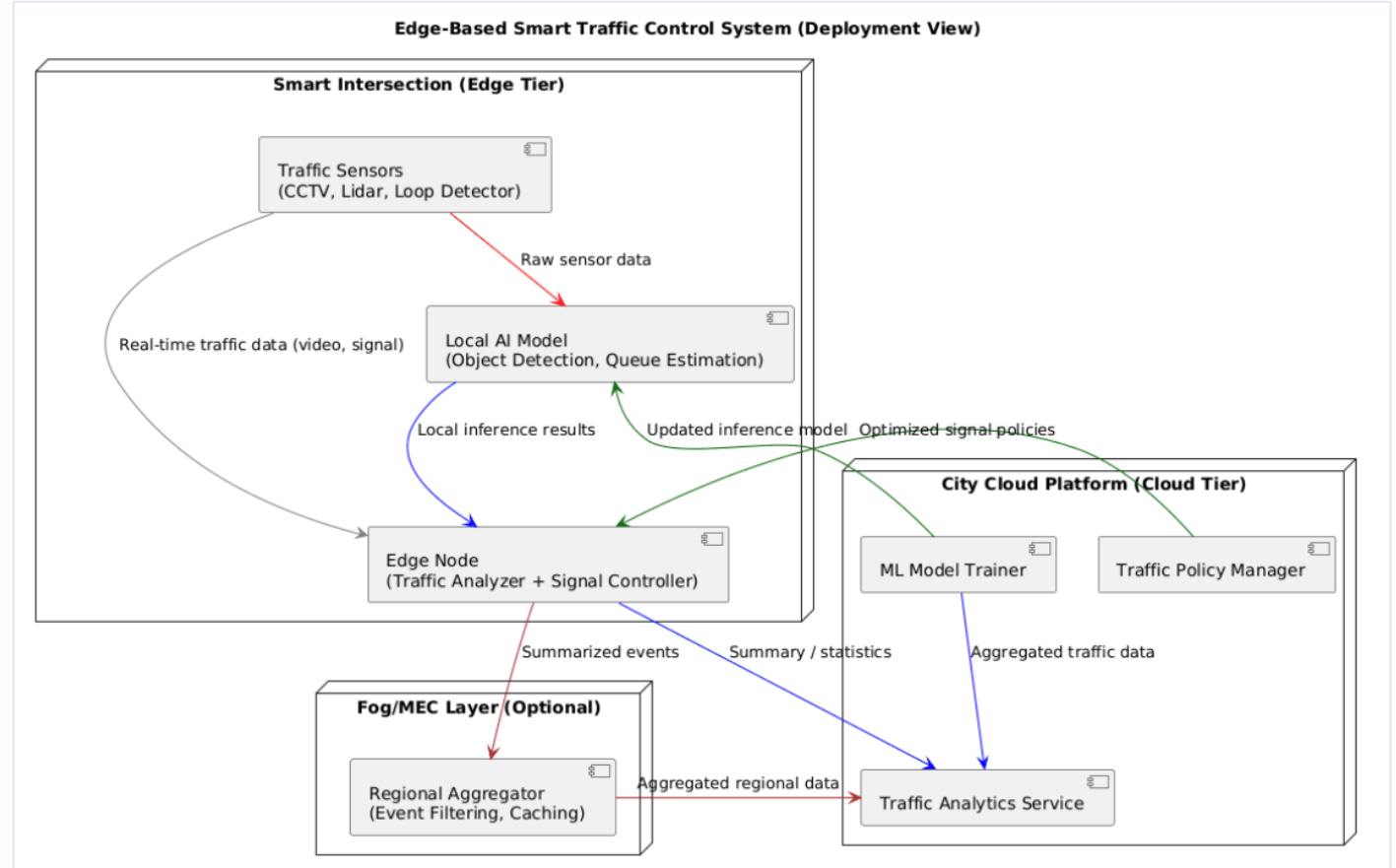


Edge Device – Edge Node – Cloud 의 계층 관계

Edge 예제 — 스마트시티 실시간 교통 제어

■ 전체 흐름 요약

- 1. 교차로(Edge)에서 센서(CCTV, Lidar, Loop Detector) 데이터 수집
- 2. Local AI 모델이 즉시 분석하고 Edge Node 가 신호등 제어
- 3. 요약 정보를 Fog/MEC 계층으로 전달 (선택)
- 4. 클라우드(City Cloud Platform)는 도시 전체 데이터를 학습
- 5. 새 모델·정책을 Edge 에 다시 보내 최적화

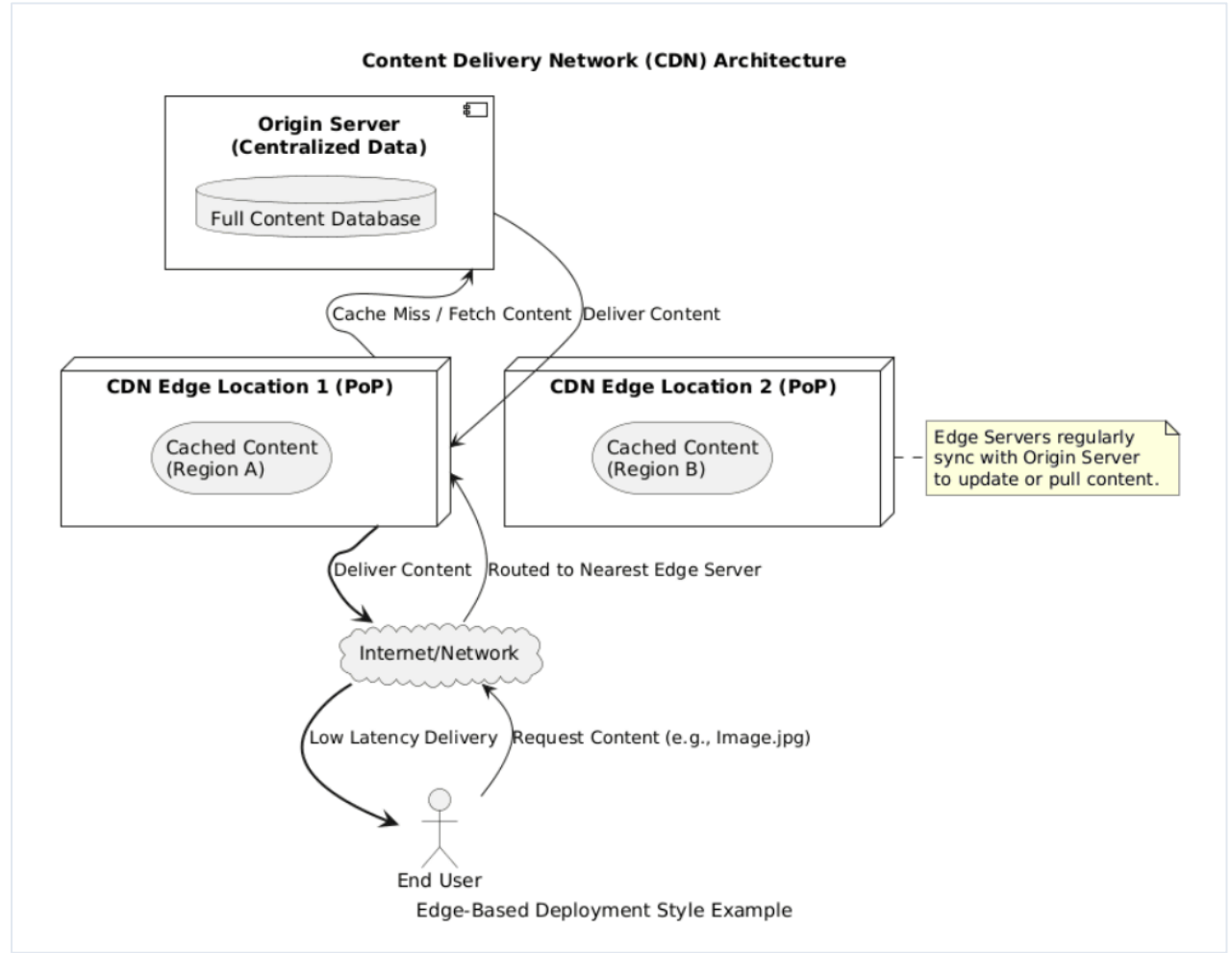


Edge Tier – Fog/MEC – Cloud Tier 배치도

Edge 예제 — Content Delivery Network (CDN)

■ CDN 구조

- Origin Server 가 전체 콘텐츠(Full Content Database)를 보관
- 각 지역의 CDN Edge Location(PoP)이 콘텐츠를 캐시하고, 사용자 요청은 가장 가까운 edge server 로 라우팅된다
- Cache miss 시에만 origin 에서 가져오며, edge server 는 주기적으로 origin 과 동기화한다
- 사용자는 낮은 latency 로 콘텐츠(이미지 등)를 받는다



Origin Server – CDN Edge Location – End User

Edge-Based Consequences: 장점과 약점

■ Benefits (장점)

- Privacy
 - 데이터를 edge 에 두면 수집 데이터의 소유권을 서비스 제공자에서 최종 사용자로 옮길 수 있다
- Speed
 - 분석용 연산 자원을 최종 사용자 가까이에 두어 통신 속도를 높인다
- Availability
 - 로컬 네트워크에서 연산을 수행하므로 중앙 site 가 멈춰도 지역 site 는 독립적으로 계속 운영된다

■ Weaknesses (약점)

- Security
 - 더 많은 데이터가 core 밖에서 처리되므로 유출·도난·침해 위험이 크게 늘어난다
- Investment Cost
 - 로컬 하드웨어가 더 많이 필요해 edge 인프라 구축 비용이 크다
 - 전체 효율은 높아질 수 있지만 상당한 초기 투자가 필요하다

1.9

Summary (Styles)

Architectural Style 을 정리하고, 품질 속성과의 관계를 되짚어 본다

- Architectural Style 의 정의와 결정 요소
- 7개 category, 23개 style 카탈로그
- Style 과 Quality Attribute 의 관계

Architectural Styles 요약 (1)

■ Architectural Style 이란

- 비슷한 소프트웨어 Architecture 들을 추상화한 것
- 다음 세 가지로 결정된다
 - element 유형의 집합
 - element 사이 관계(relationship)의 집합
 - topology 와 동작에 대한 제약(constraint)의 집합

■ Style 카탈로그

- 7개 category, 23개 architectural style
- 많은 style 이 서로 겹친다(overlap)
 - 예: Buffered Message Based, Broker, MVC, SOA

핵심 Style = element 유형 + 관계 + 제약의 집합

Architectural Styles 요약 (2) — Quality Attribute

■ Style 과 Quality Attribute(품질 속성)

- Architectural style 은 Architecture 품질을 추론(reason)하는 데 도움을 준다
- 하나의 style 이 여러 quality attribute 에 긍정·부정 영향을 동시에 준다
 - 예: Layered style — modifiability 에는 긍정, performance 에는 부정
- 하나의 quality attribute 에 대해서도 한 Architecture 가 긍정·부정 효과를 모두 가질 수 있다
 - 예: Repository style — 어떤 data client 의 기능 변경에는 modifiable(긍정)
 - 그러나 data store 의 데이터 구조와 client 사이의 높은 의존성(부정)

■ 결론

- Architectural style/pattern 은 원하는 quality attribute 를 달성할 Architecture 해법을 찾고 분석하는 데 도움을 준다

핵심 Style 선택은 곧 품질 속성 간의 trade-off 선택이다

2.1

Architecture Views

4+1 · Siemens · SEI View Model — 복잡한 시스템을 관심사별 여러 View 로 나누어 표현한다

- Architectural View 와 Structure 의 개념
- 4+1 View Model (Kruchten)
- Siemens View Model (Conceptual · Module · Execution · Code)
- SEI View Model (C&C · Module · Allocation)
- 세 View Model 의 대응 관계

Architectural View 란?

■ 정의

- Architectural View(Architecture 뷰)는 시스템 Architecture 를 특정 관심사(concern)나 측면에 초점을 맞춰 표현한 것이다

■ 왜 여러 View 가 필요한가

- 현대 시스템은 매우 복잡해서 하나의 구조로는 전체를 파악하기 어렵다
- Stakeholder(이해관계자)마다 시스템에 대한 관심사가 서로 다르다
 - 건축에서 구조 기술자는 Structural View, 배관공은 Plumbing View, 전기 기술자는 Electrical View 를 본다



하나의 건물도 보는 사람에 따라 Structural / Plumbing / Electrical View 로 나뉜다

핵심 View 는 '누가, 어떤 관심사로 시스템을 보는가'에 따라 Architecture 를 잘라 낸 표현이다

Structure 와 View

■ Structure(구조)

- 소프트웨어나 하드웨어 안에 실제로 존재하는 요소(element)들의 집합 그 자체
- Software Architecture 정의: 시스템을 추론(reason)하는 데 필요한 구조들의 집합으로, 소프트웨어 요소, 요소 간 관계, 그리고 둘의 속성으로 이루어진다

■ View(뷰)

- Structure 를 표현(representation)한 것을 그 시스템의 View 라고 부른다
- Software Architecture 문서는 이러한 View 들의 모음으로 구성된다

핵심 Structure 는 실제로 존재하는 것, View 는 그것을 그려 낸 표현이다 — Architecture 문서 = View 의 모음

4+1 View Model 개요

■ 제안자: Philippe Kruchten (1995)

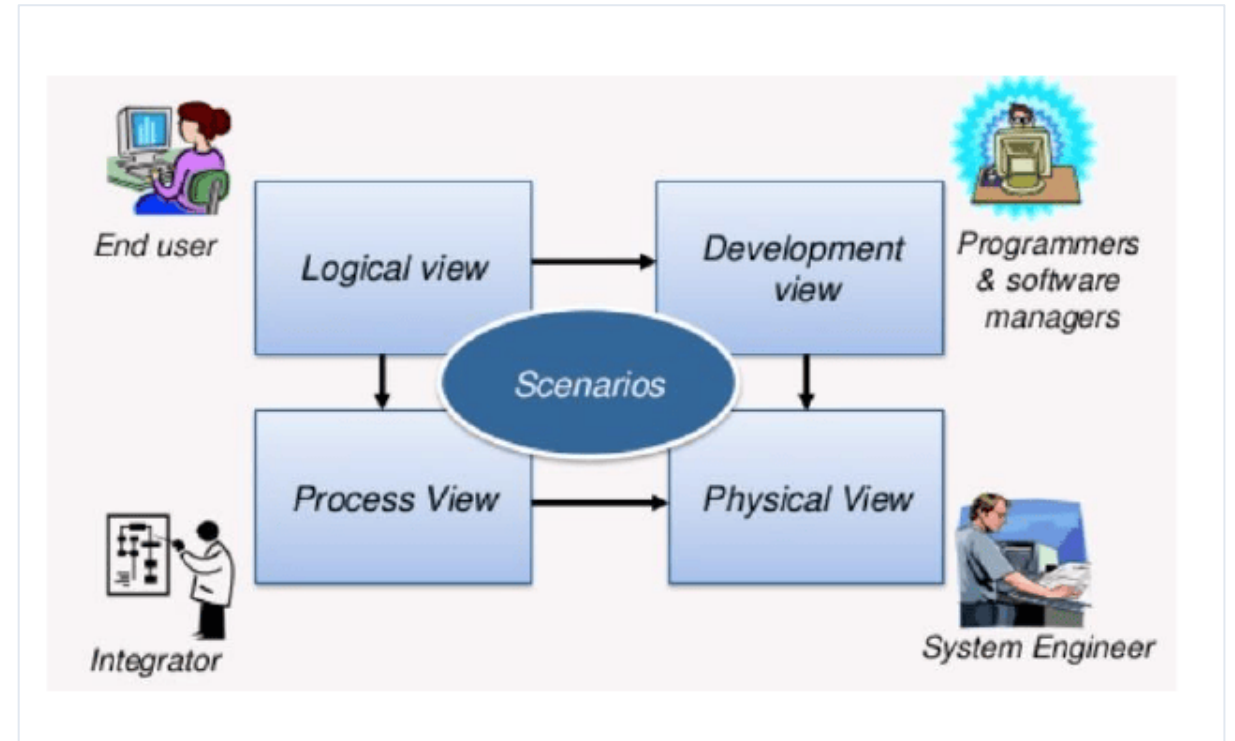
- 소프트웨어 집약 시스템의 Architecture 를 여러 개의 동시적인(concurrent) View 로 기술한다

■ 4개의 View

- Logical View — 최종 사용자(End user) 관점
- Development View — 개발자·소프트웨어 관리자 관점
- Process View — 통합자(Integrator) 관점
- Physical View — 시스템 엔지니어 관점

■ +1 View

- 선택된 Use Case 나 Scenario 로 Architecture 를 설명하고 검증한다 — 이것이 '+1' View 이다



4+1 View Model: 4개 View 의 중심에 Scenarios 가 있다

Logical View

■ 무엇을 보여주나

- 시스템의 구성 요소(component, 객체)와 그들 사이의 상호작용·관계를 보여준다

■ Viewer

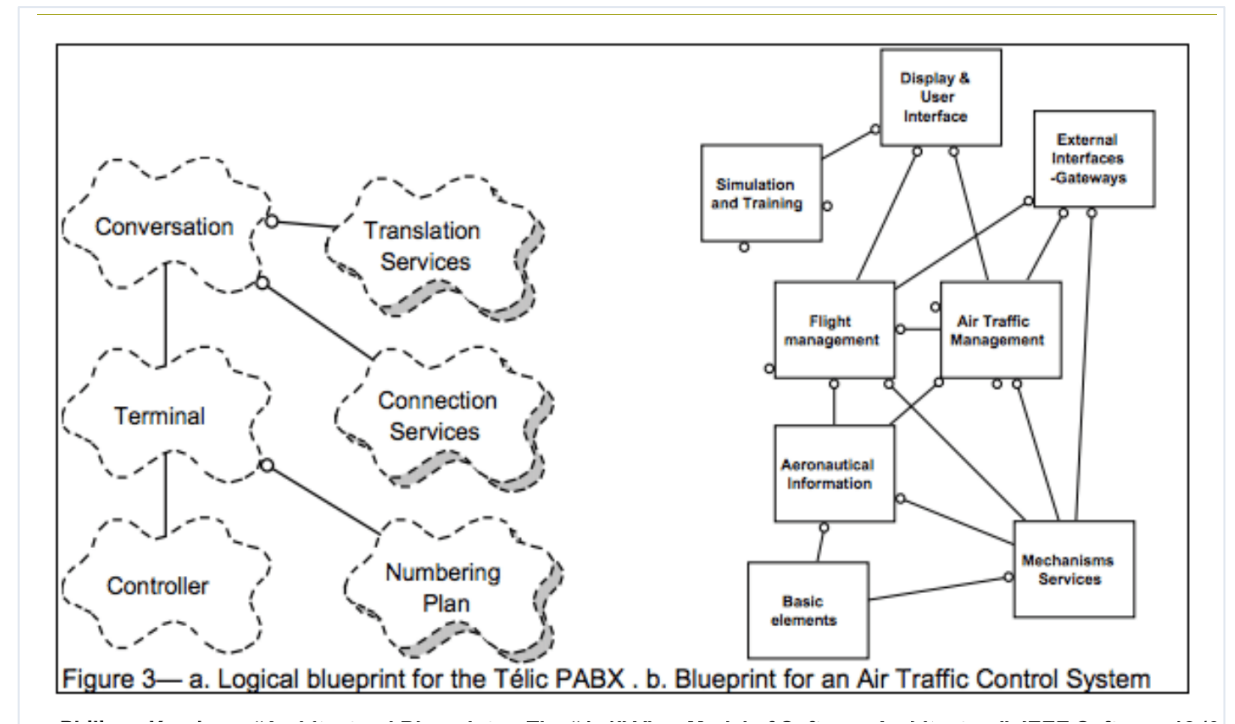
- End-user(최종 사용자)

■ 고려 사항

- 기능 요구사항(Functional requirement)
- 구현 수준이 아닌 논리적 수준(logical level)의 모델링 요소

■ 관련 UML 다이어그램

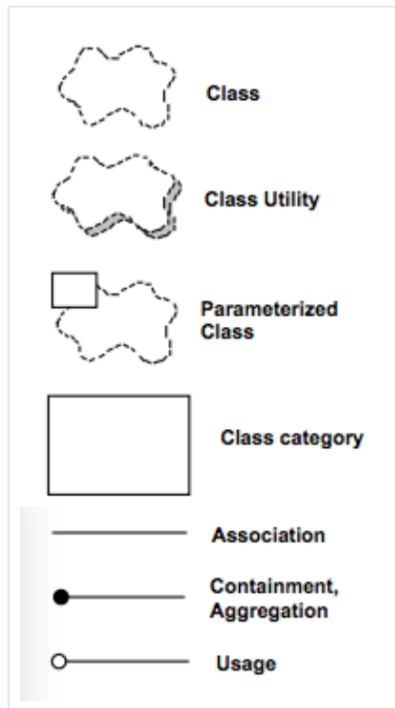
- Class diagram, Object diagram, Sequence diagram, Communication diagram



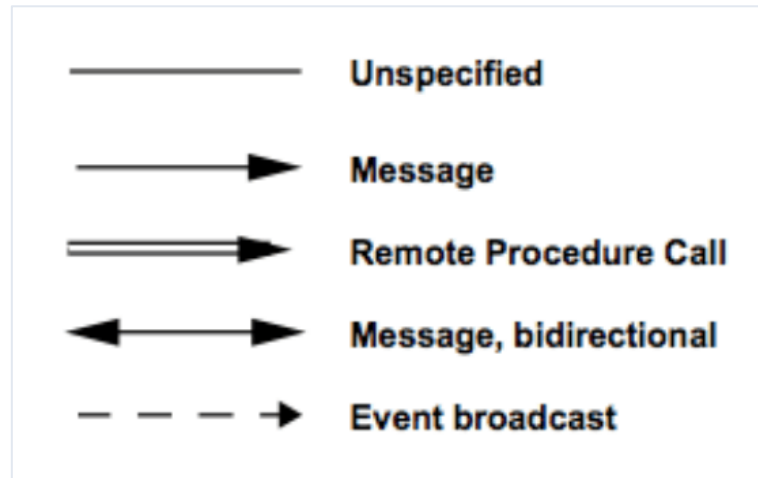
Logical View 예: (a) Télec PABX 의 논리 청사진, (b) 항공 교통 관제 시스템

4+1 예제 그림의 표기법 (Notation)

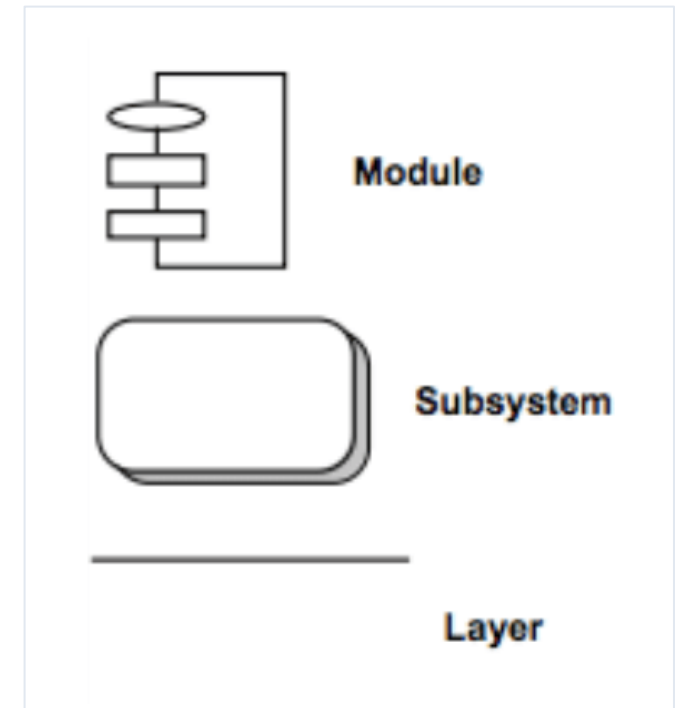
- Kruchten 의 원 논문 예제는 UML 이전의 Booch 표기법을 사용한다 — Logical View 는 Class / Class Category, Process View 는 Process / Periodic process, Development View 는 Module / Subsystem / Layer 로 표현한다
- 연결선은 Unspecified, Message, Remote Procedure Call, 양방향 Message, Event broadcast 로 구분한다



Logical View 의 표기



Process View 의 연결 표기



Development View 의 표기

Process View

■ 무엇을 보여주나

- 시스템의 프로세스(process)들과 그 프로세스들이 서로 어떻게 통신하는지를 보여준다

■ Viewer

- Integrator(통합 담당자)

■ 고려 사항

- 비기능 요구사항 — 동시성(concurrency), 성능(performance), 확장성(scalability)

■ 관련 UML 다이어그램

- Activity diagram, Component diagram

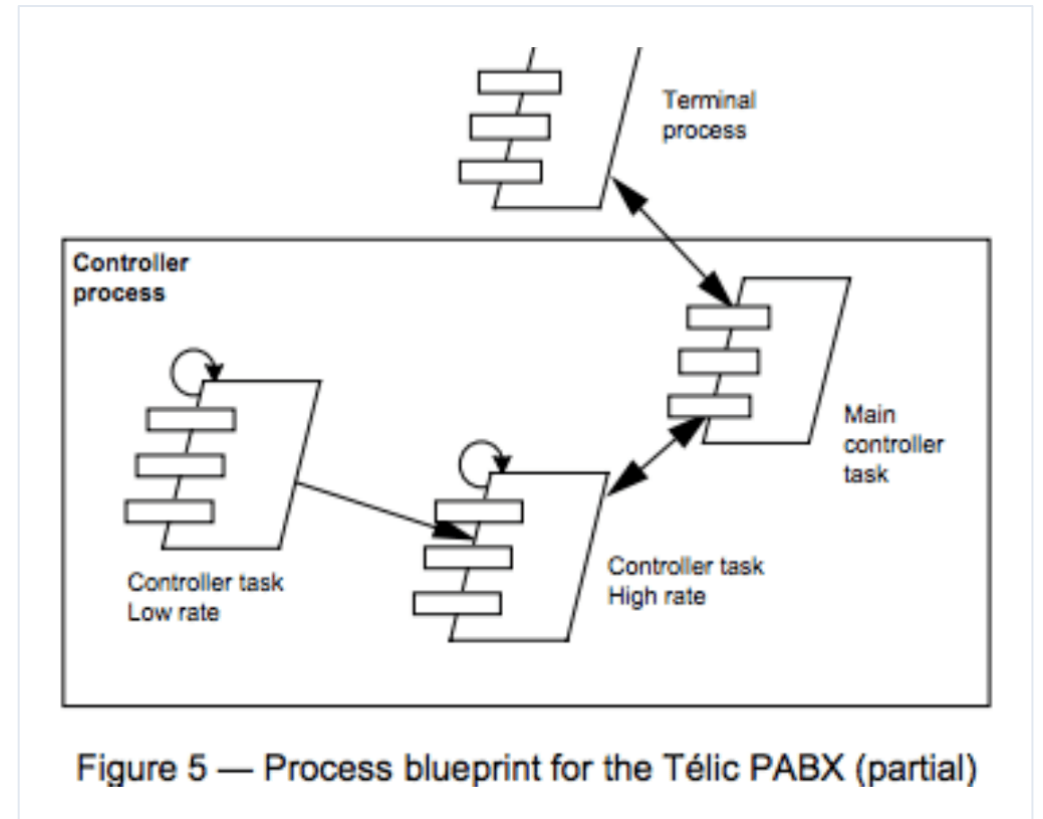


Figure 5 — Process blueprint for the Téléc PABX (partial)

Process View 예: Téléc PABX 의 Controller process 와 Terminal process

Development View

■ 무엇을 보여주나

- 시스템을 구성하는 building block 관점 — 예: class, package, subsystem

■ Viewer

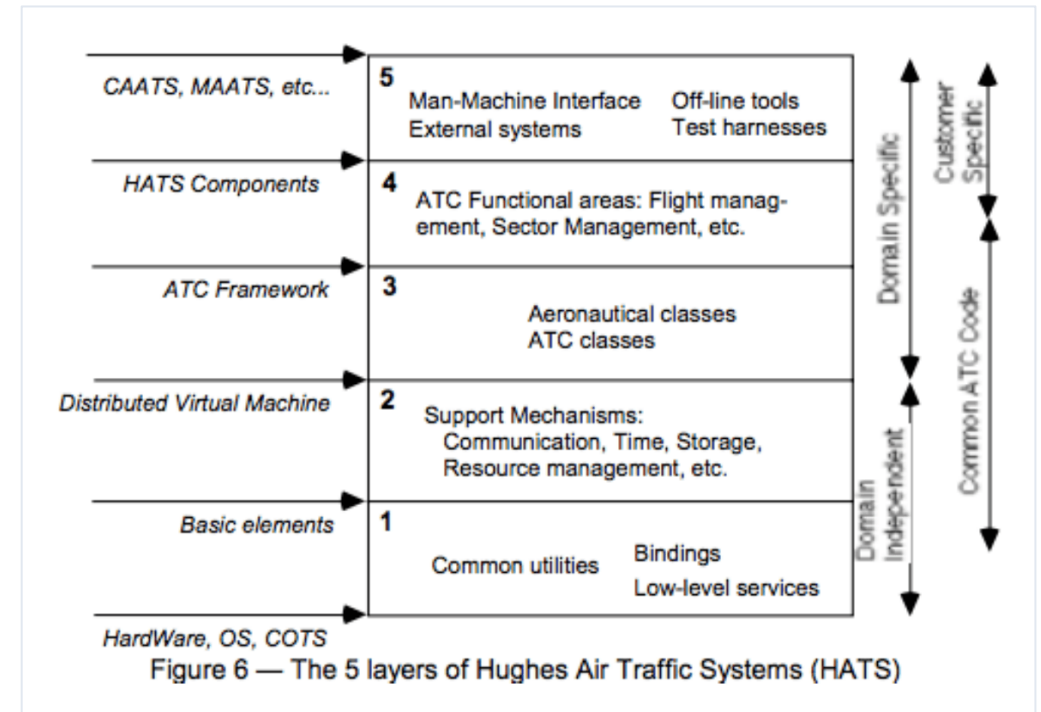
- Developer(개발자)와 Project manager

■ 고려 사항

- 비기능 요구사항 — 적응성(adaptability), 재사용(reuse), 소프트웨어 관리, 도구의 제약

■ 관련 UML 다이어그램

- Class diagram, Package diagram, Component diagram



Development View 예: Hughes 항공 교통 시스템(HATS)의 5개 Layer

Physical View

■ 무엇을 보여주나

- 시스템의 실행 환경(execution environment) — 소프트웨어가 어떤 하드웨어 노드에 배치되어 실행되는가

■ Viewer

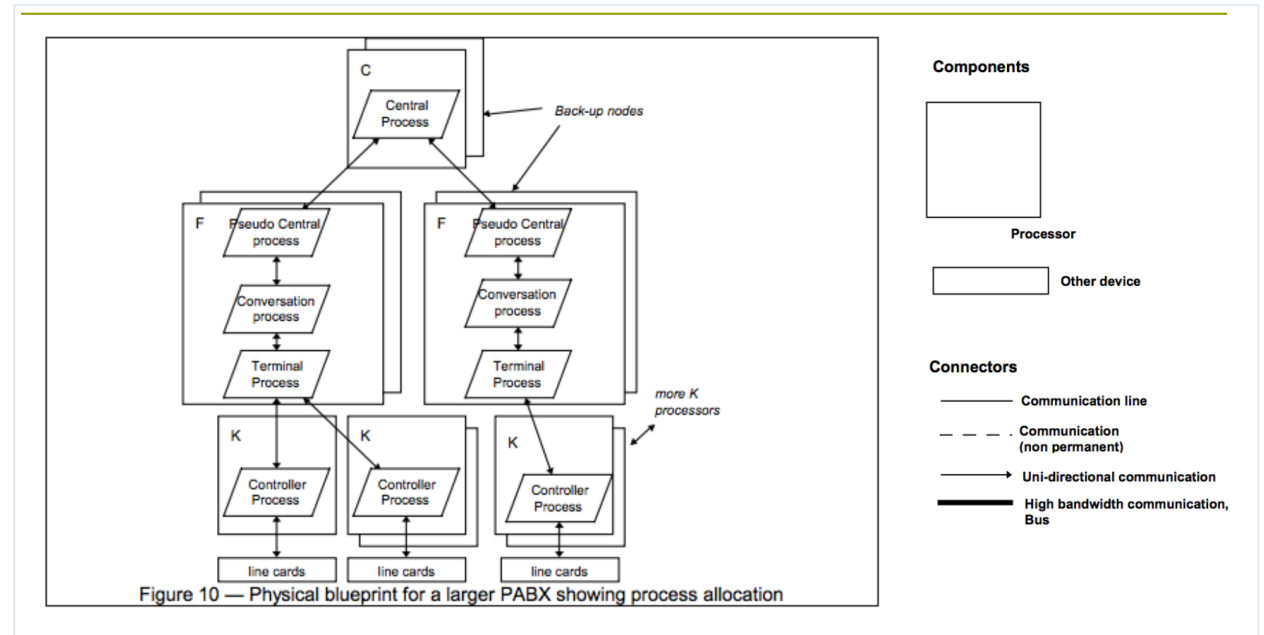
- System engineer(시스템 엔지니어)

■ 고려 사항

- 비기능 요구사항 — 동시성, 성능, 확장성
- 하드웨어의 제약(토폴로지, 통신)

■ 관련 UML 다이어그램

- Deployment diagram



Physical View 예: 대형 PABX 의 process 배치 청사진

Scenario / Use Case View (+1)

■ 무엇을 보여주나

- 설계를 검증(validate)하고 설명(illustrate)하기 위해 사용하는 View
- 다른 View 의 내용과 중복(redundant)되지만, 4개 View 를 하나로 묶어 주는 역할을 한다

■ Viewer

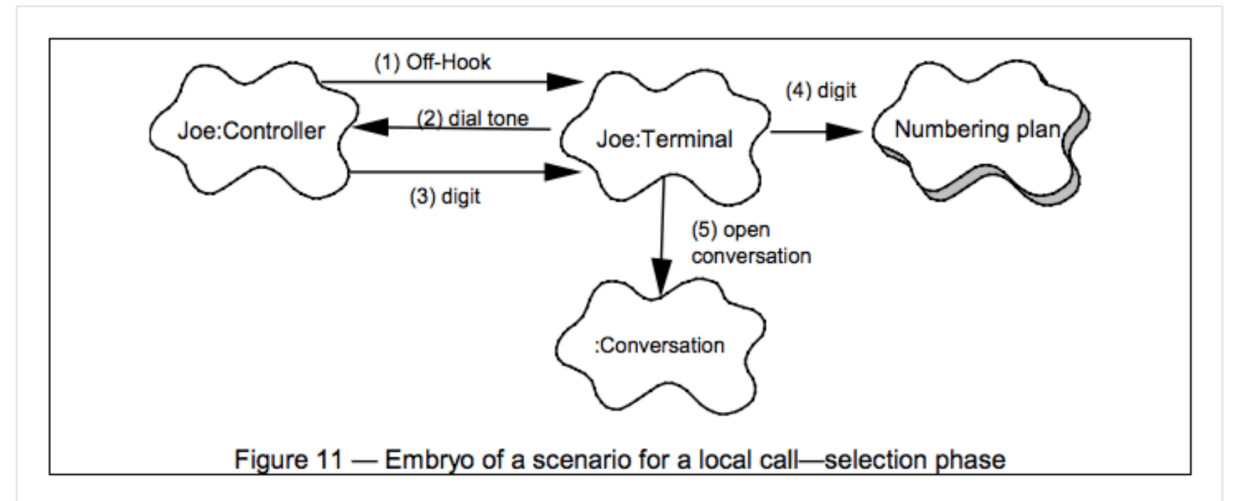
- 다른 모든 View 의 사용자와 평가자(Evaluator)

■ 고려 사항

- 시스템의 일관성(consistency)과 타당성(validity)

■ 관련 UML 다이어그램

- Use case diagram, Sequence diagram, Communication diagram

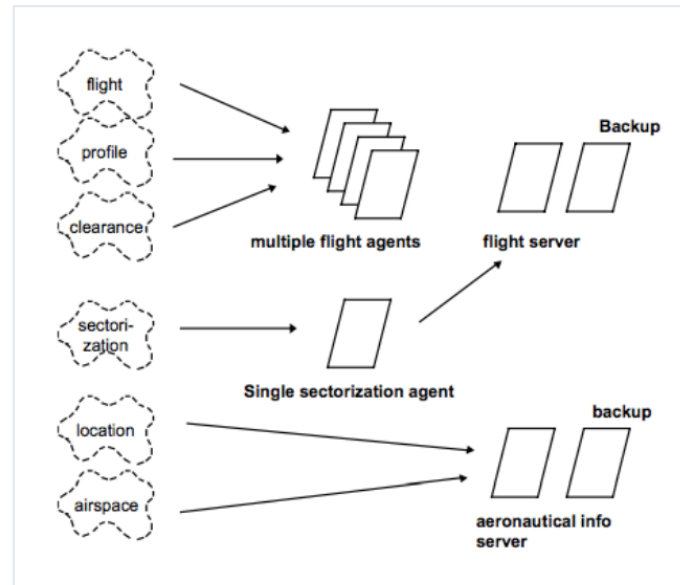


Scenario 예: 국내 통화 선택 단계의 시나리오 (Off-Hook → dial tone → digit → 대화 개시)

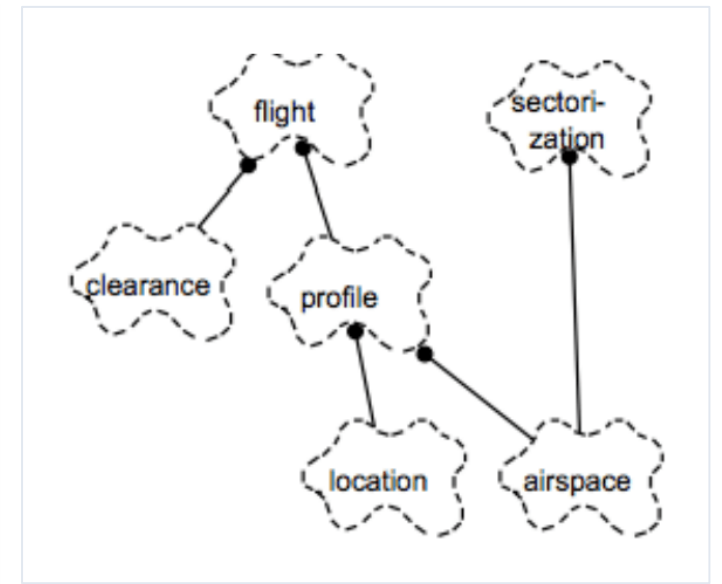
View 간 대응 — Logical → Process

■ Logical View 에서 Process View 로

- Logical View 는 동시성(concurrency)을 다루지 않는다
- 동시성과 동기화를 다루려면 Logical View 의 class 와 객체들을 Process View 의 process 에 매핑해야 한다
 - 예: flight, profile, clearance class → 여러 flight agent process 와 flight server 로 매핑
 - sectorization class → 단일 sectorization agent 로, location·airspace → aeronautical info server 로 매핑



Logical class 를 process 에 매핑

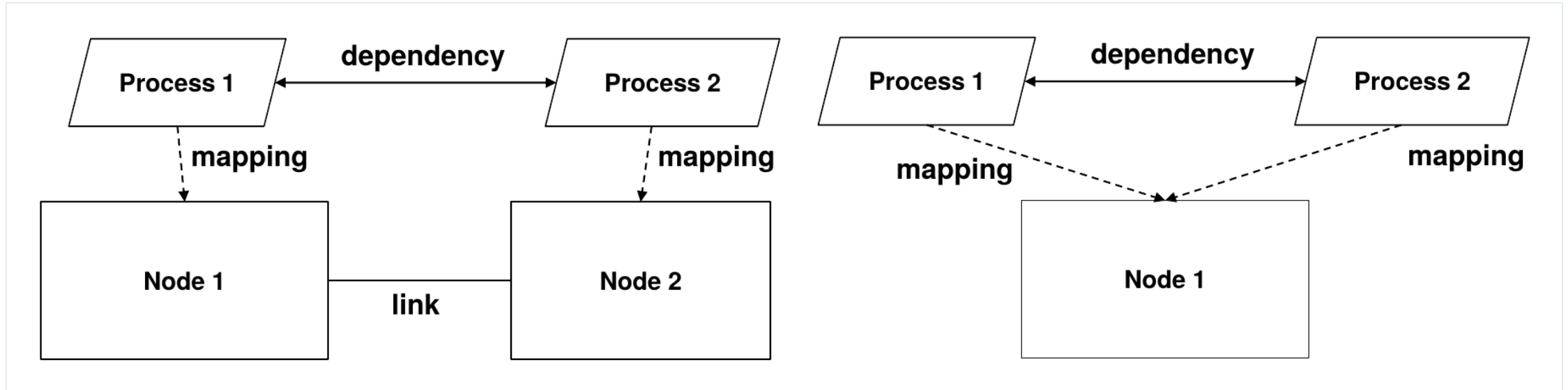


Logical View 의 class 관계

View 간 대응 — Process → Physical

■ Process View 에서 Physical View 로

- process 와 process group 을 물리적 컴퓨터 네트워크의 처리 노드(processing node)에 매핑하면 Physical View 가 된다
- Component 사이에 의존성(dependency)이 있으면, 그 component 들이 배치된 노드 사이에도 대응하는 링크(link)가 있어야 한다
 - Process 1 과 Process 2 가 서로 다른 Node 에 배치되면 Node 1 – Node 2 사이에 link 가 필요하다
 - 같은 Node 에 배치되면 별도의 link 가 필요 없다

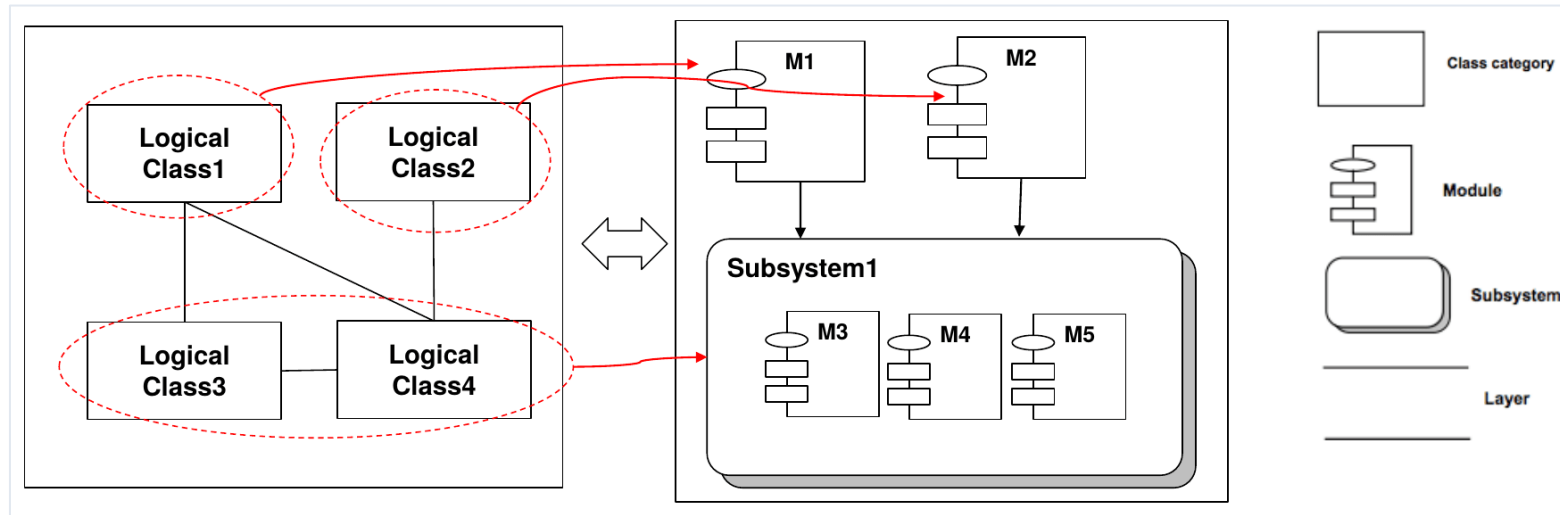


process 의 dependency 는 node 간 link 로 대응된다 (좌: 두 노드에 분산, 우: 한 노드에 배치)

View 간 대응 — Logical → Development

■ Logical View 에서 Development View 로

- Logical View 는 Development View 의 기초이기도 하다
- 보통 class 하나가 module 하나에 대응한다
- 여러 class 의 모음은 package 나 subsystem 으로 묶인다
 - 예: Logical Class1, Class2 → module M1, M2 / Logical Class3, Class4 → Subsystem1 (M3, M4, M5)



Logical class → module / subsystem 매핑

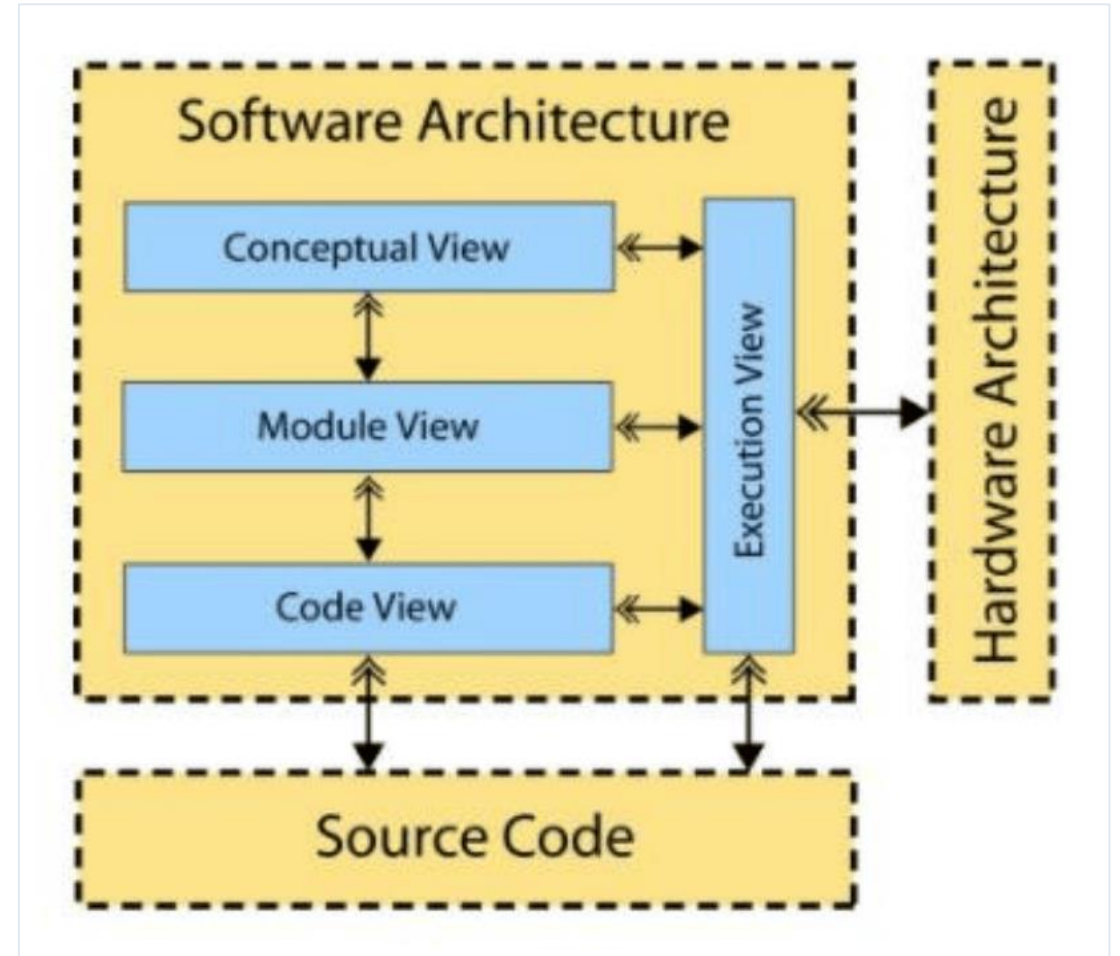
Siemens View Model 개요

■ 4개의 View 로 구성

- Conceptual View — 기능을 개념적 component 와 connector 로 분해
- Module View — 개념 요소를 subsystem 과 module 로 매핑
- Execution View — module 을 런타임 플랫폼 요소와 하드웨어에 매핑
- Code View — 실행 요소와 module 을 소스·배포 component 로 구성

■ 설계 흐름(Design Flow)

- Conceptual → Module → Code 순으로 구체화되며, Execution View 는 세 View 모두와 연결되어 Hardware Architecture 와 맞물린다
- 각 단계의 결과는 이전 단계로 피드백(Feedback)된다



Siemens View Model: 파란 상자 = View, 점선 상자 = Architecture, 화살표 = Design Flow / Feedback

Conceptual View 와 Meta-model

■ Conceptual View

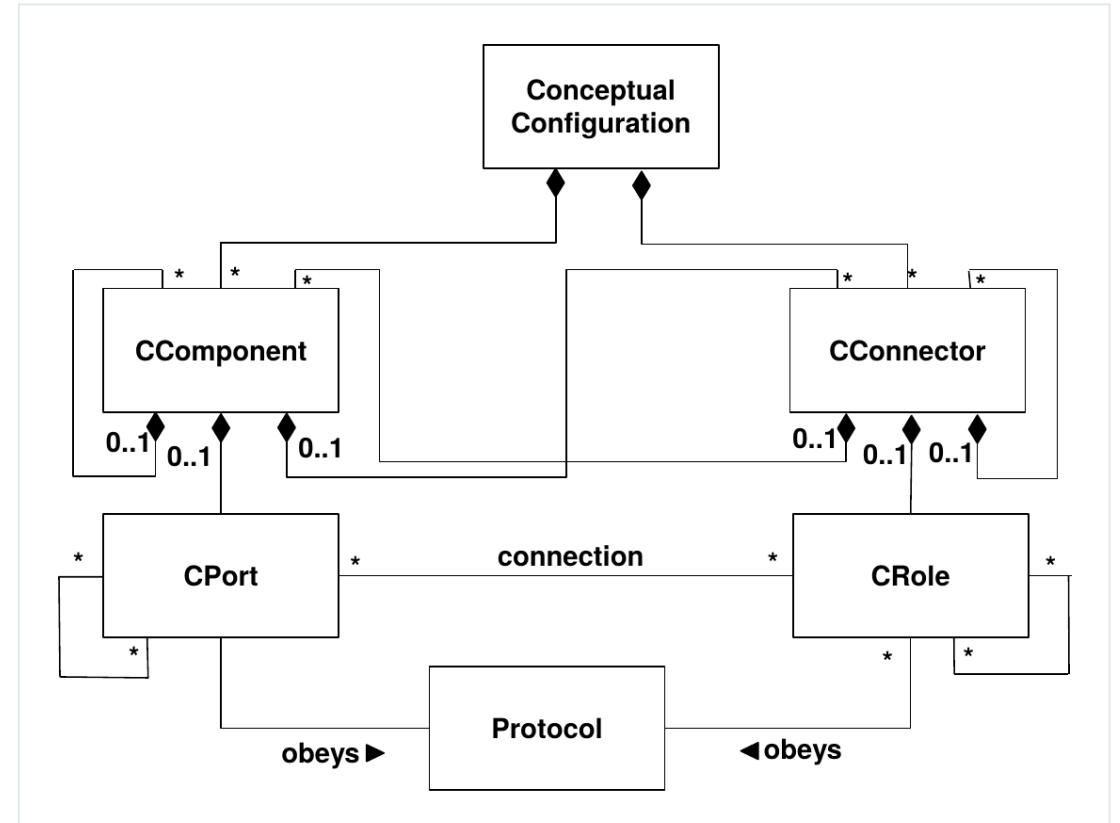
- 시스템의 기능(functionality)을 Architecture 요소에 매핑한다
 - 기능 분해를 위한 개념적 component(CComponent)
 - 조정과 데이터 교환을 위한 connector(CConnector)
- 특정 소프트웨어·하드웨어 기술에 비교적 독립적이다

■ Engineering concerns(설계 관심사)

- 시스템 기능을 component 와 connector 에 어떻게 매핑할 것인가
- 성능 같은 런타임 Quality Attribute 를 어떻게 다룰 것인가

■ Meta-model

- Conceptual Configuration 은 CComponent 와 CConnector 로 구성되고, CPort 와 CRole 이 connection 으로 연결되며 Protocol 을 따른다(obeyes)

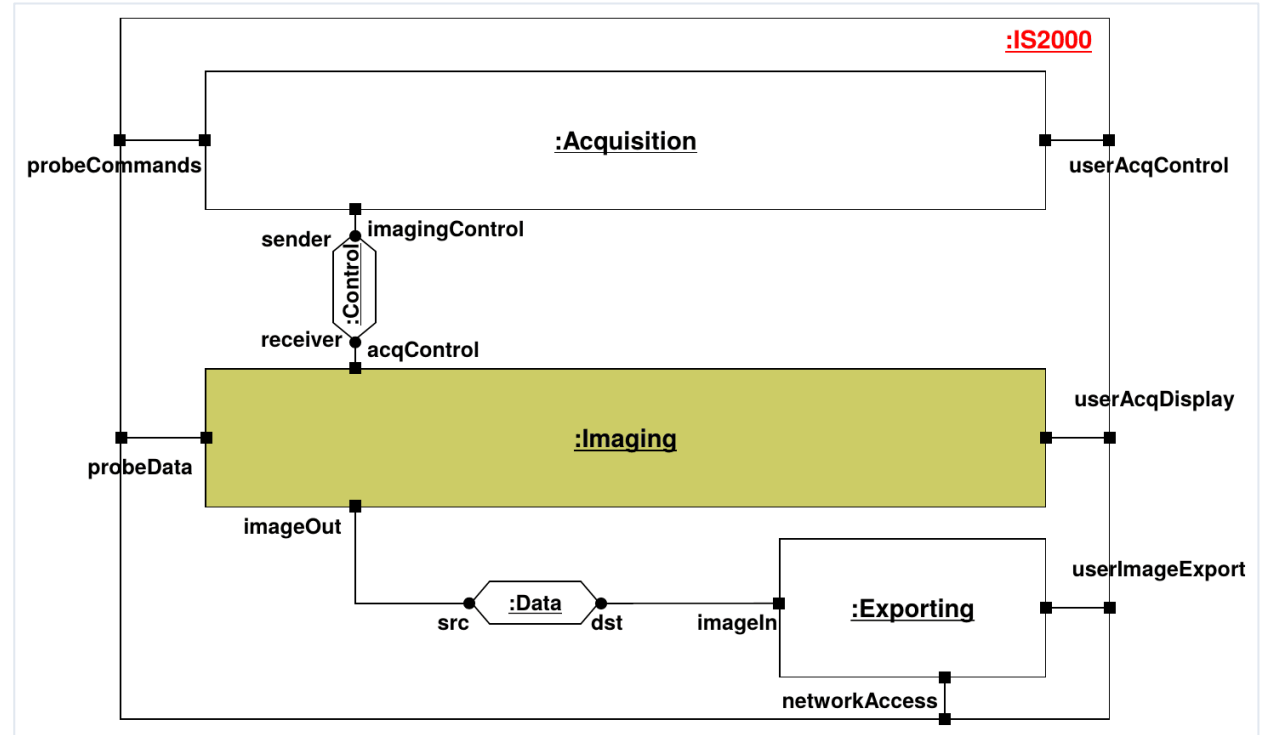


Conceptual architecture ∩ meta-model

Conceptual View 예제 — 상위 수준 구성 (IS2000)

■ IS2000 영상 시스템의 conceptual configuration

- Component: Acquisition, Control, Imaging, Exporting
 - Acquisition 이 probeCommands 로 장비를 제어하고, Control 이 imagingControl / acqControl 로 조정한다
 - Imaging 이 probeData 를 받아 imageOut 을 만들고, Data connector 를 거쳐 Exporting 으로 전달한다
- Connector: Control(sender/receiver), Data(src/dst)
- 외부 port: userAcqControl, userAcqDisplay, userImageExport, networkAccess



IS2000 의 상위 수준 conceptual configuration

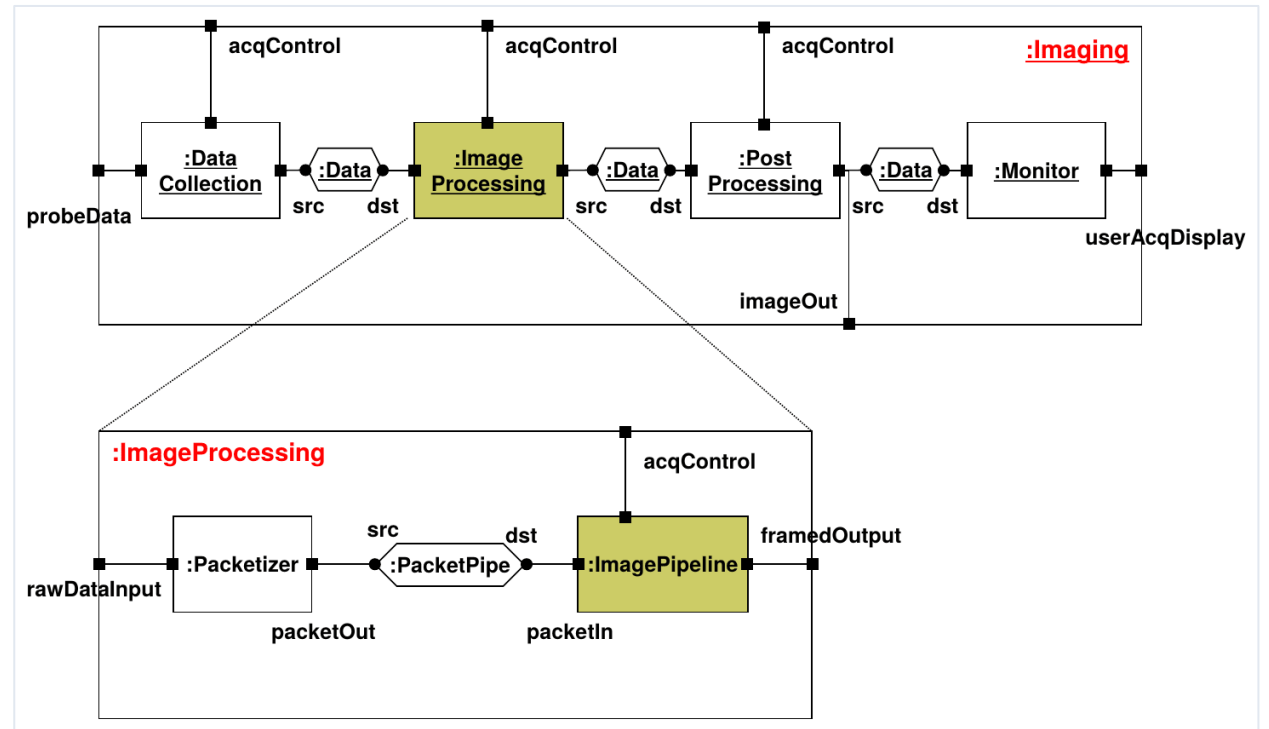
Conceptual View 예제 — Imaging 의 상세화

■ Imaging component 의 내부

- Data Collection → Image Processing → Post Processing → Monitor 가 Data connector(src/dst)로 파이프라인처럼 이어진다
- 각 단계는 acqControl port 로 제어를 받는다
- probeData 가 들어와 imageOut 과 userAcqDisplay 로 나간다

■ Image Processing 의 내부

- Packetizer → PacketPipe → ImagePipeline 으로 구성
- rawDataInput 을 packet 으로 나누어 framedOutput 을 만든다



Imaging 과 그 하위 ImageProcessing 의 refinement

Conceptual View 예제 — ImageProcessing 의 상세화

■ ImagePipeline 의 내부 구조

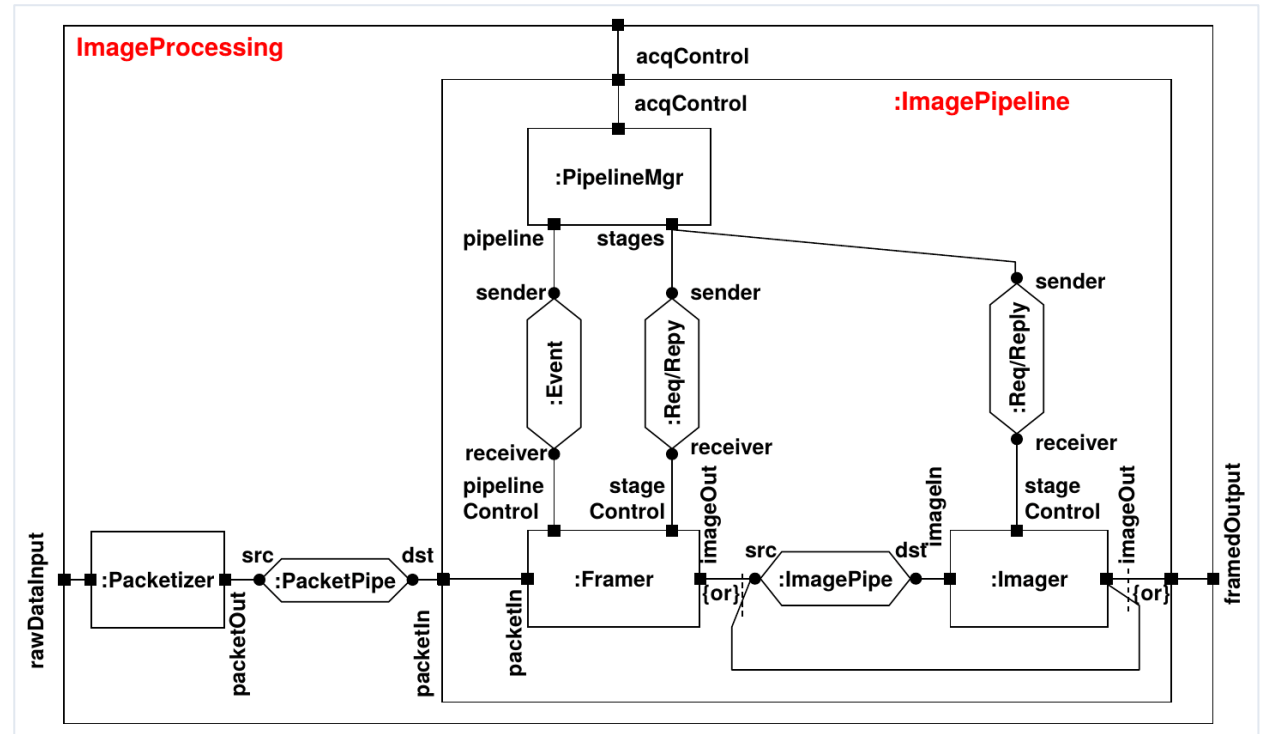
- PipelineMgr 가 pipeline / stages port 로 각 단계를 관리한다
- Framer 와 Imager 가 처리 단계(stage)이며, ImagePipe connector(src/dst)로 이어진다
 - {or} 표시: Framer 와 Imager 사이에는 대안적 연결이 존재한다

■ 제어용 connector

- Event connector: PipelineMgr → pipelineControl(sender/receiver)
- Req/Reply connector: PipelineMgr ↔ 각 stage 의 stageControl

■ 데이터 흐름

- rawDataInput → Packetizer(packetOut) → PacketPipe → packetIn → Framer → ImagePipe → Imager → framedOutput

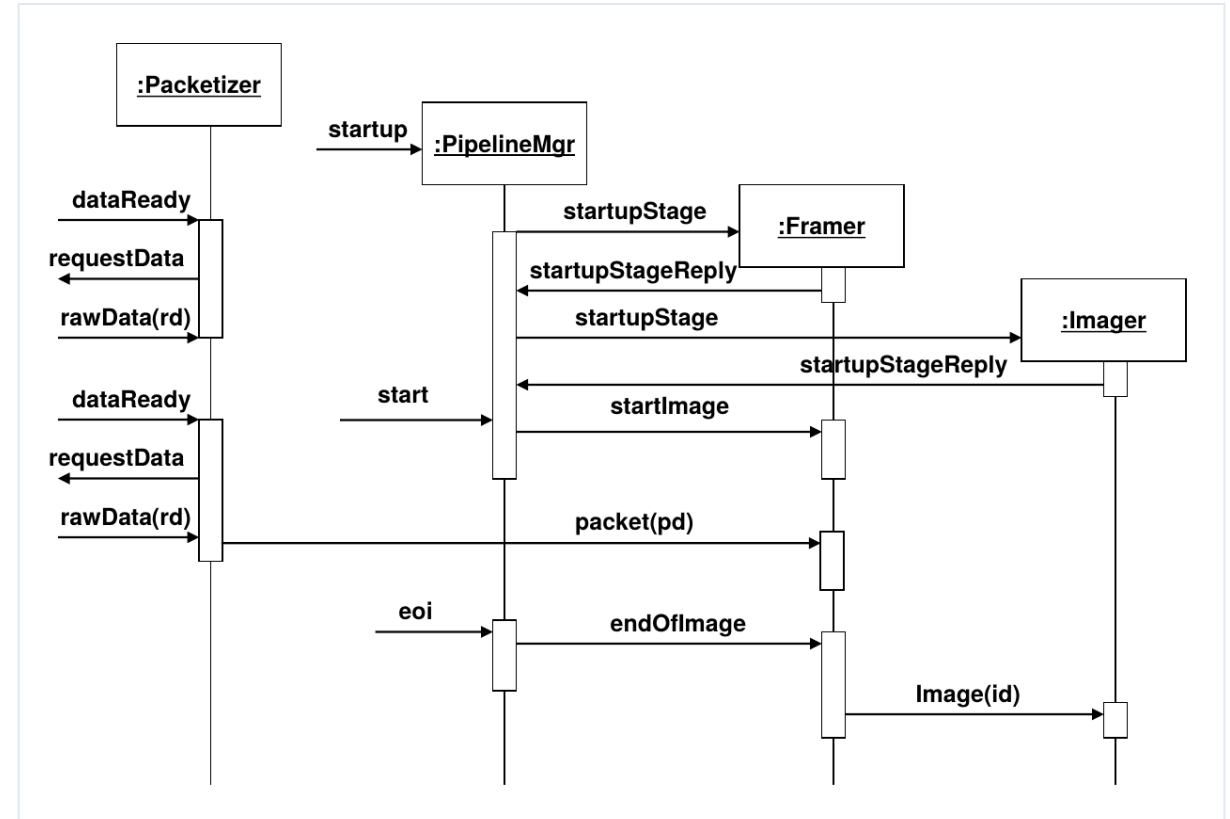


ImageProcessing 의 refinement

Conceptual View 예제 — ImageProcessing 의 행위 모델링

■ Sequence diagram 으로 표현한 행위

- startup: PipelineMgr 가 각 stage(Framer, Imager) 에 startupStage 를 보내고 startupStageReply 를 받는다
- start: PipelineMgr 가 Framer 에 startImage 를 보내 영상 처리를 시작한다
- Packetizer 는 dataReady 가 오면 requestData 로 rawData(rd)를 요청·수신하고, packet(pd)를 Framer 에 넘긴다
- eoi(end of image): Framer 가 endOfImage 를 보내면 Imager 가 Image(id)를 만든다



ImageProcessing 의 behavior modeling (sequence diagram)

핵심 Conceptual View 는 구조뿐 아니라 component 사이의 상호작용(행위)도 함께 모델링한다

Module View 와 Meta-model

■ Module View

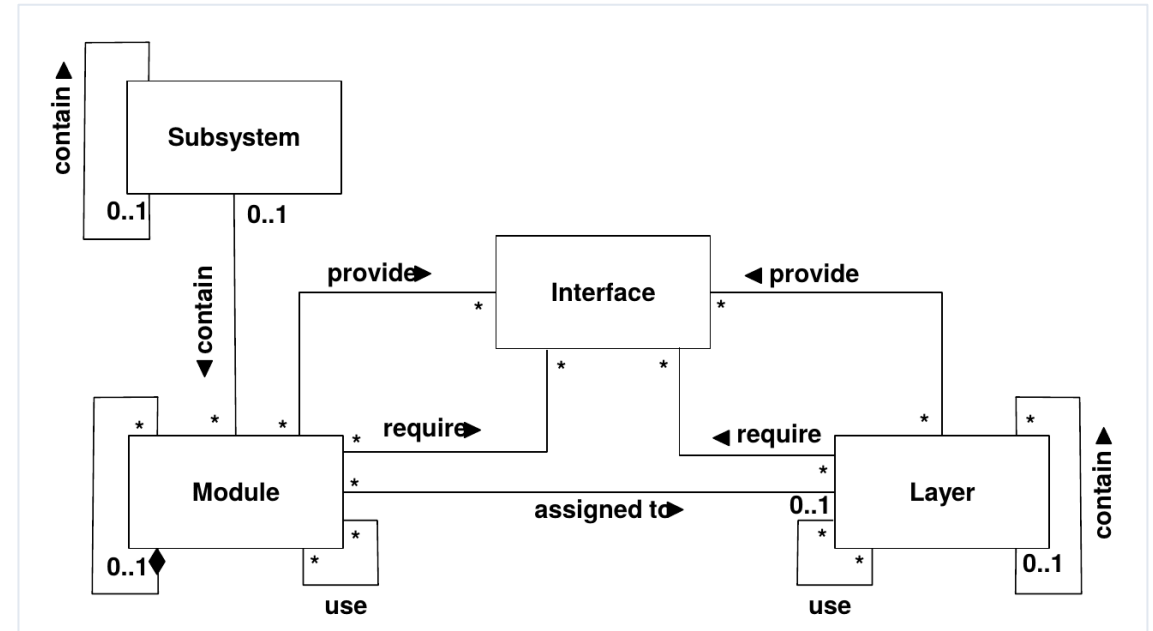
- Conceptual View 의 component 와 connector 를 subsystem 과 module 로 매핑한다
- Architect 는 개념적 해법을 오늘날의 소프트웨어 플랫폼과 기술로 어떻게 실현할지 다룬다

■ Engineering concerns

- module 사이의 의존성을 어떻게 최소화할까
- module 과 subsystem 의 재사용을 어떻게 최대화할까
- COTS 소프트웨어, 플랫폼, 표준의 변경으로부터 제품을 어떻게 격리할까

■ Meta-model

- Subsystem 은 Module 을 포함(contain)하고, Module 은 Interface 를 제공(provide)·요구(require)하며 Layer 에 할당(assigned to)된다



Module architecture 의 meta-model

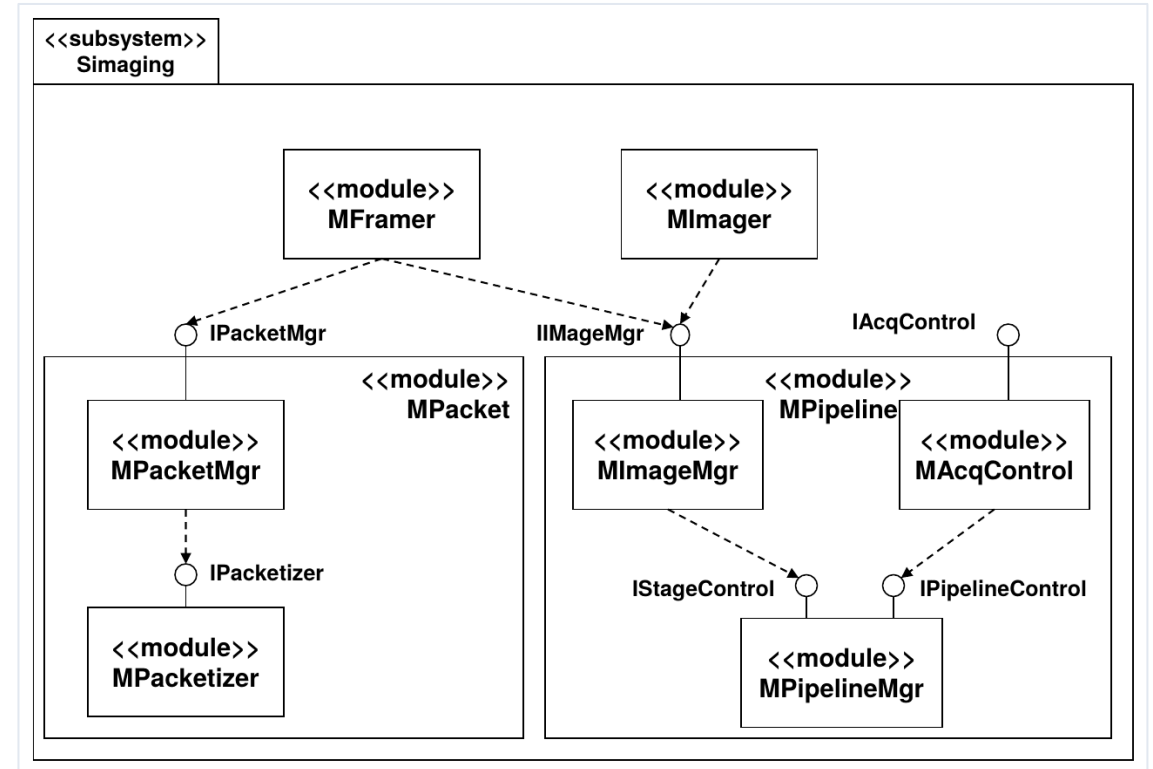
Module View 예제

■ Subsystem Imaging 의 구성

- Module: MFramer, MImager, MPacket(MPacketMgr, MPacketizer), MPipeline(MImageMgr, MAcqControl, MPipelineMgr)
- Interface: IPacketMgr, IImageMgr, IAcqControl, IPacketizer, IStageControl, IPipelineControl
- MFramer 와 MImager 는 IPacketMgr, IImageMgr interface 를 통해 하위 module 을 사용한다

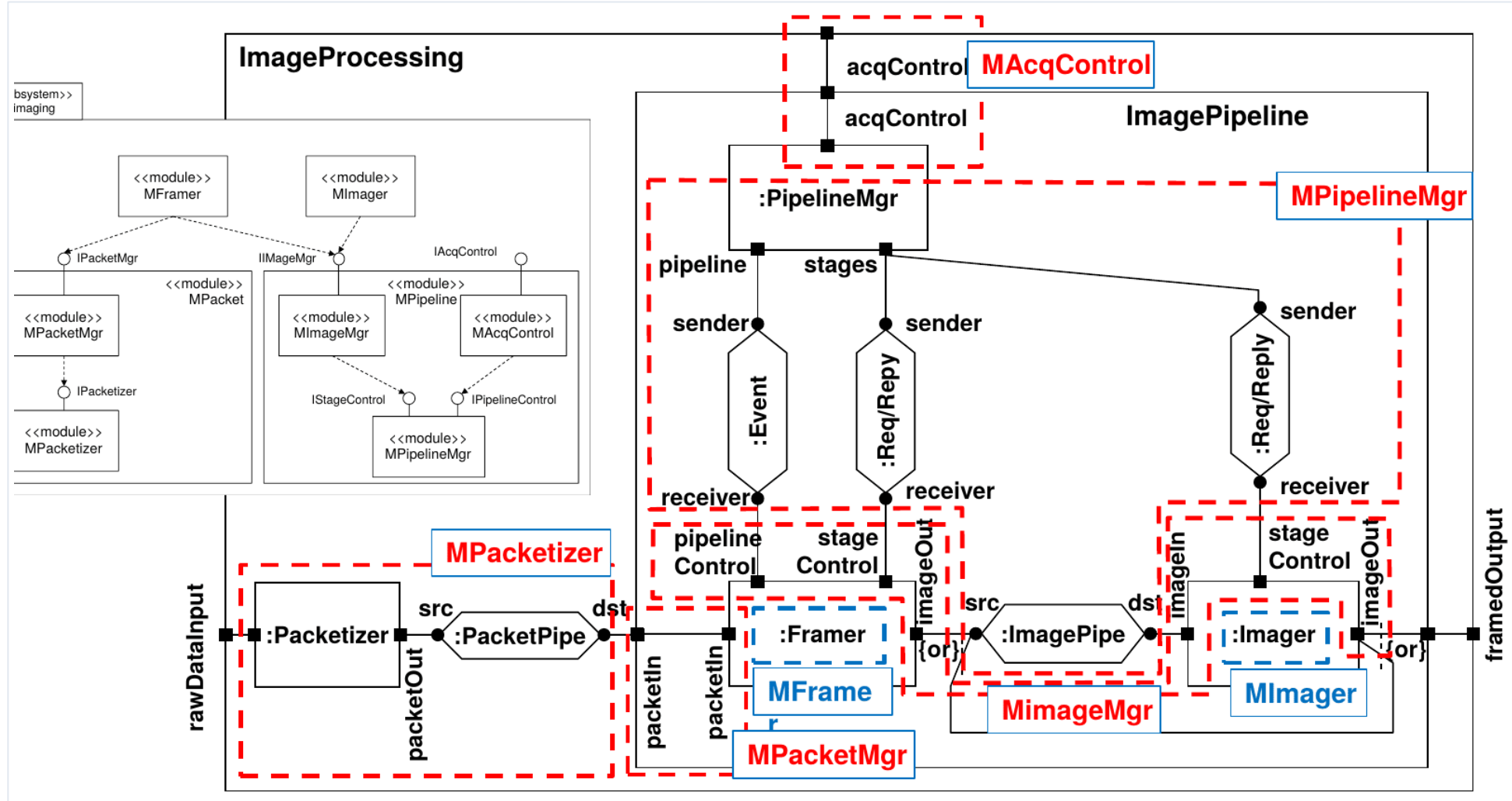
■ 읽는 법

- <<subsystem>>, <<module>> stereotype 으로 종류를 구분
- 동그라미(lollipop)는 제공 interface, 점선 화살표는 의존(require)을 뜻한다



Imaging subsystem 의 module 구조

Conceptual 요소를 Module 로 매핑하기



Conceptual View 위에 module 경계를 겹쳐 그린 그림 — 빨간 점선 = module 경계, 파란 이름 = 대응하는 module (좌상단 작은 그림은 Module View)

Conceptual 요소 ↔ Module 요소 대응표

- ImageProcessing component 전체가 SIImaging subsystem 이 되고, 각 component(Packetizer, PipelineMgr, Framers, Imager)는 하나의 module 이 된다
- packetIn port 와 PacketPipe connector 는 MPacketMgr 로, acqControl port 는 MAcqControl 로, 제어 port 와 imageIn/imageOut 은 MImageMgr 로 묶인다 — 하나의 module 이 여러 conceptual 요소를 담을 수 있다

Conceptual 요소 (이름)	종류	Module 요소 (이름)	종류
ImageProcessing	Component	SIImaging	Subsystem
Packetizer / packetOut / PacketPipe, src, dst	Component / Port / Connector 와 role	MPacketizer	Module
packetIn	Port	MPacketMgr	Module
acqControl	Port	MAcqControl	Module
PipelineMgr / pipeline, stages / ImagePipe, src, dst / Client-Server, sender, receiver / Event, sender, receiver	Component / Port / Connector 와 role	MPipelineMgr	Module
pipelineControl, stageControl, imageIn, imageOut	Port	MImageMgr	Module
Framer	Component	MFramer	Module
Imager	Component	MImager	Module

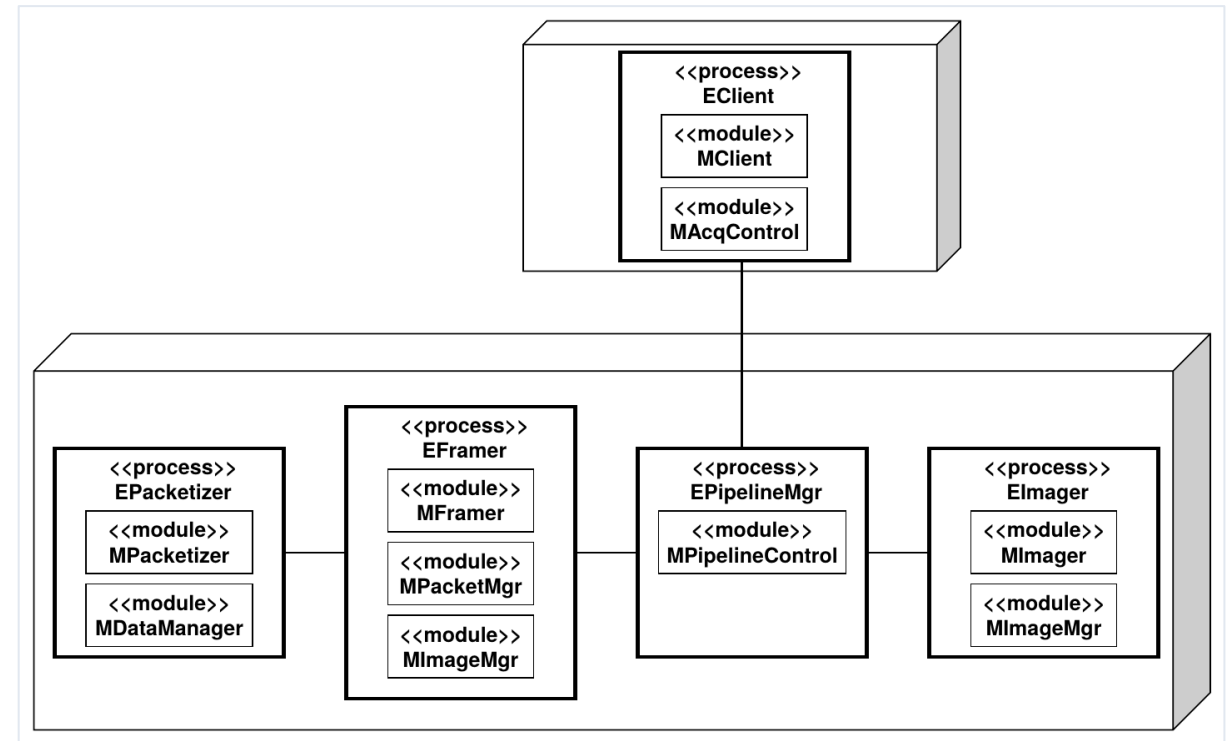
Execution View 와 예제

■ Execution View 가 기술하는 것

- module 이 런타임 플랫폼이 제공하는 요소(process, thread 등)에 어떻게 매핑되는가
- 그 런타임 요소가 하드웨어 Architecture 에 어떻게 매핑되는가
- 시스템의 런타임 엔티티와 그 속성(예: 메모리 사용량)

■ Engineering concerns

- 성능, 복구(recovery), 재구성(reconfiguration) 요구사항을 어떻게 만족시킬까
- 자원 사용을 어떻게 균형 있게 할까
- 필요한 동시성, 복제(replication), 분산을 어떻게 달성할까



Execution View 예: module 들을 <<process>> EClient, EPacketizer, EFrmer, EPipelineMgr, Elmager 에 배치

Code View

■ Code View 가 기술하는 것

- 시스템을 구현하는 소프트웨어가 어떻게 조직되어 있는가

■ Architect 가 결정하는 것

- Execution View 의 런타임 엔티티를 배포 component(예: 실행 파일)에 어떻게 매핑할까
- Module View 의 module 을 소스 component 에 어떻게 매핑할까
- 소스 component 로부터 배포 component 를 어떻게 만들어 낼까(build)

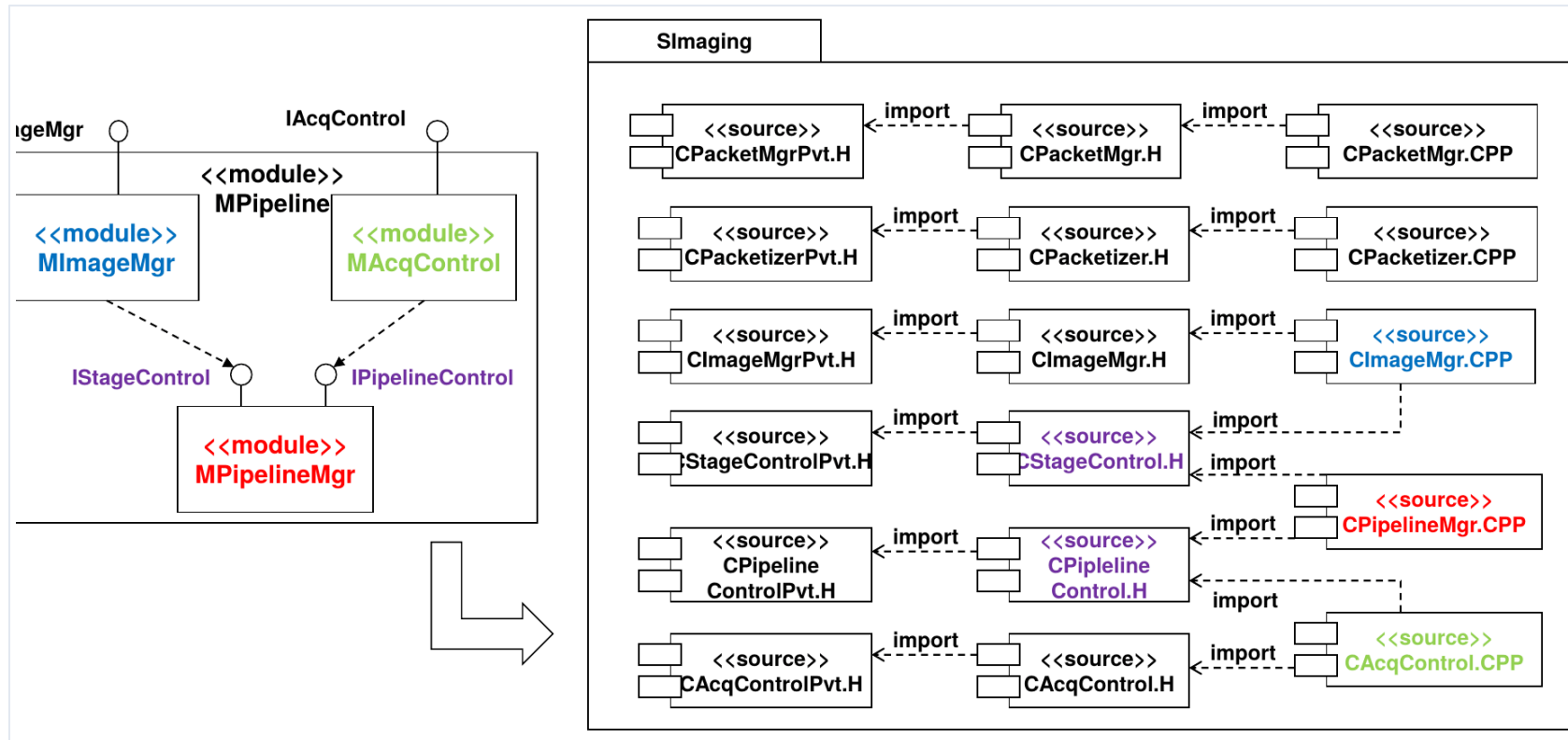
■ Engineering concerns

- 제품 업그레이드에 드는 시간과 노력을 어떻게 줄일까
- 제품 버전과 릴리스를 어떻게 관리할까
- 빌드 시간을 어떻게 줄일까

핵심 Code View 는 module 과 런타임 요소를 실제 소스 파일·실행 파일로 연결하고, 빌드·버전·릴리스를 다룬다

Code View 예제

- MPipeline subsystem 의 module(MImageMgr, MAcqControl, MPipelineMgr)이 각각 <<source>> 파일 3개 (xxxPvt.H → xxx.H → xxx.CPP, import 관계)로 구현된다



Module → 소스 파일(.H / .CPP) 매핑. 색은 어떤 module 에 속하는 소스인지 나타낸다

Siemens View Model 정리

- 네 View 는 Conceptual → Module → Execution → Code 순으로 점점 구현에 가까워지며, 각 View 마다 architect 가 답해야 할 engineering concern 이 다르다

View	무엇을 매핑하나	주요 Engineering concerns
Conceptual View	기능 → 개념적 component 와 connector (기술에 독립적)	기능을 component/connector 에 매핑, 런타임 Quality Attribute(성능 등) 처리
Module View	component/connector → subsystem 과 module	의존성 최소화, 재사용 최대화, COTS·플랫폼·표준 변경으로부터 격리
Execution View	module → 런타임 플랫폼 요소(process 등) → 하드웨어	성능·복구·재구성 요구 만족, 자원 균형, 동시성·복제·분산
Code View	런타임 엔티티 → 배포 component, module → 소스 component	업그레이드 시간·노력 절감, 버전·릴리스 관리, 빌드 시간 단축

SEI View Model 개요

■ Component-and-Connector View (C&C View)

- 런타임 행위를 가진 요소 (component)와 그들 사이의 상호 작용(connector)의 집합으로 구조화
- 시스템이 실행 중에 어떻게 동작하는지를 보여준다

■ Module View

- 코드 단위(module)와 그 관계의 집합으로 구조화
- 구현 단위의 조직을 보여준다

■ Allocation View

- 소프트웨어 시스템이 환경의 비(非)소프트웨어 구조(하드웨어, 파일 시스템, 조직)와 어떻게 관계되는지를 보여준다

핵심 SEI(Carnegie Mellon Software Engineering Institute)는 세 종류의 View 로 Architecture 를 문서화한다

C&C View (Component-and-Connector View)

■ C&C View 란

- 런타임 엔티티와 그들 사이에 일어날 수 있는 상호작용의 그림을 제공한다

■ Component — 런타임 엔티티

- 주요 처리 단위(processing unit)와 데이터 저장소(data store)를 나타낸다
 - 예: process, thread, object, data store 등

■ Connector — 상호작용

- component 사이의 복잡한 통신 형태를 표현한다
 - 예: client 와 server component 사이의 통신 프로토콜
 - 예: event broadcast, N-way 데이터 동기화

■ Configuration

- component 와 connector 가 서로 연결되어 런타임 시스템 구성(configuration)을 이룬다

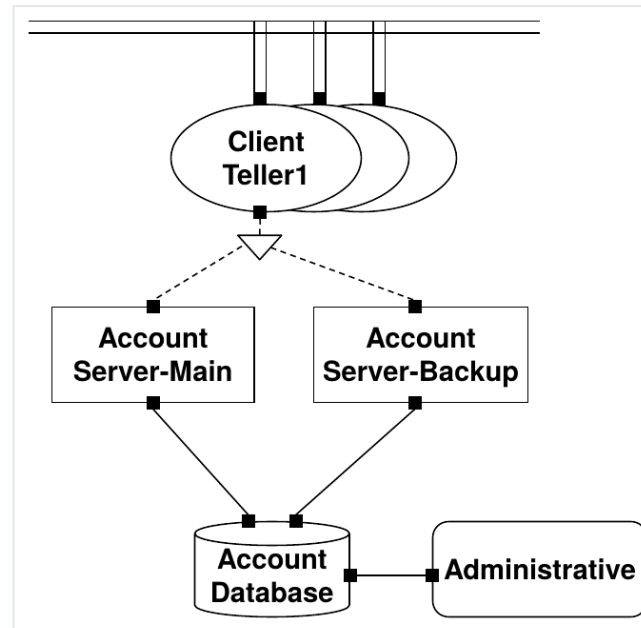
C&C View 예제 — 조감도(bird's-eye view)

■ 계좌 시스템의 C&C View

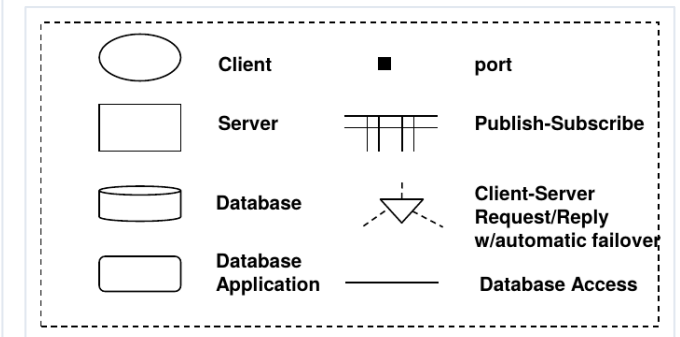
- 여러 Client Teller 가 계좌 저장소 서버(Account Server)와 상호작용한다
- Main 서버가 다운되면 Backup 서버가 넘겨받는다 (automatic failover)
- Administrative component 는 관리자가 공유 데이터 저장소(Account Database)에 접근·유지보수하게 해 준다

■ 범례(legend)가 꼭 필요하다

- Client, Server, Database, Database Application 은 component 종류
- Publish-Subscribe, Client-Server Request/Reply, Database Access 는 connector 종류



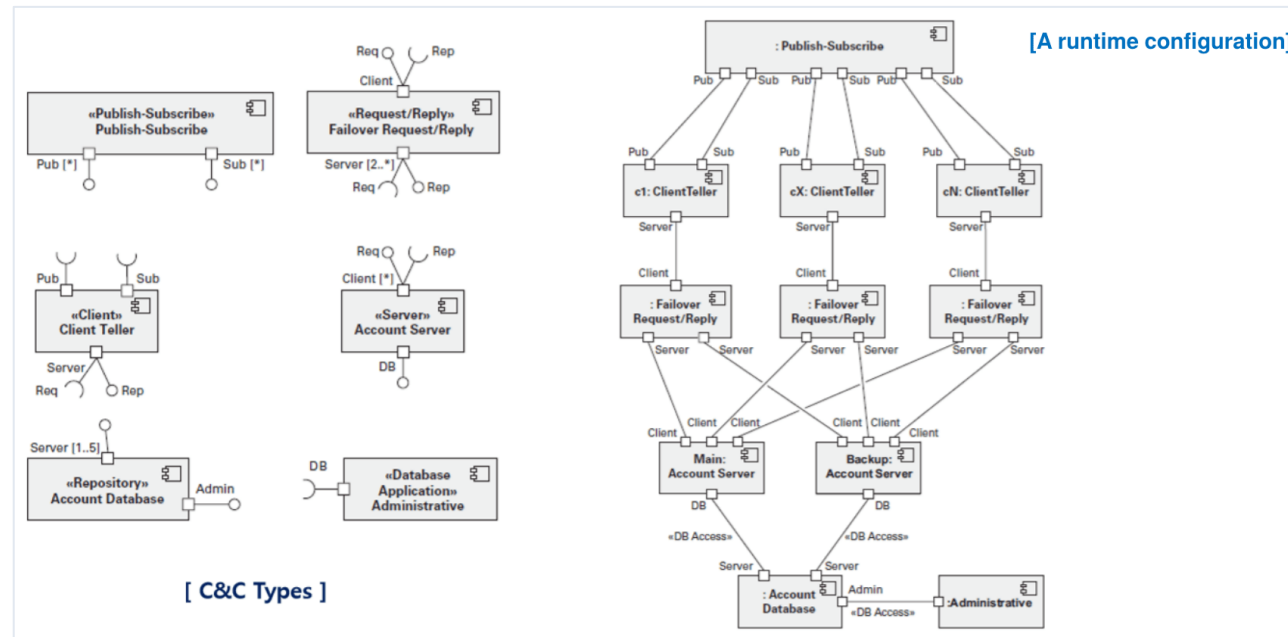
계좌 시스템 C&C View



범례

C&C View 예제 — C&C Type 과 런타임 구성

- 왼쪽: C&C Type 정의 — «Publish-Subscribe», «Request/Reply» Failover, «Client» Client Teller, «Server» Account Server, «Repository» Account Database, «Database Application» Administrative
- 오른쪽: 실제 런타임 구성 — c1..cN ClientTeller 가 Publish-Subscribe 와 Failover Request/Reply 를 거쳐 Main / Backup Account Server, Account Database 에 연결된다



C&C Types(좌)와 runtime configuration(우). 출처: Documenting Software Architectures: Views and Beyond (2nd Ed.)

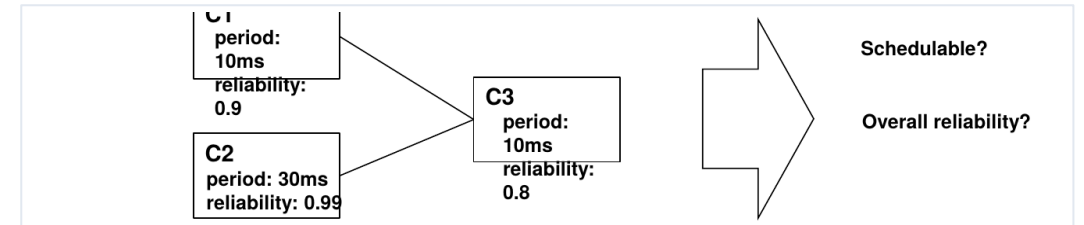
C&C View 는 무엇에 쓰는가

■ 용도

- 시스템이 어떻게 동작하는지(how the system works) 보여준다
- 성능, 신뢰성(reliability), 가용성(availability) 같은 런타임 Quality Attribute 를 architect 가 추론하도록 돕는다

■ 예

- 시스템 전체가 실시간 스케줄링 요구를 만족하는지 판단하려면 각 process 의 주기(cycle time)를 알아야 한다
- 시스템 전체 신뢰성을 추정·계산하려면 개별 요소의 신뢰성을 알아야 한다
 - C1(period 10ms, reliability 0.9), C2(30ms, 0.99), C3(10ms, 0.8) → 스케줄 가능한가? 전체 신뢰성은?



각 component 의 period 와 reliability로부터 전체 시스템의 스케줄 가능성과 신뢰성을 분석한다

Module View

■ Module View 란

- 시스템의 module 집합과 그들 사이의 관계를 기술한다

■ Module — 주요 구현 단위

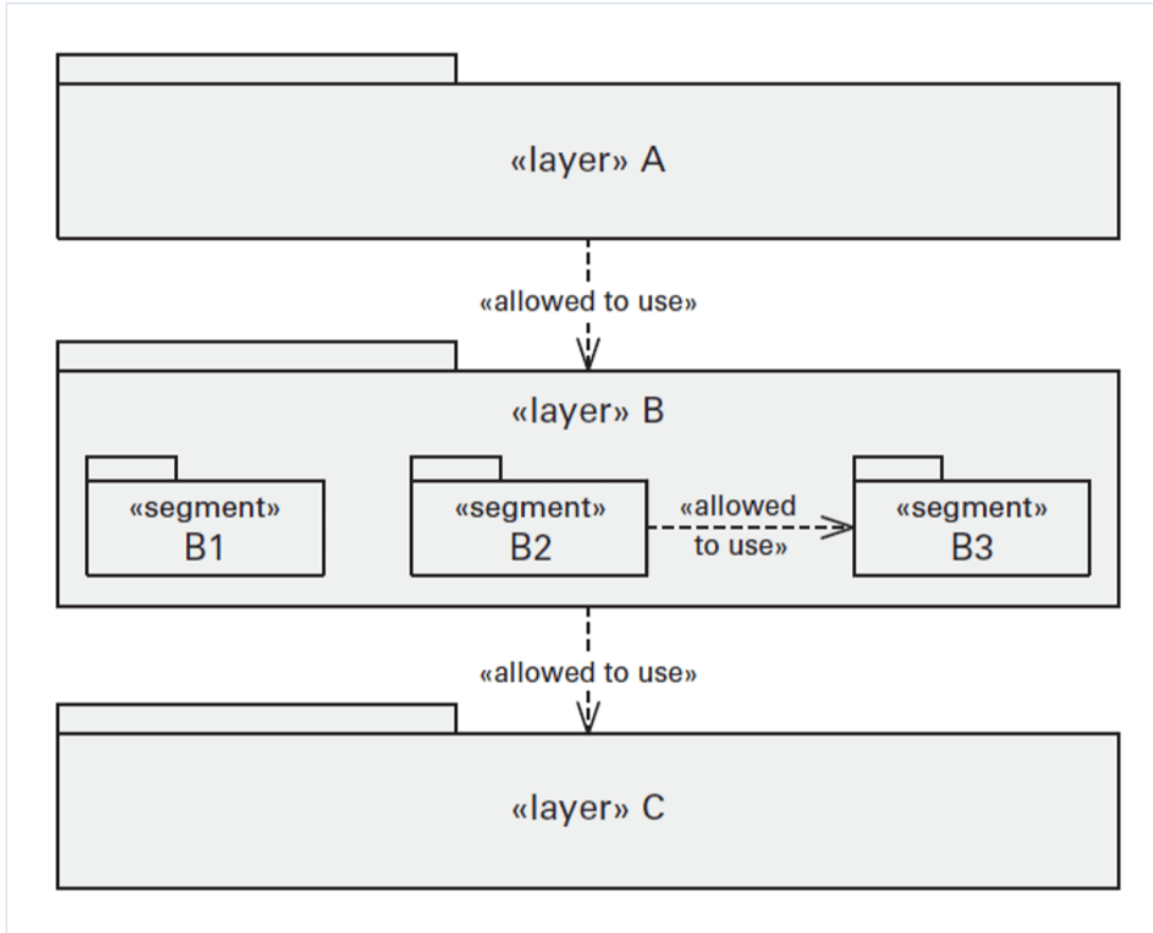
- 일관된 책임(responsibility)의 집합을 제공하는 소프트웨어 구현 단위
- class, procedure, layer 또는 코드 단위의 어떤 분해든 module 이 될 수 있다

■ 여러 가지 구조

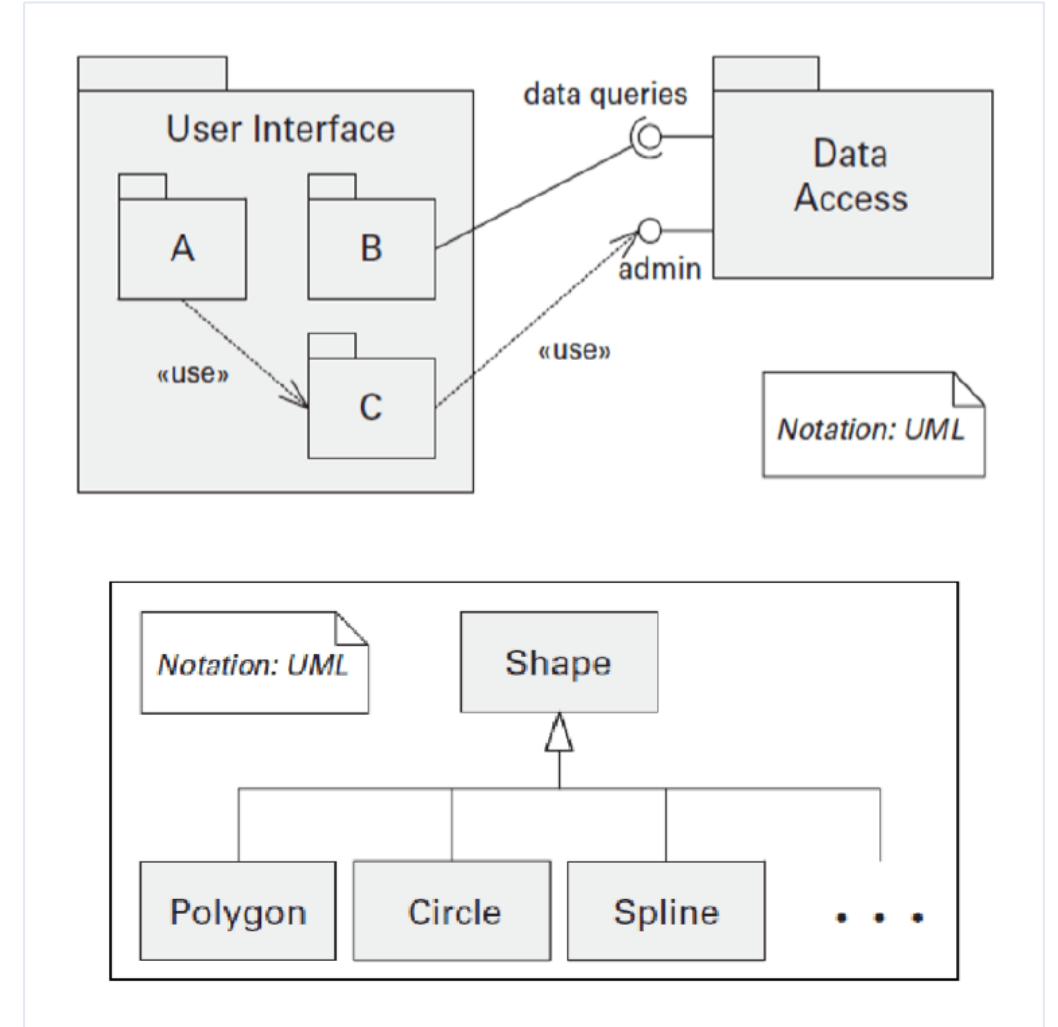
- module 사이의 관계(relation) 종류에 따라 module 을 서로 다르게 구조화할 수 있다
 - 예: Uses 구조, Layered 구조, Generalization 구조

핵심 Module View 는 '코드가 어떻게 나뉘어 있고 서로 어떤 관계인가'를 보여주는 정적(static) View 이다

Module View 예제



Layered 구조: «layer» A → B → C 의 allowed-to-use 관계



Uses 구조(User Interface → Data Access)와 Generalization 구조(Shape ← Polygon, Circle, Spline)

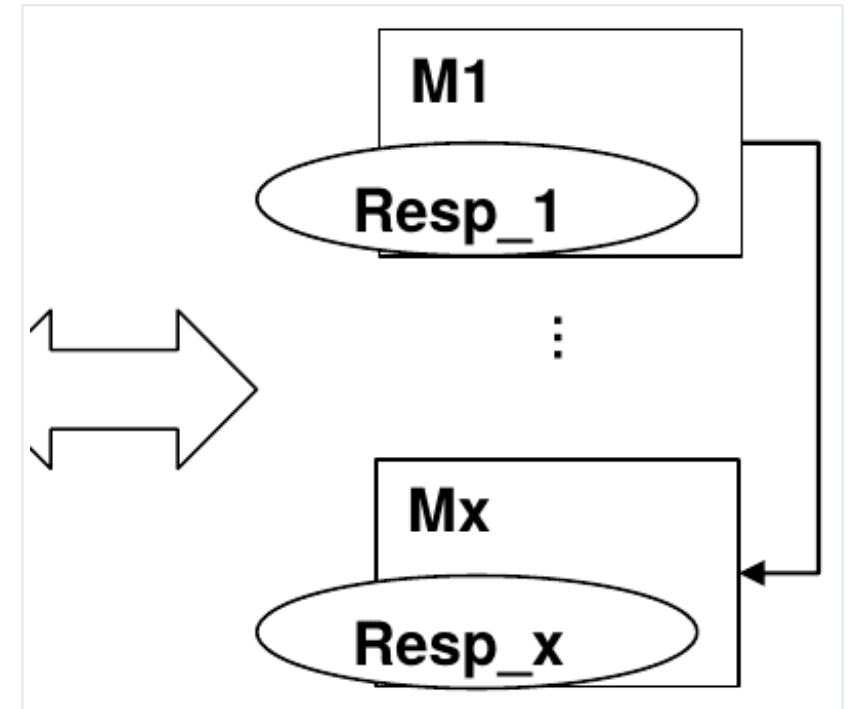
Module View 는 무엇에 쓰는가

■ Module View 의 용도

- Construction(구축): 소스 코드의 청사진(blueprint)
- Analysis(분석): 요구사항 추적(traceability)과 변경 영향 분석(impact analysis)
 - 요구사항 R1..Rx 가 어떤 module M1..Mx 의 책임(Resp)으로 이어지는지 추적
- Communication(의사소통): 시스템 책임을 위에서 아래로 제시하면 시스템을 모르는 사람도 이해하기 쉽다

■ Module View 로 할 수 없는 것

- 런타임 행위와 런타임 품질(성능, 신뢰성 등)의 분석 — 이것은 C&C View 의 몫이다



요구사항 ↔ module 책임의 추적

Allocation View

■ Allocation View 란

- 소프트웨어 요소(Module View 또는 C&C View 의 요소)와 환경 요소(environment element) 사이의 매핑을 보여준다

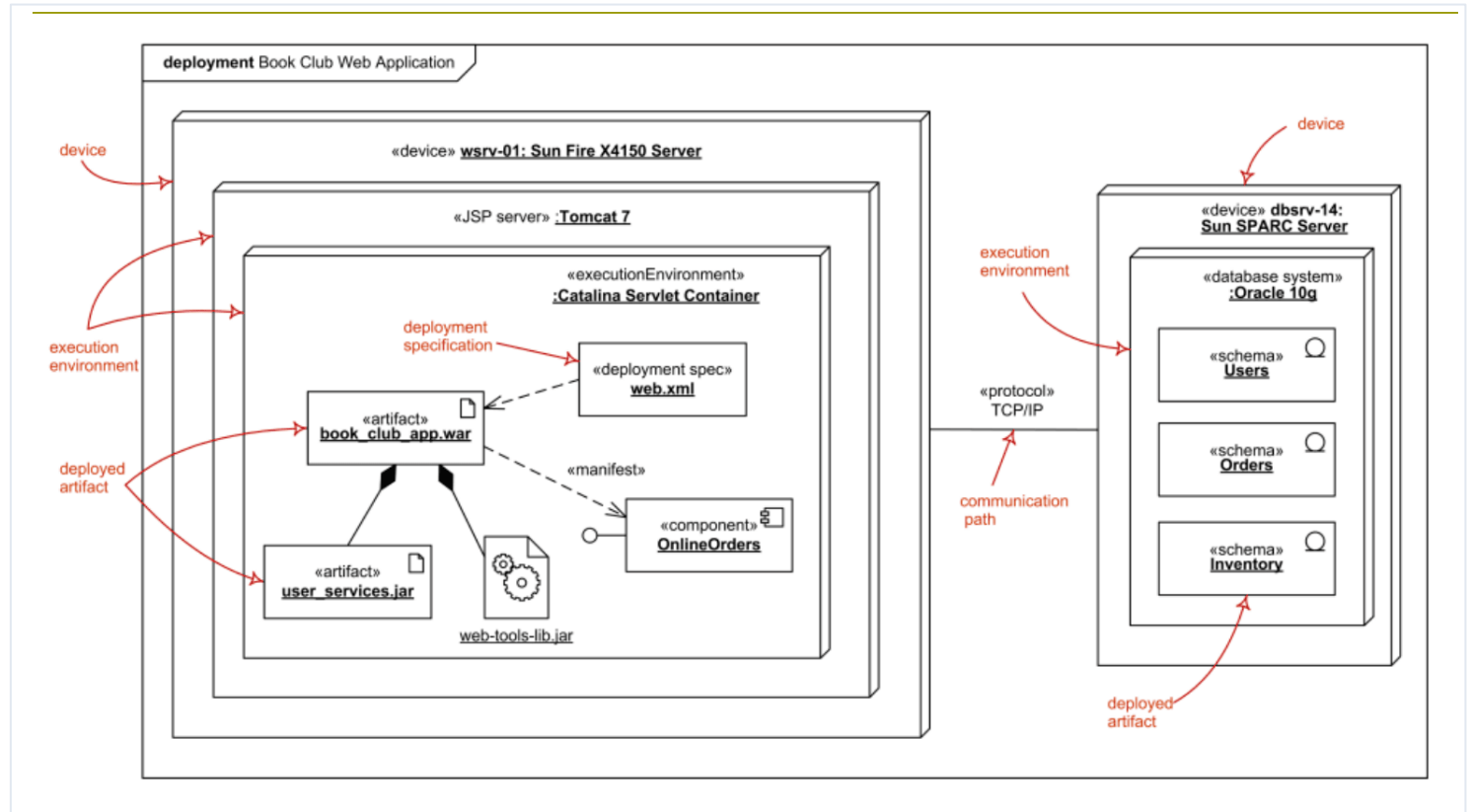
■ 세 가지 Allocation Style

- Deployment style: component 와 connector 를 소프트웨어가 실행되는 하드웨어에 매핑
- Installation style: 소프트웨어 component 를 운영 환경의 파일 시스템 구조에 매핑
- Work assignment style: 소프트웨어 module 을 사람, 팀, 조직 단위에 매핑

핵심 Allocation View 는 소프트웨어를 하드웨어·파일 시스템·조직 같은 '소프트웨어 바깥의 것'과 연결한다

Deployment Style 예제

- Book Club 웹 애플리케이션의 deployment: «device» Sun Fire 서버 위의 «JSP server» Tomcat 7 → «executionEnvironment» Catalina Servlet Container 에 «artifact» book_club_app.war 등이 배치되고, TCP/IP communication path 로 «database system» Oracle 10g 가 있는 Sun SPARC 서버와 연결된다



UML deployment diagram: device, execution environment, deployed artifact, communication path

Deployment Style 은 무엇에 쓰는가

■ 성능·신뢰성·보안 분석에 사용

- Performance: 소프트웨어를 하드웨어에 할당하는 방식을 바꿔 성능을 조정한다
- Reliability: 처리 요소나 통신 채널의 고장이 신뢰성에 직접 영향을 준다
- Security: 하드웨어 구성과 소프트웨어 할당 방식이 보안에 영향을 준다

■ 비용 산정 지원

- 하드웨어 구매 옵션을 평가할 때 비용 추정의 근거로 Deployment View 를 사용할 수 있다

핵심 소프트웨어를 어느 하드웨어에 두느냐가 성능·신뢰성·보안과 비용을 좌우한다

세 View Model 의 대응 관계

- 이름은 다르지만 세 모델 모두 '논리 구조 · 런타임(프로세스) · 배치 · 개발 단위'라는 공통 관심사를 View 로 나눈다
- SEI 의 Allocation View 는 Deployment / Installation / Work Assignment 세 style 로 세분된다

View 의 성격	4+1 View Model	SEI View Model		Siemens View Model
Logical / Conceptual View	Logical View	Module View		Conceptual View
Process / Concurrency View	Process View	C&C View		Execution View
Deployment View	Physical View	Allocation View	Deployment Style	—
Installation View	—		Installation Style	Code View
Work Assignment View	—		Work Assignment Style	—
Development / Module View	Development View	Module View		Module View

2.2

Documenting Architecture

SEI View Model 을 이용한 Architecture 문서화 — 왜, 무엇을, 어떻게 문서화하는가

- Architecture Documentation 의 목적
- Box-and-line 다이어그램의 문제점
- 좋은 Architecture 문서화의 기본 원칙
- View 문서화의 구성 요소 (Primary Presentation, Element Catalog 등)
- SEI View Model 을 이용한 문서화

Architecture 문서화의 목적

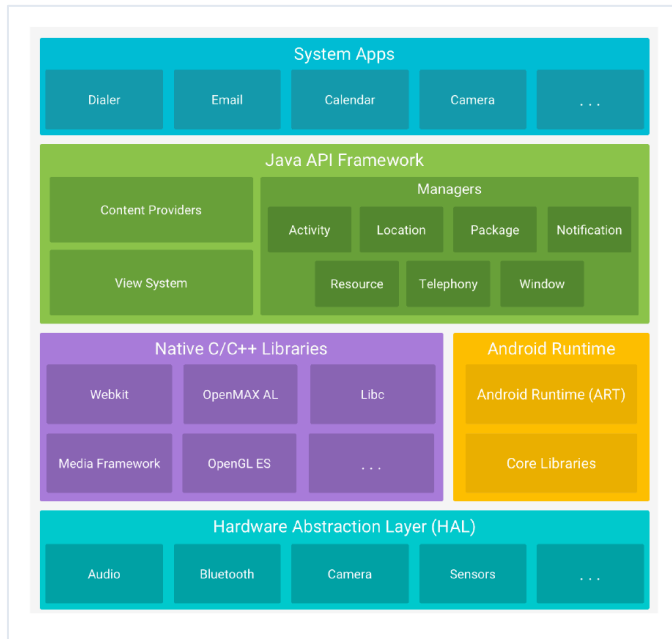
■ Architecture 문서는 다음 역할을 한다

- 교육 수단: 새로 합류한 팀원이나 고객에게 시스템을 가르치는 자료
- 설계 결정의 저장소: 프로젝트의 미래를 위해 중요한 설계 결정을 architect 가 기록해 두는 곳
- 시스템 구축의 기초: 구현자에게 무엇을 구현해야 하는지 알려 준다

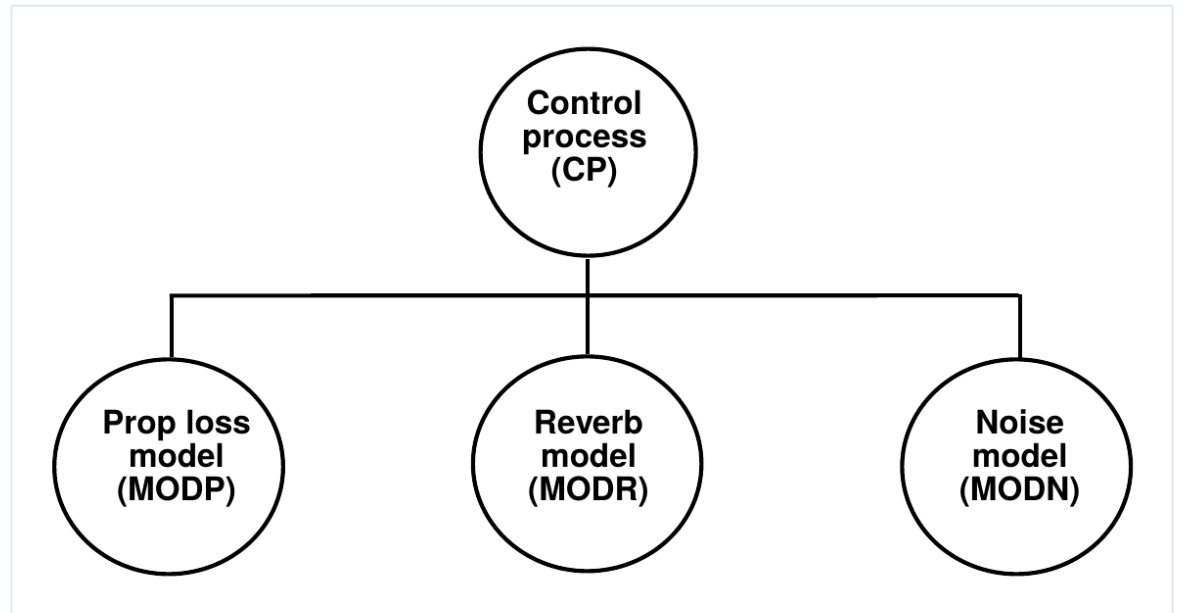
핵심 문서는 교육 · 설계 결정의 기록 · 구현의 기준, 세 가지 목적을 동시에 만족해야 한다

Architecture 를 설명하는 다이어그램 — 이 그림들은 충분한가?

- 왼쪽: Android 플랫폼 Architecture(System Apps / Java API Framework / Native Libraries · Android Runtime / HAL) — <https://developer.android.com/guide/platform>
- 오른쪽: Control process(CP)가 Prop loss / Reverb / Noise model 과 선으로 이어진 그림 — 상자와 선이 무엇을 뜻하는지 알 수 있는가?



Android 플랫폼 Architecture



전형적인 box-and-line 다이어그램

이 다이어그램의 무엇이 문제인가?

■ 모호한 box-and-line 다이어그램

- 동그라미와 사각형은 무엇을 의미하는가?
- 요소들 사이의 interface 나 상호작용은 무엇인가?
- 배치(layout)에는 어떤 의미가 있는가?

■ 근거(rationale)가 없다

- 설계 대안(design alternative)에 대한 논의와 선택 이유가 없다

■ 결론

- box-and-line 다이어그램만으로는 Architecture 가 아니다 — 그것은 출발점일 뿐이다

핵심 상자와 선만 있는 그림은 Architecture 의 시작점일 뿐, Architecture 문서가 아니다

좋은 Architecture 문서화의 기본 원칙

■ 그래픽이 의미를 효과적으로 전달한다

- 요소마다 다른 모양·색을 쓰고, 그 의미를 설명하는 범례(legend)를 둔다
- connector 의 종류를 명확히 구분한다

■ 다이어그램은 적절한 수준의 상세함을 가진다

- 각 View 는 한 페이지에 들어간다
- 필요하면 여러 View 로 분리한다
- 계층(hierarchy)을 활용한다

■ View 사이의 구분이 명확하다

- 관심사의 분리(separation of concerns)
- 한 View 에 서로 다른 관심사를 섞지 않는다

Documenting Architecture 개요

■ Architecture 문서화란

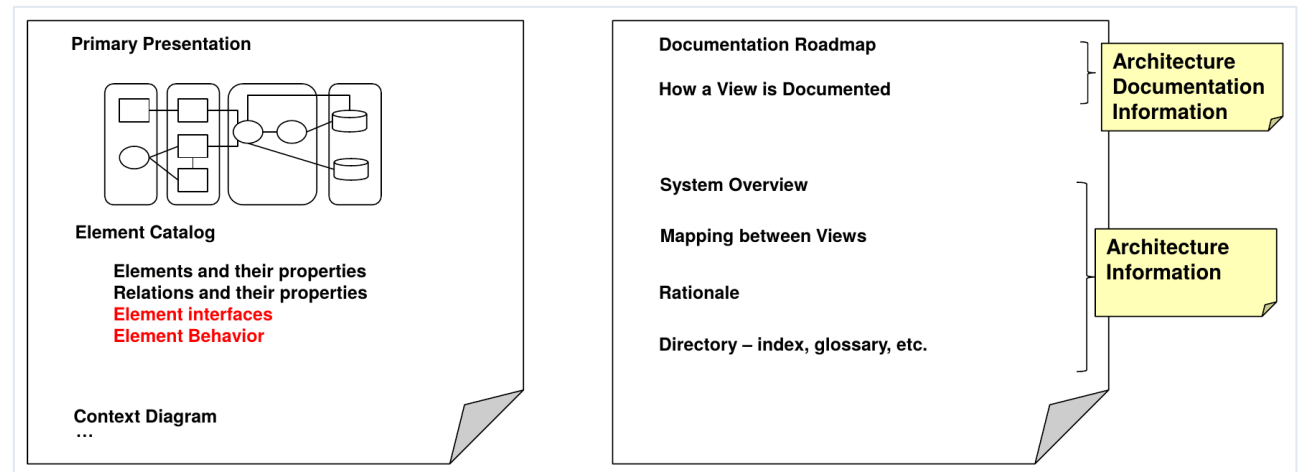
- 관련 View 들을 문서화하고, 여기에 여러 View 에 걸쳐 적용되는 문서를 덧붙이는 일이다

■ View 하나의 문서(좌)

- Primary Presentation(주 표현), Element Catalog(요소와 속성, 관계와 속성, 요소 interface, 요소 behavior), Context Diagram 등

■ View 를 넘어서는 문서(우)

- Architecture Documentation Information: Documentation Roadmap, View 문서화 방법
- Architecture Information: System Overview, View 간 매핑, Rationale, Directory(index, glossary 등)



View 문서(좌)와 View 를 넘어서는 문서(우)의 구성

핵심 Architecture 문서 = 각 View 의 문서 + View 들을 묶어 주는 공통 문서

C&C View 문서화 — 개요

■ C&C View 란

- Component 와 Connector 가 어떻게 연결되어 runtime configuration 을 이루는지 보여주는 view
- Component 의 port 와 Connector 의 role 이 서로 붙어(attachment) 시스템이 구성된다

구분	요소	설명
Elements	Component	runtime 에 존재하는 주요 처리 단위와 data store. interface 로 port 집합을 가진다
	Connector	Component 사이의 상호작용. interface 로 role 집합을 가진다
Relations	Attachment	Component 의 port 를 Connector 의 role 에 연결한다
	Interface Delegation	Component 의 port 를 내부 sub-architecture 의 port 와 연결한다

핵심 C&C View = runtime 에 존재하는 Component 와 Connector 의 연결 구조

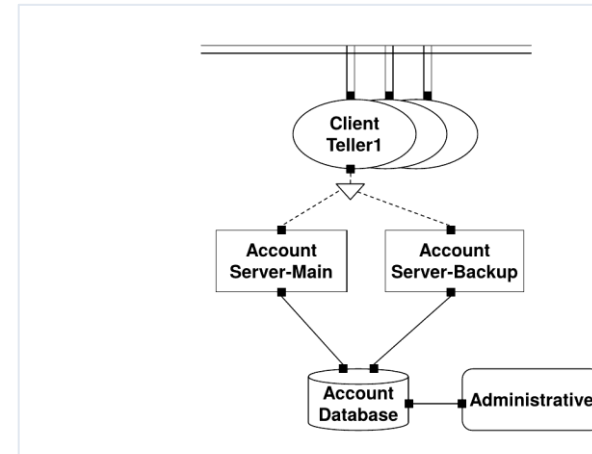
C&C View 예제 — Bird's-eye View

■ 계좌 관리 시스템의 전체 구조

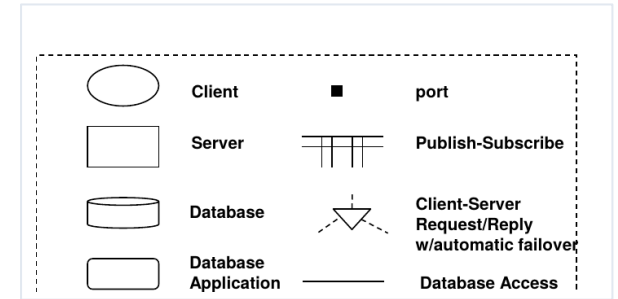
- 여러 Client Teller 가 Account Server(repository server)와 상호작용한다
- Main server 가 죽으면 Backup server 가 자동으로 넘겨받는다(failover)
- Administrative component 는 관리자가 공유 data store 를 관리하게 해 준다

■ 범례(legend)를 반드시 함께 둔다

- Client, Server, Database 등 component 종류와 Publish-Subscribe, Request/Reply 등 connector 종류를 기호로 구분한다



계좌 시스템 C&C View

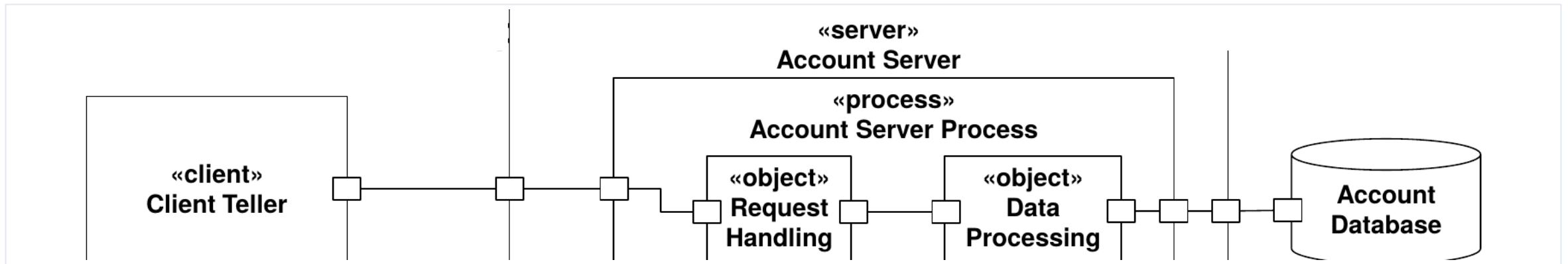


범례(Key)

C&C View 의 Component 는 무엇인가?

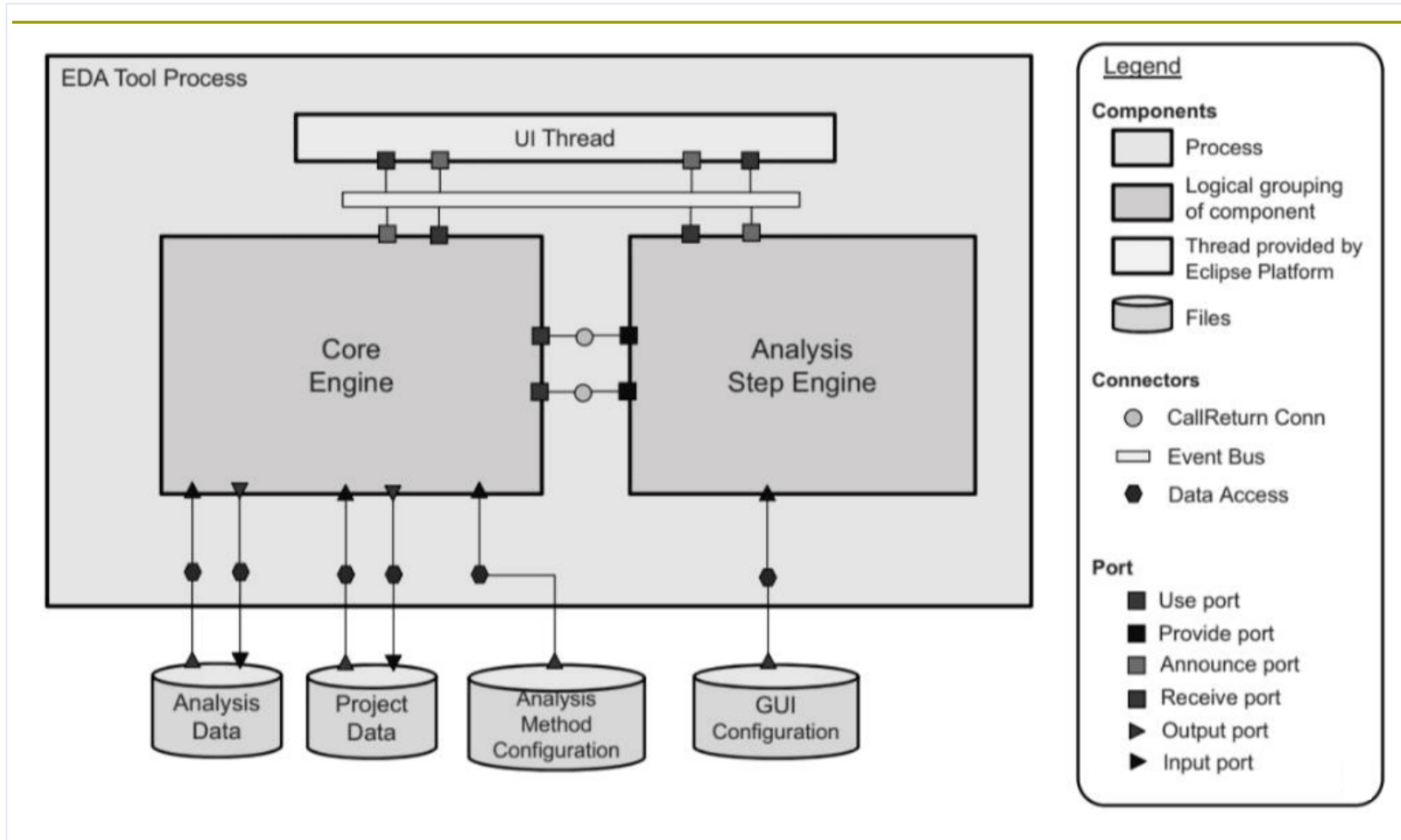
■ C&C View 의 요소는 runtime 에 실제로 존재한다

- 분산 시스템의 계산·저장 요소: client, server, data store
- 동시에 실행되는 실행 단위: process, thread
- 주요 처리 단위(논리적·개념적 관점): object
 - object 는 Module View 의 요소(class)와 혼동하기 쉬우니 주의한다



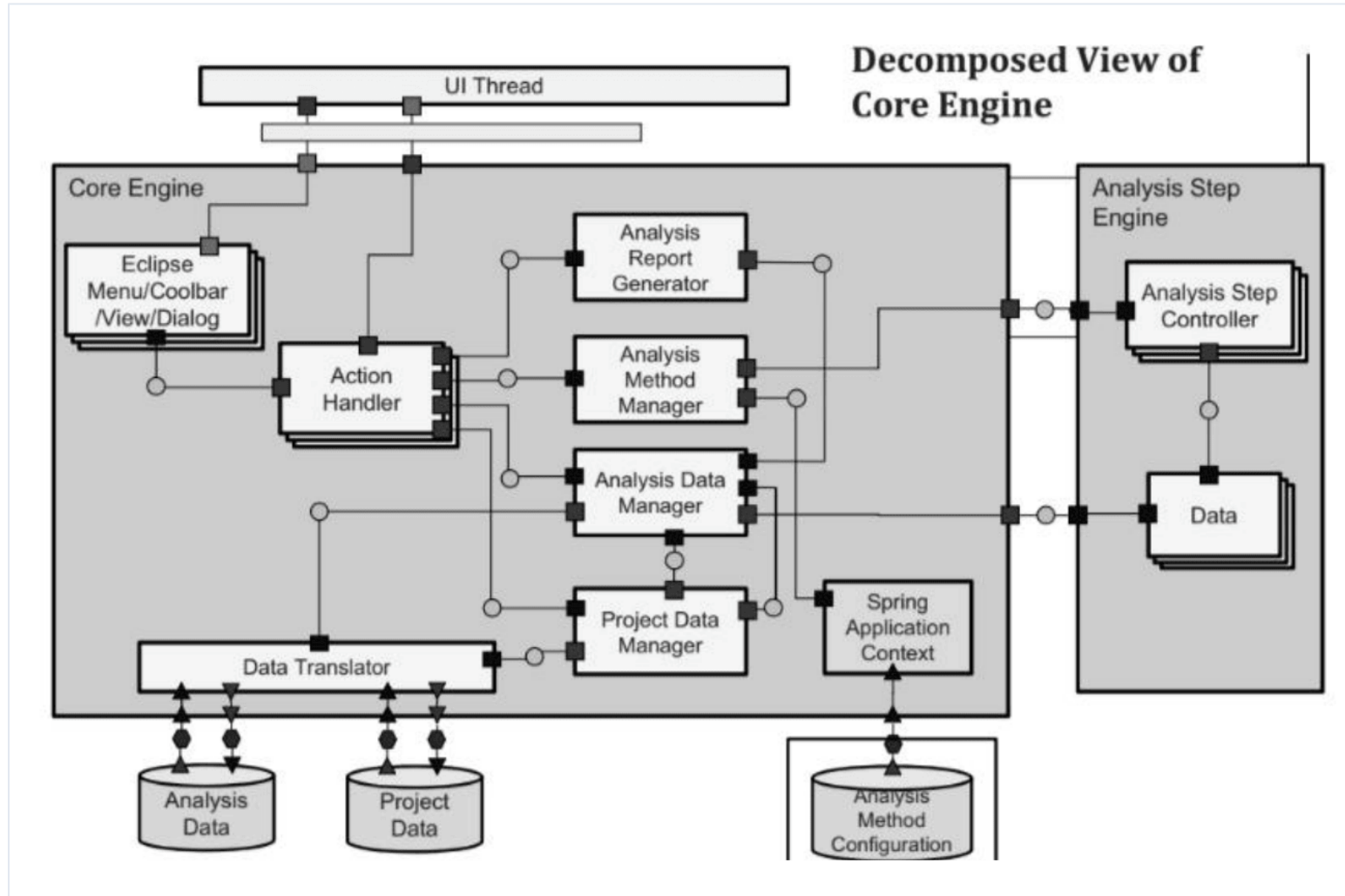
server 안에 process, 그 안에 object 가 중첩된 component 표현 (stereotype 으로 종류 구분)

High-level C&C View 예제 — EDA Tool



EDA Tool Process 의 최상위 C&C View: Core Engine 과 Analysis Step Engine 이 UI Thread, 데이터 파일과 연결된다. 오른쪽 Legend 가 process, port, connector 종류를 설명한다

Decomposed C&C View 예제 — Core Engine



앞 그림의 Core Engine 을 한 단계 분해한 C&C View: Action Handler, 여러 Manager, Data Translator 가 내부 component 이며 외부 port 로 상위 view 와 연결된다

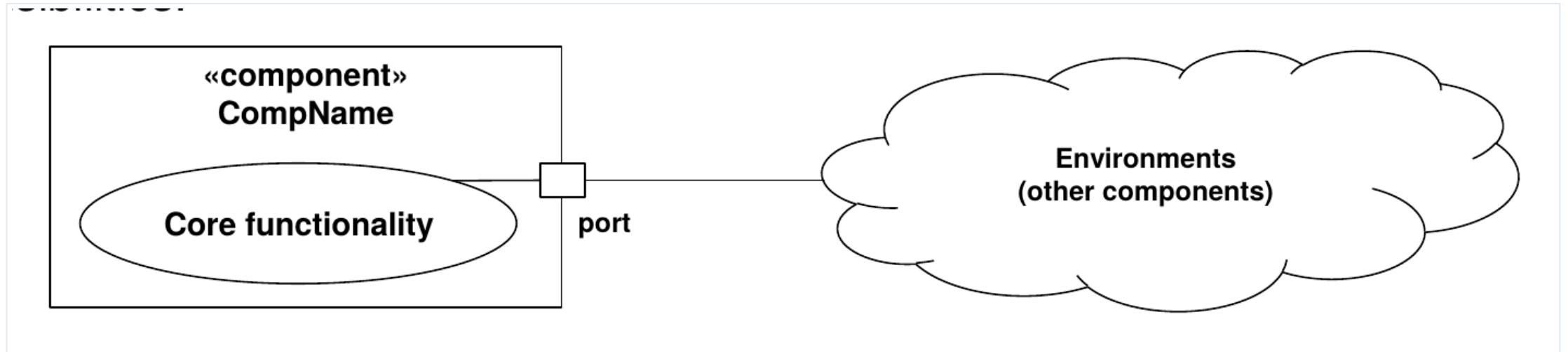
Component 정의 방법

■ Component Name

- component 가 맡은 기능(intended function)이 드러나도록 이름을 붙인다

■ Port 집합 (component 의 interface)

- port 는 component 가 환경(다른 component)과 상호작용하는 특정 지점
- component 는 핵심 기능에 집중하고, 상호작용은 port 를 통해 관리한다
 - 책임이 명확히 분리된다(core functionality vs. interaction)

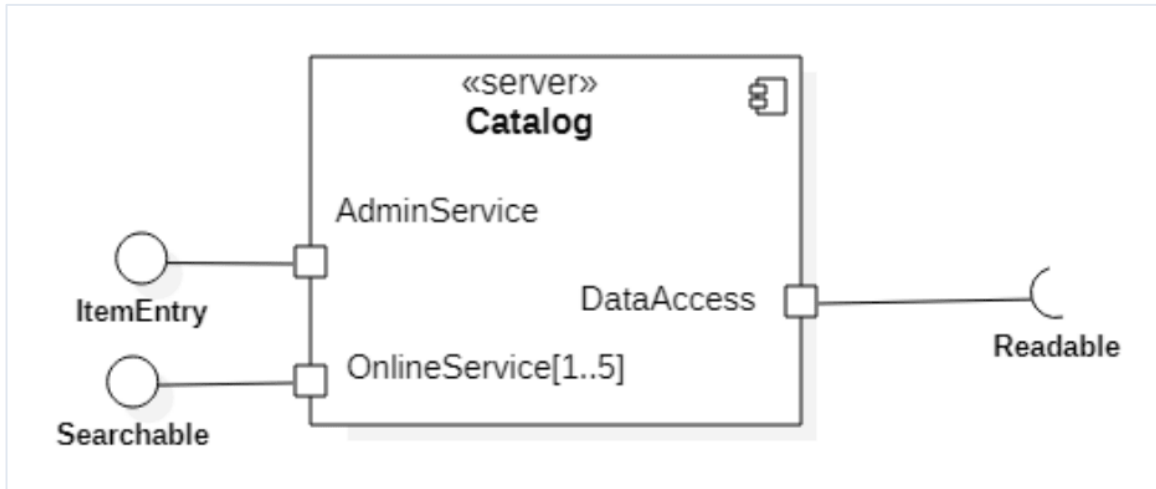


component = 핵심 기능 + port, port 를 통해 환경과 상호작용

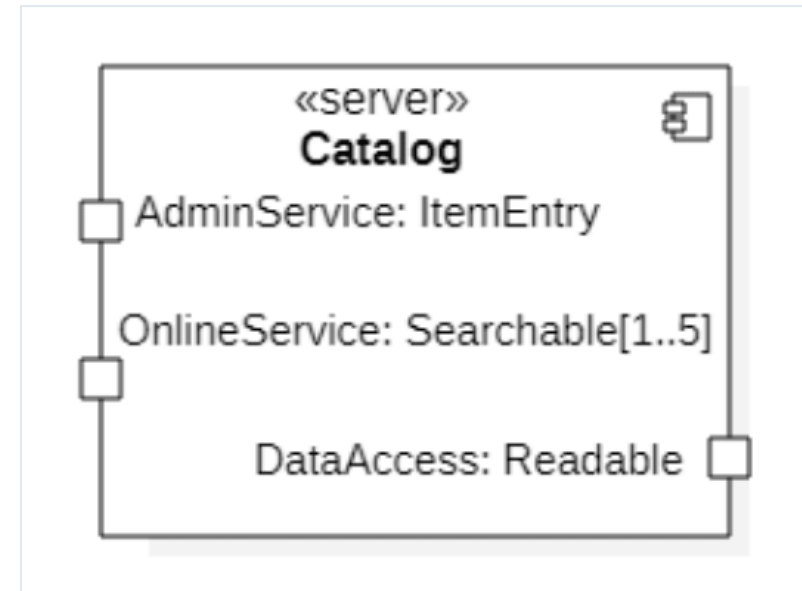
Component Type 표기법 (UML Component)

■ UML component 로 component type 을 표현한다

- port 를 작은 사각형으로 표시하고, 이름과 다중도(multiplicity)를 붙일 수 있다 (예: OnlineService[1..5])
- required/provided service(interface)를 lollipop-socket 기호로 표시할 수 있다
- stereotype(«server» 등)으로 component type 을 구분한다



port 와 provided/required interface 표시



port 이름: interface 형태로 표시

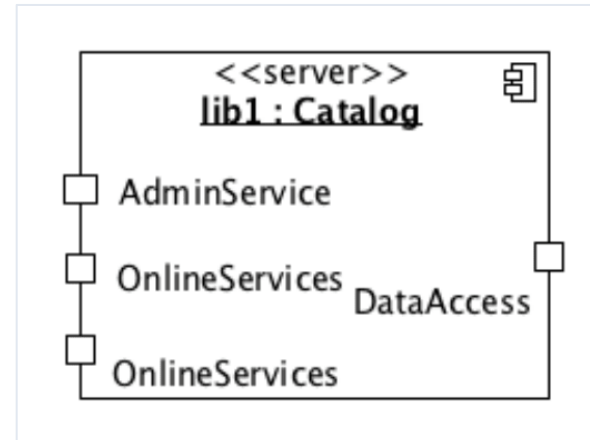
Component Instance 표기법

■ UML component instance

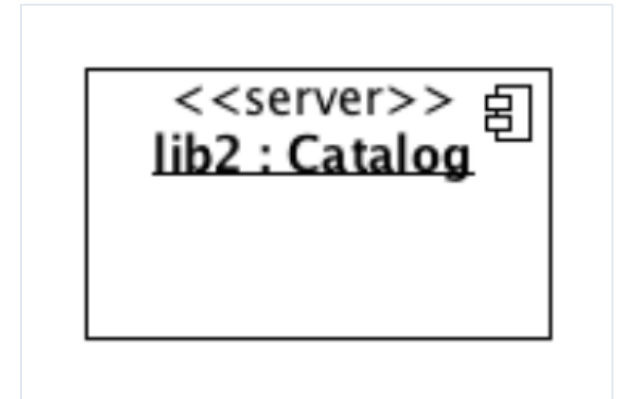
- 이름에 ':' 을 넣어 type 과 구분한다
 - 이름 있는 instance: "lib1:Catalog"
 - 익명 instance: ":Catalog"

■ port 표시

- type 에 정의된 port 와 일관되게 표시한다
- 같은 type 의 port 가 여러 개 있으면 각각 그린다
(예: OnlineServices 두 개)



lib1 : Catalog (named instance)

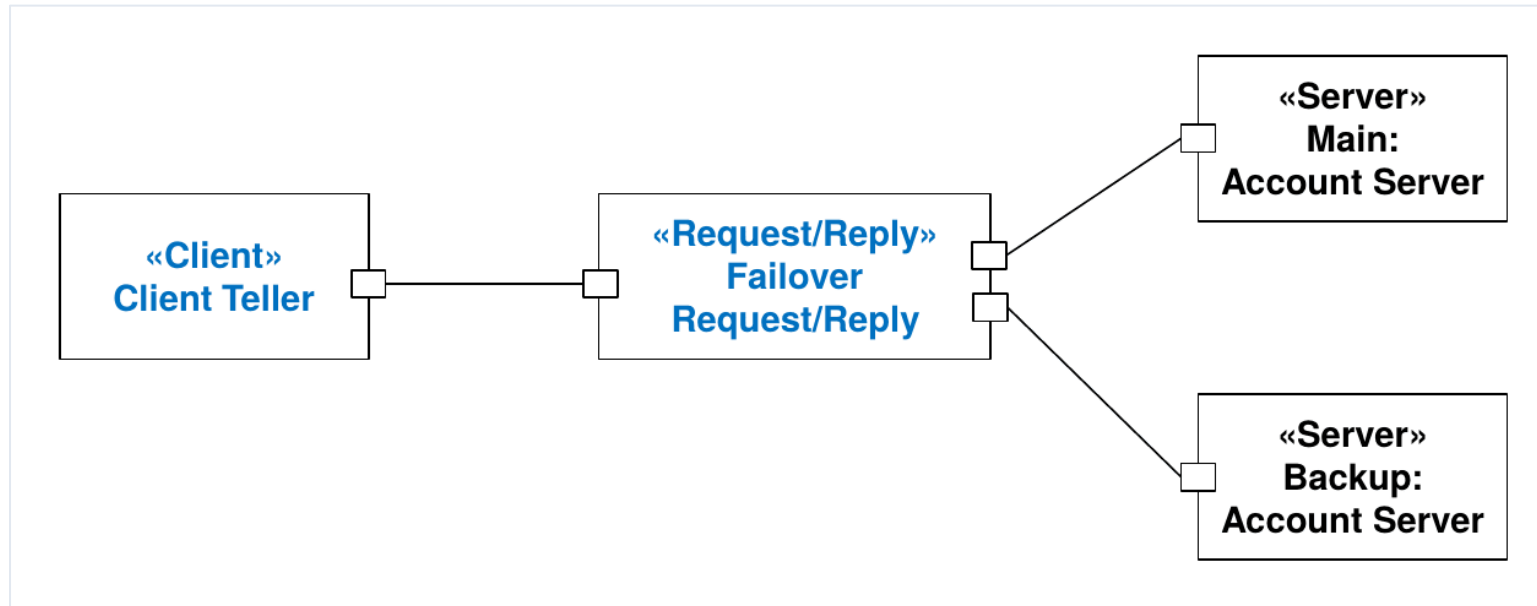


lib2 : Catalog

Component 모델링 가이드라인 (1)

■ Guideline 1: type 과 instance 를 한 다이어그램에 섞지 않는다

- 아래 그림은 잘못된 예: 'Client Teller' 는 type 인데 'Main: Account Server', 'Backup: Account Server' 는 instance 이다
- type 다이어그램과 instance 다이어그램을 분리해 그려야 한다



type(Client Teller)과 instance(Main:, Backup:)가 섞인 잘못된 예

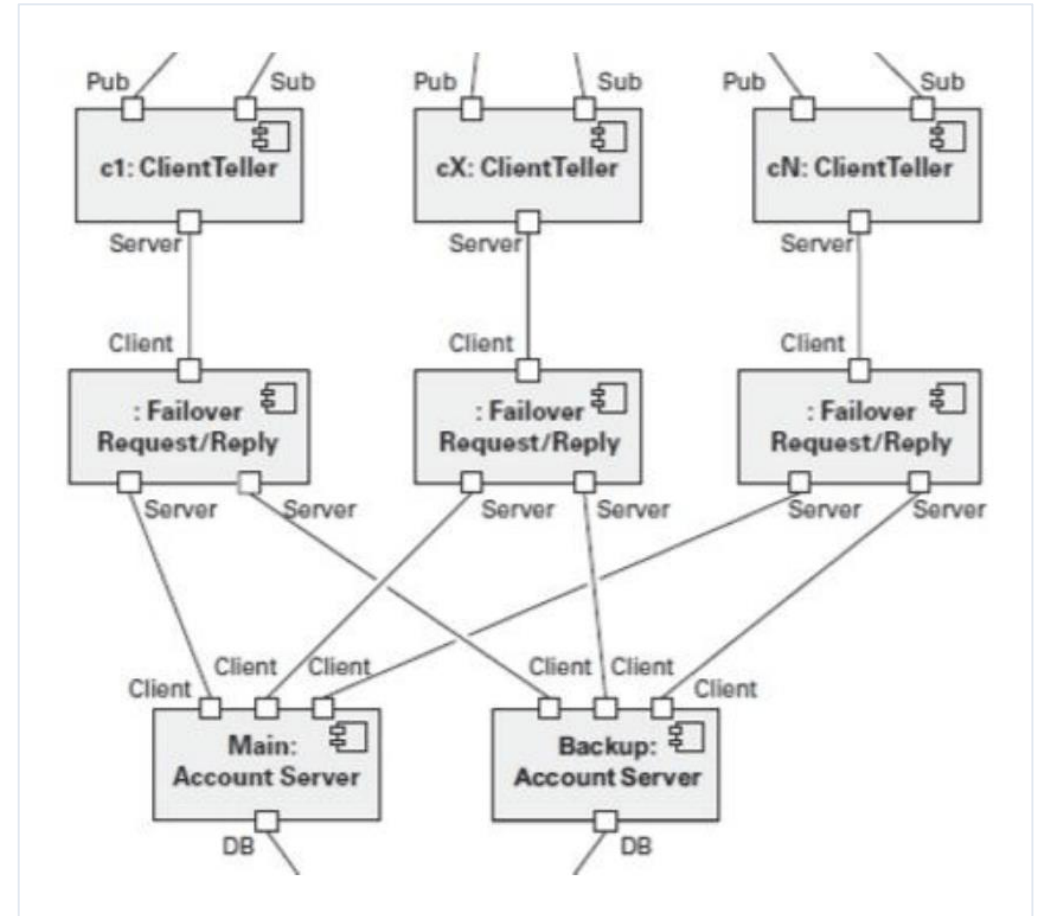
Component 모델링 가이드라인 (2)

■ Guideline 2: component 에 이름 있는 port 를 사용한다

- 어느 port 가 어느 것인지 명백한 경우가 아니면 port 이름을 붙인다

■ port 이름 규칙

- port 이름은 그 component 와 상호작용하는 환경 요소의 역할(role)로 붙일 수 있다
 - 예: ClientTeller 의 port 이름은 Pub, Sub, Server / Failover Request/Reply 의 port 는 Client, Server

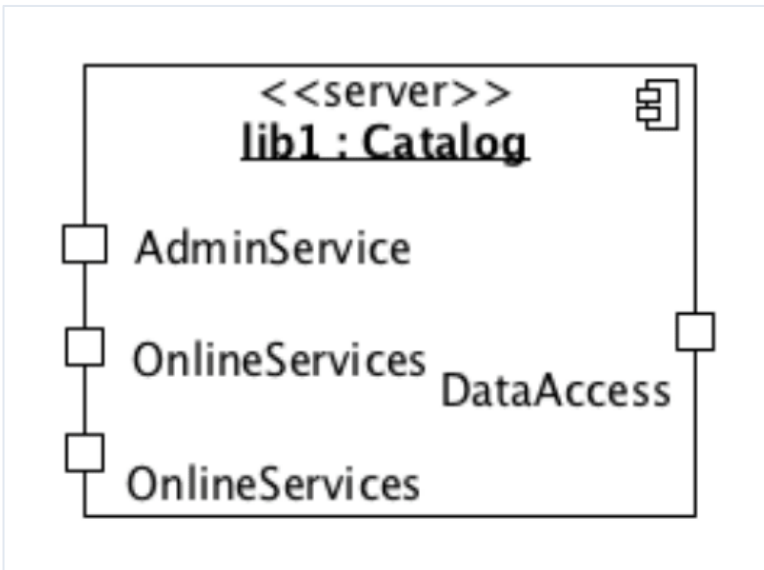


named port 를 사용한 instance 다이어그램

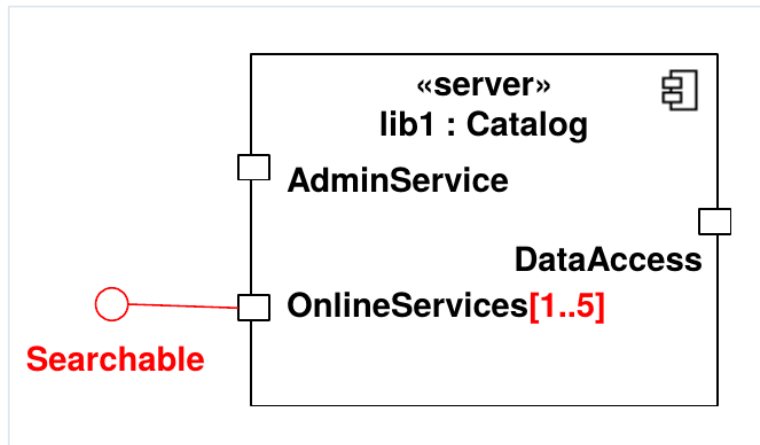
Component 모델링 가이드라인 (3)

■ Guideline 3: instance 에는 interface 를 그리지 않는다

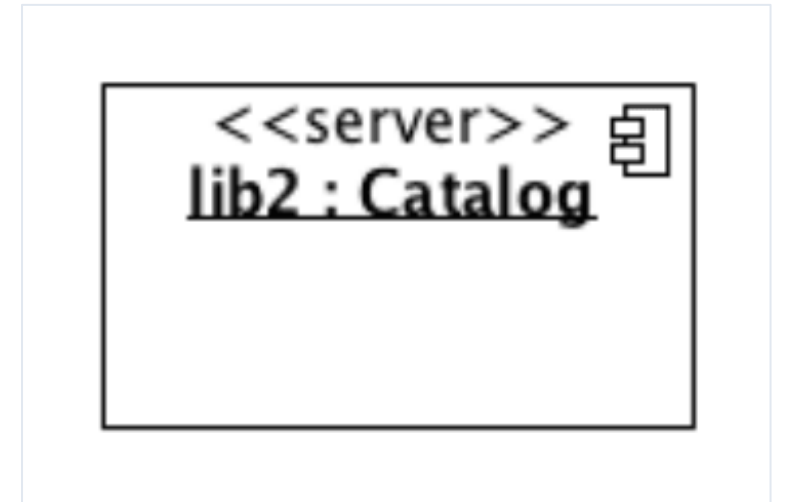
- provides/requires lollipop 은 component type 에만 사용한다
- component instance 의 port 에는 multiplicity 를 붙이지 않는다
 - 아래 가운데 그림의 'Searchable' lollipop 과 '[1..5]' 는 instance 에서 잘못된 표기



올바른 instance 표기



잘못된 예: interface 와 multiplicity

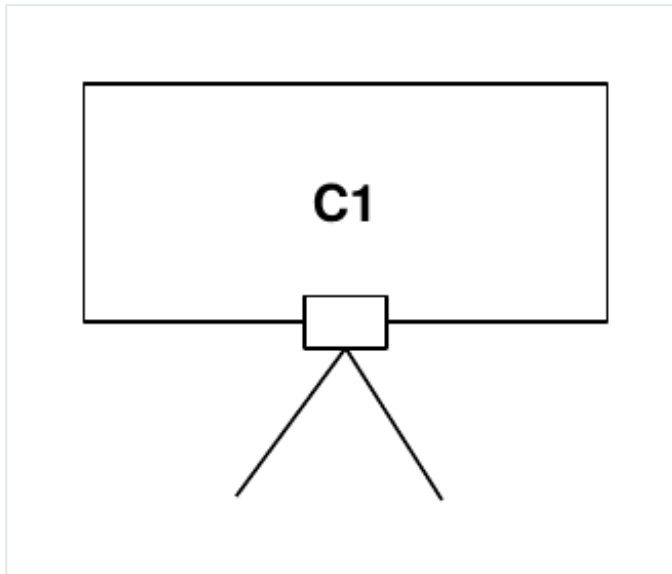


올바른 instance 표기

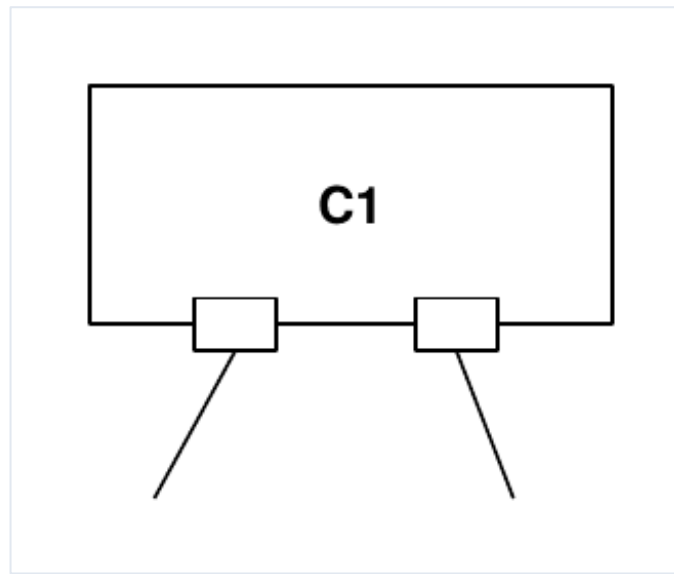
Component 모델링 가이드라인 (4)

■ 여러 component 가 하나의 component(C1)와 상호작용하는 세 가지 방법

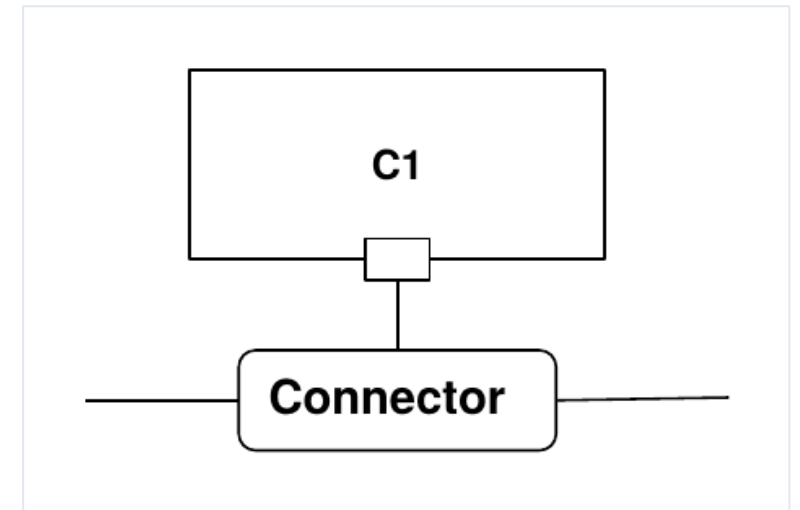
- 1. C1 이 하나의 interface(port)를 두고, 모든 환경 요소가 그 port 를 통해 상호작용한다
- 2. C1 이 환경 요소마다 별도의 interface(port)를 둔다
- 3. C1 은 port 하나만 두고, 그 port 에 붙은 Connector 가 환경과의 중재(mediation)를 담당한다



1. 단일 interface



2. 환경별 interface



3. Connector 가 중재

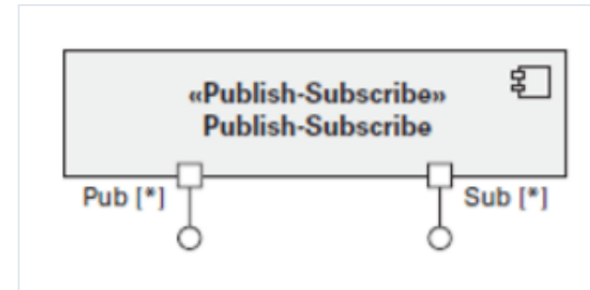
Connector 정의 방법

■ Connector Name

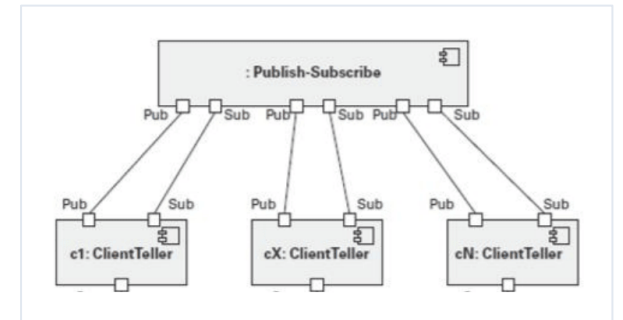
- 상호작용의 성격이 드러나도록 붙인다
 - 예: service invocation, event multicast, transaction 지향 통신 채널

■ Role 집합 (connector 의 interface)

- role 은 component 가 상호작용을 위해 connector 를 어떻게 사용하는지 정의한다
- role 은 이름과 multiplicity 를 가질 수 있다
 - 예: Publish-Subscribe connector 의 Pub[*], Sub[*] role



Publish-Subscribe connector type

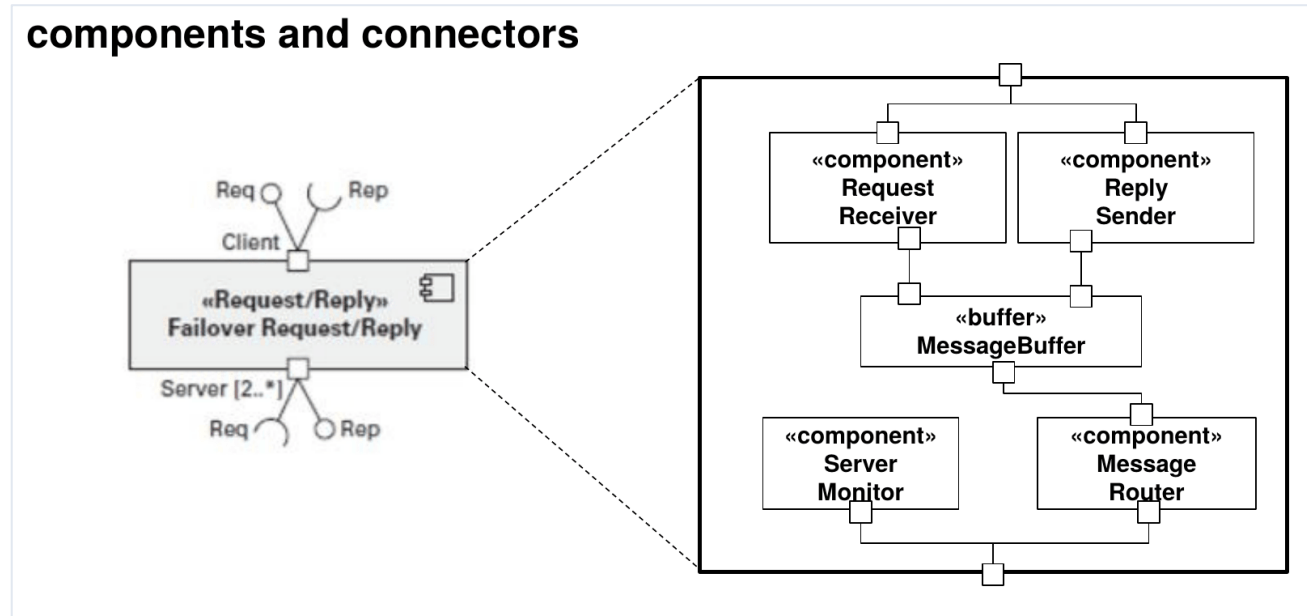


connector instance 와 Pub/Sub role

복잡한 Connector 의 분해

■ Connector 도 내부 구조를 가질 수 있다

- 복잡한 connector 는 여러 component 와 connector 의 집합으로 분해해 표현할 수 있다
 - 예: Failover Request/Reply connector = Request Receiver, Reply Sender, Message Buffer, Server Monitor, Message Router 로 구성



«Request/Reply» Failover Request/Reply connector 의 내부 구조

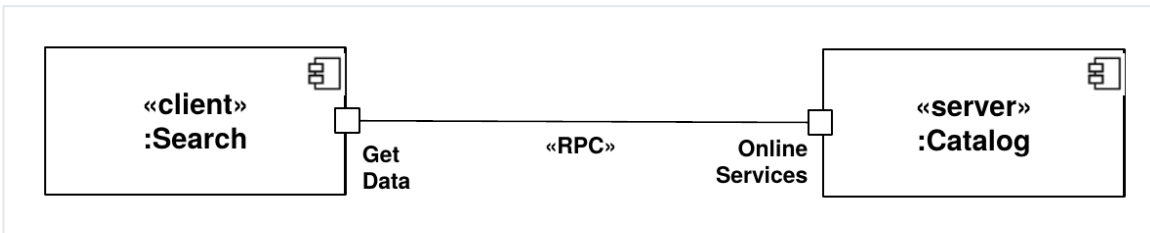
Connector 표기법 (1): UML Connector / UML Component

■ UML connector 로 표현

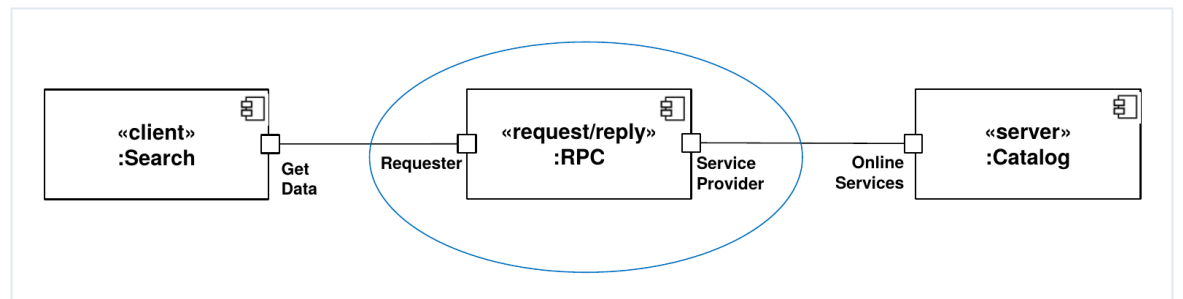
- 두 component 의 port 를 잇는 단순한 선. stereotype(«RPC»)으로 connector type 을 나타낸다
- UML connector 는 내부 구조·속성·행위 설명을 가질 수 없다
- role 은 선의 양 끝에 라벨을 붙여 표시한다

■ UML component 로 표현

- publish/subscribe 처럼 connector 가 복잡할 때 유용하다
- role(Requester, Service Provider)을 명시적으로 나타낼 수 있다



UML connector (선 + stereotype)



UML component 로 표현한 «request/reply» :RPC connector

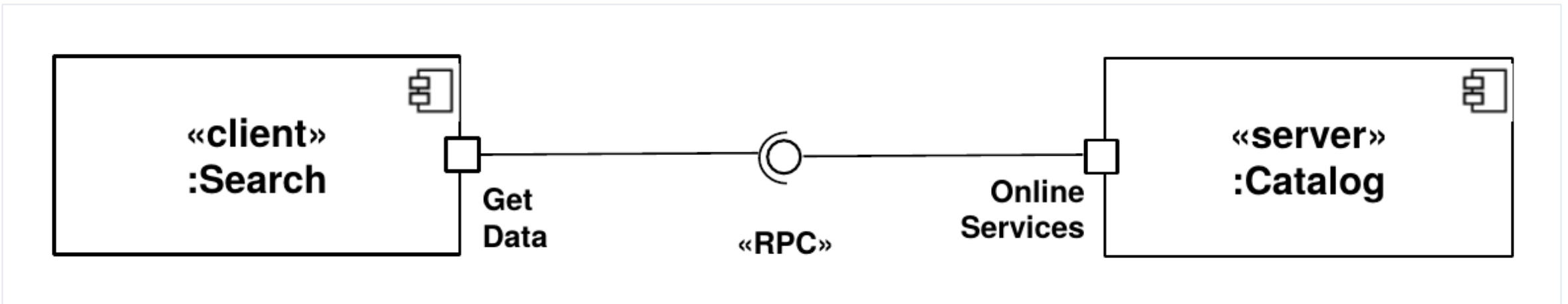
Connector 표기법 (2): Assembly Connector

■ Assembly connector 는 call-return connector 에만 사용한다

- 한 port 의 required interface 가 다른 port 의 provided interface 와 호환될 때 두 port 를 연결한다 (socket-lollipop 결합)

■ provided/required 직관과 맞지 않는 connector 에는 부적합하다

- 예: pipe-and-filter 의 filter 입력 port 는 provided 인지 required 인지 명확히 대응되지 않는다

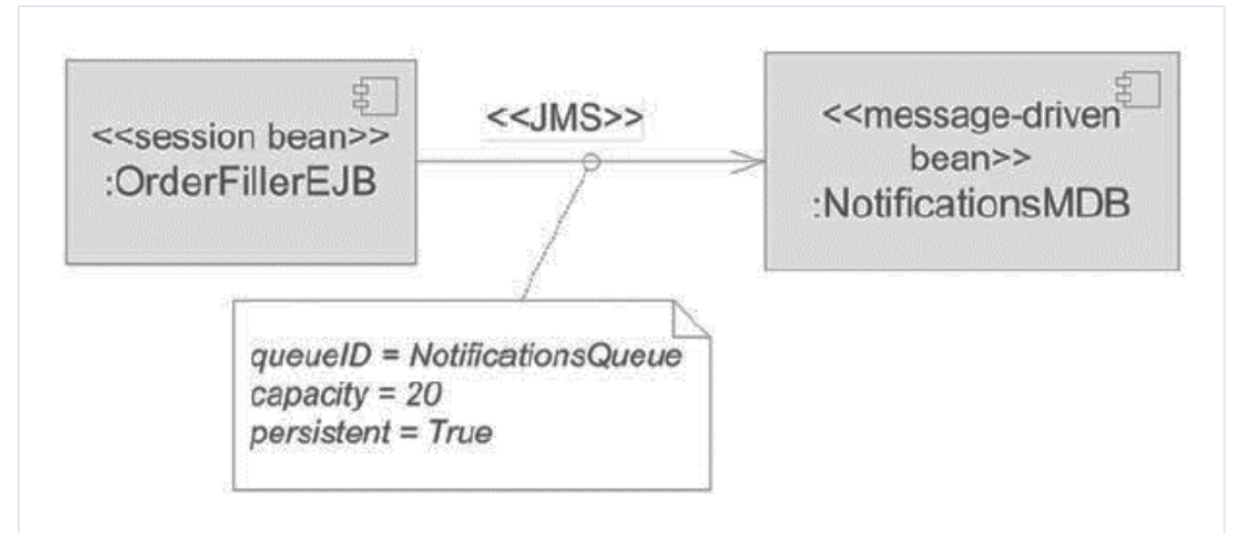


assembly connector 로 표현한 «RPC» 연결

Connector 표기법 (3): Tagged Values

■ Tagged value 로 정보를 추가한다

- connector 에 더 많은 정보를 덧붙일 수 있는 방법
- connector type 에 대한 stereotype 을 만들고, 그 stereotype 에 속성(tagged value)을 정의한다
 - 예: «JMS» connector 의 queueID, capacity, persistent 속성

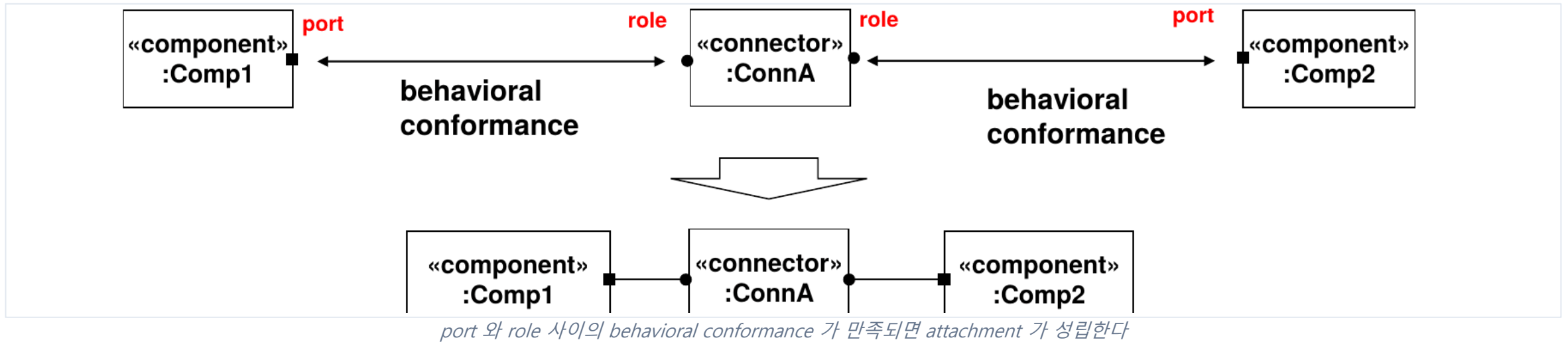


«JMS» connector 의 tagged values

Attachment Relation (연결 관계)

■ 어떤 connector 가 어떤 component 에 붙는지 나타낸다

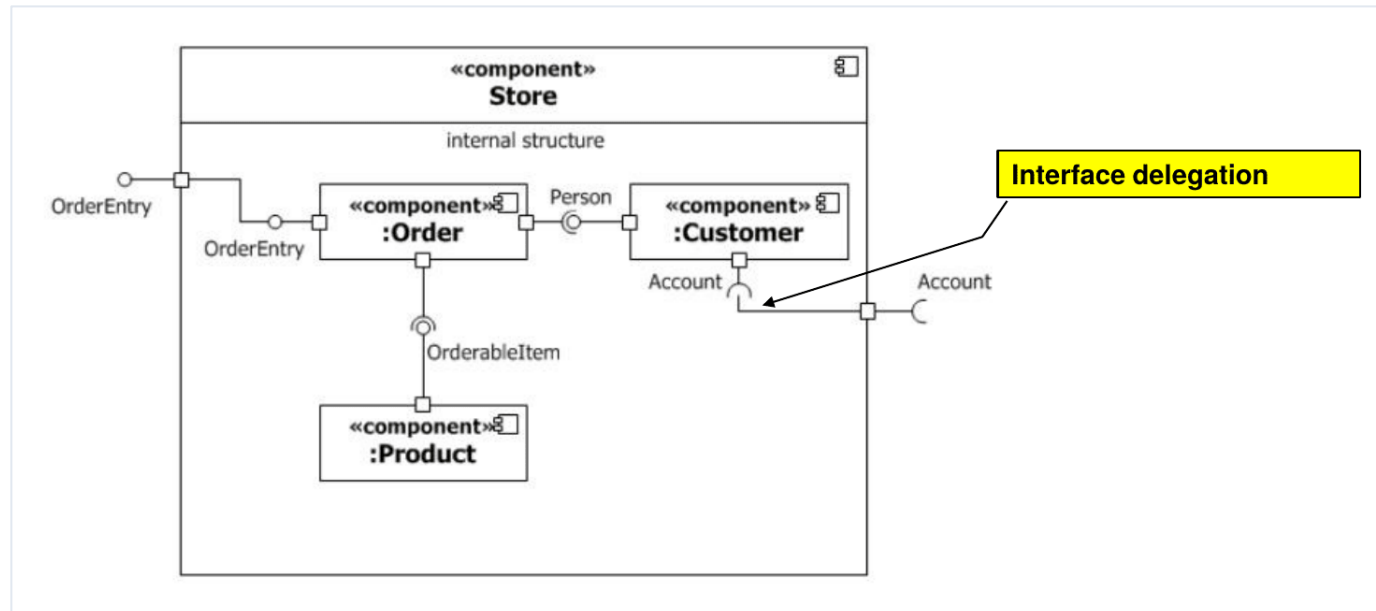
- attachment 로 시스템이 component 와 connector 의 그래프로 정의된다
- component 의 port 가 connector 의 role 에 붙는다(attach)
 - 조건: port 의 protocol 이 그 role 이 기대하는 행위(behavior)와 일치해야 한다 (behavioral conformance)



Interface Delegation Relation (위임 관계)

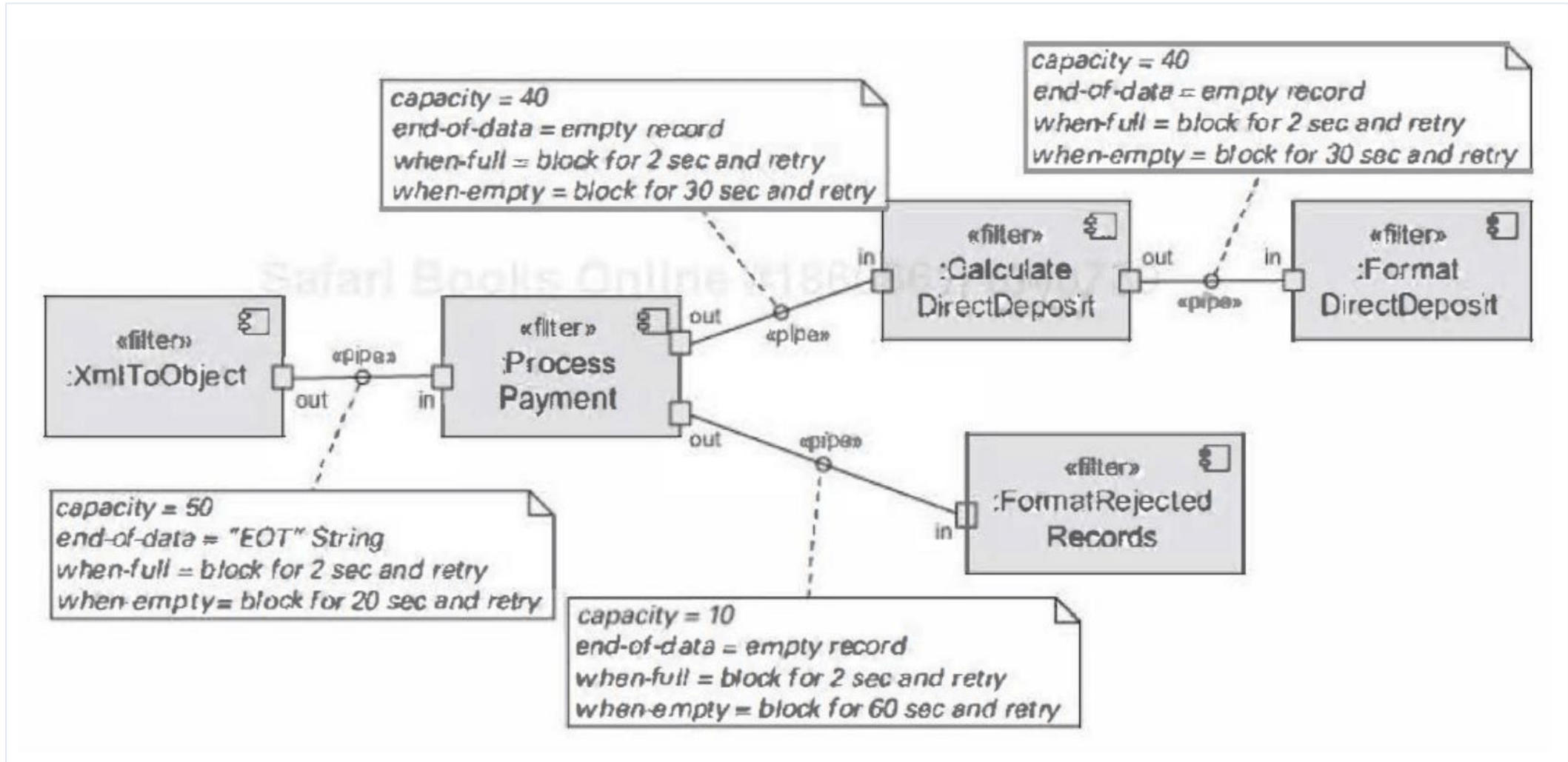
■ 내부 port/role 을 외부 port/role 과 연결한다

- component 나 connector 를 분해했을 때, 바깥쪽 port 가 안쪽 sub-architecture 의 어느 port 로 이어지는지 나타낸다
 - 예: Store component 의 외부 port OrderEntry, Account 가 내부 :Order, :Customer component 의 port 로 위임된다



UML delegation connector 로 표현한 interface delegation

C&C Style 표기 예 (1): Data Flow Style

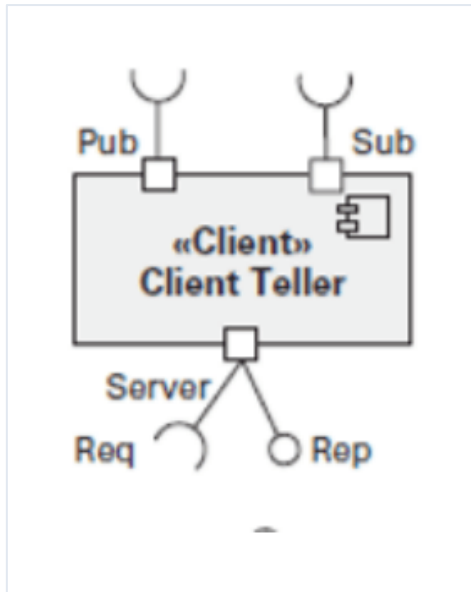


Pipe and Filter 스타일의 C&C View: «filter» component 들이 «pipe» connector 로 연결되고, 각 pipe 의 capacity, end-of-data, when-full/when-empty 속성이 note 로 기술된다

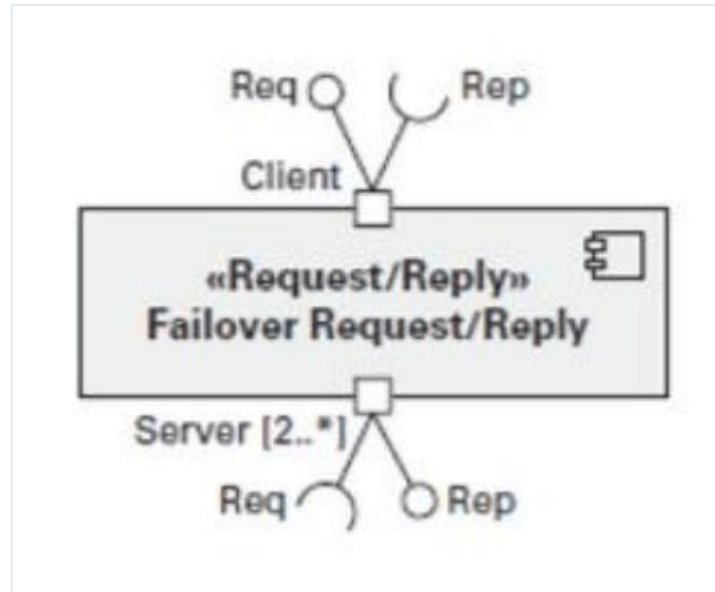
C&C Style 표기 예 (2): Call-Return Style — Type

■ Call-Return 스타일의 component type 과 connector type

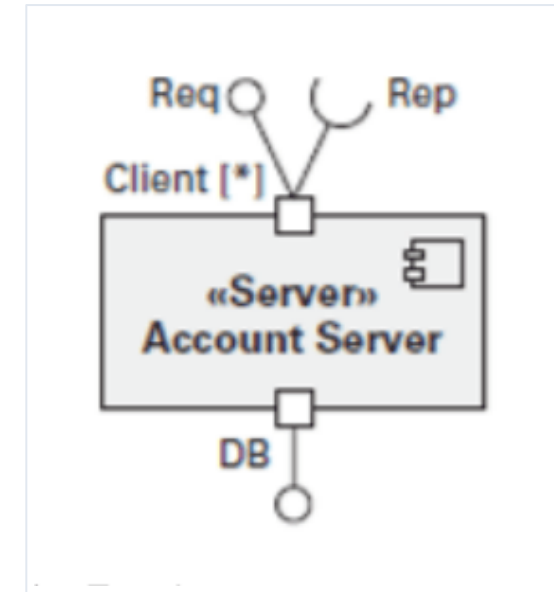
- «Client» Client Teller: Pub/Sub port 와 Server port(Req 제공, Rep 요구)
- «Request/Reply» Failover Request/Reply connector: Client role 과 Server[2..*] role
- «Server» Account Server: Client[*] port 와 DB port



Component type: Client

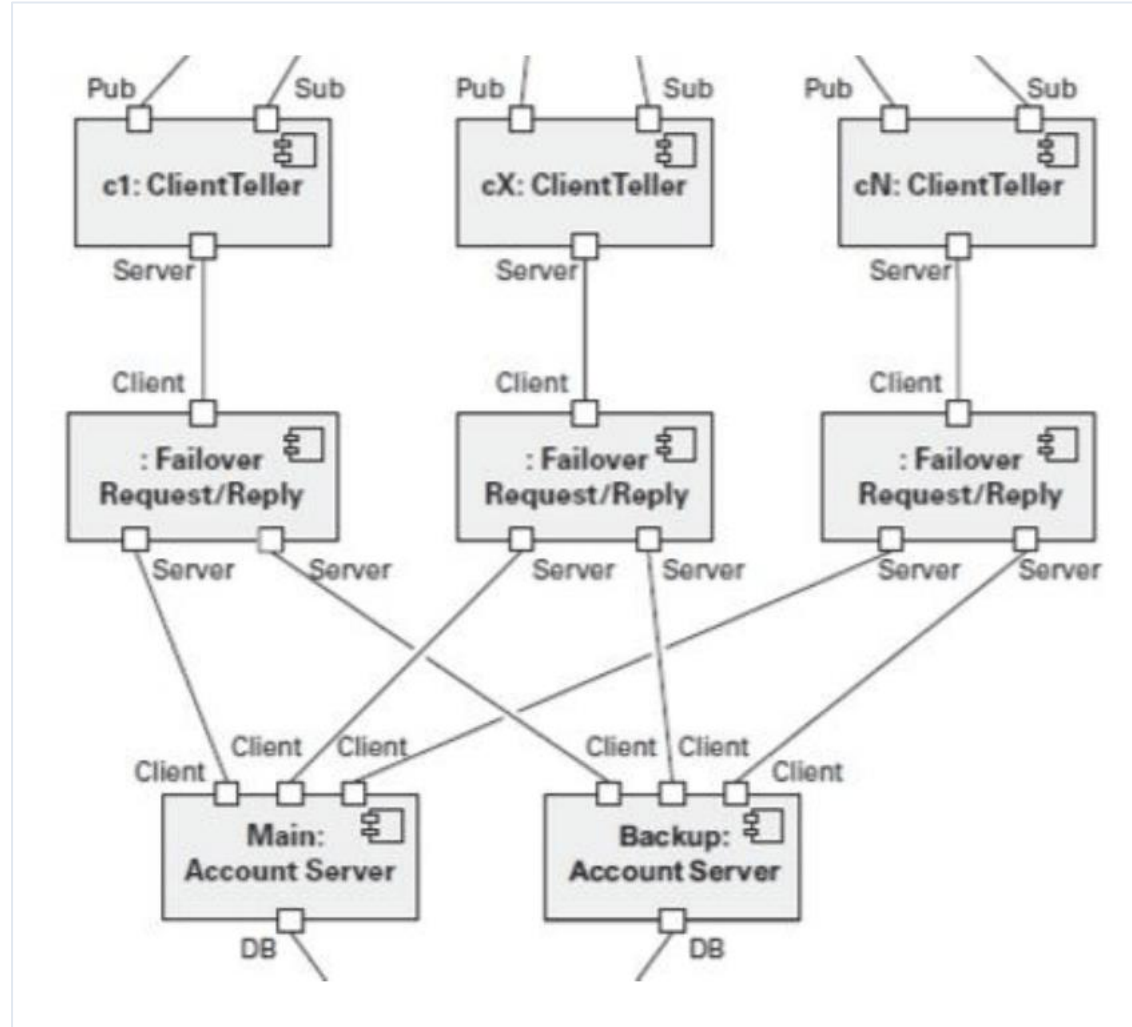


Connector type



Component type: Server

C&C Style 표기 예 (2): Call-Return Style — Architecture

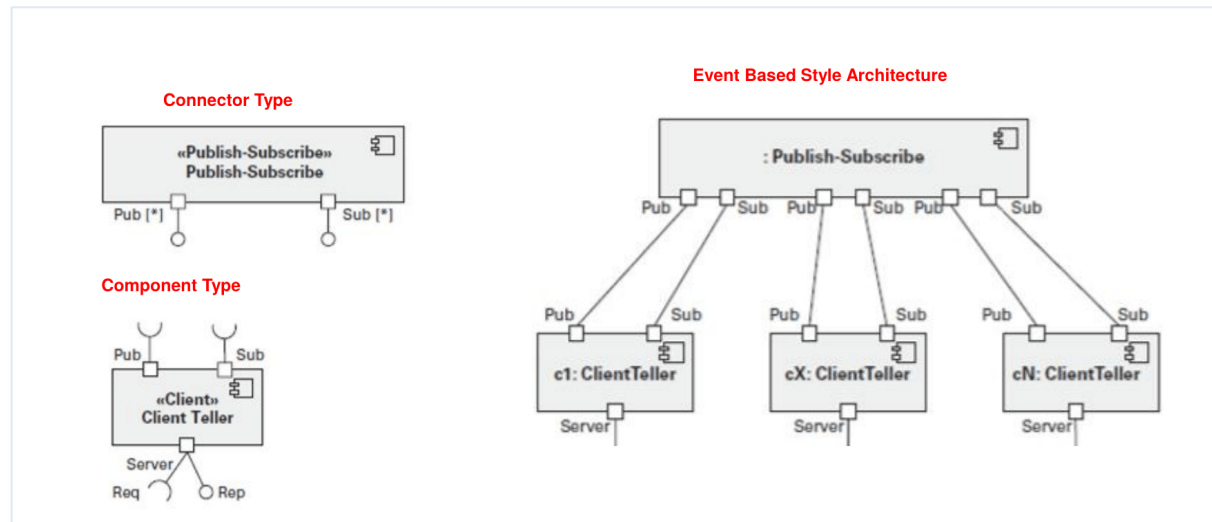


Call-Return 스타일 Architecture: ClientTeller instance 들이 :Failover Request/Reply connector 를 통해 Main/Backup Account Server 와 연결된다

C&C Style 표기 예 (3): Event Based Style

■ Publish-Subscribe 기반 표기

- Connector type: «Publish-Subscribe» connector 는 Pub[*], Sub[*] role 을 가진다
- Component type: «Client» Client Teller 는 Pub, Sub port 로 event 를 주고받는다
- Architecture: 하나의 :Publish-Subscribe connector 에 여러 ClientTeller instance 가 Pub/Sub role 로 붙는다

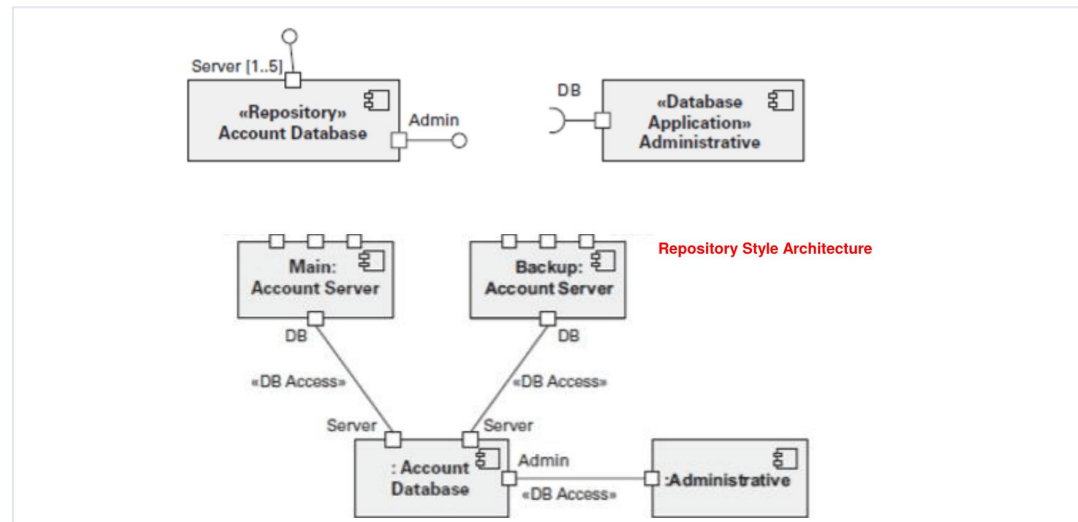


Event Based 스타일의 connector type, component type, Architecture

C&C Style 표기 예 (4): Repository Style

■ 공유 데이터 저장소 중심 표기

- Component type: «Repository» Account Database 는 Server[1..5] port 와 Admin port 를 가진다
- «Database Application» Administrative 는 DB port 로 repository 에 접근한다
- Architecture: Main/Backup Account Server 와 :Administrative 가 «DB Access» connector 로 :Account Database 에 연결된다



Repository 스타일의 component type 과 Architecture

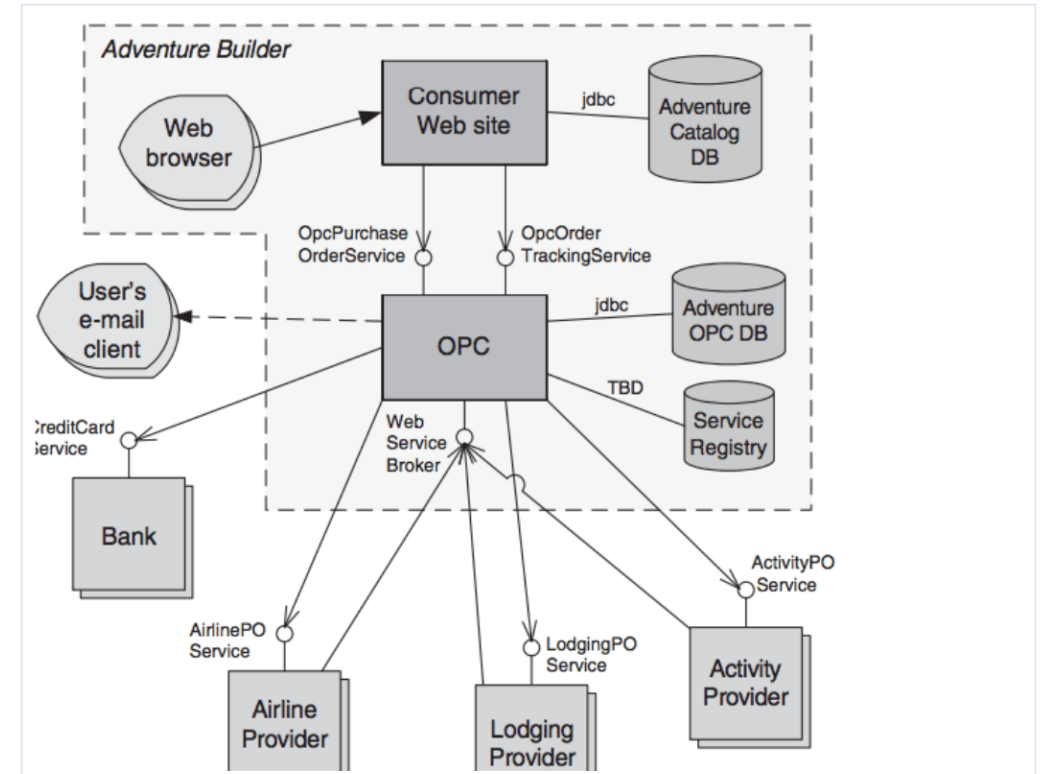
Adventure Builder 예제 — 시스템 개요

■ Adventure Builder 시스템

- 휴가 상품을 구성하기 위해 외부 service provider(항공, 숙박, 액티비티)와 상호작용한다
- 결제 처리를 위해 은행 서비스(Bank)와 상호작용한다
- 중앙의 OPC(Order Processing Center) component 가 내부·외부 service consumer 와 provider 사이의 상호작용을 조정한다

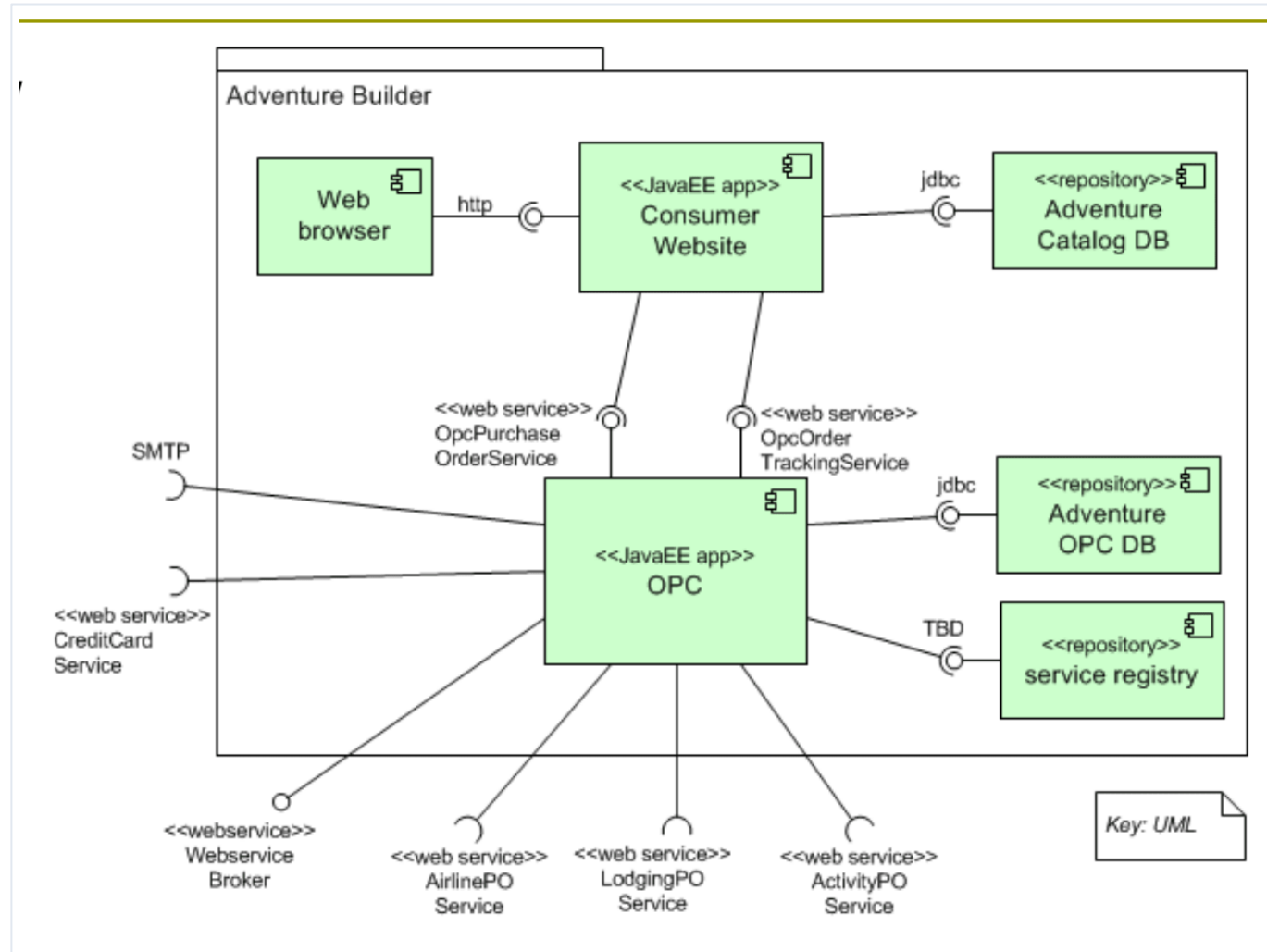
■ 범례(Key)

- Client-side application, Java EE application, External Web service provider, Data repository 와 HTTP/SOAP/SMTP/Data access connector 를 구분한다



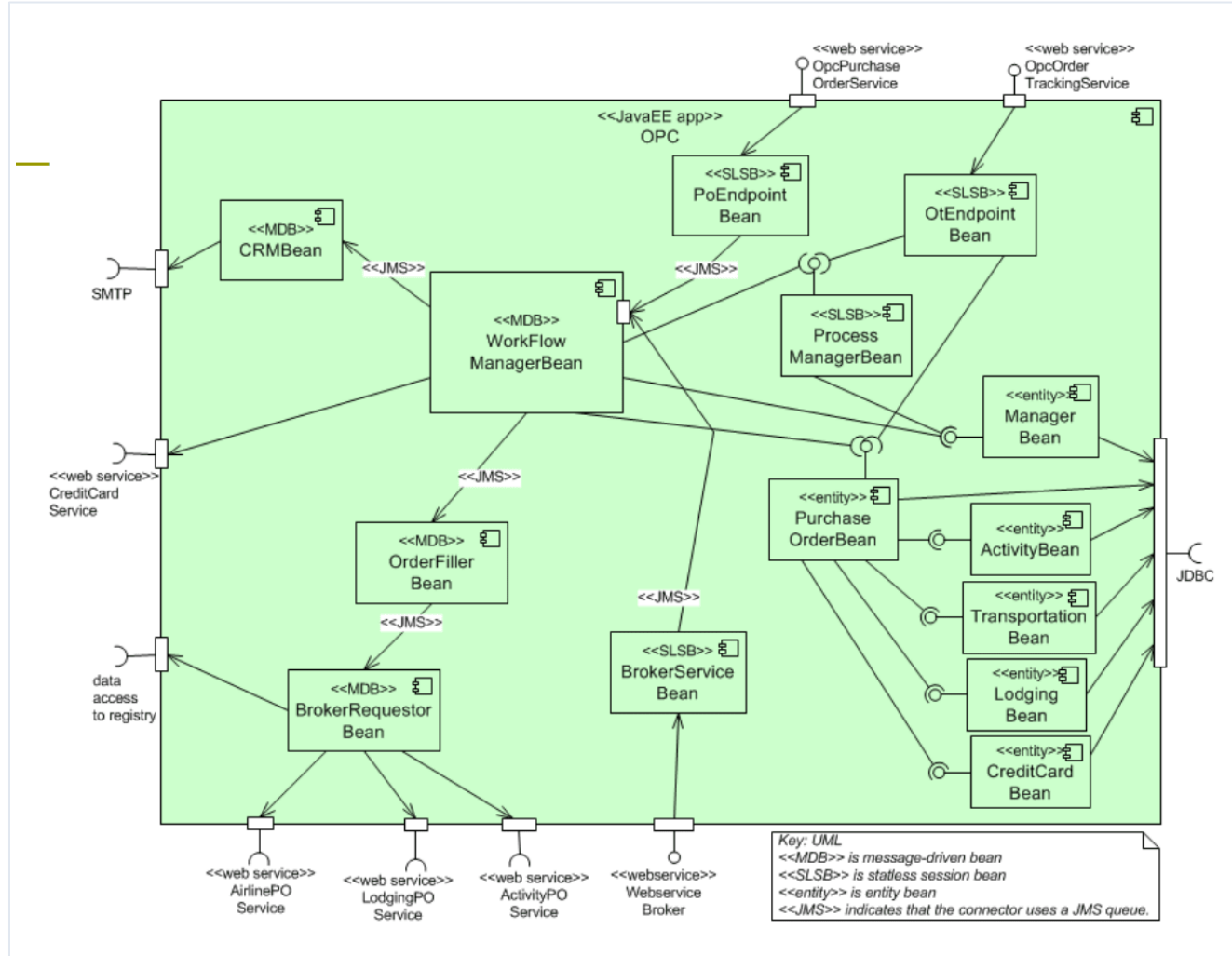
Adventure Builder 의 개략 C&C View (Bass et al.)

Adventure Builder 예제 — Top Level View



Top Level SOA View (UML): Web browser → «JavaEE app» Consumer Website → «JavaEE app» OPC, 그리고 OPC 가 «repository» DB 들과 «web service» 들(AirlinePO, LodgingPO, ActivityPO, CreditCard)에 연결된다

Adventure Builder 예제 — OPC C&C View



OPC 내부의 C&C View: «MDB»(message-driven bean), «SLSB»(stateless session bean), «entity» bean 들이 «JMS» connector 와 web service 로 연결된다. 오른쪽 아래 Key 로 stereotype 을 설명한다

C&C View 문서화 요약

■ C&C View 의 요소는 runtime 에 존재한다

- 분산 시스템의 처리·저장 요소: client, server, data store
- 동시 실행 단위: process, thread
- 주요 처리 단위(논리·개념적): object

■ C&C Style 의 분류

- Data flow style: Pipe and Filter
- Call-return style: Client-Server, Multi-tier, SOA
- Event-based style: Publish-Subscribe
- Repository style

C&C View 요약 — UML 로 표현하기

C&C 개념	UML 표기
Component type	UML component
Component instance	UML component instance (':' 로 구분)
Port	UML port
Connector	UML connector. 대안: UML component, UML assembly connector, tagged association
Attachment 관계	UML component 와 UML connector 사이의 연결
Interface delegation 관계	UML delegation connector

Module View 문서화 — 개요

■ Module View 란

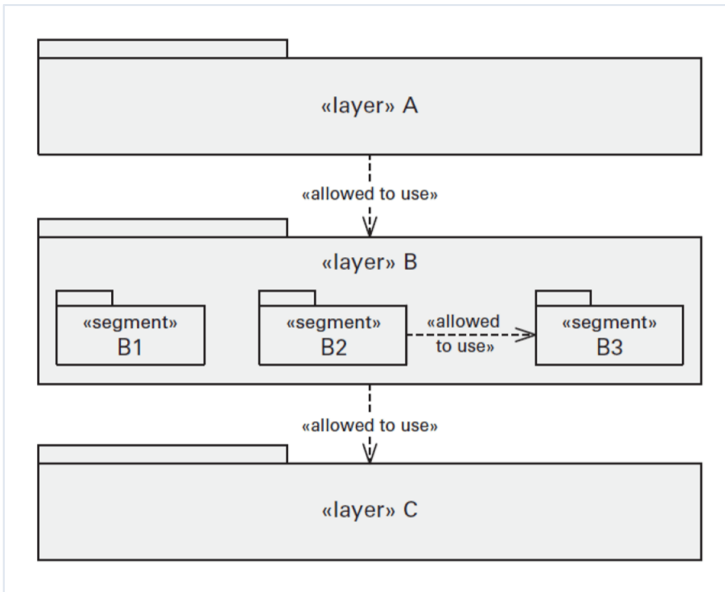
- 시스템을 이루는 module 의 집합과 그 사이의 관계를 기술하는 view
- 구축(construction), 분석(analysis), 의사소통(communication)에 사용한다

구분	요소	설명
Elements	Module	일관된 책임(responsibility) 집합을 제공하는 소프트웨어 구현 단위
Relations	Is part of	submodule(부분)과 aggregate module(전체) 사이의 part/whole 관계
	Depends on	여러 형태의 의존 관계. 예: uses, allowed to use, crosscuts, data entity 관계
	Is a	일반화/특수화(generalization/specialization) 관계

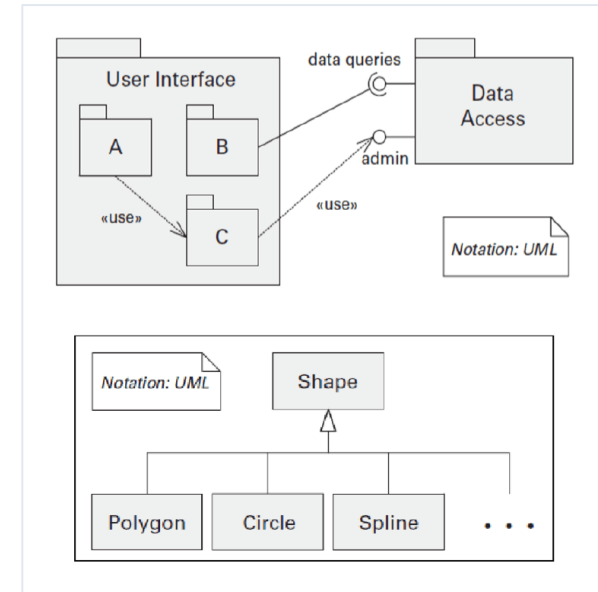
Module View 예제

■ 같은 시스템도 관계에 따라 여러 Module View 로 표현한다

- Layered Structure: «layer» A, B, C 와 «allowed to use» 관계, layer 안의 «segment»
- Uses Structure: User Interface 의 A, B, C 가 «use» 관계로 Data Access 에 의존
- Generalization Structure: Shape 를 Polygon, Circle, Spline 이 특수화



Layered Structure



Uses Structure(위), Generalization Structure(아래)

Module 이란 무엇인가?

■ 정의

- 일관된 책임(responsibility) 집합을 제공하는 소프트웨어 구현 단위
- Responsibility: 그 module 이 담당하는 기능(feature/functionality)의 종류

■ 다양한 형태의 module

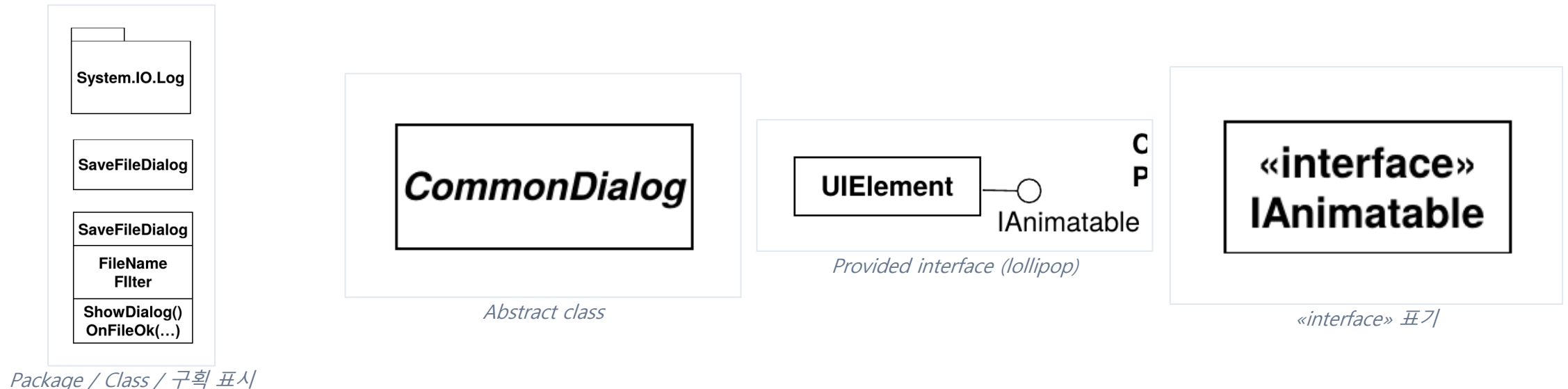
- 절차적·함수형 프로그래밍의 procedure 또는 function
- 객체지향 프로그래밍의 class
- 관점지향(aspect-oriented) 프로그래밍의 aspect
- Layer: module 들의 논리적 묶음
- Data entity: 정보를 담는 객체

핵심 Module 은 '책임'을 기준으로 나눈 구현 단위이며, 언어·패러다임에 따라 형태가 다르다

Module View 표기법 — Module 표현

■ UML 로 module 표현하기 (stereotype 포함)

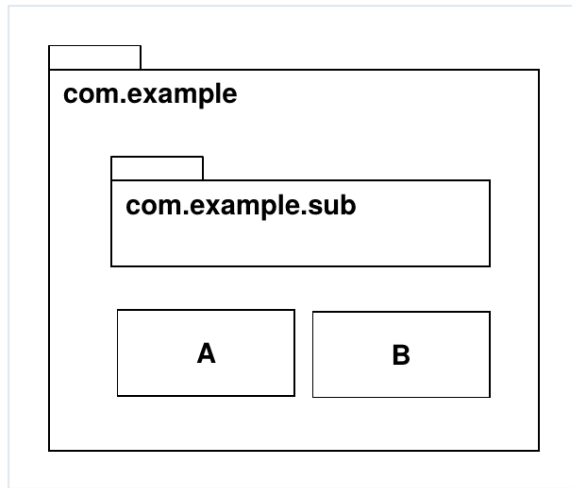
- Package(System.IO.Log), Subsystem, Class(SaveFileDialog)로 module 을 표현한다
- Class 는 attribute·operation 구획(compartment)을 함께 보일 수 있다
- Abstract class 는 이탤릭 이름(CommonDialog), provided interface 는 lollipop(UIElement-IAnimatable) 또는 «interface» class 로 표시한다



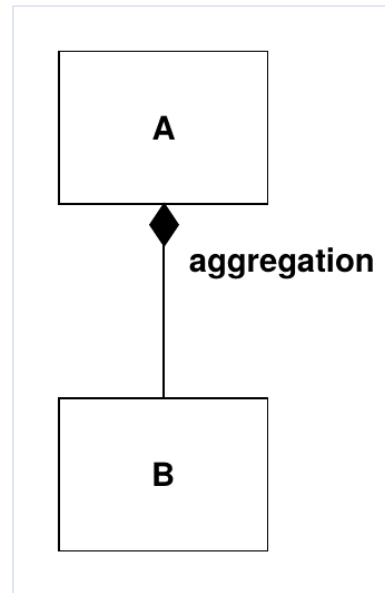
Module View 표기법 — Module 간 관계

■ 세 가지 관계의 UML 표기

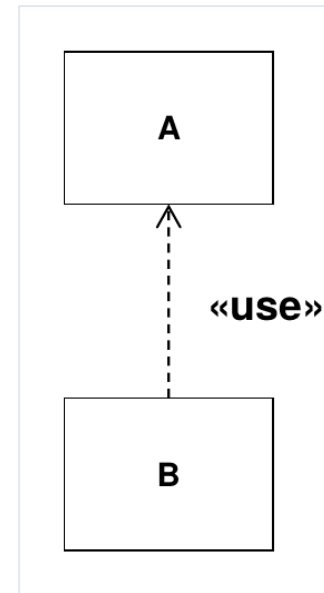
- Is-Part-Of: package 안에 포함(`com.example` $\supset$ `com.example.sub`, A, B) 또는 aggregation(검은 마름모)
- Depends-On: 점선 화살표 + `«use»` stereotype
- Is-A: 실선 + 빈 삼각형(class 상속), 점선 + 빈 삼각형(interface 실현)



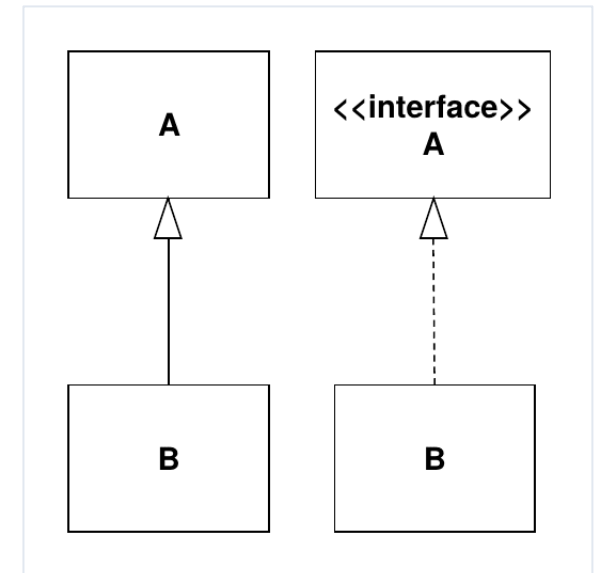
Is-Part-Of (포함)



Is-Part-Of (aggregation)



Depends-On («use»)



Is-A (상속 / 실현)

Module View 의 스타일

■ Decomposition Style

- is-part-of 관계에 초점

■ Uses Style

- depends-on 을 uses 로 특수화

■ Generalization Style

- is-a 관계에 초점

■ Layered Style

- allowed-to-use 관계로 layer 구성

■ Data Model Style

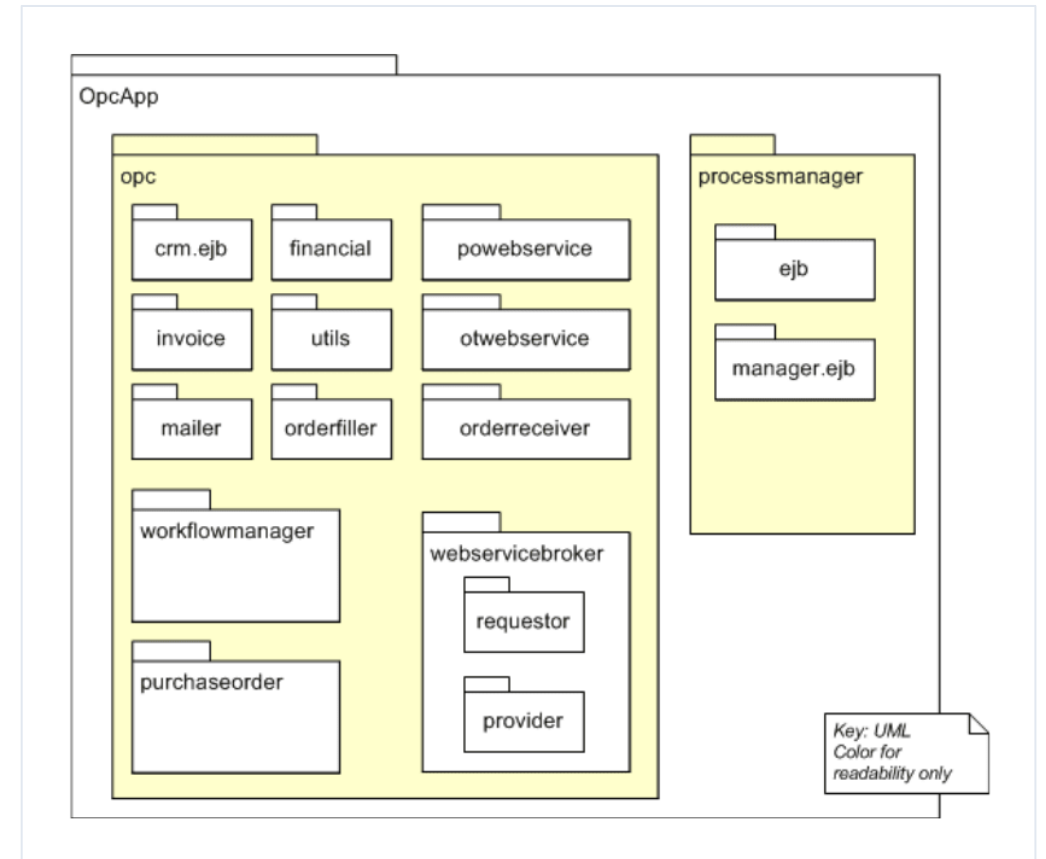
- data entity 와 그 관계

핵심 각 스타일은 기본 Module View 를 제약하여, 일부 요소와 관계 type 의 특수화된 버전을 추가한다

Decomposition Style

■ is-part-of 관계에 초점을 둔 스타일

- 시스템의 기능을 이해하기 쉬운 크기의 조각으로 나누고, 이를 재귀적으로 정제해 점점 더 상세한 내용을 전달한다
- 시스템을 학습하는 과정을 지원하는 데 매우 적합하다
- 보통 시스템의 work assignment view(작업 할당 view)의 입력이 된다



OPC Module Decomposition View (SEI Adventure Builder SAD)

Decomposition 기준과 비형식 표기법

■ Module 을 분해하는 기준

- 특정 Quality Attribute 의 달성
- Build-versus-buy 결정 (직접 만들 부분 vs. 구매할 부분)
- Product line 구현
- 팀 할당: 서로 다른 팀에 맡길 수 있도록 module 을 분리

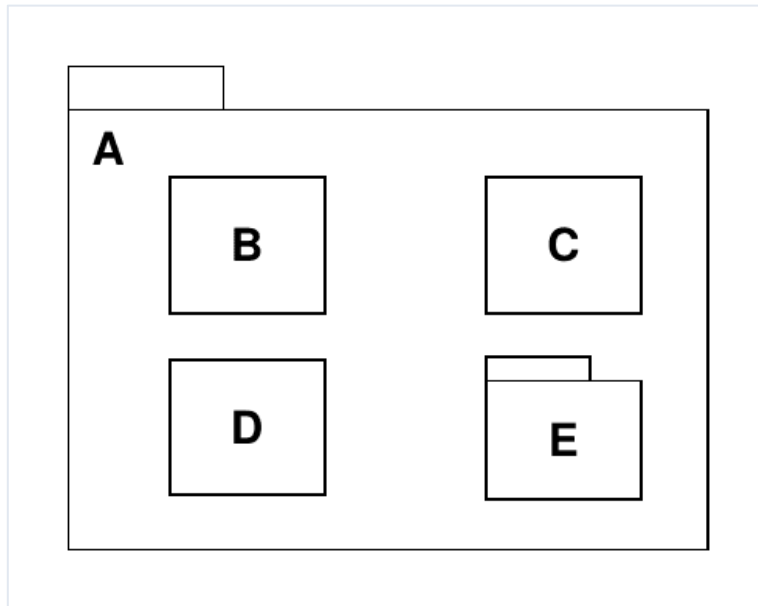
■ Informal Notation (비형식 표기)

- 이름 붙은 상자 안에 다른 이름 붙은 상자를 넣어 표현한다
- 포함 관계를 들여쓰기(indentation)로 표현할 수도 있다
 - 예 (A-7E): Hardware-Hiding Module ⊃ Extended Computer Module ⊃ Data Module, Input/Output Module, Computer State Module ...
 - Behavior-Hiding Module ⊃ Function Driver Module, Shared Services Module ...
 - Software Decision Module ⊃ Application Data Type Module ...

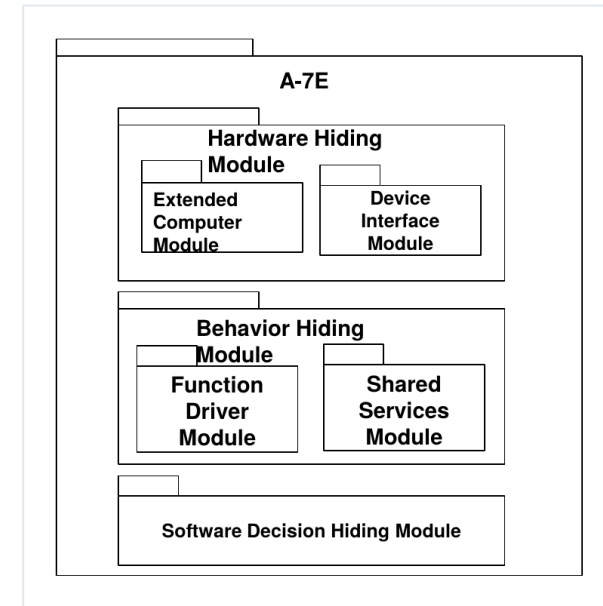
Decomposition Style 의 UML 표기법

■ UML Package 로 포함 관계를 표현한다

- package A 안에 B, C, D, package E 를 넣어 A 의 submodule 임을 나타낸다
- A-7E 예: 들여쓰기 목록(Hardware-Hiding, Behavior-Hiding, Software Decision Hiding Module)을 중첩 package 로 그대로 옮길 수 있다



Package 중첩 표기



A-7E 의 Decomposition View (UML package)

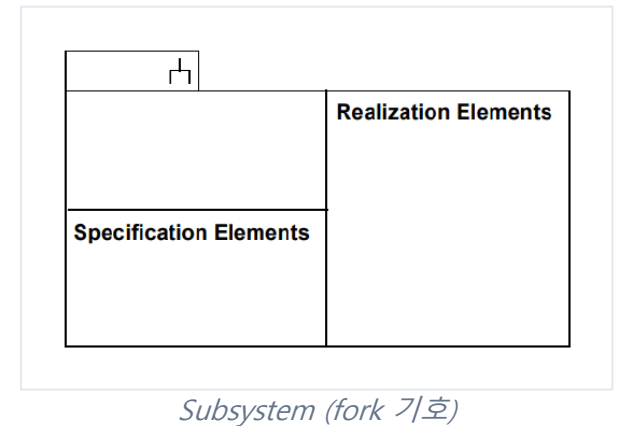
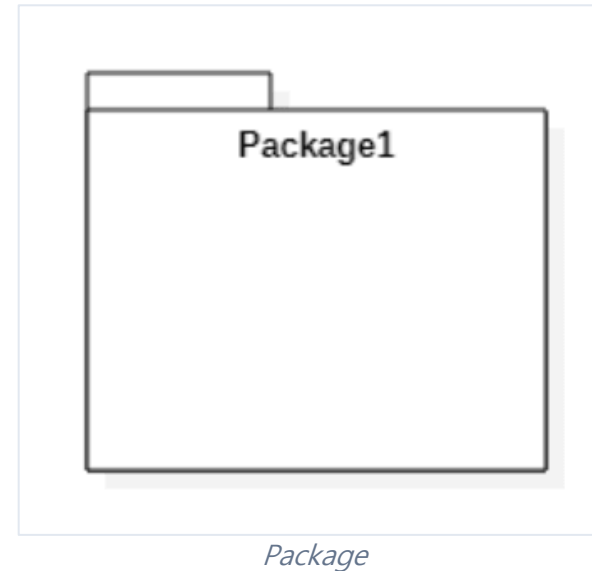
UML 의 Subsystem 과 Package

■ Package

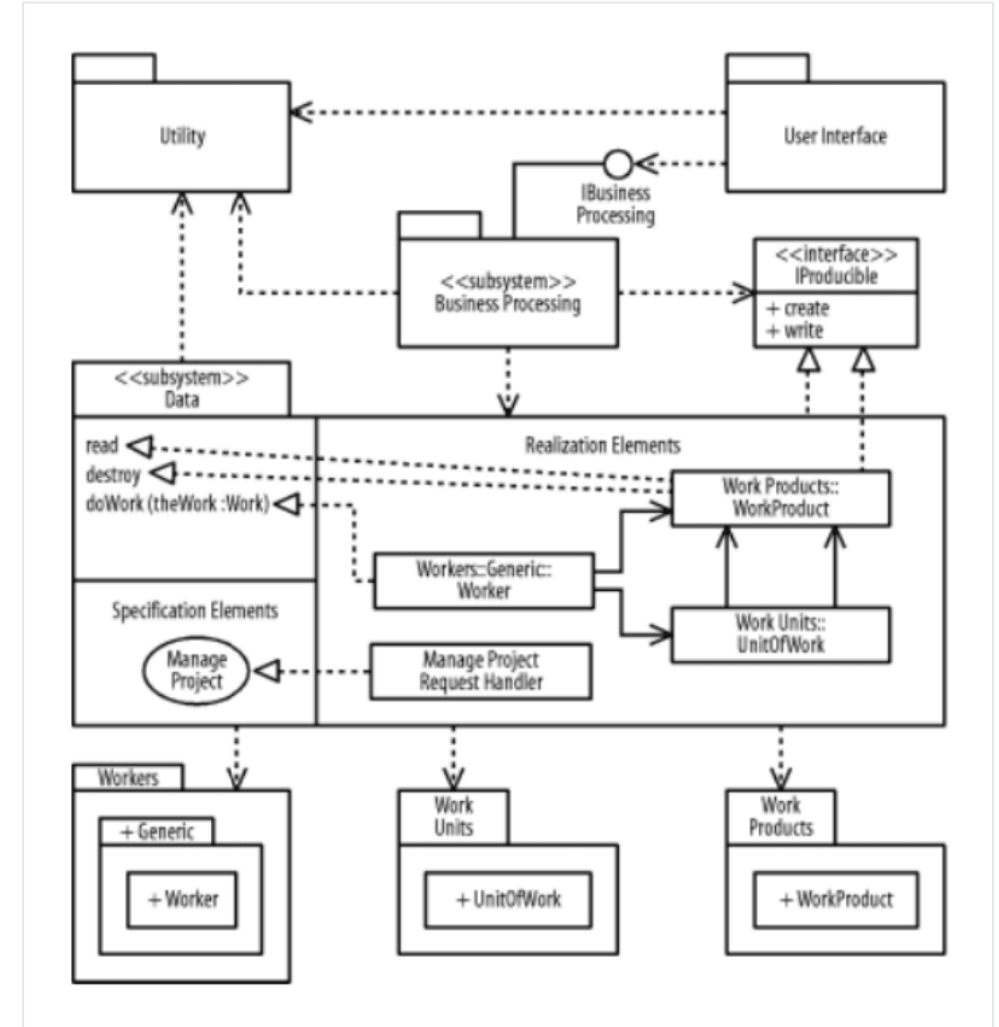
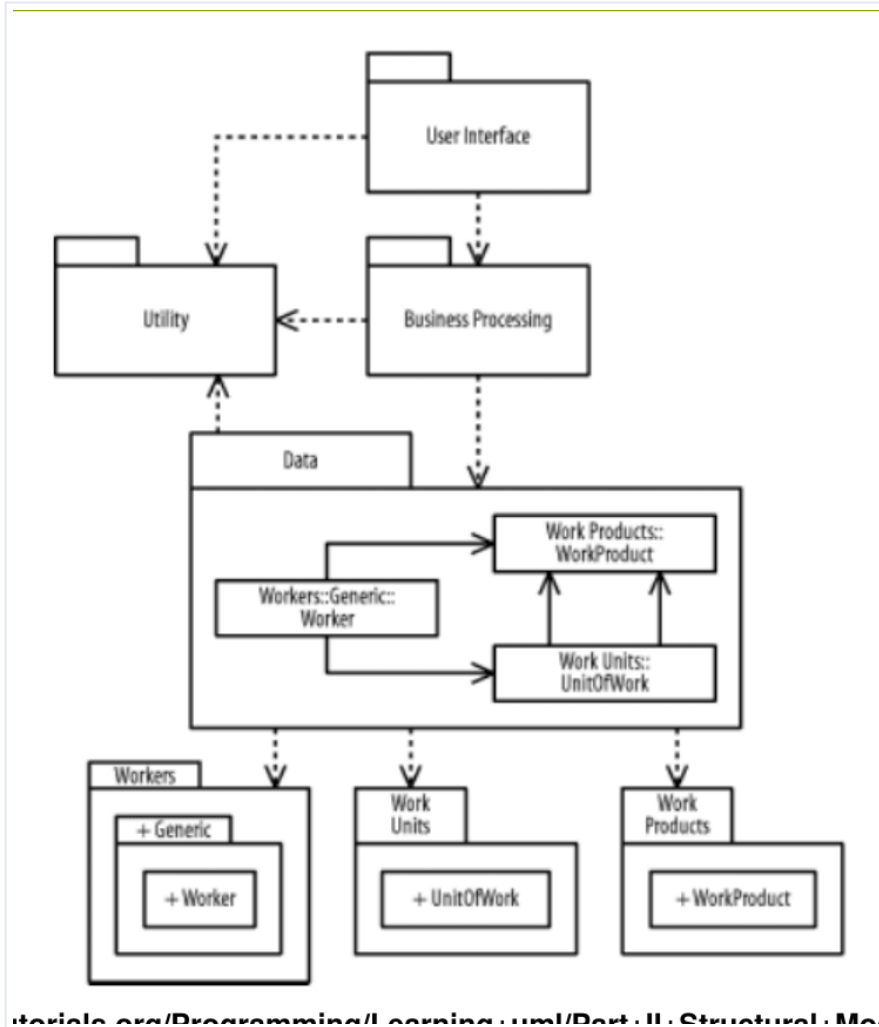
- 요소들을 의미 있게 묶은 단위(grouping)

■ Subsystem

- 전체 시스템 임무 중 응집된 기능 부분집합을 수행하는 시스템의 일부
- 독립적으로 실행할 수 있다
- 점진적으로 개발·배포할 수 있다
- 표기는 package 와 기본적으로 같고, 큰 사각형의 오른쪽 위에 fork 기호를 추가한다
 - Specification Elements 와 Realization Elements 구획을 가질 수 있다



Package vs. Subsystem 예제



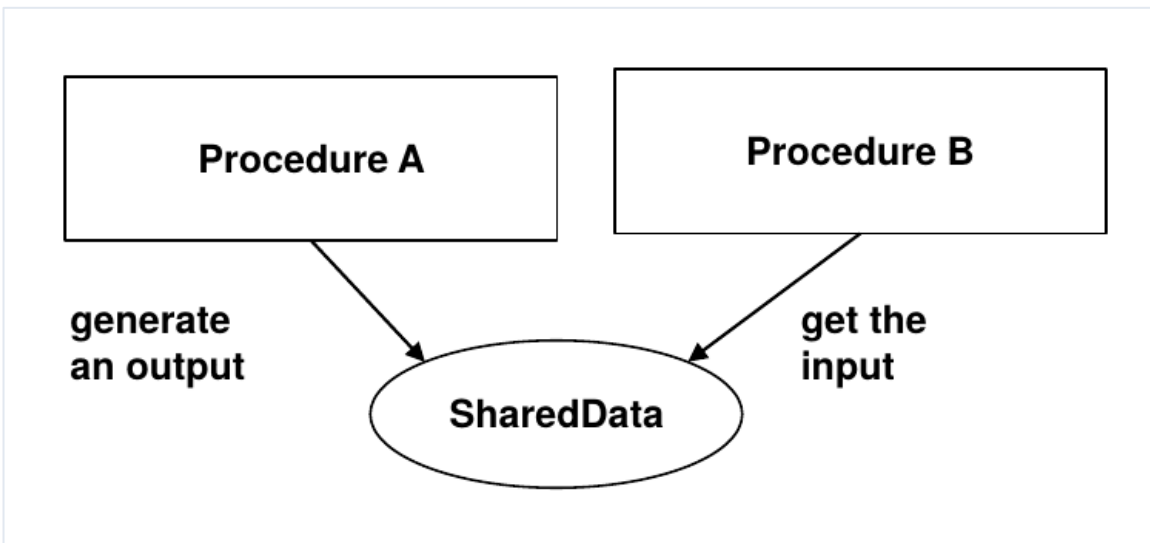
Package 로 표현: User Interface, Business Processing, Data, Utility 가 의존 관계로 연결

Subsystem 으로 표현: «subsystem» Business Processing, Data 가 «interface» 를 통해 Specification/Realization 요소를 분리

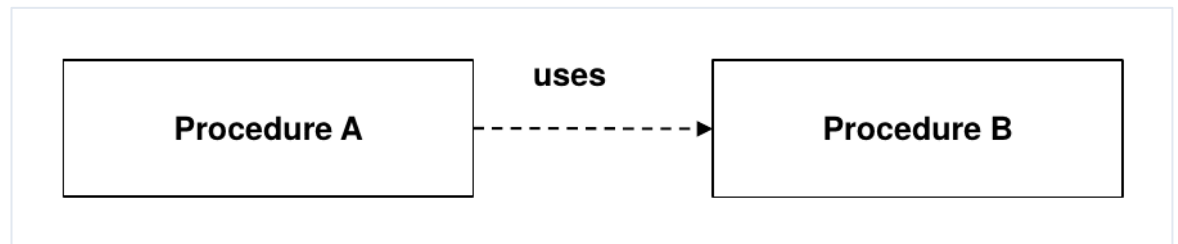
Uses Style

■ depends-on 관계를 uses 로 특수화한 스타일

- 개발자에게 자기 담당 부분이 올바르게 동작하려면 어떤 다른 module 이 반드시 존재해야 하는지 알려 준다
- uses 관계가 항상 module 간 호출(call) 관계와 일치하지는 않는다
 - 예: Procedure A 가 SharedData 에 출력을 쓰고 Procedure B 가 읽는다면, 호출은 없어도 A 는 B 를 uses 한다



호출 없이 공유 데이터로 연결된 두 procedure

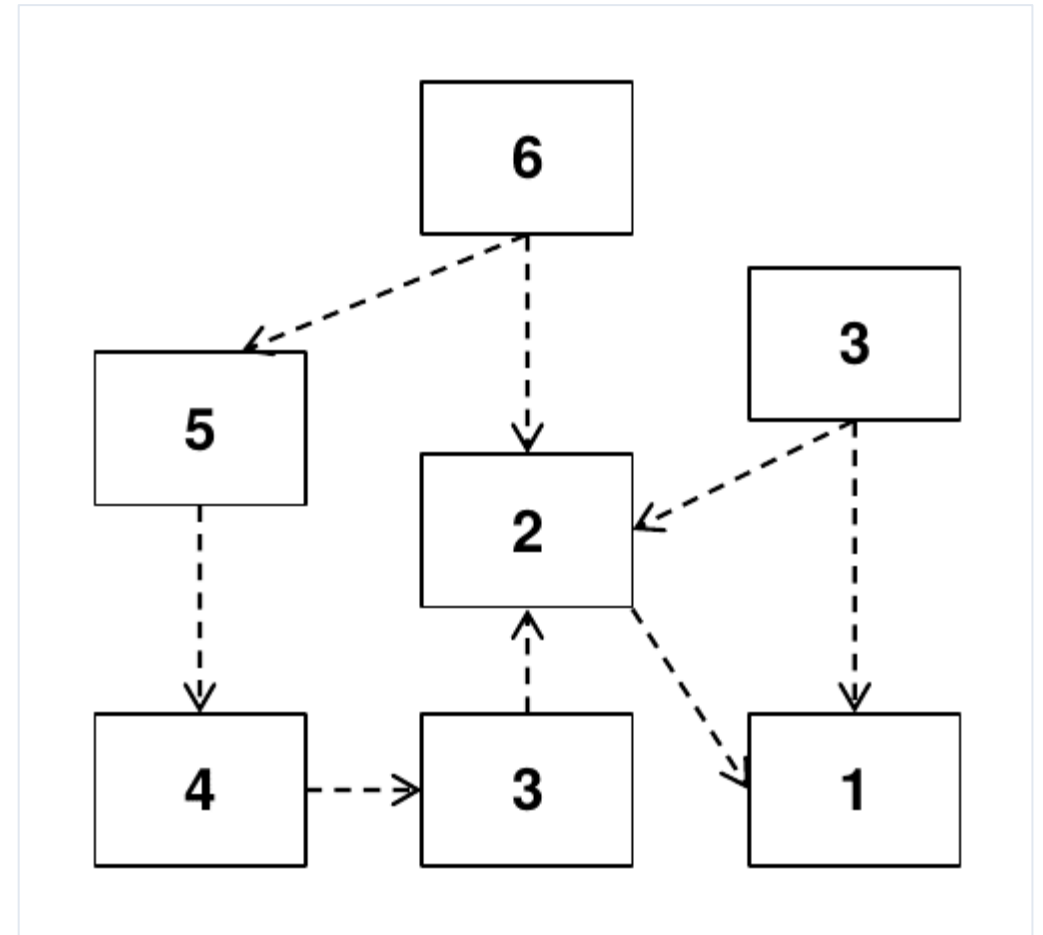


이때의 uses 관계

Uses Style 의 용도

■ 계획(planning)에 유용하다

- 점진적 개발(incremental development): 어떤 module 부터 만들어야 하는지 순서를 정한다
- 시스템 확장과 부분집합(subset) 구성
- 디버깅과 테스트
- 특정 변경이 미치는 영향 범위 파악



uses 그래프: 1 을 먼저 만들고 2, 3 ... 순으로 쌓아 올릴 수 있다

Uses Style 의 표기법

■ Informal Notation

- 행렬(matrix) 형태: (x, y) 칸의 표시는 module x 가 module y 를 uses 함을 뜻한다
- 두 열(two-column) 표: 왼쪽에 사용하는 요소, 오른쪽에 그것이 사용하는 요소를 나열한다

■ UML Notation

- dependency(점선 화살표)에 «uses» stereotype 을 붙인다



User Interface «uses» Data Base

Uses Style 예제 — Matrix 와 Two-column Table

■ 같은 uses 관계를 두 가지 비형식 표기로 나타낸 예

- Matrix: INGST CSCI 행에서 STMGT, SDSRV, DCCI, ADSRV 열에 1 이 표시 → INGST 가 이들을 uses
- Two-column table: INGST CSCI 가 사용하는 요소를 subsystem 별로 나열

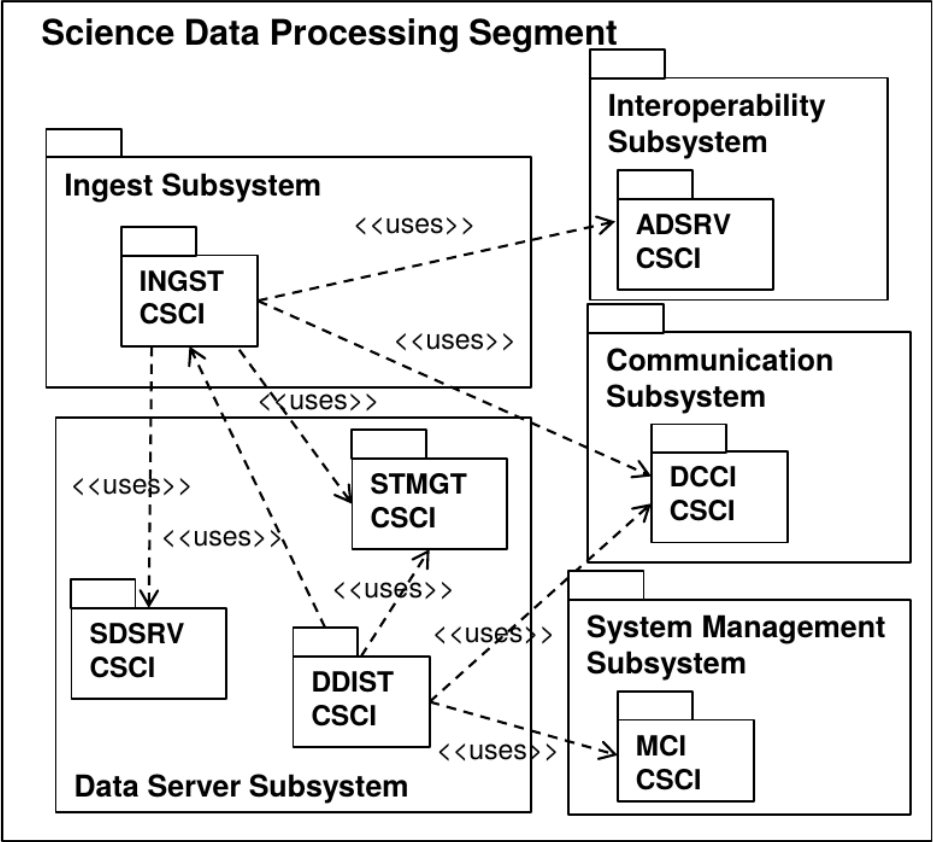
	INGST CSCI	STMGT CSCI	SDSRV CSCI	DDIST CSCI	DCCI CSCI	ADSRV CSCI	MCI CSCI
INGST CSCI		1	1		1	1	
STMGT CSCI							
SDSRV CSCI							
DDIST CSCI	1	1			1		1
DCCI CSCI							
ADSRV CSCI							
MCI CSCI							

Matrix form

SDPS Element Uses This Element		
Science Data Processing Segment		
[c.]	Ingest Subsystem	
	INGST CSCI	ADSRV CSCI in the Interoperability Subsystem
		STMGT CSCI in the Data Server Subsystem
		SDSRV CSCI in the Data Server Subsystem
		DCCI CSCI in the Communications Subsystem
	[etc.]	Other CSCIs within the Ingest Subsystem
	Data Server Subsystem	
	DDIST CSCI	MCI CSCI in the System Management Subsystem
		DCCI CSCI in the Communications Subsystem
		STMGT CSCI in the Data Server Subsystem
		INGST CSCI in the Ingest Subsystem
	[etc.]	Other CSCIs within the Data Server Subsystem
	other subsystems within the Science Data Processing Segment	

Two-column table

Uses Style 예제 — UML 표기와 Two-column Table



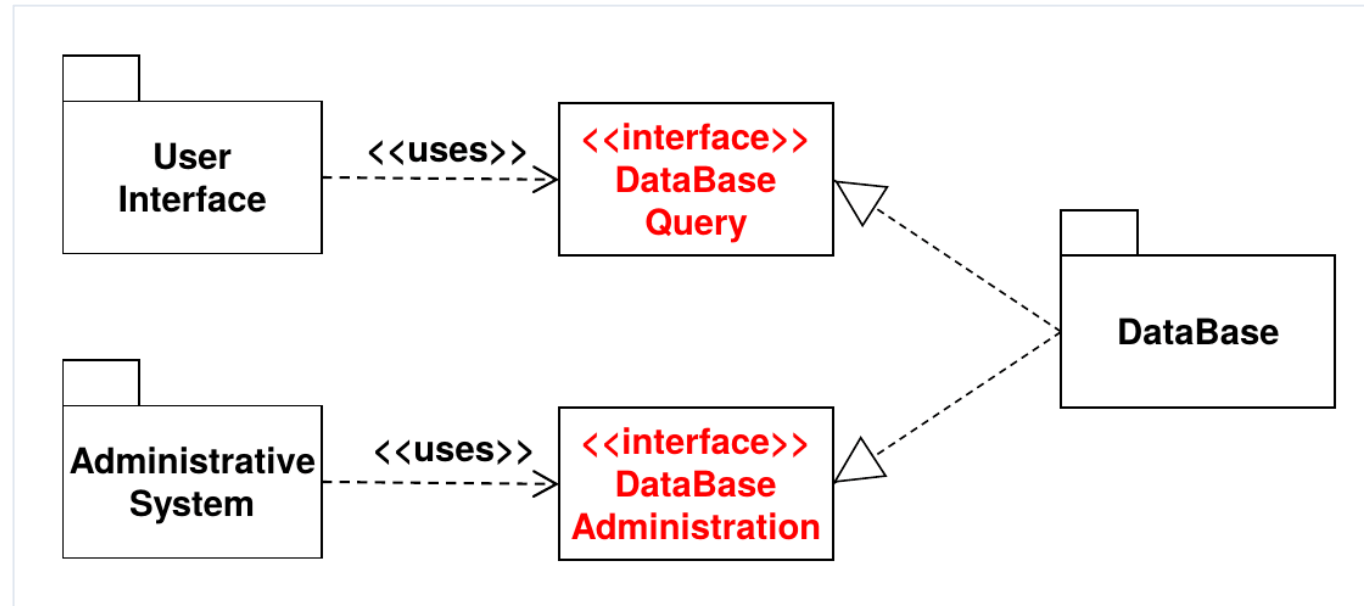
SDPS Element Uses This Element		
Science Data Processing Segment		
	Ingest Subsystem	
	INGST CSCI	ADSRV CSCI in the Interoperability Subsystem
		STMGT CSCI in the Data Server Subsystem
		SDSRV CSCI in the Data Server Subsystem
		DCCI CSCI in the Communications Subsystem
	[etc.]	Other CSIs within the Ingest Subsystem
	Data Server Subsystem	
	DDIST CSCI	MCI CSCI in the System Management Subsystem
		DCCI CSCI in the Communications Subsystem
		STMGT CSCI in the Data Server Subsystem
		INGST CSCI in the Ingest Subsystem
	[etc.]	Other CSIs within the Data Server Subsystem
[etc.]	other subsystems within the Science Data Processing Segment	

Science Data Processing Segment: 왼쪽은 subsystem 별 CSCI 사이의 «uses» 관계를 UML 로, 오른쪽은 같은 내용을 two-column table 로 표현

Uses Style 예제 — Interface 를 명시한 UML 표기

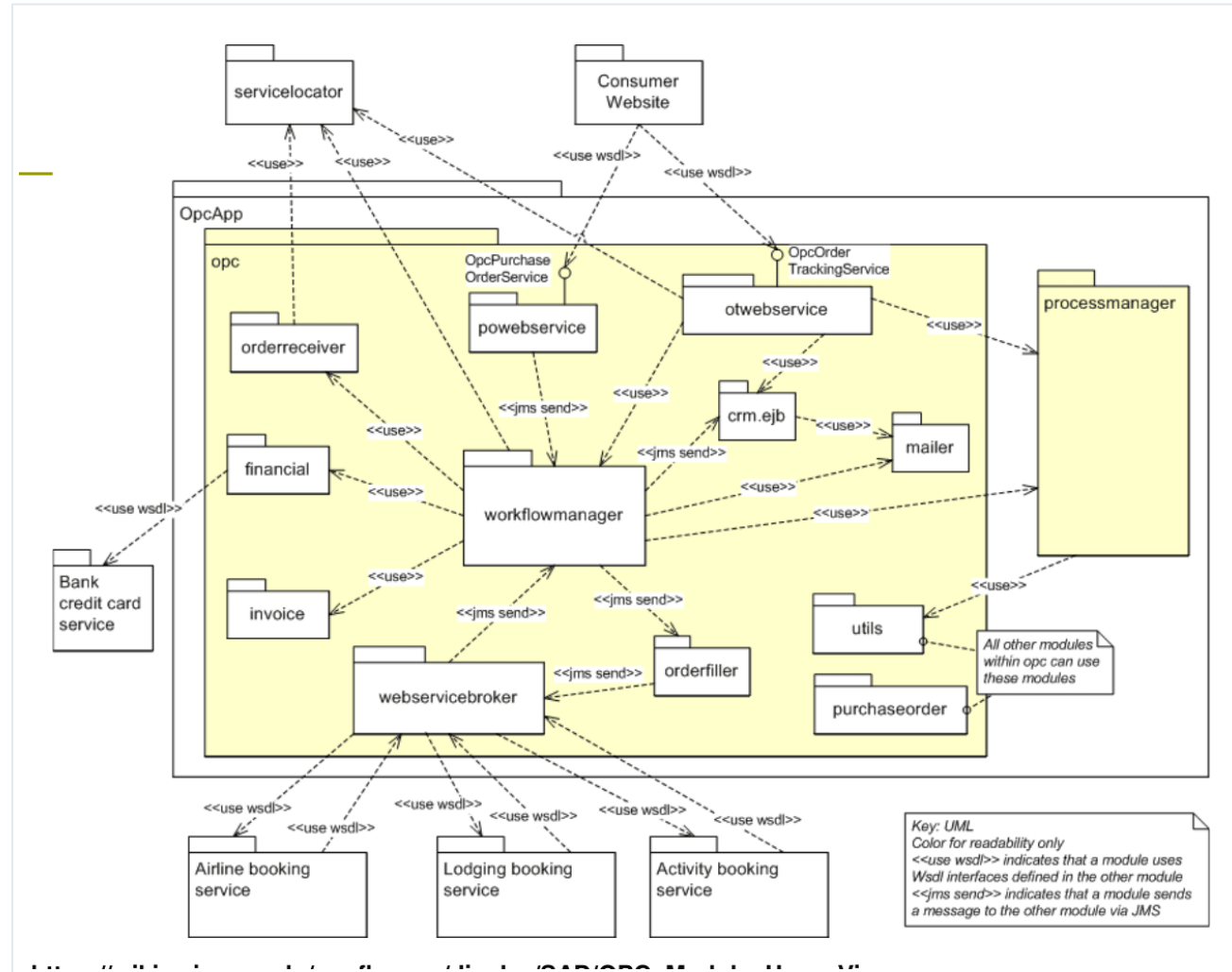
■ UML 에서는 uses 관계와 함께 interface 를 명시적으로 보일 수 있다

- User Interface 는 «interface» DataBase Query 를, Administrative System 은 «interface» DataBase Administration 을 uses 한다
- 두 interface 모두 DataBase module 이 실현(realize)한다 → 사용자는 interface 에만 의존한다



«uses» + «interface» 를 함께 표현한 예

Uses Style 예제 — OPC Module Uses View



Adventure Builder OPC 의 Module Uses View: workflowmanager 를 중심으로 opc 안의 module 들과 외부 web service(Airline/Lodging/Activity booking, Bank credit card, Consumer Website) 사이의 «uses» 관계 (SEI SAD)

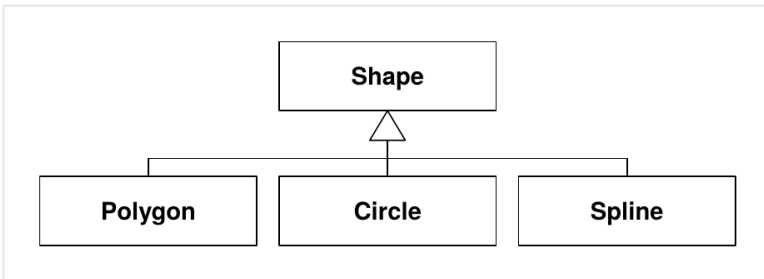
Generalization Style 과 표기법

■ Elements / Relations / Constraints

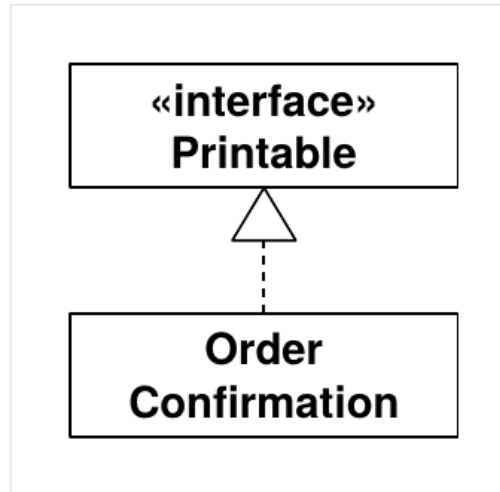
- Element: Module. 'abstract' 속성을 가질 수 있다 → interface 는 있으나 구현이 없는 module
- Relation: Generalization. class 상속, interface 상속, interface 실현(realization)으로 더 특수화할 수 있다
- Constraint: generalization 관계에 cycle(순환)이 있으면 안 된다

■ UML 표기

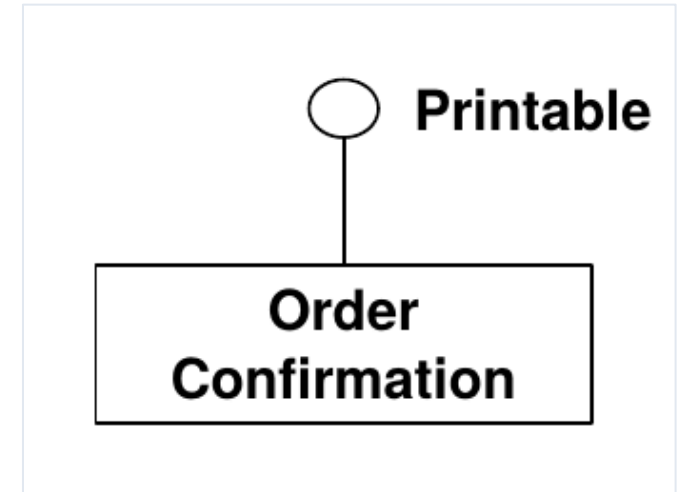
- class/interface 상속: 실선 + 빈 삼각형
- interface 실현: 점선 + 빈 삼각형, 또는 lollipop



Class 상속



Interface 실현

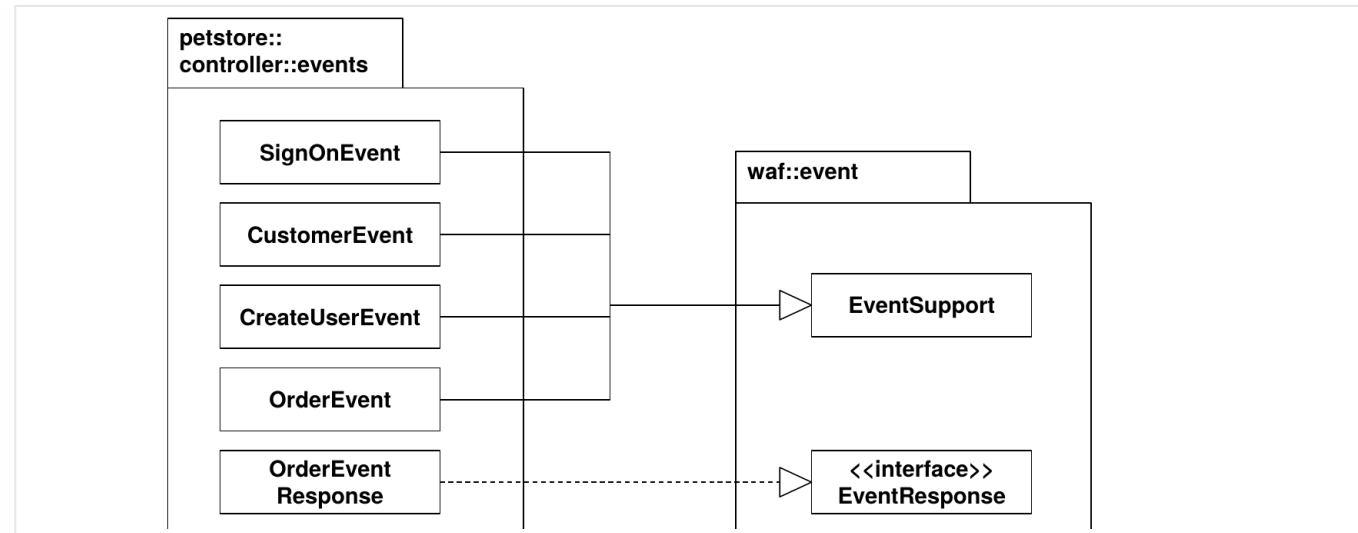


lollipop 표기

Generalization Style 의 용도 (1): 확장과 진화

■ Architecture 와 개별 요소의 extension 과 evolution 에 사용한다

- web application framework(waf)가 event-handling 서비스를 제공한다
- PetStore 같은 응용은 시스템 내 다른 module 과의 상호작용에 쓰이는 응용 고유 event 를 정의해야 한다
 - petstore::controller::events 의 SignOnEvent, CustomerEvent 등이 waf::event 의 EventSupport 를 상속하고, OrderEventResponse 는 «interface» EventResponse 를 실현한다

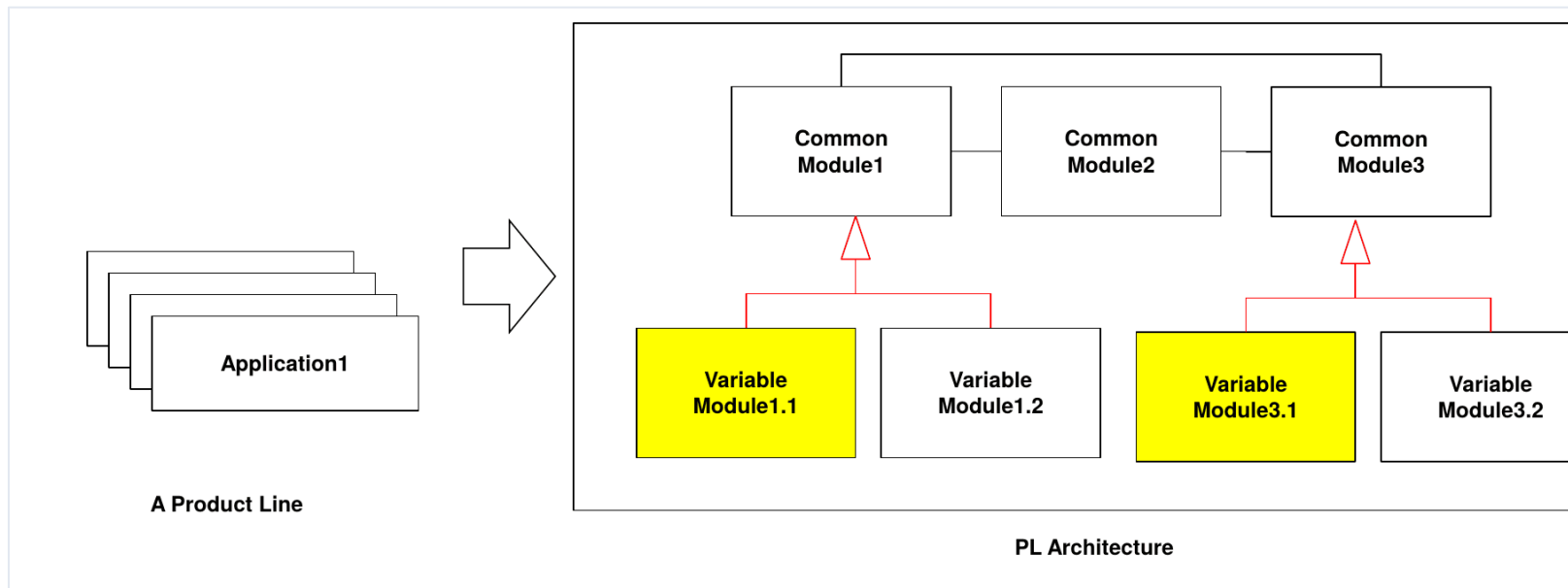


framework 의 event 를 특수화해 응용 event 를 정의

Generalization Style 의 용도 (2): 공통점과 변이

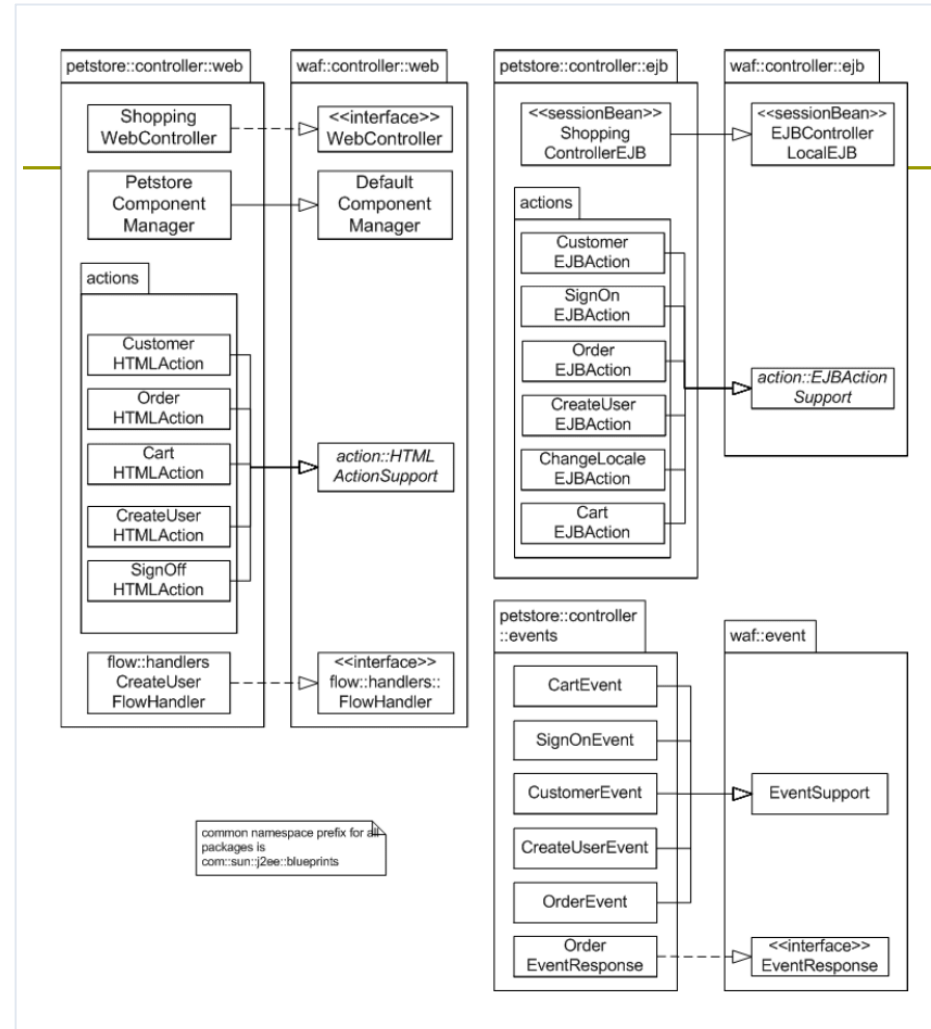
■ Commonalities and variations

- 공통점(commonality)은 상위 module 에 정의하고, 변이(variation)는 그 module 의 자식(children)으로 정의한다
- Product Line: Common Module 1~3 이 공통 부분, Variable Module 1.1/1.2, 3.1/3.2 가 제품별 변이. 각 Application 은 필요한 Variable Module 을 골라 조합한다



Product Line Architecture 에서의 generalization 활용

Generalization Style 예제 — PetStore



Generalization View – Package petstore (SEI SAD): petstore::controller 의 web, ejb, events package 가 waf 의 <<interface>> WebController, EJBController, EventSupport 등을 상속·실현한다

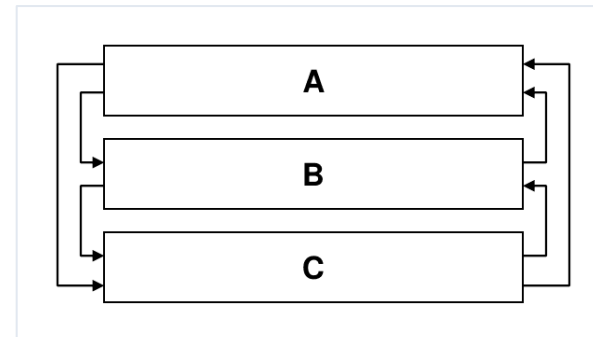
Layered Style

■ Layering 이란

- 소프트웨어를 layer 로 나누는 것. layer 는 응집된 서비스 집합을 제공하는 module 의 묶음이다
- A, B, C 가 서로 마음대로 사용하는 구조는 layer 처럼 보이지만 layering 이 아니다

■ Layer 는 소스 코드만 보고 도출할 수 없다

- Module A «uses» Module B 일 때, B 가 A 보다 아래 layer 여야 하는가? 같은 layer 인가? 코드로는 알 수 없다
- Layering 은 coupling, cohesion, 변경 가능성을 근거로 module 을 layer 에 배정하는 의식적인 설계 결정이다



layering 처럼 보이지만 아닌 예

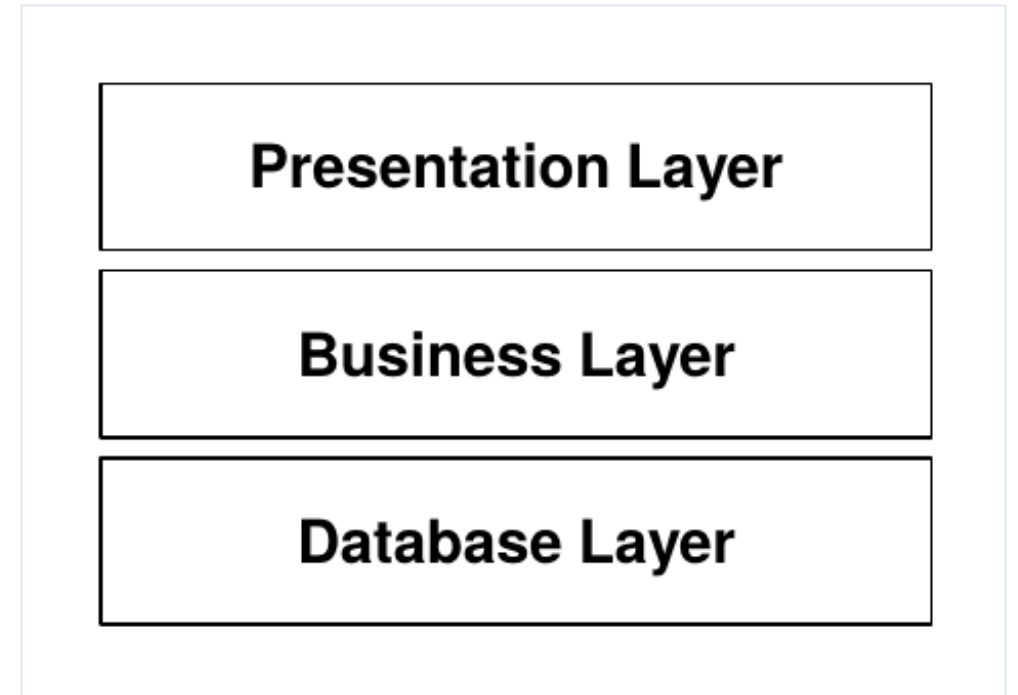


uses 만으로는 layer 를 정할 수 없다

Layered Style 의 용도

■ Layered Style 이 도움이 되는 목적

- Modifiability(변경 용이성)와 Portability(이식성) 향상
- 복잡도 관리와 개발자에게 코드 구조를 쉽게 전달
- 재사용(reuse) 촉진
- 관심사의 분리(separation of concerns) 달성



Presentation / Business / Database Layer

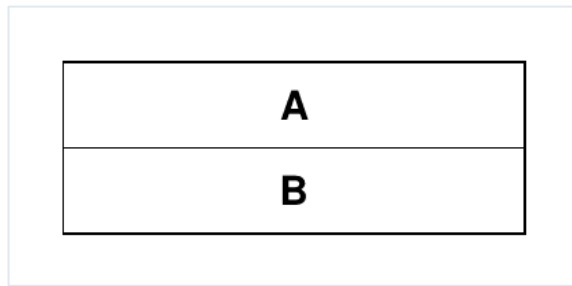
Layered Style 의 비형식 표기 (1): Stack, Segmented

■ Stack

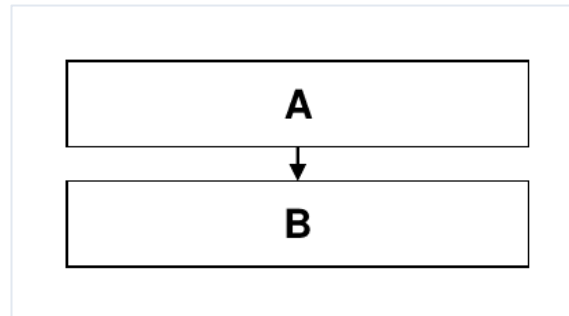
- layer 를 위아래로 쌓은 상자. 위 layer 가 아래 layer 를 사용할 수 있다 (화살표로 명시하기도 한다)

■ Segmented Layers

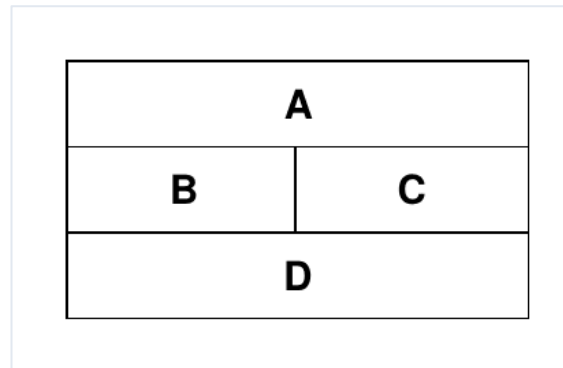
- 한 layer 를 여러 segment(B, C)로 나눈다. segment 사이의 허용 관계(B→C)도 화살표로 보일 수 있다



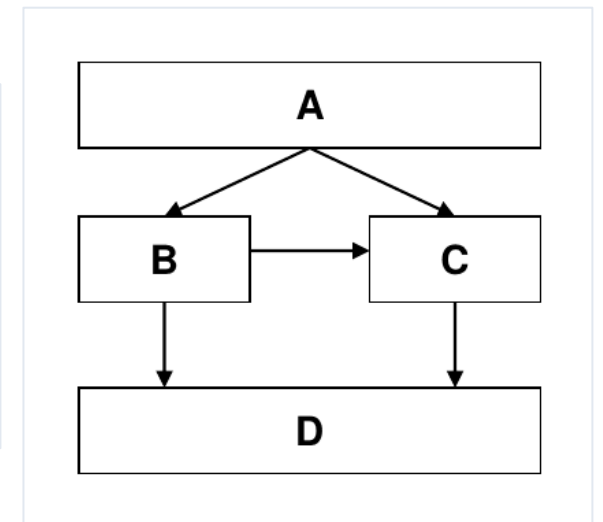
Stack



Stack (화살표)



Segmented



Segmented (화살표)

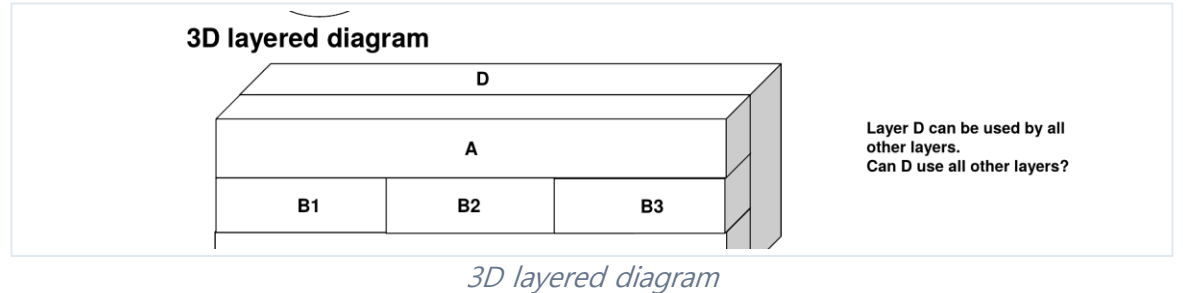
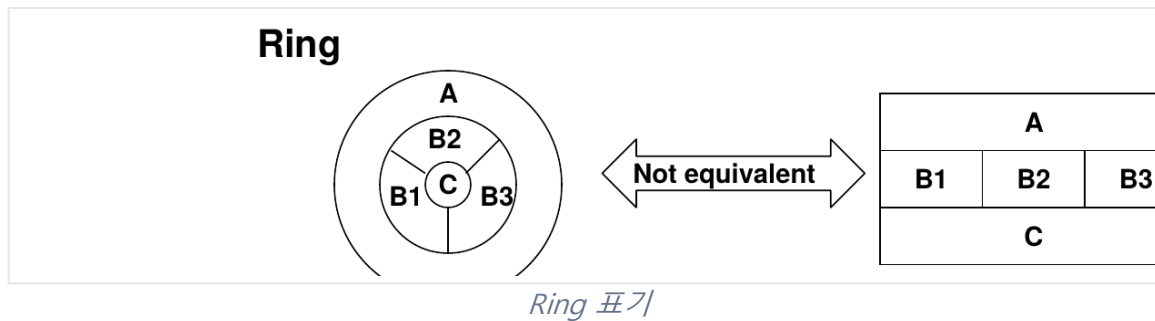
Layered Style 의 비형식 표기 (2): Ring, 3D

■ Ring

- 동심원으로 layer 를 표현. 바깥쪽이 상위 layer. 단, 왼쪽 ring 과 오른쪽 stack 은 동등하지 않다(segment B1, B2, B3 사이의 관계가 다르다)

■ 3D layered diagram

- layer D 는 다른 모든 layer 가 사용할 수 있다. 그러나 D 가 다른 모든 layer 를 사용할 수 있는지는 그림만으로 불분명하다



Layered Style 의 비형식 표기 (3): Sidecar, Contents

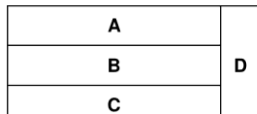
■ Layers with a Sidecar

- layer D 를 A, B, C 옆에 세워 둔 형태. D 가 다른 module 을 사용하고 또 사용될 수 있는지 명확히 정의해야 한다

■ Contents and Interface

- layer L 안의 module M1, M2, M3 와 layer 가 외부에 제공하는 interface 를 함께 표현한다

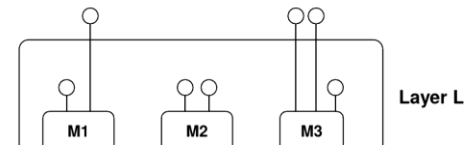
Layers with a Sidecar



Is D allowed to use and be used by other modules?

Layers with a Sidecar

Contents and Interface



Contents and Interface

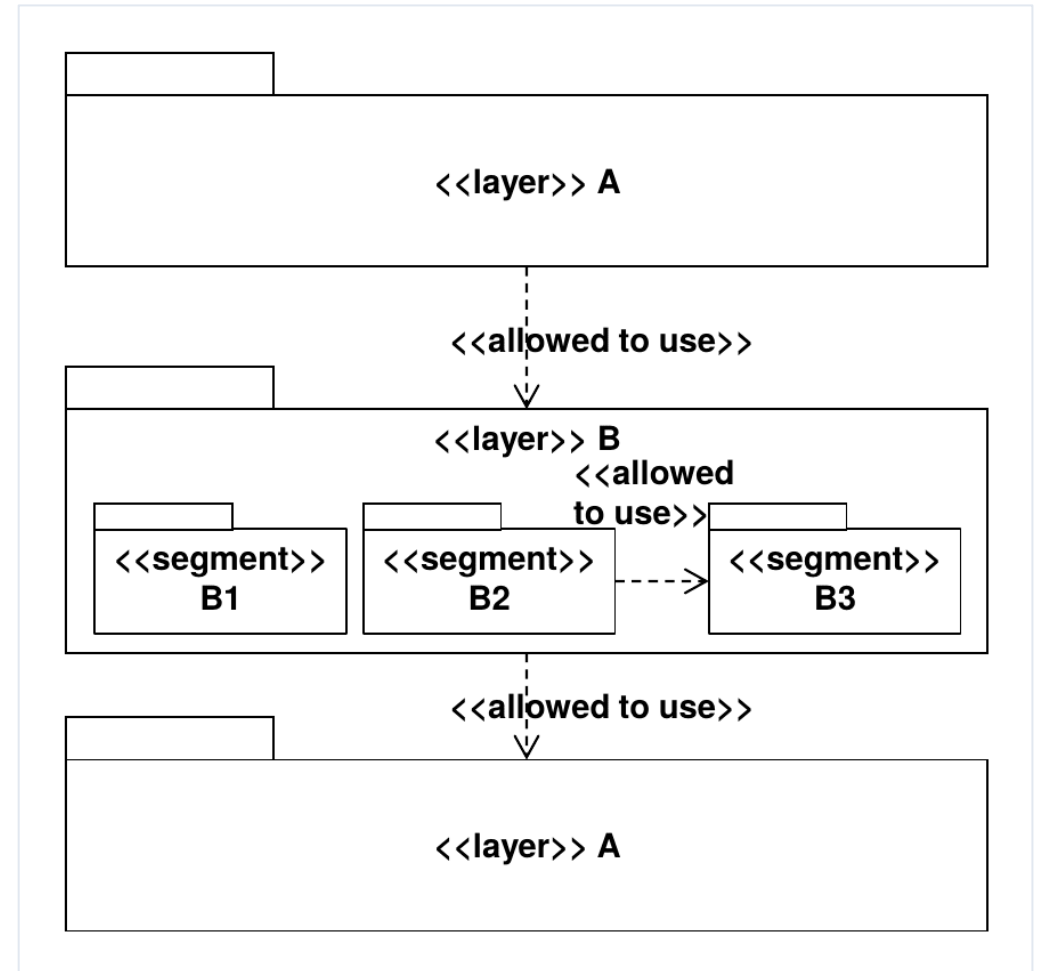
Layered Style 의 UML 표기

■ UML package 와 access dependency

- layer 는 «layer» stereotype 을 붙인 UML package 로 표현한다
- allowed-to-use 관계는 UML access dependency 의 stereotype(«allowed to use»)으로 표현한다
- layer 안의 segment 는 «segment» package 로 표현한다

■ UML access dependency 의 의미

- package 안의 요소는 같은 package 의 다른 요소에 접근할 수 있다
- package 가 다른 package 에 access 하면, 접근되는 package 의 public 요소가 모두 보인다
- access dependency 는 전이적(transitive)이지 않다

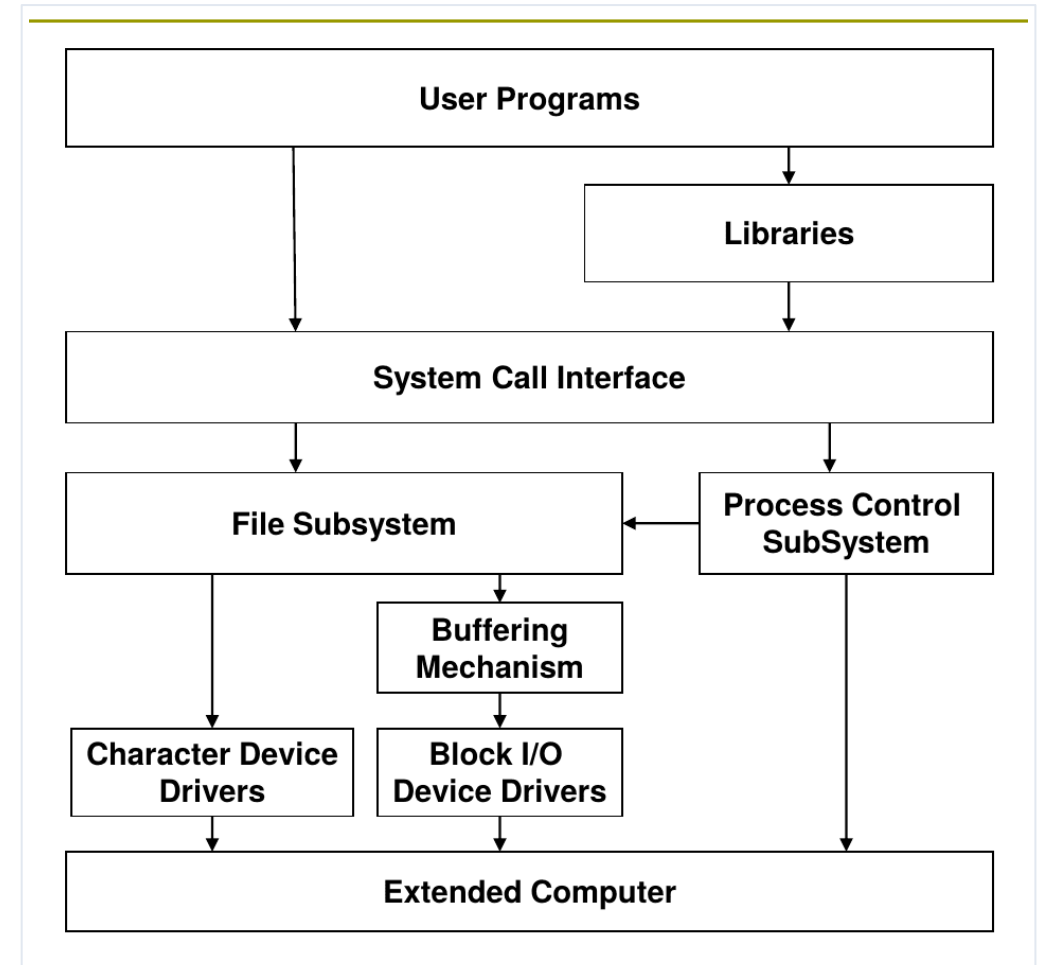


«layer», «segment», «allowed to use»

Layered Style 예제 — Unix System V

■ Unix System V 의 layered view

- User Programs → Libraries → System Call Interface 순으로 아래 layer 를 사용한다
- System Call Interface 아래에 File Subsystem 과 Process Control Subsystem 이 나란히 있다
- File Subsystem 은 Buffering Mechanism 을 거쳐 Character Device Drivers / Block I/O Device Drivers 를 사용한다
- 맨 아래 Extended Computer(하드웨어 추상화) layer. 화살표는 allowed-to-use 관계

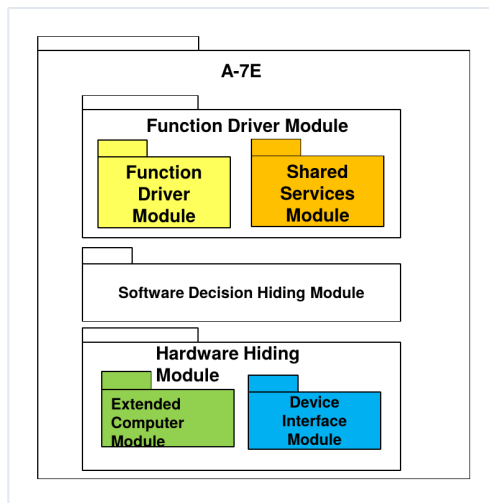


Unix System V 의 layer 구조

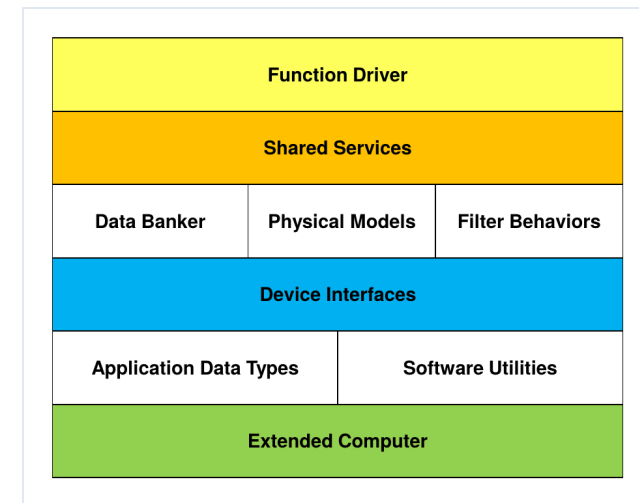
Decomposition Style vs. Layered Style

■ 같은 A-7E 시스템의 두 가지 view

- Decomposition View: Function Driver Module, Software Decision Hiding Module, Hardware Hiding Module 이 각각 submodule 을 포함한다 (is-part-of)
- Layered View: Function Driver → Shared Services → Data Banker/Physical Models/Filter Behaviors → Device Interfaces → Application Data Types/Software Utilities → Extended Computer 순의 allowed-to-use 관계
- 색으로 두 view 의 같은 module(Function Driver, Shared Services, Extended Computer, Device Interface)을 대응시켰다



Decomposition View



Layered View

Data Model Style

■ Data Model Style 이란

- data entity 와 그 관계로 표현한 정적 정보 구조

■ Conceptual Data Model

- 문제 영역(problem domain)에서 인식되는 entity 와 관계에 초점

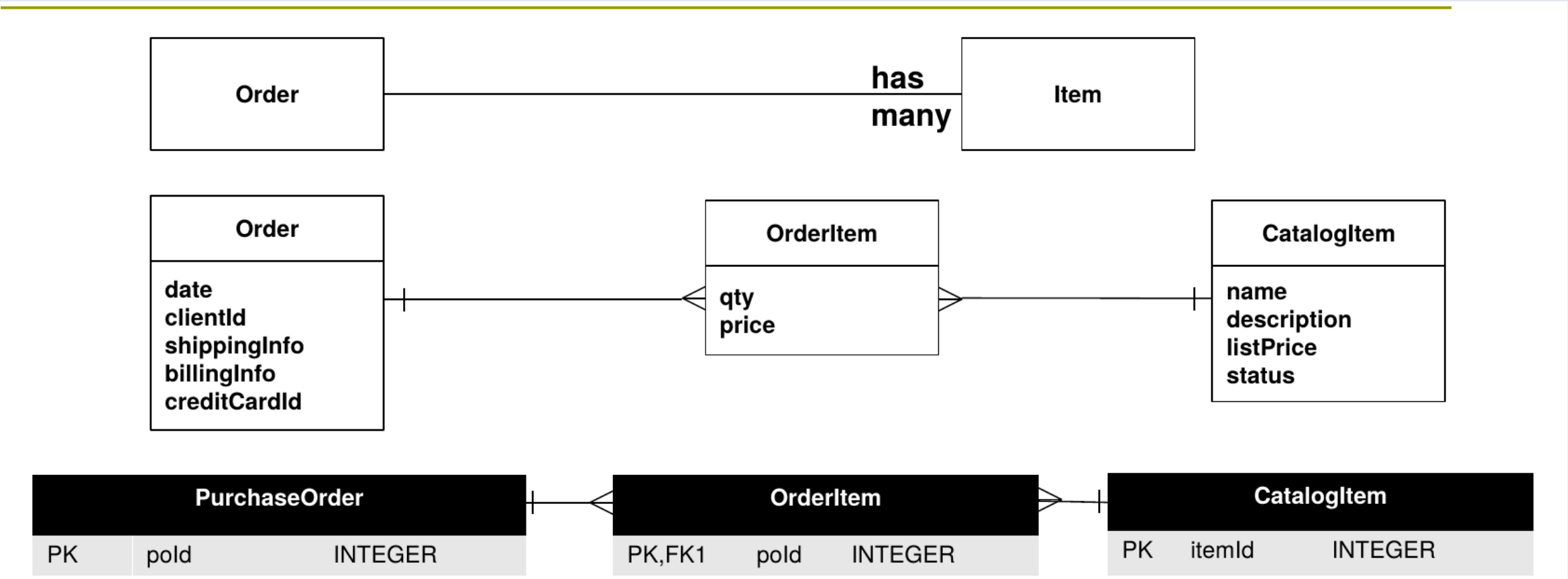
■ Logical Data Model

- conceptual model 을 데이터 관리 기술(예: 관계형 DB)에 맞게 발전시킨 것

■ Physical Data Model

- entity 분할·병합, 데이터 중복, 식별 key 와 index 생성 같은 최적화를 포함

Data Model Style 예제 — 세 수준의 모델



Conceptual: Order 가 여러 Item 을 가진다 → Logical: Order, OrderItem, CatalogItem 과 속성 → Physical: PurchaseOrder, OrderItem, CatalogItem 테이블에 PK/FK, 자료형, index 가 추가된다

Data Model Style — 요소, 관계, 용도

■ Elements

- Data entity: 시스템에 저장되거나 어떤 형태로든 표현되어야 하는 정보를 담는 객체

■ Relations

- 논리적 연관(association): one-to-one, one-to-many, many-to-many
- Generalization / Specialization
- Aggregation

■ Data Model 의 용도

- 시스템에서 사용하는 데이터의 구조를 기술한다
- 데이터 모델 변경의 영향 분석(impact analysis)을 수행한다
- 중복과 불일치를 피해 데이터 품질을 확보한다
- 데이터에 접근하는 module 의 구현을 안내한다

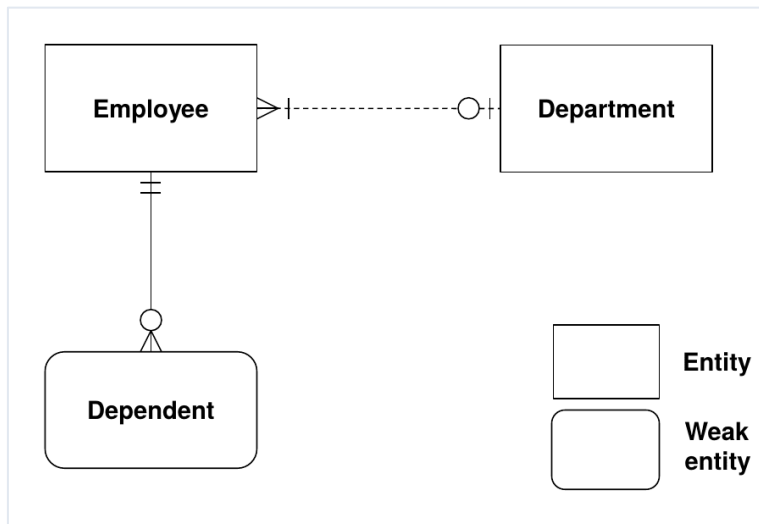
Data Model Style 의 표기법

■ Crow's Foot ERD Notation

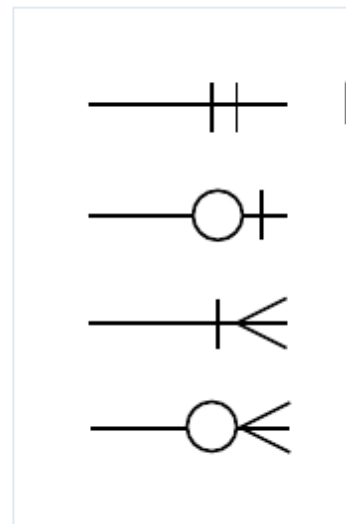
- 사각형 = entity, 모서리 둥근 사각형 = weak entity
- 선 끝 기호로 cardinality 표현: exactly one, zero or one, one or more, zero or more
- 실선 = identifying relationship, 점선 = non-identifying relationship

■ UML

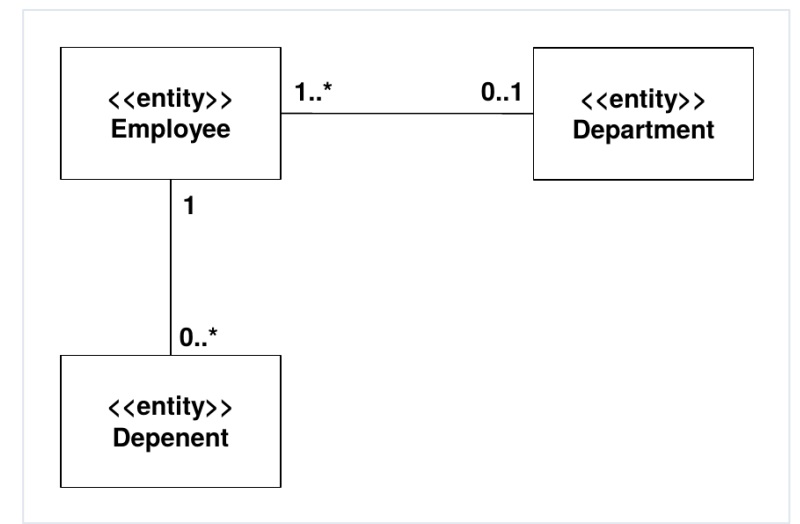
- UML class diagram 으로 표현. «entity» stereotype 과 다중도(1..*, 0..1, 0..*)를 사용한다



Crow's Foot ERD



cardinality 기호

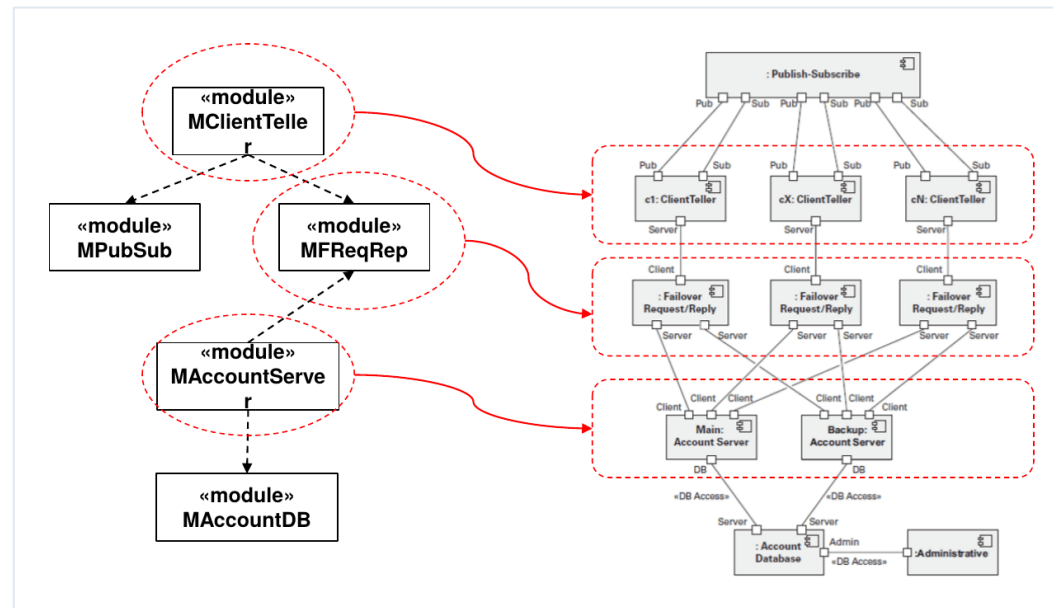


UML class diagram

C&C View 와 Module View 의 관계 (1)

■ 같은 code module 이 C&C View 의 여러 요소에서 실행될 수 있다

- «module» MClientTeller 하나가 c1, cX, cN 세 ClientTeller instance 로 실행된다
- «module» MReqRep 하나가 세 개의 :Failover Request/Reply connector 로, MAccountServer 가 Main/Backup 두 server 로 실행된다

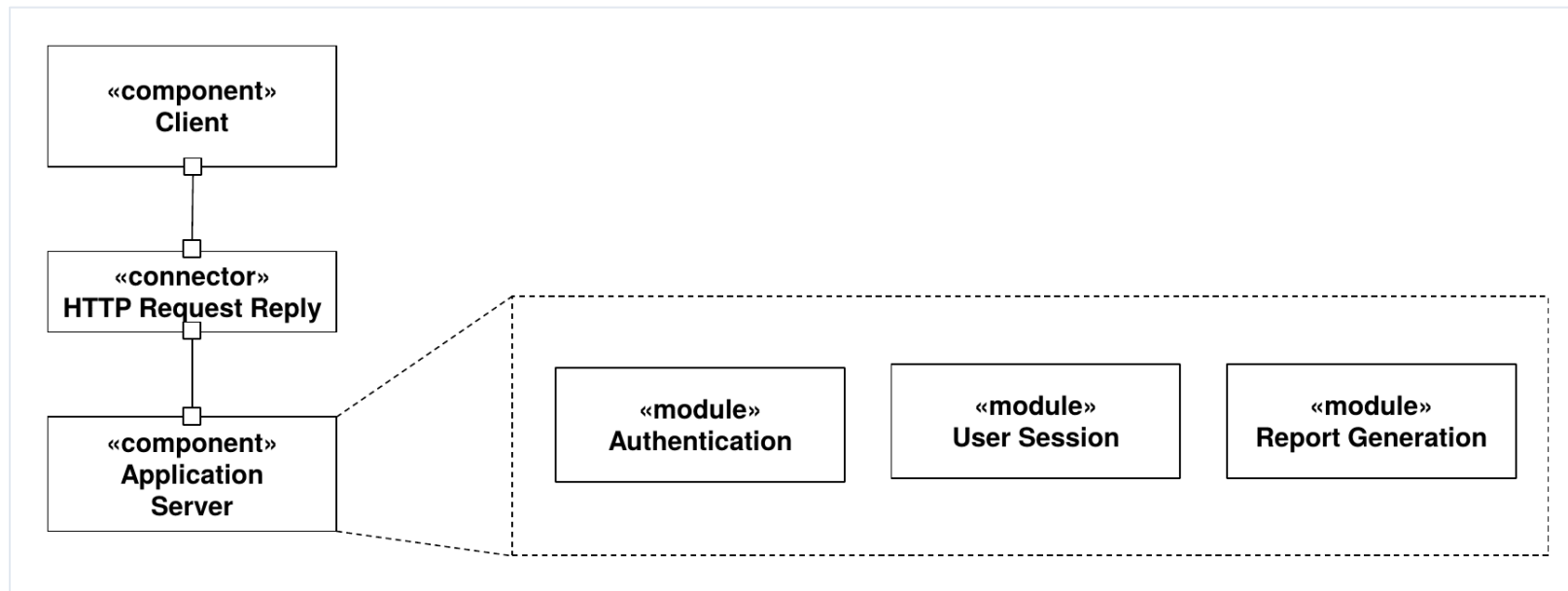


Module(왼쪽) → C&C 요소(오른쪽) 의 1:N 대응

C&C View 와 Module View 의 관계 (2)

■ C&C View 의 component 하나가 여러 module 의 코드를 실행할 수 있다

- «component» Application Server 는 «module» Authentication, User Session, Report Generation 세 module 의 코드로 구성된다
- 즉 component 와 module 은 1:1 이 아니며, N:M 대응이 일반적이다

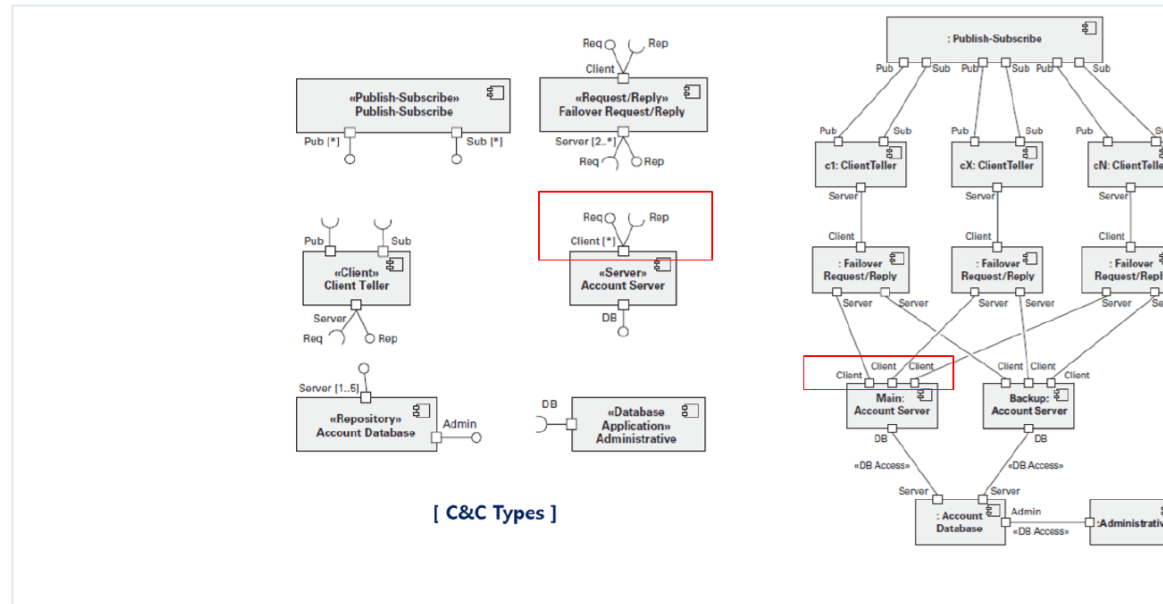


Component(왼쪽) → Module(오른쪽) 의 1:N 대응

C&C View 와 Module View 의 관계 (3)

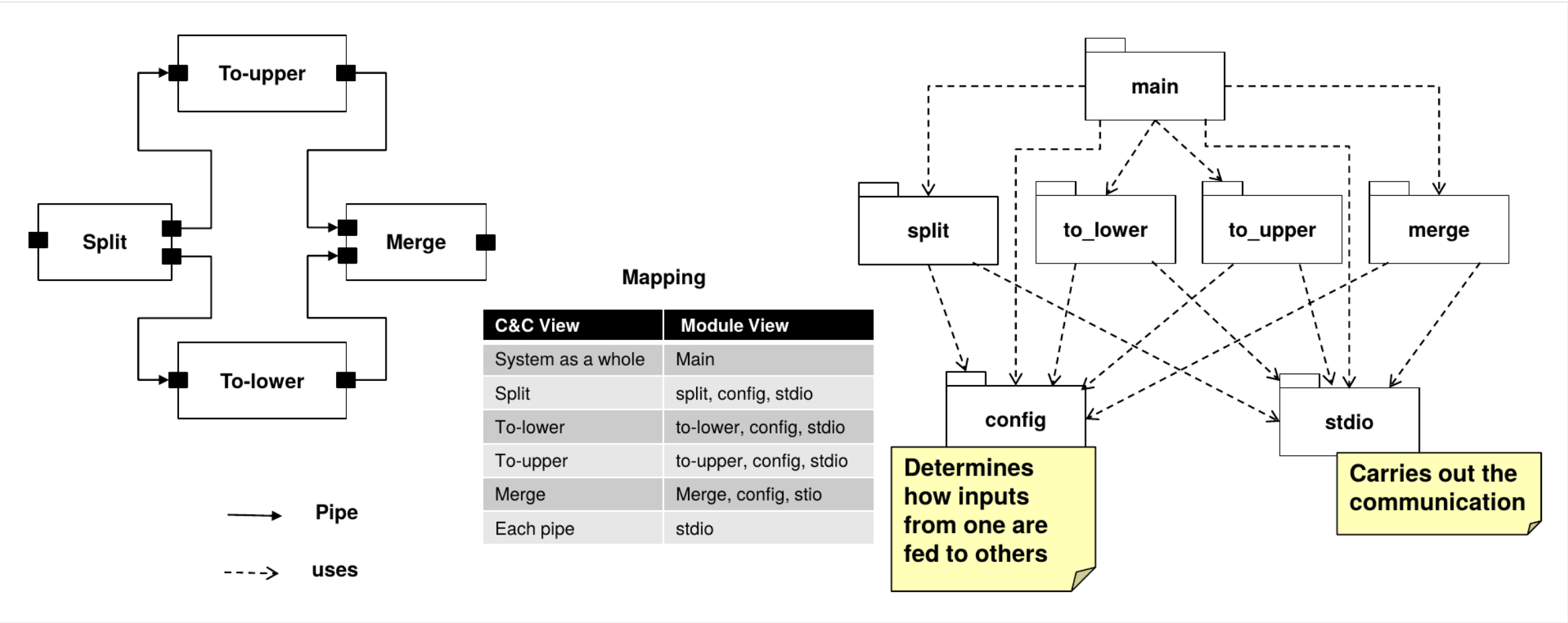
■ component 하나가 환경과 여러 상호작용 지점을 가질 수 있다

- 각 상호작용 지점(port)은 같은 module interface 로 정의된다
- «Server» Account Server type 의 Client[*] port 하나가, Architecture 에서는 Main: Account Server 의 여러 Client port 로 나타난다 (붉은 상자)



C&C Types(왼쪽)와 Architecture(오른쪽): 하나의 port 정의가 여러 상호작용 지점으로

C&C View 와 Module View 의 관계 (4): Mapping



C&C View(Split, To-upper, To-lower, Merge + Pipe) 와 Module View(main, split, to_lower, to_upper, merge, config, stdio) 의 mapping 표. config 는 입력이 어디로 전달되는지 결정하고, stdio 는 통신을 수행한다

Module View 문서화 요약

■ Decomposition Style

- 책임이 module 에 어떻게 배분되는지 보인다
- 신규 인력의 학습·탐색 도구, work assignment view 의 입력

■ Uses Style

- module 이 서로 어떻게 의존하는지 보인다
- 점진적 개발과 유용한 subset 구성에 도움

■ Generalization Style

- 한 module 이 다른 module 의 일반화/특수화임을 보인다
- 객체지향 시스템에서 널리 쓰이며 상속과 공통성 활용

■ Layered Style

- 시스템을 allowed-to-use 관계로 연결된 virtual machine 집합으로 나눈다
- portability 와 modifiability 달성에 도움

■ Data Model Style

- data entity 와 관계로 데이터 구조를 기술한다
- 구현을 안내하고 data-centric 시스템의 성능·변경을 개선

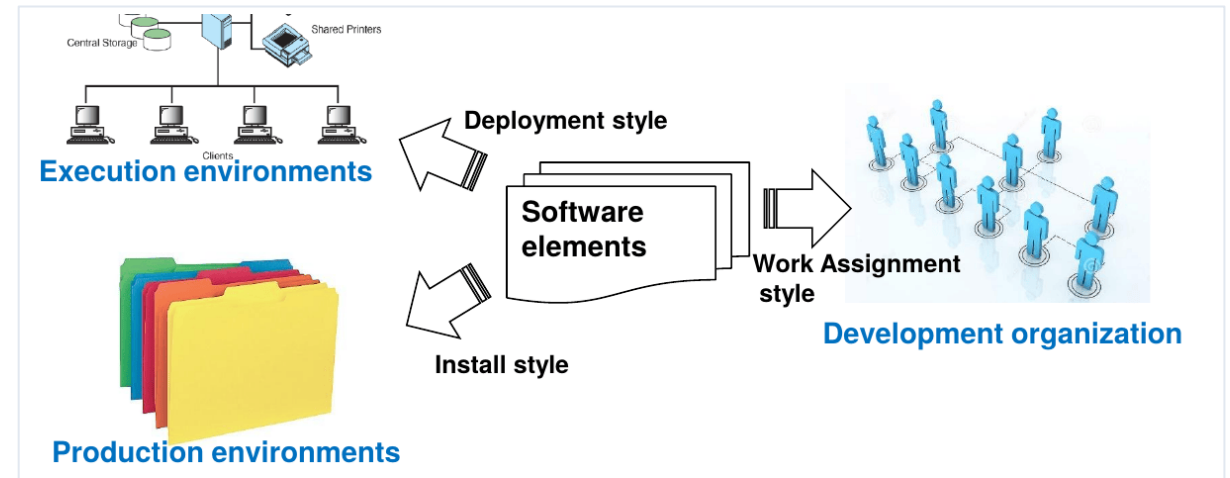
Allocation View의 스타일

■ Allocation View(할당 뷰)란

- 소프트웨어 요소와 소프트웨어 환경의 비소프트웨어 요소(하드웨어, 파일 시스템, 조직) 사이의 매핑을 표현한다
- 구체적인 용도와 표기법은 스타일마다 다르다

■ 세 가지 스타일

- Deployment Style: 소프트웨어 요소 → 실행 환경(하드웨어)
- Install Style: 소프트웨어 요소 → 운영(production) 환경의 파일/폴더
- Work Assignment Style: 소프트웨어 요소 → 개발 조직(팀)



소프트웨어 요소를 실행 환경·운영 환경·개발 조직에 매핑하는 세 스타일

Deployment Style — 정의와 Elements

■ 정의

- Architecture 의 Component 와 Connector 를 컴퓨팅 플랫폼의 하드웨어에 매핑한다
- 보통 C&C 스타일의 요소가 컴퓨팅 플랫폼에 할당된다

■ Software element (C&C View 의 요소)

- 필요(required) 속성: 처리 요구사항, 자원 요구사항, fault tolerance(결함 허용) 요구

■ Environmental element (컴퓨팅 하드웨어)

- processor, memory, disk, network 등
- 제공(provided) 속성:
 - CPU 속성: 클럭 속도, 프로세서 수 등
 - Memory 속성: 메모리 크기와 속도 특성
 - Disk 등 저장 장치 용량
 - Bandwidth(대역폭) 등

핵심 소프트웨어가 요구하는(required) 속성과 하드웨어가 제공하는(provided) 속성을 맞추어 보는 것이 배치 설계의 핵심이다

Deployment Style — Relations

■ Allocated-to (정적)

- 소프트웨어 요소가 어느 물리 장치에 상주하는지 나타낸다

■ Migrates-to (동적)

- 소프트웨어 요소가 한 프로세서에서 다른 프로세서로 이동할 수 있다

■ Copy-migrates-to (동적)

- 소프트웨어 요소가 자신의 복사본을 새 프로세서로 보낸다

■ Execution-migrates-to (동적)

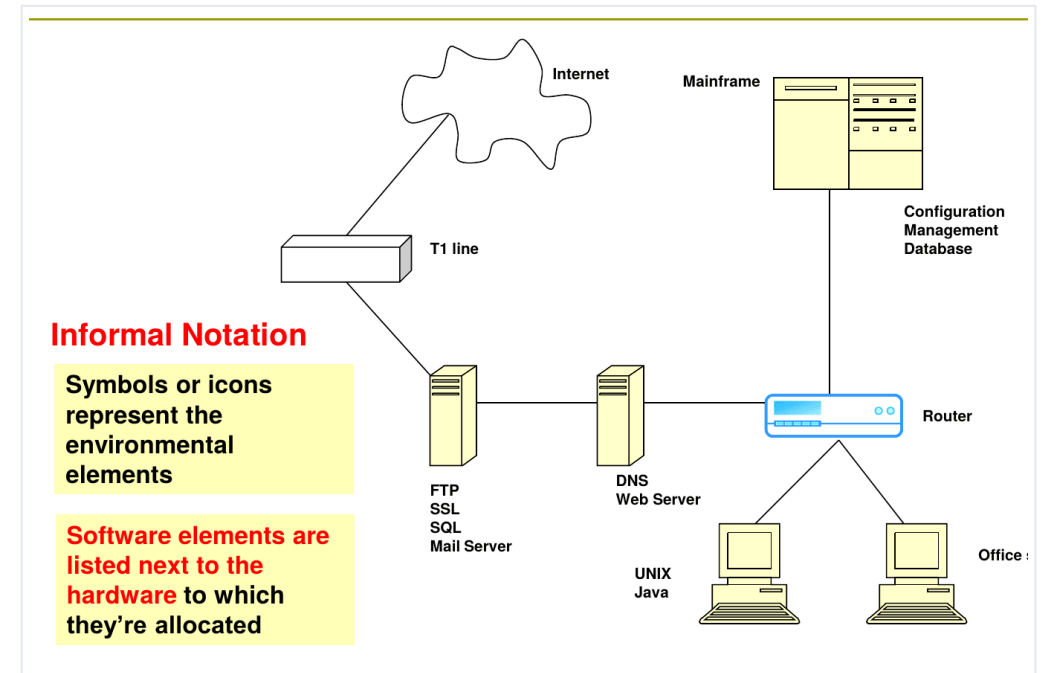
- 복사본이 여러 프로세서에 존재하지만, 한 시점에는 하나만 활성화된다

핵심 Allocated-to 는 정적 관계이고, 나머지 세 가지는 실행 중에 바뀌는 동적 관계이다

Deployment Style 의 비형식 표기법 (Informal Notation)

■ Informal Notation

- 기호나 아이콘으로 환경 요소(하드웨어)를 표현한다
 - Mainframe, Router, Web Server, PC 등을 그림 아이콘으로 표시
- 소프트웨어 요소는 할당된 하드웨어 옆에 목록으로 적는다
 - 예: Web Server 옆에 FTP, SSL, SQL, Mail Server, DNS
- 이해하기 쉽지만 의미가 엄밀하지 않다



아이콘으로 하드웨어를 그리고 옆에 소프트웨어 요소를 나열한 예

UML Deployment Diagram

■ Deployment Diagram(배치 다이어그램)

- 소프트웨어 artifact 가 배치 대상(node)에 어떻게 배치되는지 표현한다

■ Node (배치 대상)

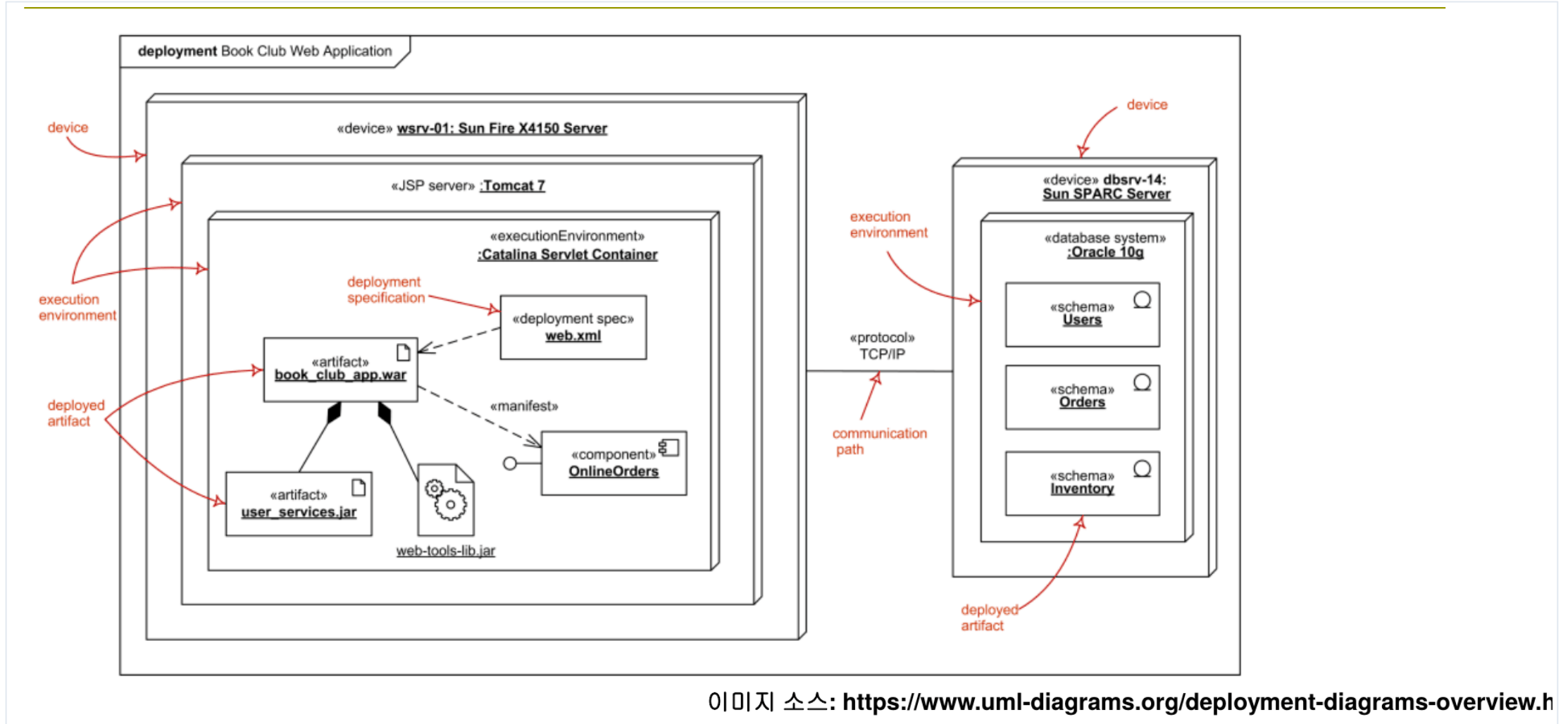
- 장치(device) 또는 소프트웨어 실행 환경(execution environment)

■ Artifact

- 개발 과정의 결과물로 나오는, 물리 세계의 구체적 요소
- 예: 실행 파일, 라이브러리, 아카이브(jar/war/ear), DB 스키마, 설정 파일 등

핵심 Deployment Diagram 은 SEI 의 Deployment Style 을 표현하는 표준 UML 표기법이다

Deployment Diagram 예제



Book Club Web Application 의 배치 다이어그램 (출처: uml-diagrams.org)

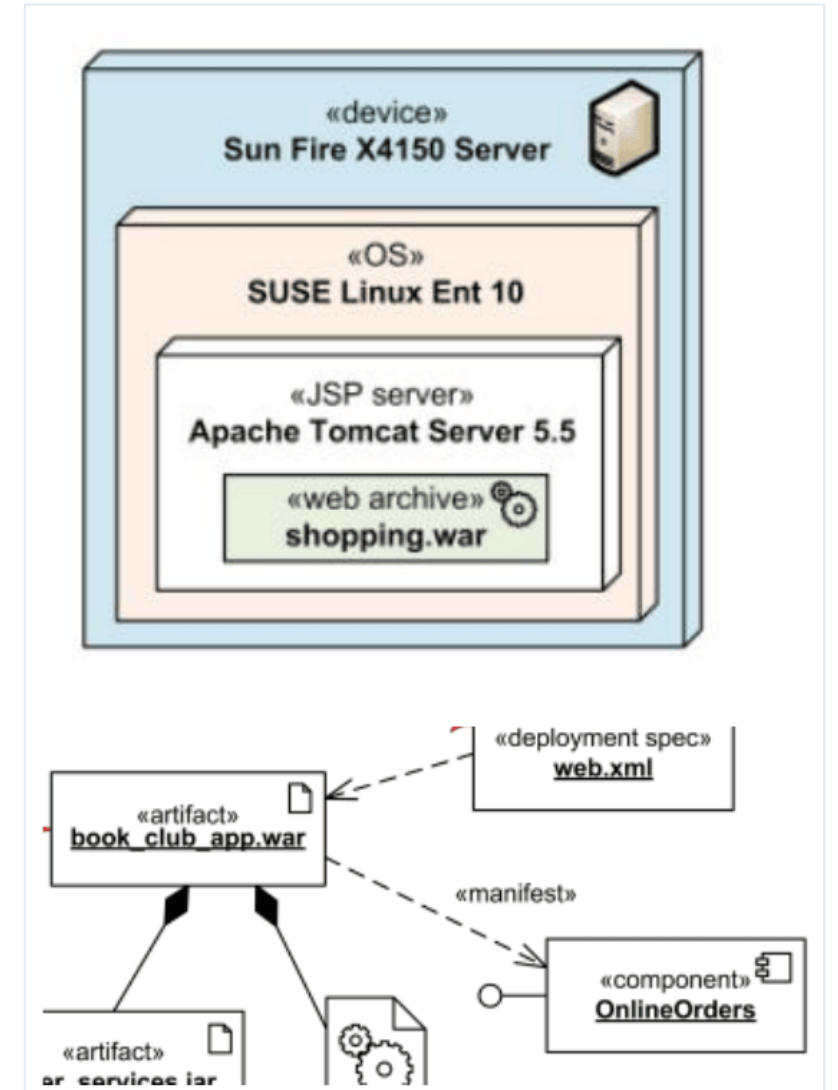
Deployment Diagram 표기법 — Node 와 Artifact

■ Node

- 3차원 상자 모양으로 그린다
- 서버 장치 안에 여러 소프트웨어 실행 환경을 중첩할 수 있다
 - 예: Sun Fire 서버 안에 OS → JSP 서버 → web archive

■ Artifact

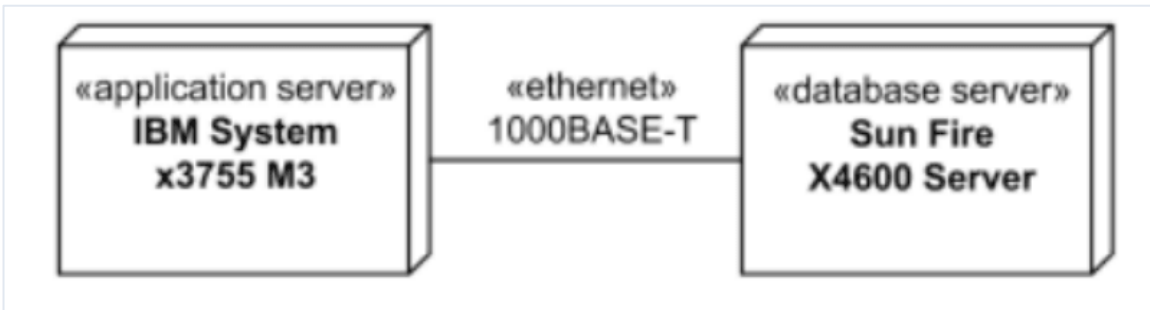
- Artifact 이름과 문서 아이콘이 있는 사각형으로 표시
- <<manifest>> 관계: 물리적 artifact 가 논리적 component 에 어떻게 대응되는지 나타낸다
 - 예: book_club_app.war 가 OnlineOrders component 를 manifest



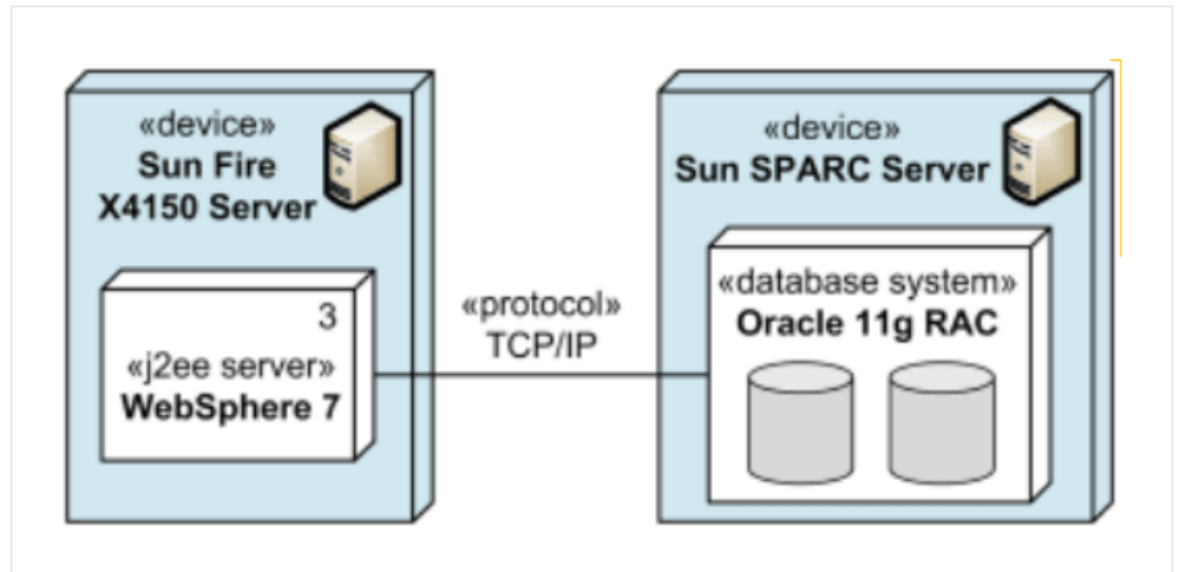
Deployment Diagram 표기법 — Communication Path

■ Communication Path(통신 경로)가 나타내는 것

- 물리 장치 node 사이의 물리적 링크 (예: ethernet 1000BASE-T)
- 소프트웨어 실행 환경 node 사이의 프로토콜 (예: TCP/IP)



장치 간 물리적 링크

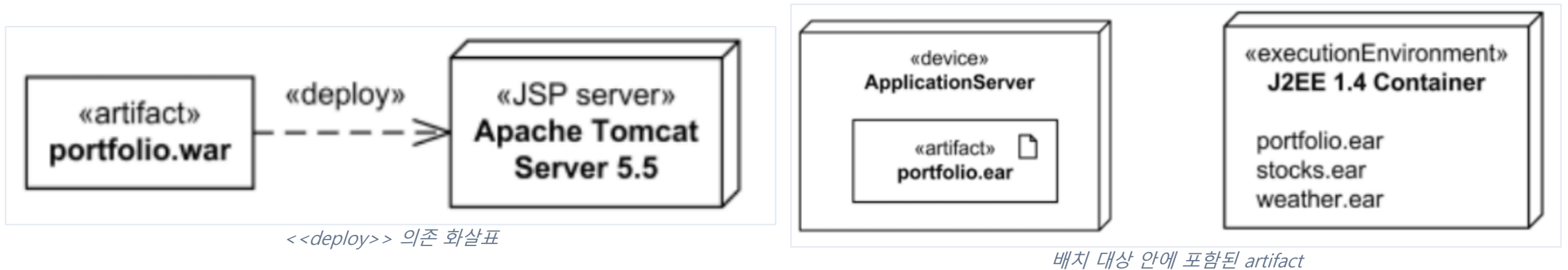


실행 환경 간 프로토콜

Deployment Diagram 표기법 — Deployment 관계

■ Deployment 는 의존 관계(dependency)

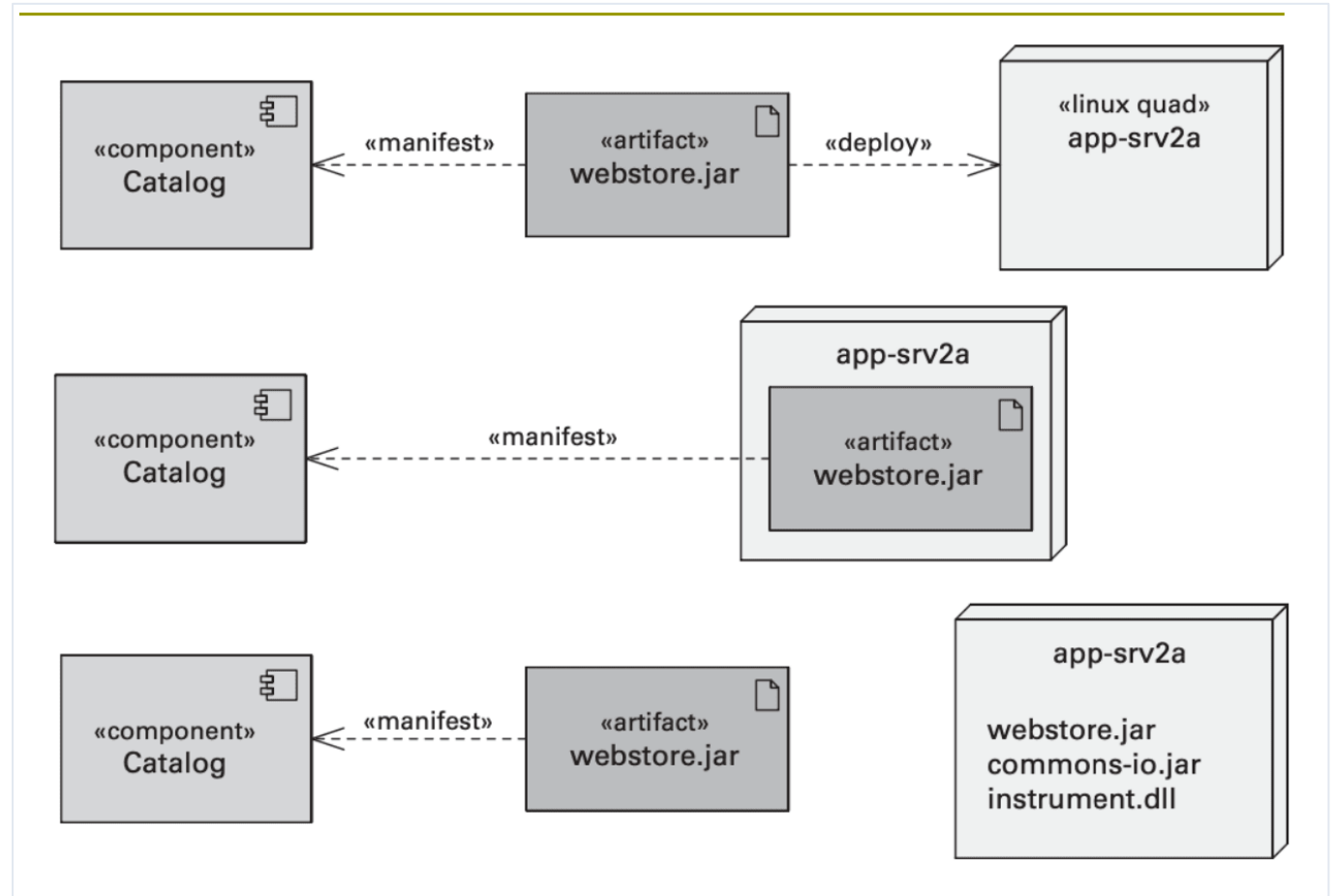
- Artifact 가 배치 대상(deployment target)에 할당되는 것을 나타낸다
- 표현 방법 1: artifact 에서 배치 대상으로 향하는 <<deploy>> 의존 화살표
- 표현 방법 2: 배치 대상 안에 artifact 를 포함시켜 그린다



Deployment Diagram 표기법 — 세 가지 UML 대안

■ 같은 배치를 표현하는 세 가지 방법

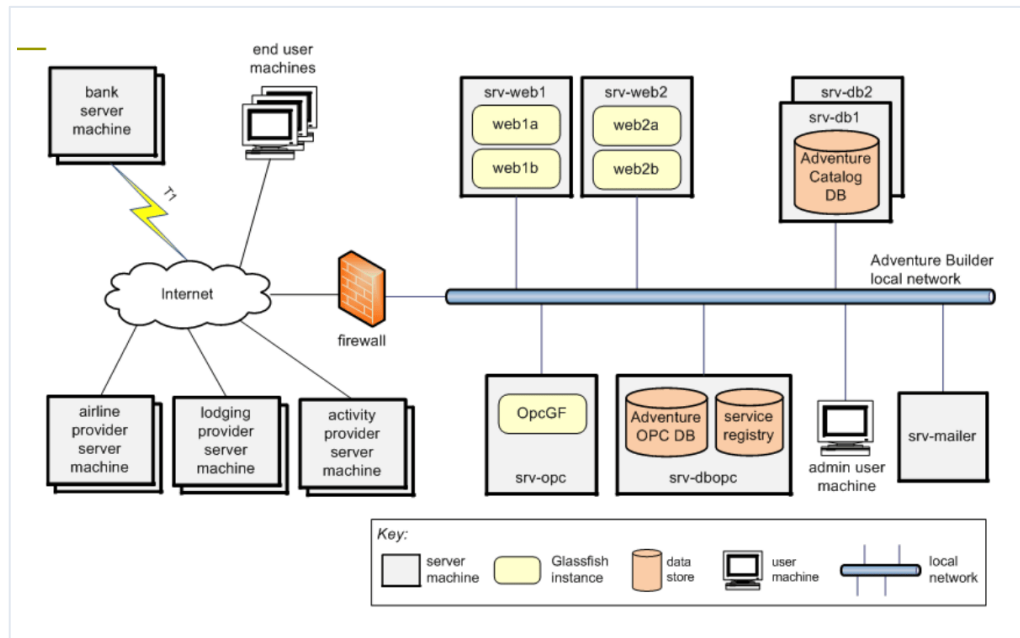
- (위) component → artifact <<manifest>>, artifact → node <<deploy>> 화살표
- (가운데) node 안에 artifact 를 포함시키고 component 와 <<manifest>> 로 연결
- (아래) node 안에 artifact 이름만 텍스트로 나열
- 상황에 맞게 가장 읽기 쉬운 형태를 고른다



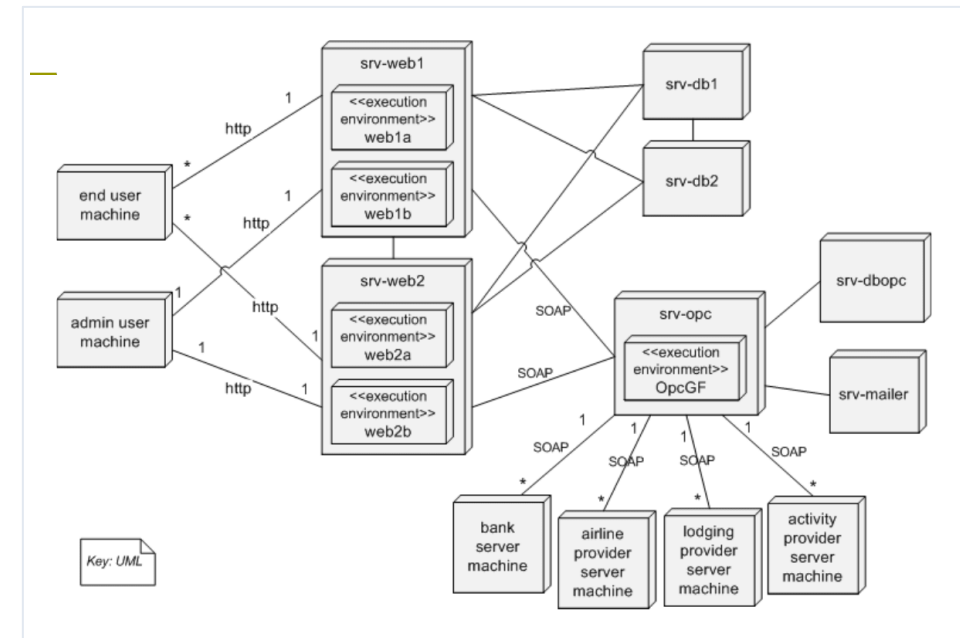
Catalog component 와 webstore.jar 를 app-srv2a 에 배치하는 세 가지 표현

Deployment Diagram 예제 — Adventure Builder

- SEI 의 Adventure Builder 예제: 사용자 PC 가 Internet 과 firewall 을 거쳐 local network 의 웹/애플리케이션/DB 서버에 접근한다
- 왼쪽은 비형식 표기(장치 그림 + Glassfish 인스턴스), 오른쪽은 UML 표기(execution environment + http/SOAP 통신 경로) (출처: wiki.sei.cmu.edu > SAD > Deployment View)



비형식 표기



UML 표기

Install Style — 정의, Elements, Relations

■ 정의

- Architecture 의 component 를 운영(production) 환경의 파일 관리 시스템에 매핑한다

■ Elements

- Software element: C&C component
 - 필요(required) 속성: 운영 환경에 대한 요구 (예: 특정 버전의 Java 런타임)
- Environment element: 파일이나 폴더 같은 configuration item(형상 항목)
 - 제공(provided) 속성: 운영 환경이 제공하는 특성 (예: 형상 관리 시스템의 이력.브랜치 처리 방식)

■ Relations

- Allocated-to: component 가 configuration item 에 할당된다
- Containment: configuration item 이 다른 item 에 포함된다

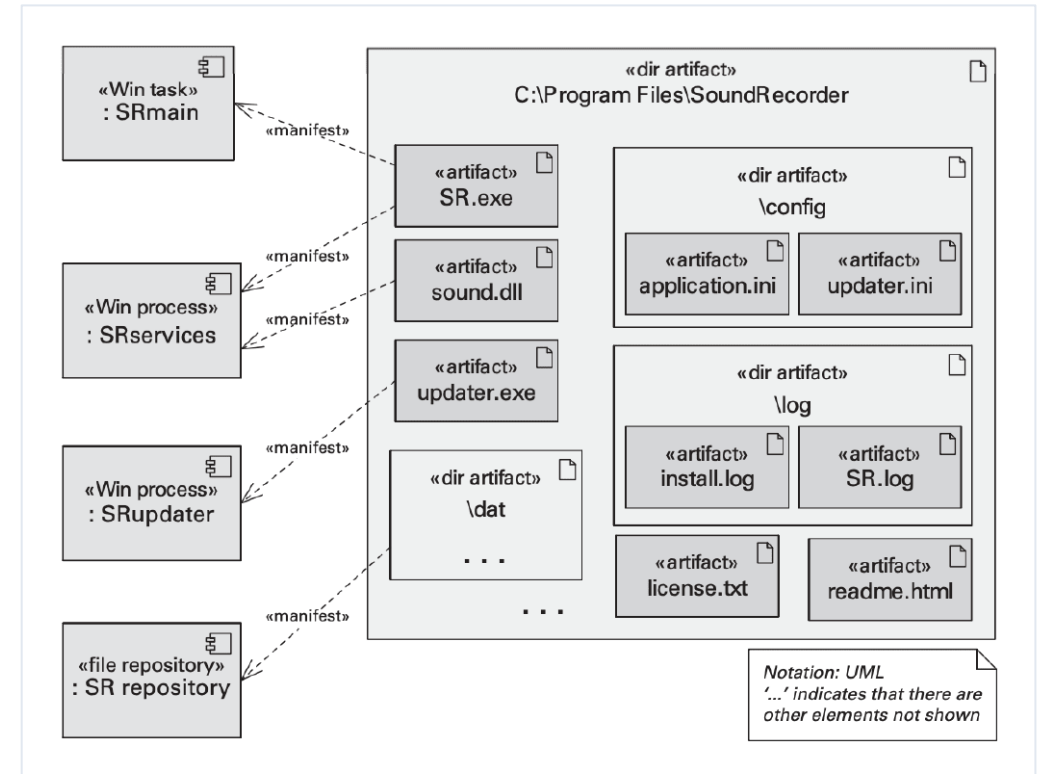
■ Constraints

- 파일과 폴더는 is-contained-in 관계를 따르는 트리 구조로 구성된다

Install Style 의 표기법 (UML)

■ UML stereotype 활용

- <<artifact>>: 파일(configuration item)을 나타낸다
- <<manifest>>: artifact 가 소프트웨어 요소의 구체적인 물리적 표현을 담고 있음을 나타낸다
- <<dir artifact>>: 폴더(디렉터리)를 나타내며, 안에 파일 artifact 를 포함시켜 트리 구조를 표현한다
- 예: SRmain 프로세스 → SR.exe, SRservices → sound.dll 등이 C:\Program Files\SoundRecorder 아래에 설치된다

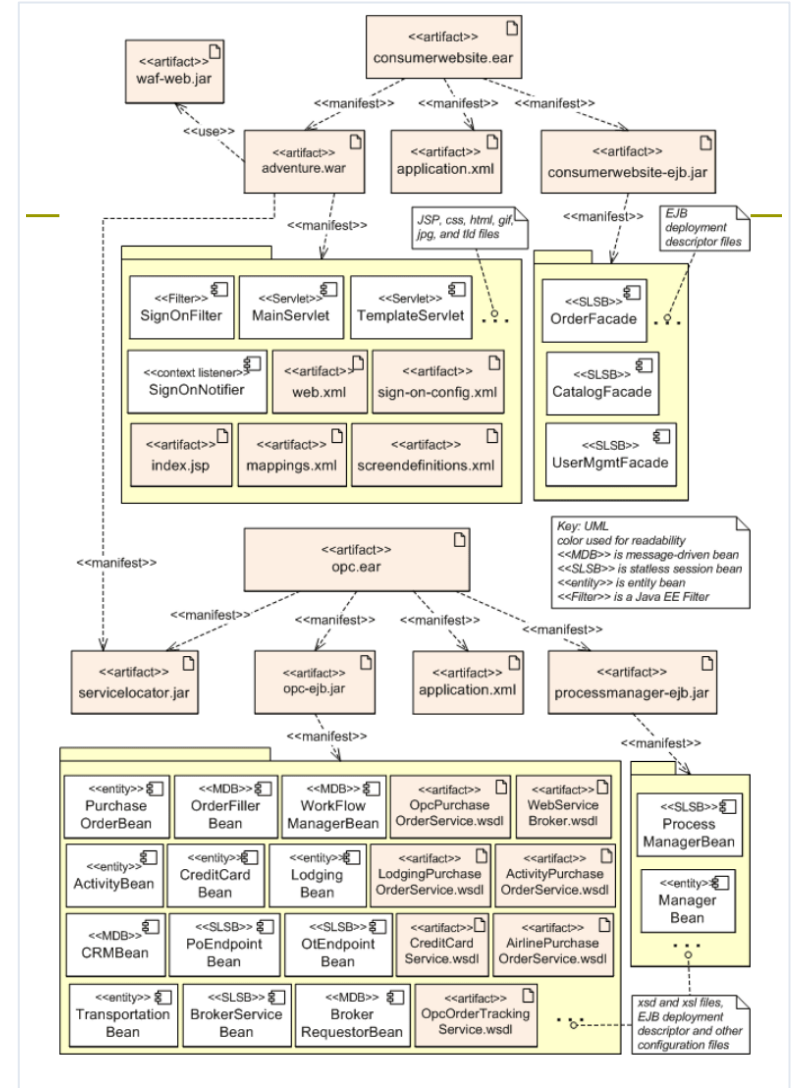


SoundRecorder 의 Install View (출처: Documenting Software Architectures, 2nd Ed.)

Install Style 예제 — Adventure Builder

■ Adventure Builder 의 Install View

- ear/war/jar 등 배포 파일을 <<artifact>> 로 표현한다
- <<manifest>> 로 각 artifact 가 어떤 component(Bean, Servlet 등)를 담는지, 그리고 포함 관계를 나타낸다
- 예: opc.ear 안에 여러 jar 가 들어가고, 각 jar 는 Purchase Order, Broker 같은 Bean 을 manifest 한다
- 출처: wiki.sei.cmu.edu > SAD > Install View



Work Assignment Style

■ 정의

- 소프트웨어 Architecture 를 개발 조직의 팀에 매핑한다

■ Elements

- Software element: module
- Environment element: 조직 단위 — 사람, 팀, 부서 등
 - 제공(provided) 속성: 보유 기술(skill set), 인력과 일정 측면의 수용 능력(capacity)

■ Relations

- Allocated-to: module 이 조직 단위에 할당된다

■ Constraints

- 보통 하나의 module 은 하나의 조직 단위에만 할당하도록 제한한다

핵심 누가 어떤 module 을 만들지 정하는 뷰이므로, 팀의 역량·인력·일정을 module 에 맞추어 계획할 때 쓴다

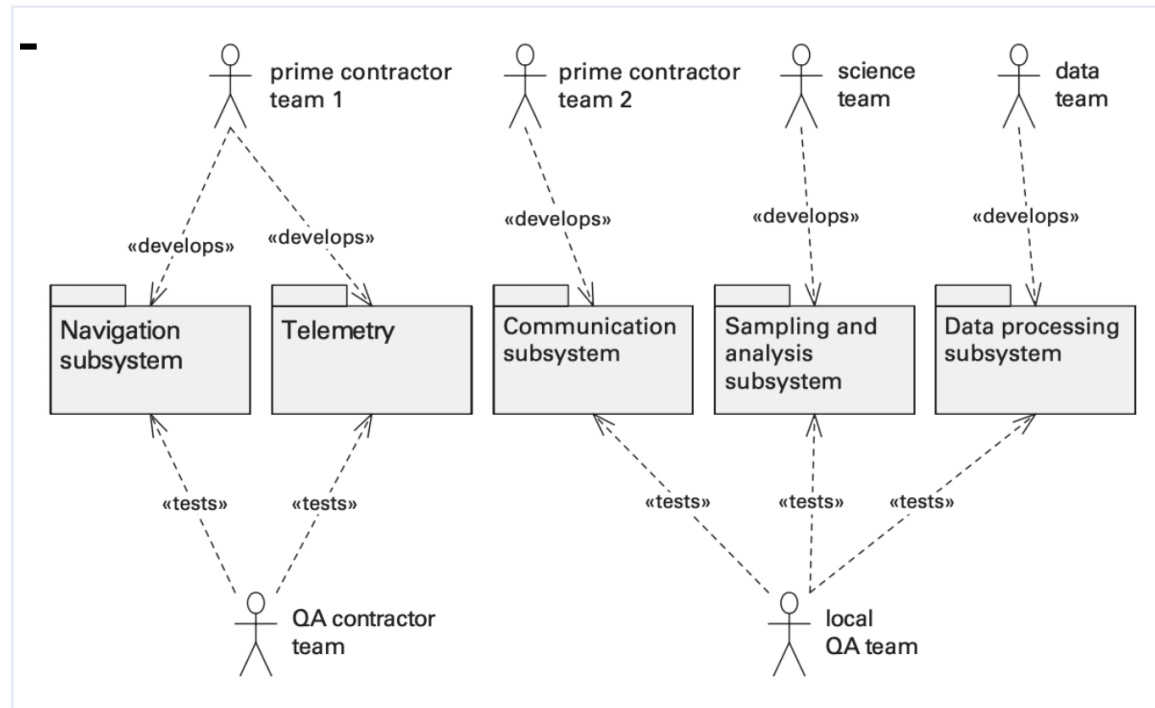
Work Assignment Style 의 표기법 — 표 형식

- 가장 흔한 표기는 표(tabular notation): module(segment/subsystem)과 담당 조직 단위를 한 행씩 대응시킨다

Segment	Subsystem (Module)	Organizational Unit
Science Data Processing Segment	Client	Science team
	Interoperability	Prime contractor team 1
	Ingest	Prime contractor team 2
	Data Management	Data team
	Data Processing	Data team
	Data Server	Data team
	Planning	Orbital vehicle team
Flight Operations Segment	Planning and Scheduling	Orbital vehicle team
	...	

Work Assignment Style 의 표기법 — UML

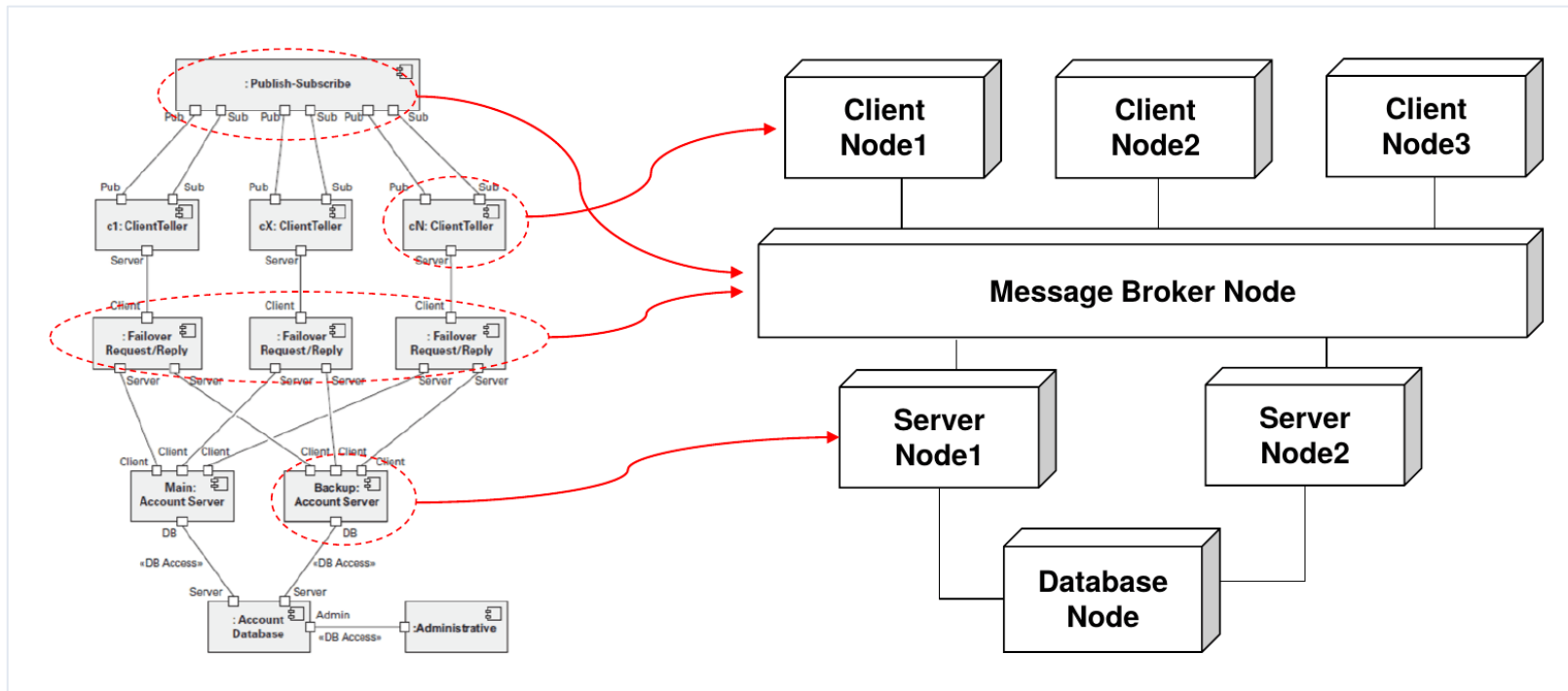
- UML 로는 조직 단위를 actor 로, module 을 package/subsystem 으로 그리고 <<develops>>, <<tests>> 같은 stereotype 의존 관계로 할당을 표현한다
- 예: prime contractor team 1 이 Navigation subsystem 을 develops, QA contractor team 이 tests



UML 표기의 Work Assignment View

C&C View 와 Allocation View 의 관계

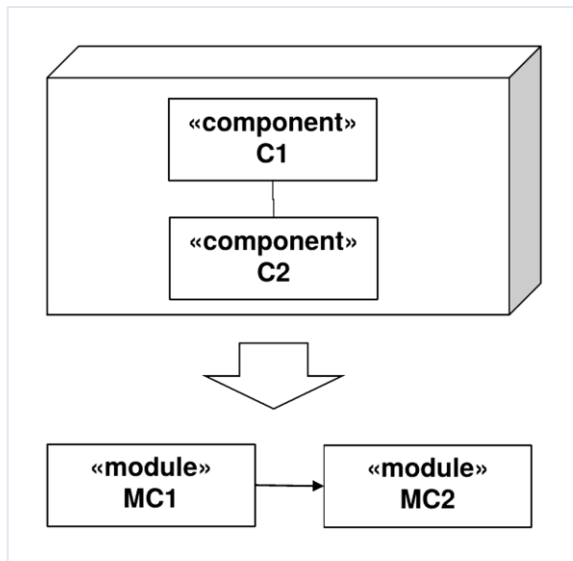
- C&C View 의 component 는 Allocation View 에서 컴퓨팅 플랫폼의 하드웨어 node 에 매핑된다
- 예: 클라이언트 component 들은 Client Node 1~3 에, 서버 component 들은 Message Broker Node 와 Server Node 에, DB 는 Database Node 에 배치된다



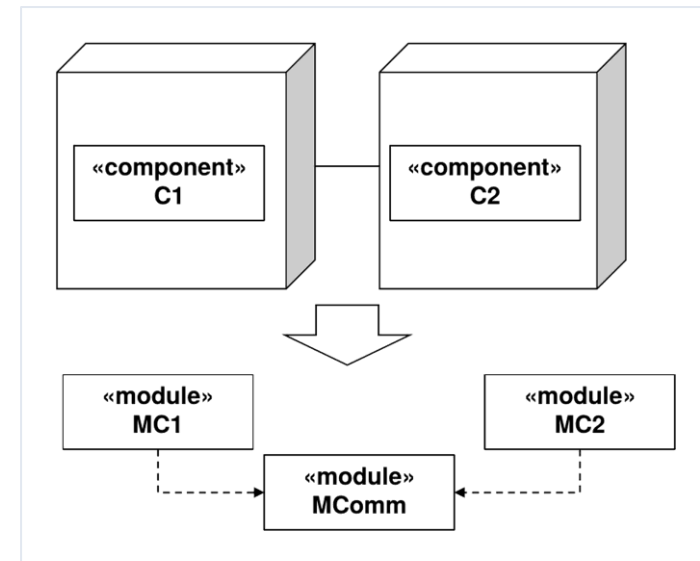
C&C View(왼쪽)의 component 가 하드웨어 node(오른쪽)에 할당되는 모습

C&C, Module, Allocation View 의 관계

- C&C component 를 하드웨어 node 에 어떻게 배치하느냐에 따라 module 설계가 달라진다
- 왼쪽: C1, C2 가 같은 node 에 있으면 module MC1 → MC2 직접 의존으로 충분하다
- 오른쪽: C1, C2 가 서로 다른 node 에 있으면 통신을 담당하는 module(MComm)이 추가로 필요하다



같은 node 에 배치



다른 node 에 배치 → MComm 추가

핵심 세 View 는 독립적이지만 서로 영향을 주므로, 배치 결정이 바뀌면 module 구조도 함께 검토해야 한다

Allocation View 요약

■ Deployment Style

- 런타임 소프트웨어 요소를 그것이 실행되는 하드웨어에 매핑한다

■ Install Style

- Architecture 의 component 를 운영 환경의 파일 시스템에 매핑한다

■ Work Assignment Style

- 소프트웨어 Architecture 를 개발 조직의 팀에 매핑한다

핵심 Allocation View 의 스타일들은 소프트웨어 구조를 소프트웨어 환경(하드웨어, 파일 시스템, 조직)의 구조에 매핑한다

2.3

Hierarchical Multi-View & C4 Model

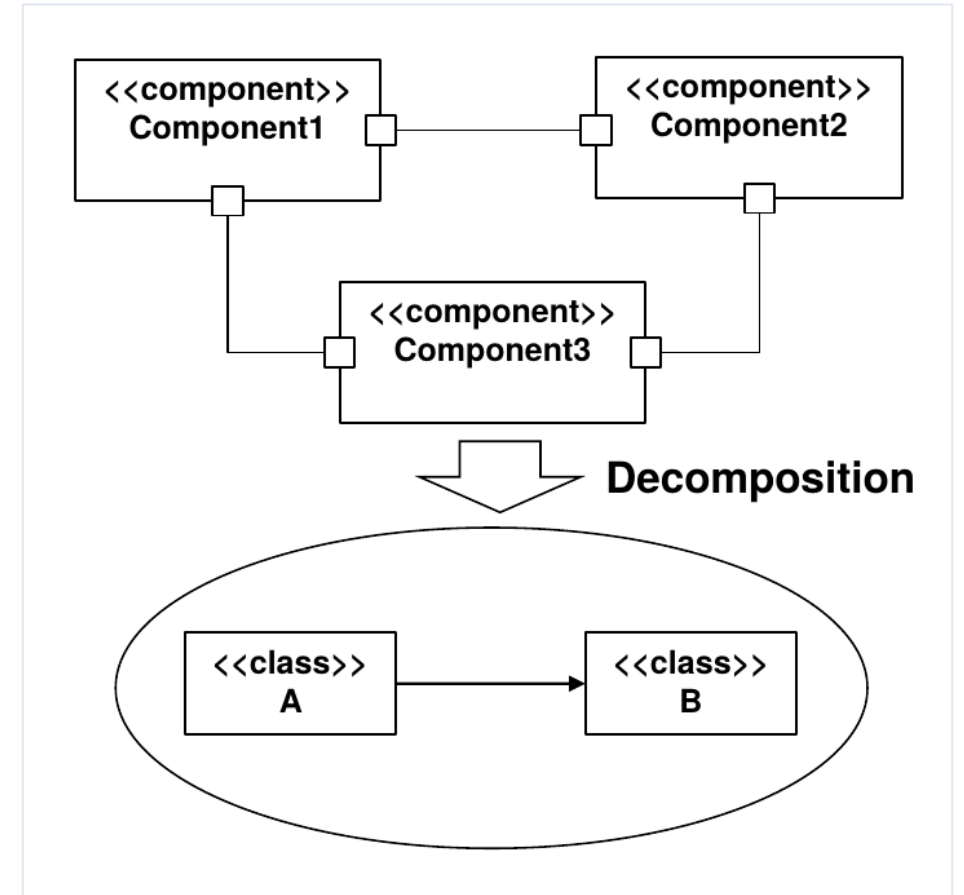
설계 단계별 계층적 분해(Hierarchical Multi-View)와 줌인/줌아웃(C4)으로 Architecture 를 문서화하는 두 가지 모델

- Hierarchical Multi-View Model: Top Level / Component Level Design
- Structure View · Behavior View · Deployment View
- C4 Model: Context · Container · Component · Code
- C4 Model 표기법과 Supporting Diagram
- SEI View Model · Hierarchical Multi-View · C4 Model 비교

Architectural Design — 반복적 분해

■ 시스템 구조는 반복적으로 분해된다

- Top Level Design: 시스템을 최상위 component 들로 나눈다 (Component1, 2, 3)
- Component Level Design: 각 component 를 다시 class 수준의 세부 요소(A, B)로 분해한다
- 각 설계 단계마다 Structure/Behavior/Deployment 관점의 뷰를 작성한다

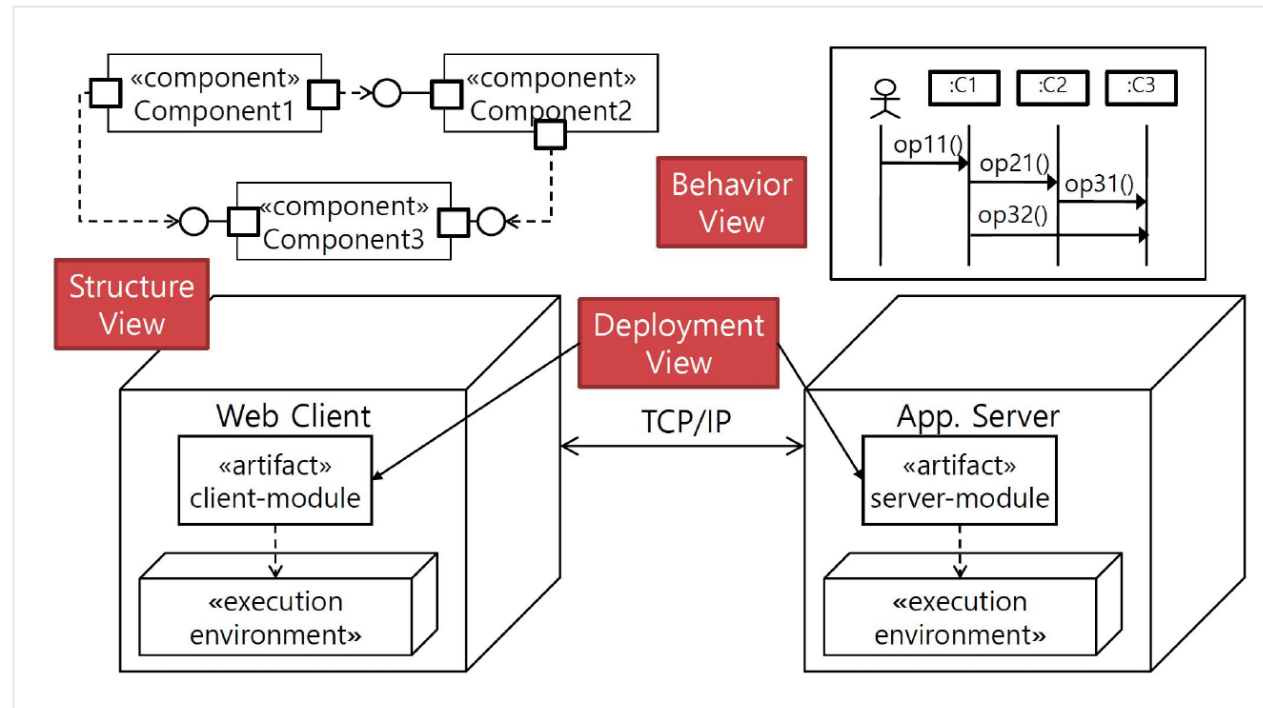


Top Level Design → Decomposition → Component Level Design

핵심 Hierarchical Multi-View Model 은 설계 단계별로 점진적으로 분해하는 과정을 강조한다

Top Level Design

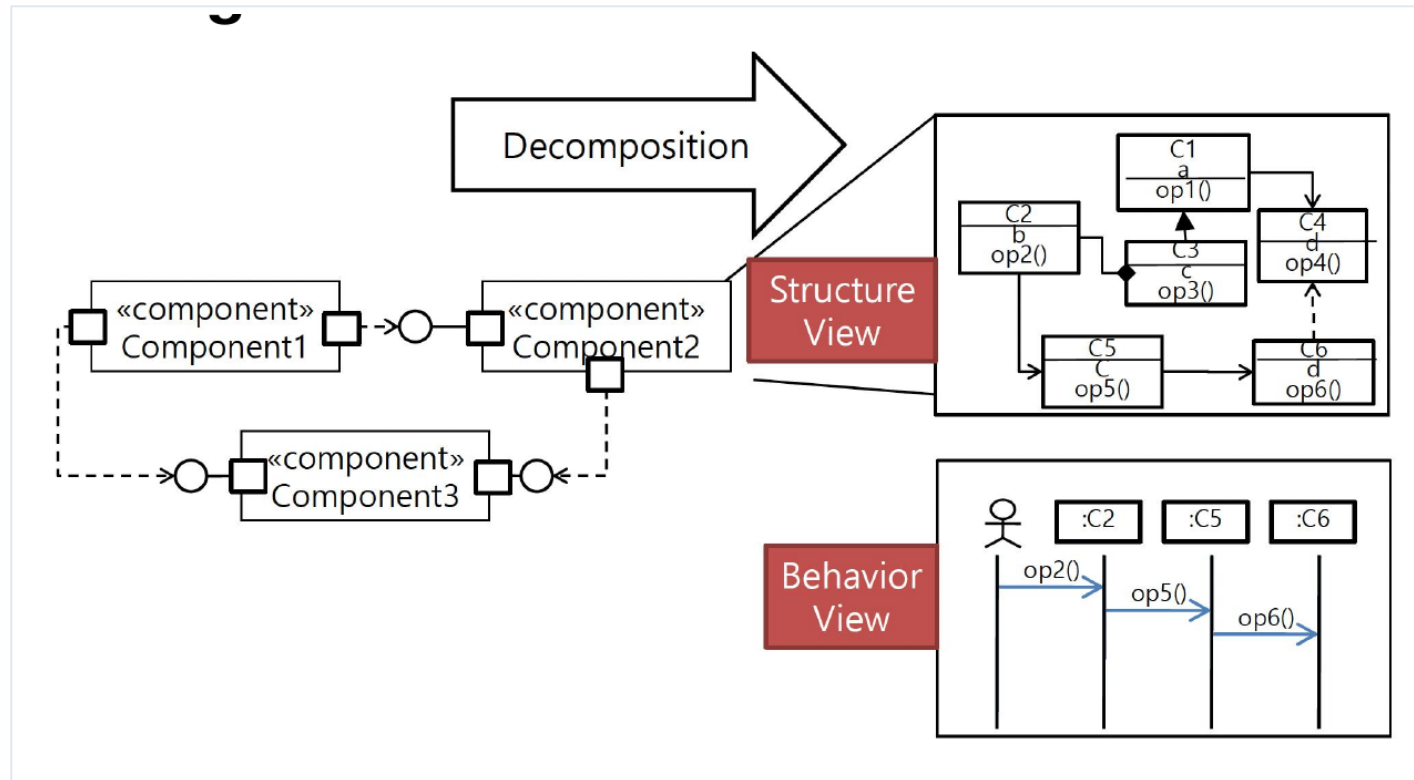
- 시스템의 첫 번째 수준 분해: 시스템 전체를 주요 component 로 나누고 그 관계를 정한다
- Structure View(component diagram), Behavior View(sequence diagram), Deployment View(deployment diagram)를 함께 작성한다



Top Level Design 의 Structure / Behavior / Deployment View

Component Level Design

- Top Level Design 의 각 component 를 더 세밀한 요소(class 등)로 분해한다
- component 내부의 Structure View(class diagram)와 Behavior View(sequence diagram)를 작성한다



Component2 를 class 수준으로 분해한 Component Level Design

Architecture 문서화를 위한 View 구성

- 설계 단계(Top Level / Component Level)와 관점(Structure / Behavior / Deployment)을 조합해 문서를 구성한다

View	Top Level Design	Component Level Design
Structure View	Component Diagram, Component Specification	Class Diagram
Behavior View	Interaction Diagram	Interaction Diagram
Deployment View	Deployment Diagram	—

핵심 Deployment View 는 시스템 전체 배치를 다루므로 Top Level Design 에서만 작성한다

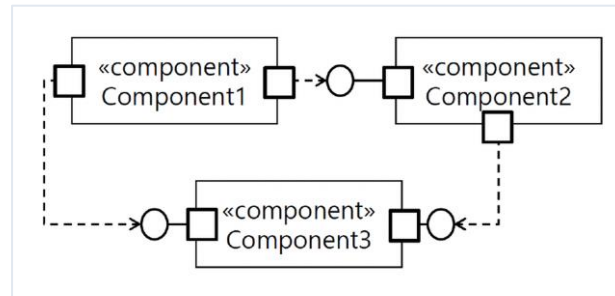
Structure View

■ UML Component Diagram (정적 구조)

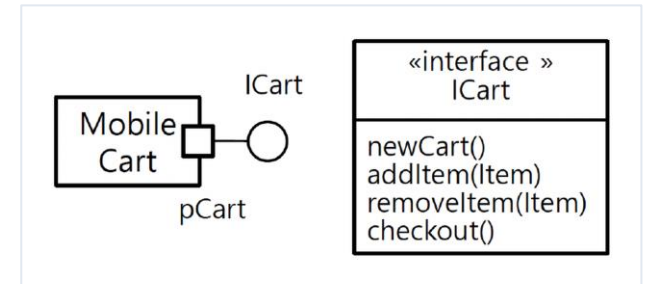
- component 와 그 사이의 제공/요구 interface 연결을 표현한다

■ Component Specification

- Port: component 가 외부와 상호작용하는 지점 (예: pCart)
- Interface Specification: port 가 제공하는 interface 의 operation 정의 (예: ICart 의 newCart, addItem, removeItem, checkout)



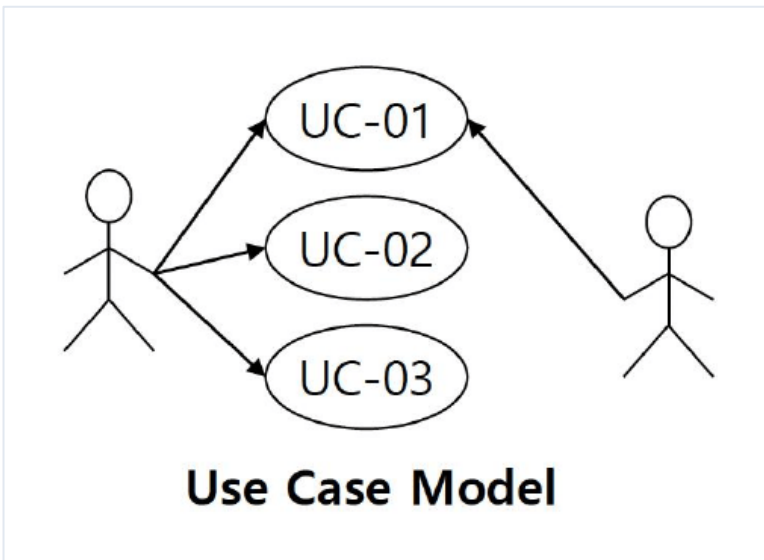
Component Diagram



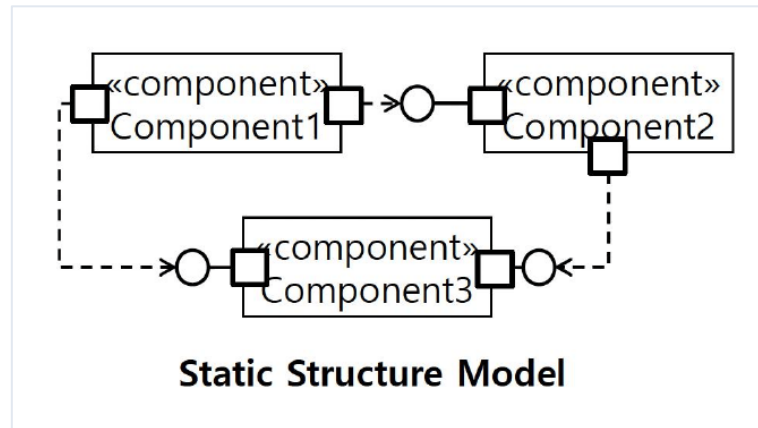
Port 와 Interface Specification

Behavior View — Use Case 별 상호작용

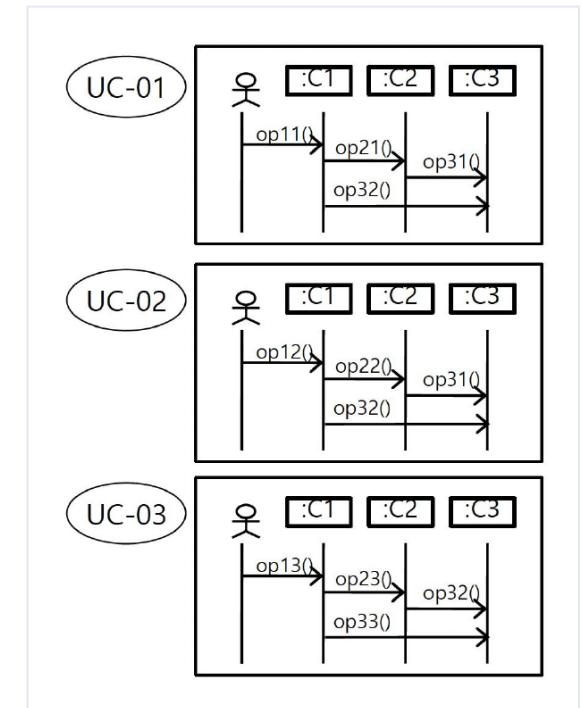
- 각 use case 마다 component 들 사이의 상호작용을 분석한다
- Use Case Model(UC-01~03)과 Static Structure Model(component 구조)을 바탕으로, use case 별 sequence diagram 을 그린다



Use Case Model



Static Structure Model

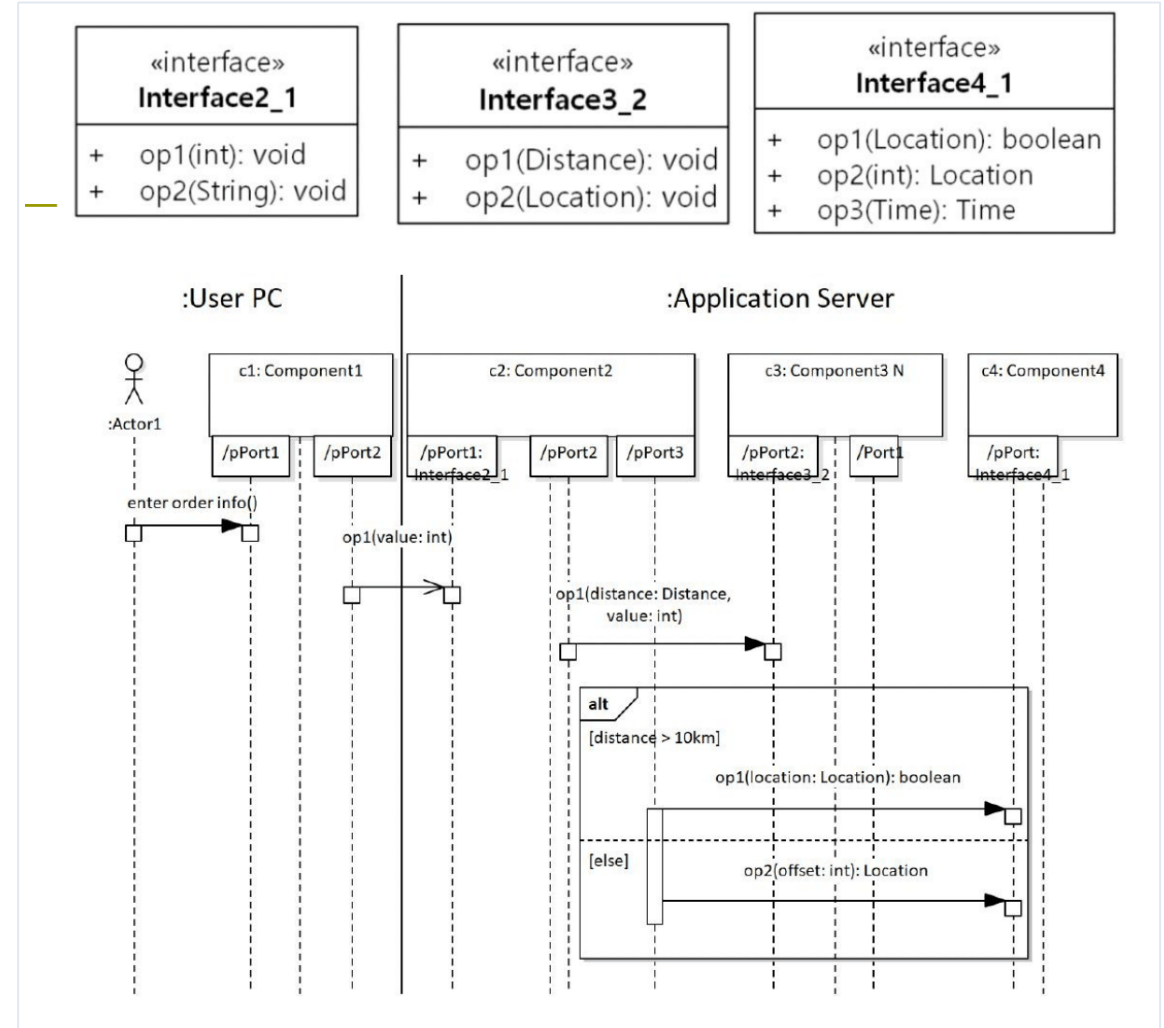


Use case 별 Interaction Diagram

Behavior View — Port 를 통한 상호작용

■ Component instance 사이의 상호작용

- component instance 들이 port 를 통해 주고받는 메시지를 sequence diagram 으로 기술한다
- 각 port 의 interface(Interface2_1, 3_2, 4_1)에 정의된 operation 이 메시지가 된다
- 실행 node(:User PC, :Application Server)를 함께 표시하면 배치와의 관계도 드러난다

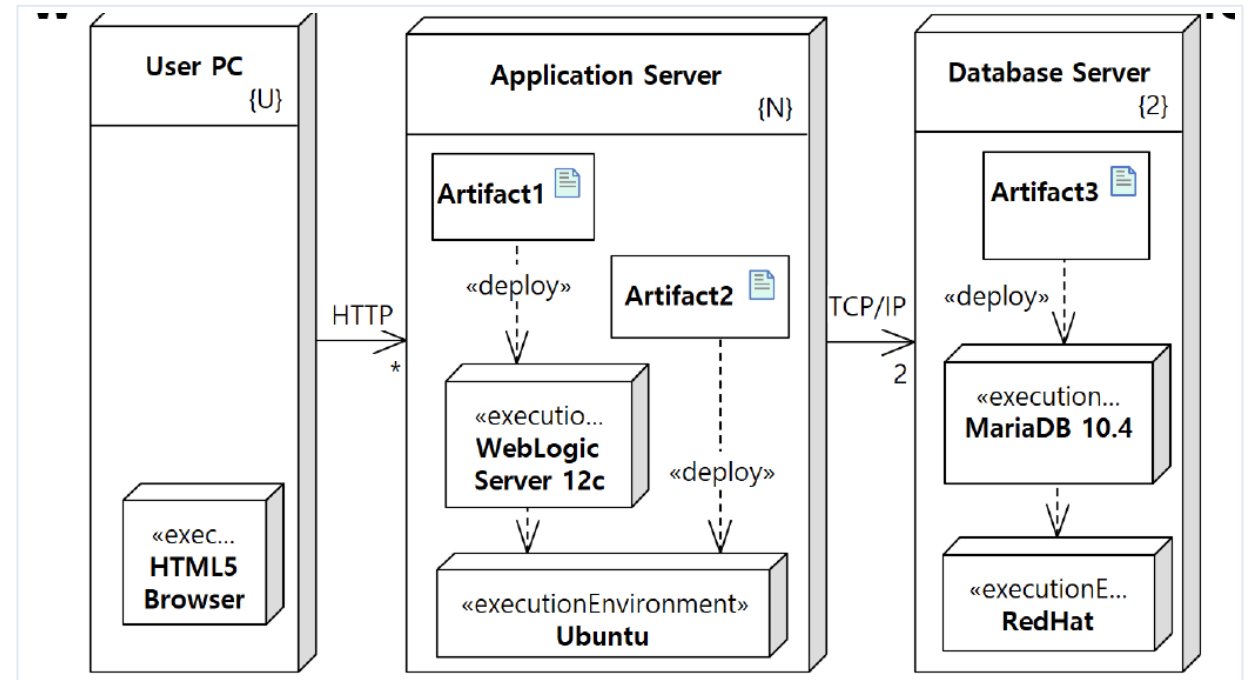


Port 와 interface 를 명시한 sequence diagram

Deployment View

■ Deployment Diagram

- 핵심 infrastructure 요소(node)와 통신 경로(HTTP, TCP/IP)를 보여 준다
- 소프트웨어 artifact 가 배치 대상(node)에 어떻게 배치되는지 보여 준다
- 예: Artifact1, 2 는 Application Server 의 WebLogic/Ubuntu 에, Artifact3 은 Database Server 의 MariaDB 에 배치



User PC – Application Server – Database Server 배치

SEI View Model vs. Hierarchical Multi-View Model

	Hierarchical Multi-View Model	SEI View Model
주요 초점	설계 단계별로 점진적인 분해 과정을 강조	이해관계자별 시스템 구조 표현을 강조
View 구성	Top Level Design: Structure View(Component Diagram), Behavior View Component Level Design: Structure View(Class Diagram), Behavior View Deployment Diagram	C&C View: 런타임 구조 (Component Diagram) Module View: 개발 구조 (Class Diagram) Allocation View
추상화 방식	포함 관계 기반의 계층적 분해: 시스템 → 컴포넌트 → 클래스	직교적/독립적(Orthogonal): 세 View 는 서로 다른 관심사를 다루며 동시에 존재한다

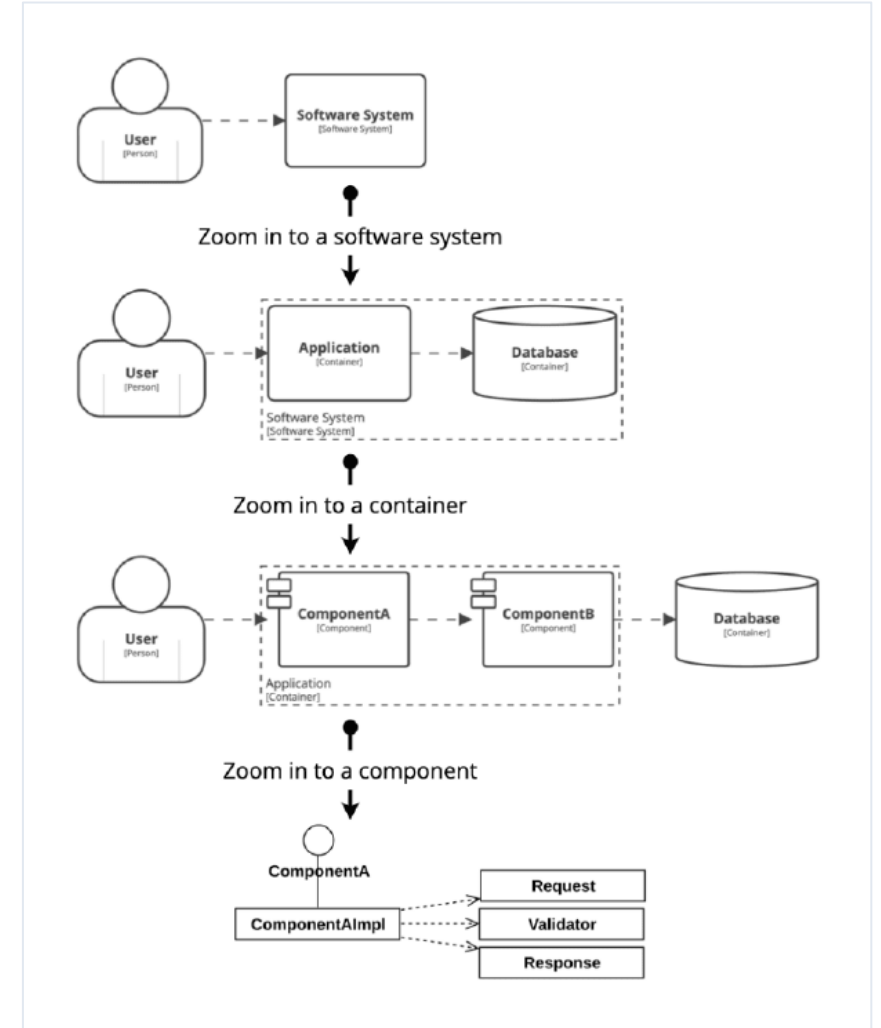
C4 Model 개요

■ C4 Model 이란

- 소프트웨어 Architecture 를 추상화 수준에 따라 4 단계 계층 (Context, Container, Component, Code)으로 문서화하는 경량 모델

■ 특징

- 다양한 이해관계자에게 각자에게 맞는 수준의 정보를 전달한다
- 줌인/줌아웃 방식: 시스템 → 컨테이너 → 컴포넌트 → 코드로 점점 확대해 들어간다
- 단순하고 명확한 표기법



Zoom in: software system → container → component

System Context Diagram

■ 무엇을 표현하는가

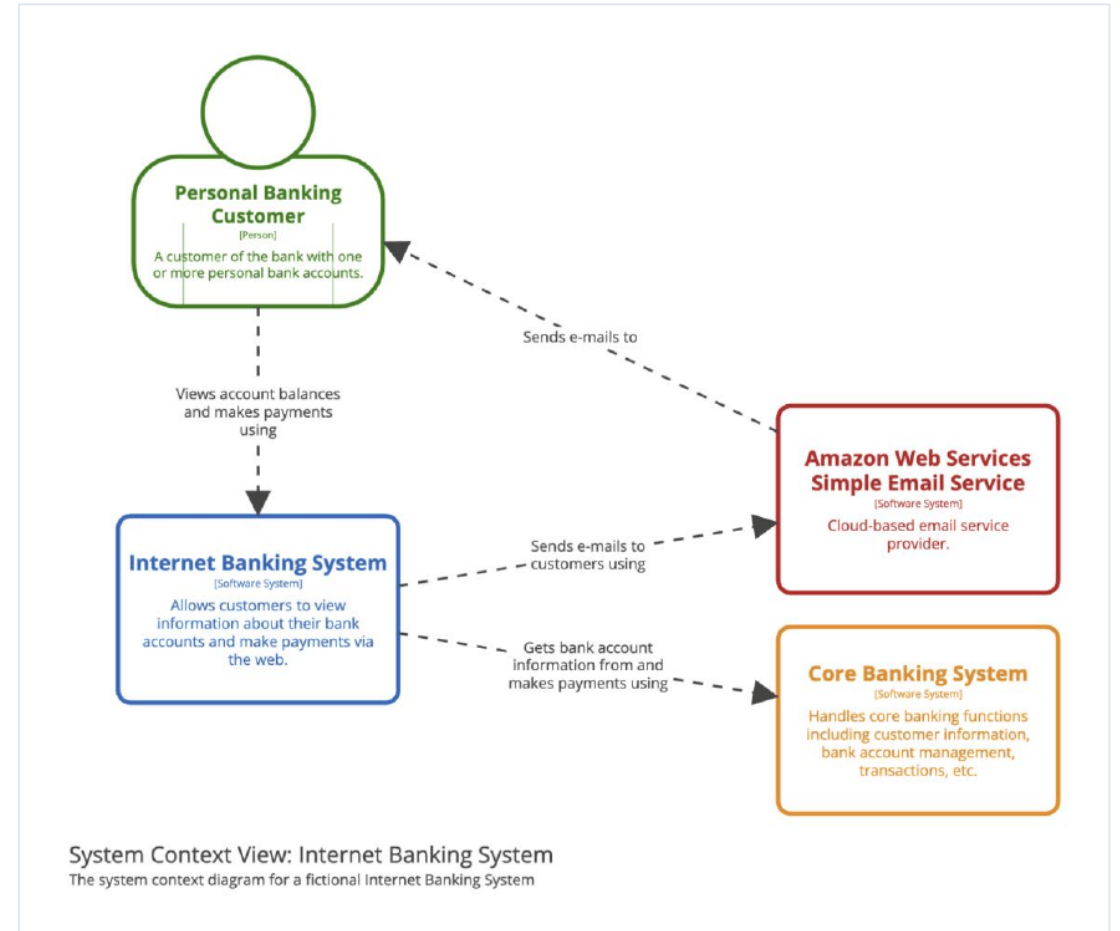
- 소프트웨어 시스템이 외부 개체(사용자, 다른 시스템)와 어떻게 상호작용하는지 표현한다

■ 모델링 요소

- 시스템(Software System)
- 사용자(Person)
- 외부 시스템

■ 이해관계자

- 비즈니스 담당자
- 프로젝트/제품 관리자
- 기술팀 모두



Internet Banking System 의 System Context Diagram

Container Diagram

■ 무엇을 표현하는가

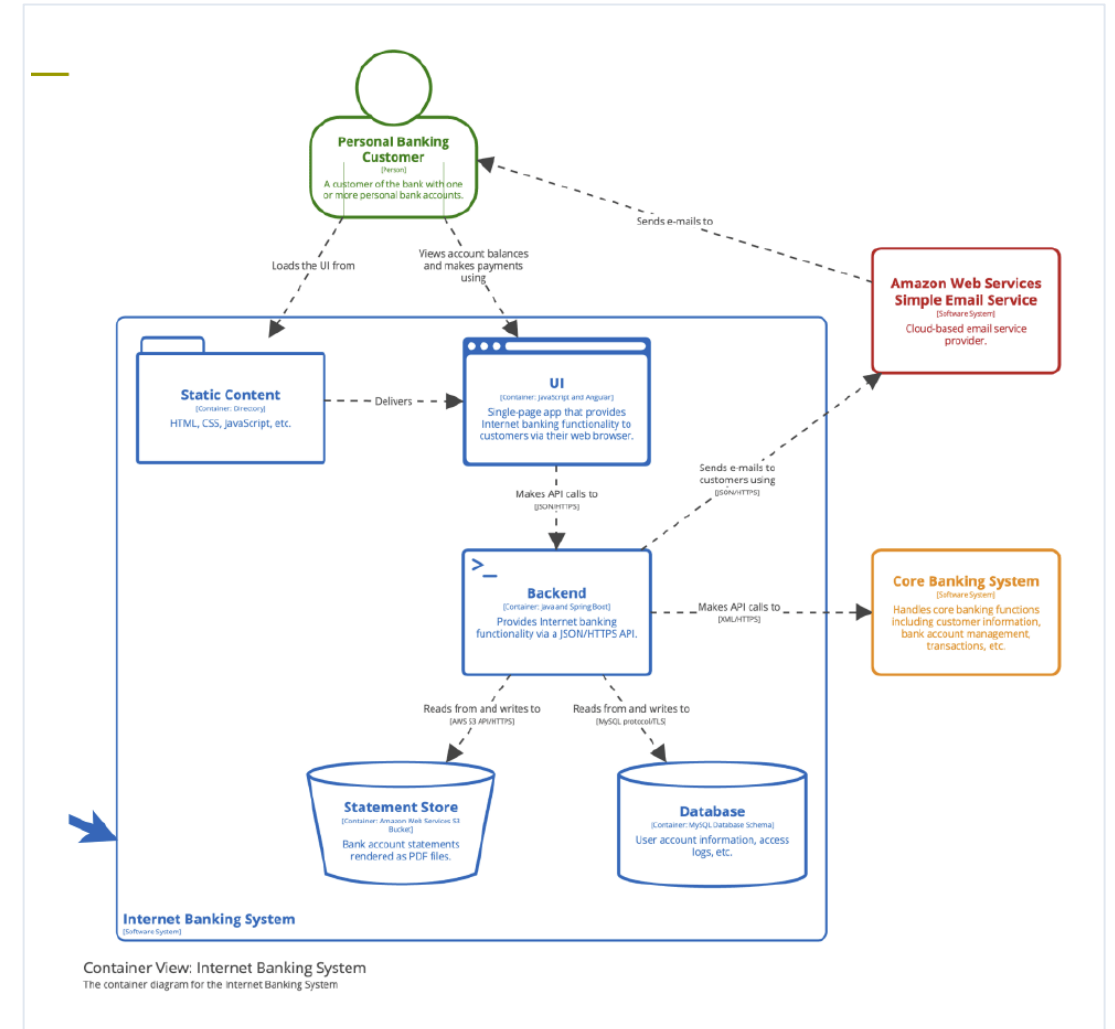
- 시스템 내부로 'zoom in'하여 시스템을 구성하는 주요 building block 들을 표현한다

■ 모델링 요소

- Container: 독립적으로 배포/실행 가능한 단위 (예: 애플리케이션, 데이터베이스)
- 사용자
- 외부 시스템

■ 이해관계자

- 개발팀, Architect, 운영팀



Internet Banking System 의 Container Diagram

Component Diagram

■ 무엇을 표현하는가

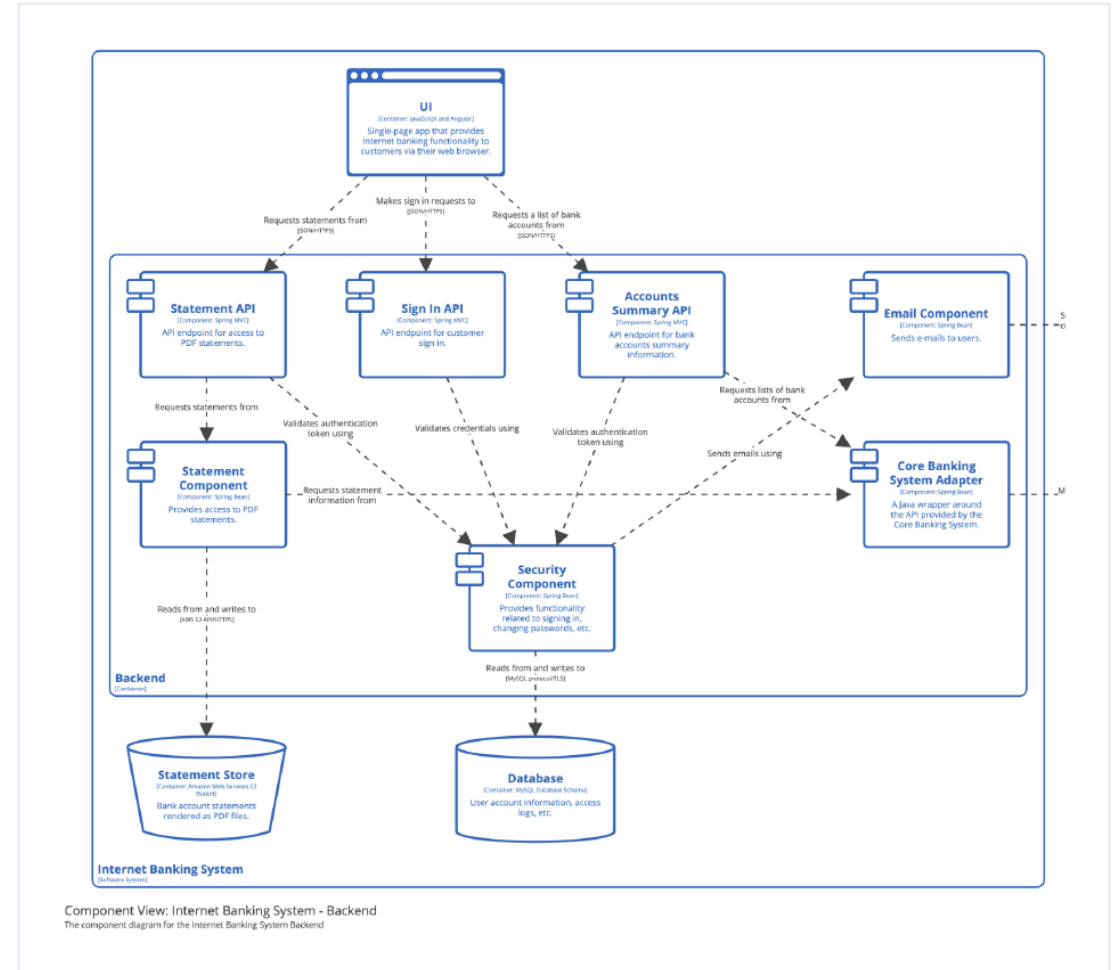
- 특정 container 내부로 '쥬인'하여 container 를 구성하는 component 들의 관계를 표현한다

■ 모델링 요소

- Component: 기능을 제공하는 코드의 논리적 묶음. 독립적으로 배포될 수 없다
- Container

■ 이해관계자

- 핵심 개발팀, Architect



Backend container 의 Component Diagram

Code Diagram

■ 무엇을 표현하는가

- 특정 component 내부로 '쥬인'하여 코드를 구성하는 가장 상세한 구조를 표현한다

■ 모델링 요소

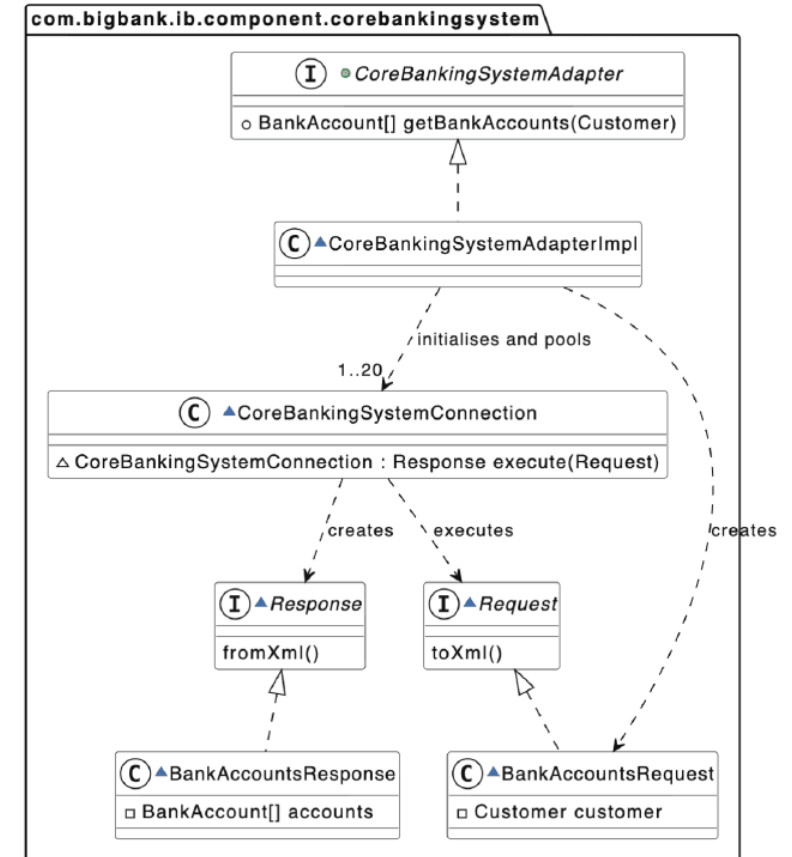
- 클래스, 인터페이스, 함수

■ 이해관계자

- 개발자

■ 참고

- 보통 UML class diagram 형태로 표현하며, 이상적으로는 코드에서 자동 생성한다

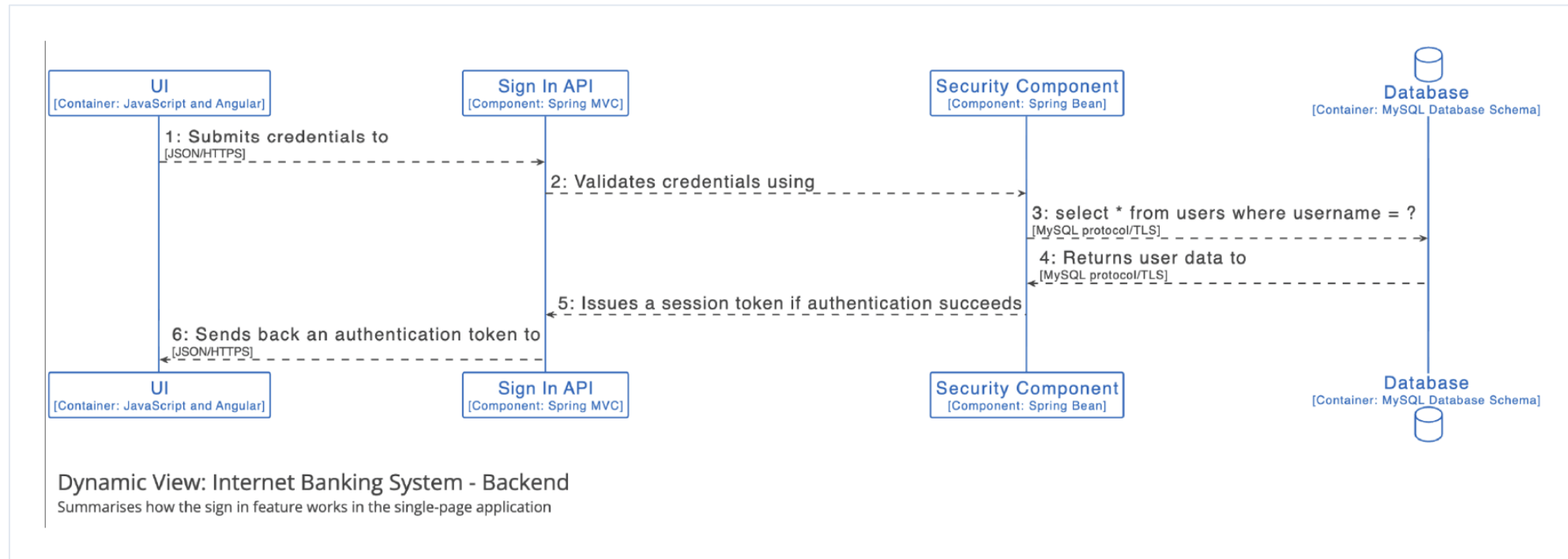


Code View: Internet Banking System - Backend - Core Banking System Adapter
A summary of the implementation details for the Core Banking System Adapter component

Core Banking System Adapter component의 Code Diagram

Supporting Diagram — Dynamic Diagram

- 4 단계 정적 다이어그램만으로 설명하기 어려운 복잡한 상호작용은 dynamic diagram(sequence/collaboration 형태)으로 보충한다
- 예: 로그인(sign in) 기능이 UI → Sign In API → Security Component → Database 순으로 어떻게 동작하는지 요약

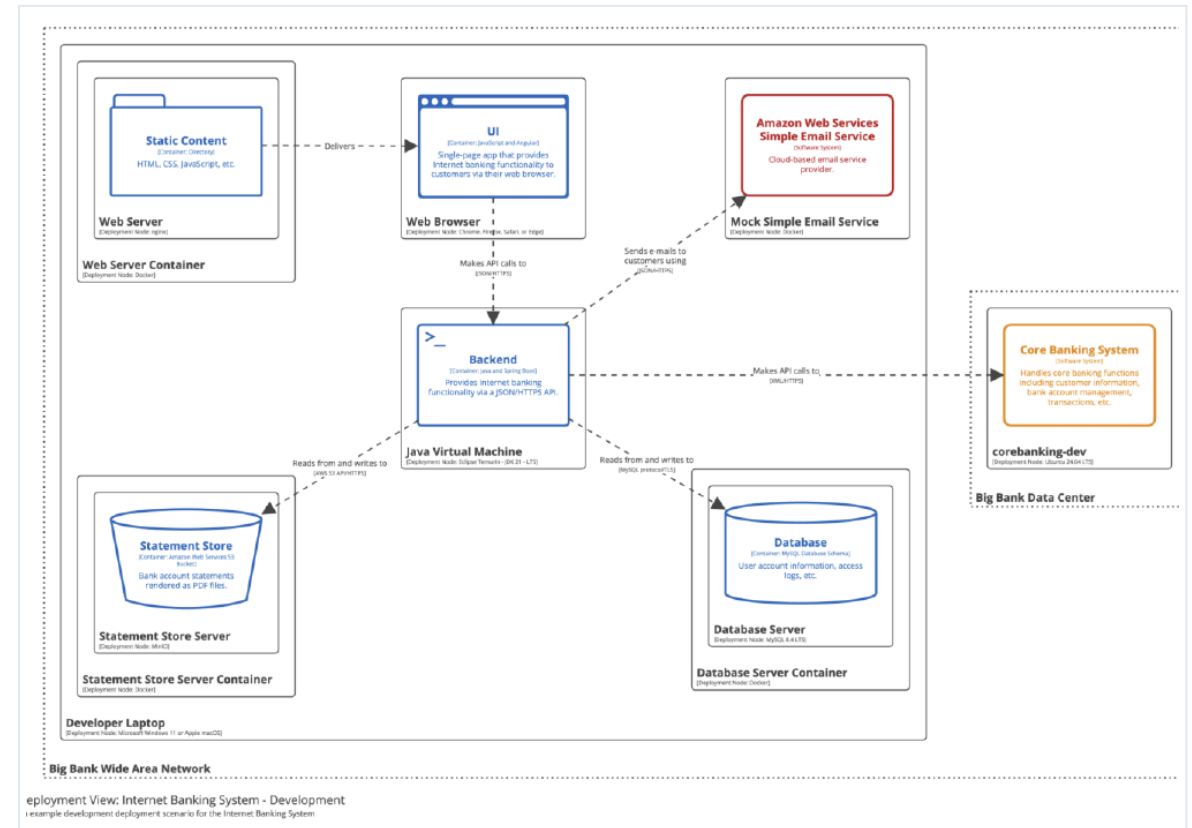


Dynamic View: Internet Banking System – Backend 의 sign in 흐름

Supporting Diagram — Deployment Diagram

■ Deployment Diagram 의 용도

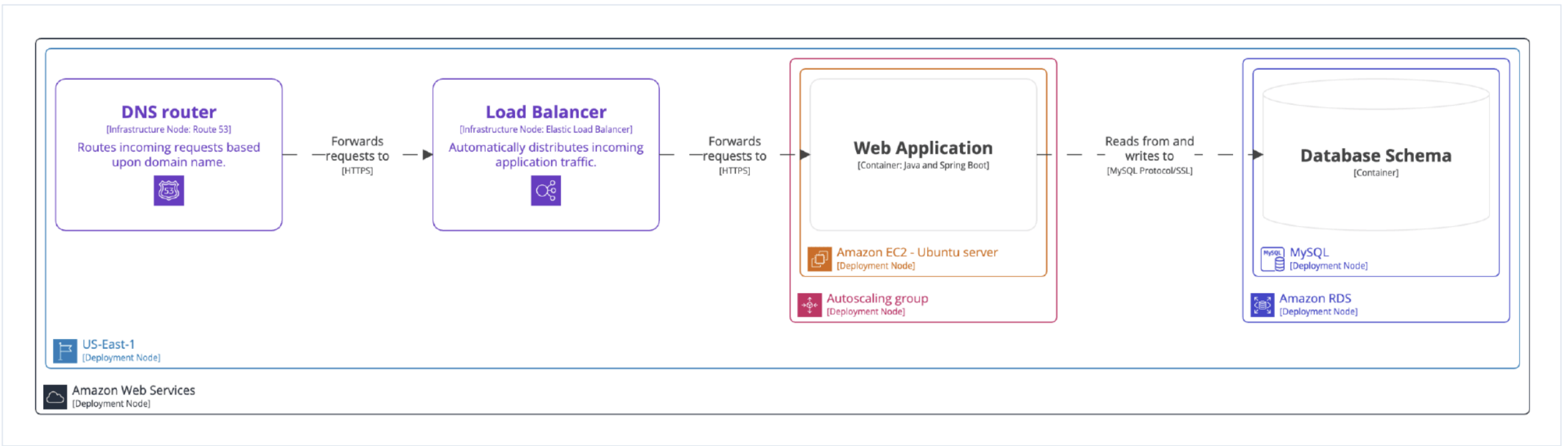
- container 가 infrastructure(서버, VM, 클라우드 서비스 등)에 어떻게 매핑되는지 기술한다
- 예: Web Server container 는 Developer Laptop 의 웹 서버에, Backend 는 JVM 에, Database 는 DB 서버에 배치
- Big Bank Data Center 의 Core Banking System 처럼 외부 시스템의 위치도 함께 표시한다



Development 환경의 Deployment Diagram (Internet Banking System)

Deployment Diagram 예제 — AWS 배치

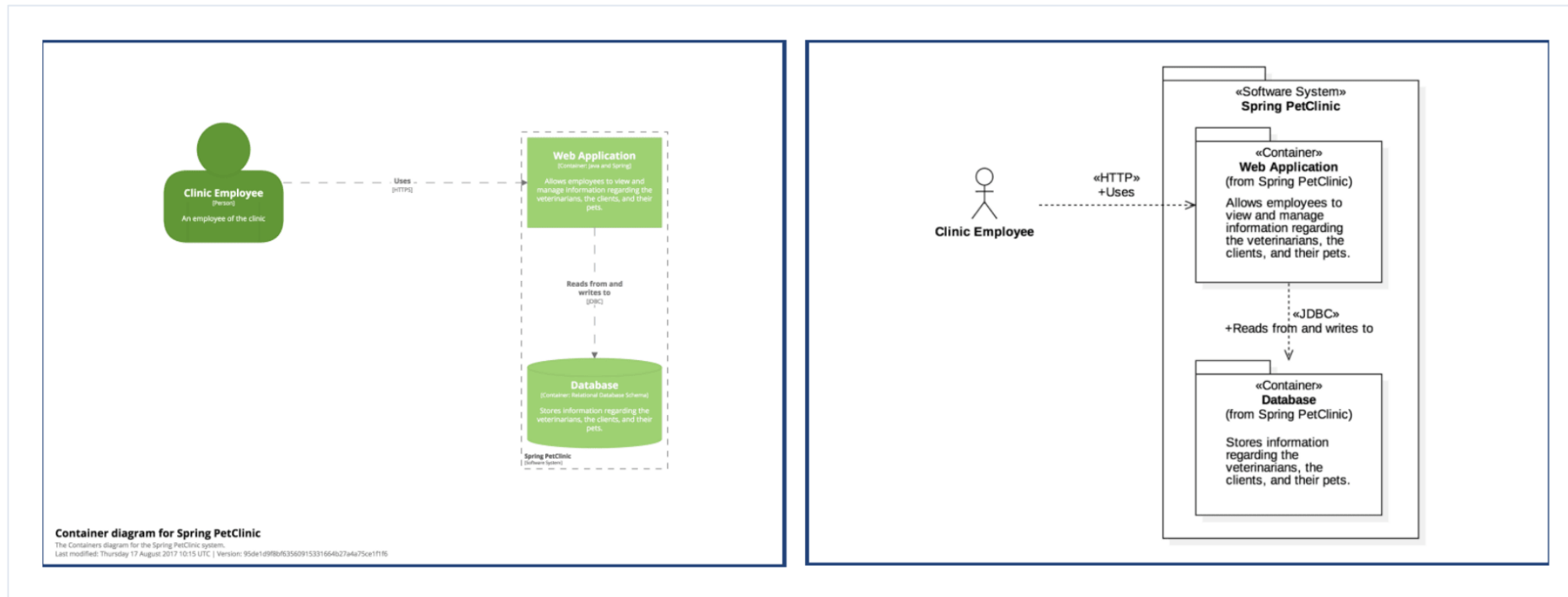
- DNS router → Load Balancer → Web Application → Database Schema 로 이어지는 배치 흐름을 표현한다
- 각 상자에 AWS 서비스 종류(Route 53, ELB, EC2, RDS 등)를 함께 적어 어떤 infrastructure 에 올라가는지 보여 준다



AWS 인프라에 배치된 Web Application 예제

C4 Model 표기법 — 자유로운 표기

- C4 Model 은 특정 표기법을 강제하지 않는다: 같은 내용을 색이 있는 상자로도, UML 스타일 stereotype 상자로도 그릴 수 있다
- 다만 요소의 이름·유형·기술·책임이 드러나도록 일관된 규칙을 지키는 것이 중요하다



같은 Container Diagram 을 두 가지 표기로 그린 예

C4 Model 표기법 — Boxes

■ Boxes (요소)에 적는 내용

- 요소 이름, 유형, 기술(해당되는 경우), 설명/책임을 함께 적는다
- 유형은 [Person], [Software System], [Container: 기술], [Component: 기술] 처럼 대괄호로 표시한다

■ 예

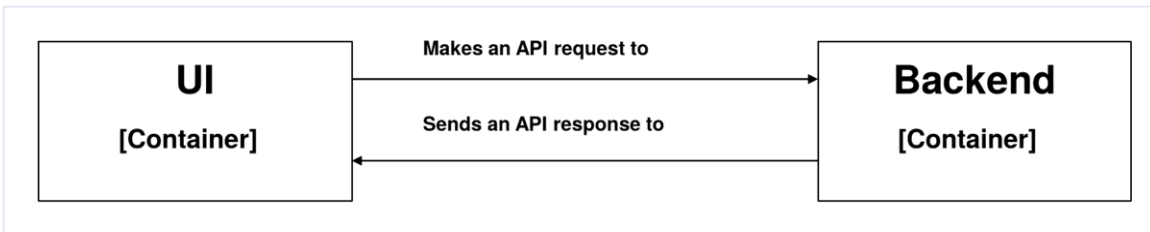
- Personal Banking Customer [Person] — 하나 이상의 개인 계좌를 가진 은행 고객
- Internet Banking System [Software System] — 고객이 웹으로 계좌 정보를 보고 송금하게 해 준다
- Backend [Container: Java and Spring Boot] — JSON/HTTPS API 로 인터넷 뱅킹 기능을 제공한다
- Core Banking System Adapter [Component: Spring Bean] — Core Banking System API 의 Java wrapper

핵심 상자 하나만 읽어도 그 요소가 무엇이고, 무슨 기술로, 무슨 책임을 지는지 알 수 있어야 한다

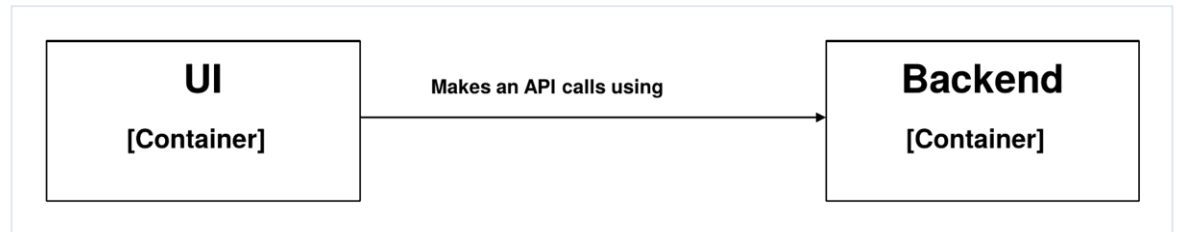
C4 Model 표기법 — Lines

■ Lines (관계)

- 가장 중요한 종속성이나 데이터 흐름을 보여 주는 단방향 선을 선호한다
- 선의 목적과 방향을 설명하는 주석을 붙인다 (예: Makes API calls using)
- 요청/응답을 각각 그린 양방향 선보다, 의도를 드러내는 단방향 선 하나가 읽기 쉽다



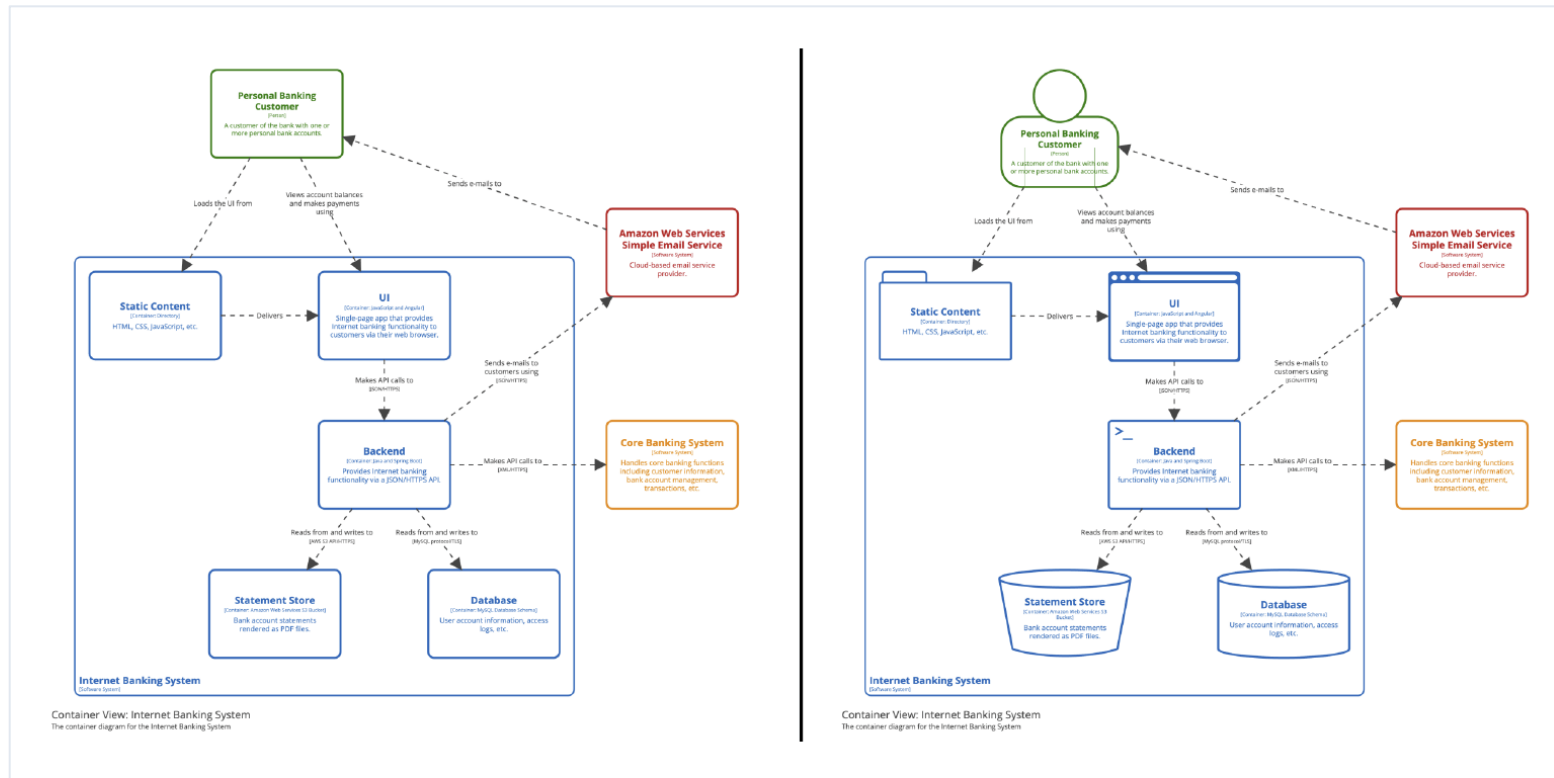
요청/응답을 따로 그린 양방향 선



목적을 적은 단방향 선 (권장)

C4 Model 표기법 — 모양·색상·크기 활용

- 모양, 색상, 크기를 사용해 다이어그램을 보완할 수 있다
- 예: 사람은 사람 모양, 데이터베이스는 원통형으로 그리고, 외부 시스템은 다른 색으로 구분해 읽기 쉽게 만든다



기본 상자 표기(왼쪽)와 모양·색상으로 보완한 표기(오른쪽)

C4 Model vs. SEI View Model

	C4 Model	SEI View Model
주요 초점	시스템의 정적 구조(static structure)를 계층적으로 분해하는 것에 집중	이해관계자별 시스템 구조 표현을 강조
View 구성	Context Diagram Container Diagram Component Diagram Code Diagram Deployment Diagram (선택)	C&C View, Allocation View C&C View Module View, C&C View Module View Allocation View
추상화 방식	포함 관계 기반의 계층적 분해: 시스템 → 컨테이너 → 컴포넌트 → 코드	직교적/독립적(Orthogonal): 세 View 는 서로 다른 관심사를 다루며 동시에 존재한다

C4 Model vs. Hierarchical Multi-View Model

	C4 Model	Hierarchical Multi-View Model
주요 초점	시스템의 정적 구조(static structure)를 계층적으로 분해하는 것에 집중	설계 단계별로 점진적인 분해 과정을 강조
View 구성	Context Diagram Container Diagram Component Diagram Code Diagram Deployment Diagram (선택) Dynamic Diagram (선택)	— Top-Level Design – Structure View (Component Diagram) Component-Level Design – Structure View (Class Diagram) (위와 같음) Deployment View (Deployment Diagram) Behavior View (Interaction Diagram)
추상화 방식	포함 관계 기반의 계층적 분해: 시스템 → 컨테이너 → 컴포넌트 → 코드	관심사 기반의 기능적 분해: 기능, 동시성, 구현, 배포, 사용 사례(시나리오)

Summary — Architectural Views

■ 지금까지 살펴본 Architecture 문서화 기법

- 기본 표현(primary representation) 문서화: C&C View, Module View, Allocation View
- 요소의 interface 와 behavior 문서화
- Architecture 결정과 그 근거(rationale) 문서화

■ 핵심 포인트

- 문서화는 단순히 상자와 선을 그리는 일이 아니다
- View 마다 서로 다른 표기법이 필요하다
- 상세 수준은 문서의 사용 맥락에 따라 달라진다
- 좋은 문서를 만드는 일은 결코 사소한 작업이 아니다

핵심 어떤 View 를, 누구를 위해, 어느 수준으로 문서화할지 먼저 정하는 것이 좋은 Architecture 문서의 출발점이다